

IBM Storage Scale Container Native Storage Access 6.0.0



Tables of Contents

Overview	1
What's new?	2
Supported features	3
Limitations	4
Technology preview	4
Planning	5
Overview	5
Hardware requirements	6
Software requirements	8
Deployment considerations	9
GPU Direct Storage	11
Networking	13
Network and firewall requirements	13
Container Network Interface (CNI) configuration	15
NVIDIA Networking	17
NVIDIA Network Operator	17
RDMA over InfiniBand	19
RDMA over Converged Ethernet (RoCE)	19
Networking Scenarios and Examples	20
RoCE deployment with hostNetwork=false	20
IBM Storage Scale container native and SELinux	29
Roles and personas	30
Container images	31
Air-gapped installs	32
Storage cluster	34
On-premise	34
Cloud	36
Red Hat OpenShift Container Platform	37
Software requirements	38
Red Hat OpenShift configuration on-premise	38
Worker node configuration	38
Infrastructure node configuration	39
Master node configuration (compact cluster)	39
Red Hat OpenShift service on AWS	40
Support matrix for IBM Storage Scale container native and ROSA	40
Red Hat OpenShift configuration on AWS	40
Validating Red Hat OpenShift configuration	42
Kubernetes	43
Software requirements	43
Configuration	43
Migration from IBM Storage Scale CSI	44
Google Kubernetes Engine (GKE)	45
Software requirements	45
Configuration	45
Installing the IBM Storage Scale container native operator and cluster	46
Labels and annotations	47
Install	48
Container Images	49
Building the kernel portability layer	50
Kubernetes resources	51
Cluster	52
Callhome	64
RemoteCluster	66
Creating secrets for storage cluster GUI users	69
Configuring Certificate Authority (CA) certificates	70
File Systems	72
Remote file system	72
Local file system	74
Encryption	78
CacheVolumeOperation	80

Validating installation	81
Verifying the IBM Storage Scale container native cluster	81
Status and events	83
Using IBM Storage Scale GUI	84
GUI Access	84
GUI User Management	84
Upgrading IBM Storage Scale container native	84
Supported upgrade paths	85
Upgrade considerations for CSI volumes	85
Upgrade	86
Verification	89
Post upgrade tasks	89
Upgrading Google Kubernetes Engine (GKE)	92
Configuring IBM Storage Scale Container Storage Interface (CSI) driver	95
Configuring storage class to use CSI driver	95
Managed CSI fields	96
Caching data from object storage	97
Caching modes	97
Kubernetes resources	97
Storage class for cache volume	98
Creating cache volumes	98
Tuning cache configuration parameters	101
Maintenance	101
Shutting down a cluster	102
IBM Storage Scale container native cluster and node maintenance	102
Nodes	102
Draining nodes	102
Adding new nodes to an existing cluster	103
Deleting nodes from a cluster	104
Starting the cluster after shutdown	107
IBM Storage Scale Storage cluster	107
Updating remote mount access key on IBM Storage Scale storage cluster	107
Updating authentication to the storage cluster	108
File systems	109
Local filesystems	109
Add disks	109
Delete disks	110
Maintenance mode	111
Cleanup	111
Removing applications	112
Cleanup Kubernetes resources	112
Remote file systems	112
Remote clusters	113
Local file systems	113
Cluster	114
Cleanup IBM Storage Scale CSI	114
Cleanup IBM Storage Scale container native	114
Cleanup nodes	115
Red Hat OpenShift	115
Kubernetes	115
Cleanup storage cluster	116
Cleanup on AWS ROSA	116
Monitoring	117
System monitor and Kubernetes readiness probe	117
Viewing and analyzing the performance data with the IBM Storage Scale bridge for Grafana	117
Support	118
Known issues	118
Pod Issues	118
A GUI pod is stuck in 3/4 running containers with multiple restarts	
Error: daemon and kernel extension do not match	119
Error: could not insert module, required key not available	120

MountVolume.SetUp failed for volume "ssh-keys"	
Unable to retrieve some image pull secrets	120
GUI or Grafana bridge pods fails to start, no data returned from pmcollector to frontend applications	
pmcollector pod is in pending state during OpenShift Container Platform upgrade or reboot	122
pmsensors that shows null after failure of pmcollector node	123
pmcollector pods are not in Running state	123
pid_limits set higher than podPidLimits, but not being honored	124
Upgrade failed due to PodsUnsafeToDelete: Unknown safety with reason	124
Core pods restart constantly because mount parameters have changed	125
Cluster Issues	125
NumaMemoryInterleave is not set to yes	125
Remote cluster Issues	125
RestError: failed to get storage cluster information. errmsg: 401 Unauthorized GET or PasswordsOutOfSync	125
Failed to establish remote cluster connection when cacert ConfigMap does not exist	126
Adding a Remote Cluster to an existing IBM Storage Scale container native cluster taking a long time to appear	126
File system issues	127
Remote file systems are defined but not mounted on all nodes	127
Remote file systems unable to mount successfully	127
Cache volume issues	127
Empty cache mode and health status in CacheVolume CR	127
Upgrade Issues	127
tls errors found in the operator log	127
Red Hat OpenShift Node issues	128
Adding a node fails - the node appears to already belong to a GPFS cluster	
Red Hat OpenShift Service on AWS	129
Error: InvalidGroup.NotFound	
Error: CSI pods in CrashLoopBackOff	129
Troubleshooting	129
Troubleshooting operator	130
Troubleshooting deployment	131
Troubleshooting custom resources	133
Remote Cluster	133
Failed to register remote storage cluster on local GUI REST-API	133
Failed login attempts timeout (403 error)	
Troubleshooting IBM Storage Scale Container Storage Interface (CSI)	133
Troubleshooting PVCs	135
Troubleshooting Pods	135
Troubleshooting Red Hat OpenShift	136
GUI mounts not getting refreshed	136
Recovery of cluster after Red Hat OpenShift node failure	137
Troubleshooting cluster maintenance	138
Gathering data about your cluster	140
Red Hat OpenShift Container Platform	140
must-gather	140
Kernel crash dumps	141
Kubernetes	141
must-gather	141
Generating GPFS trace reports	142
Generating GPFS kernel thread dumps	142
References	143
IBM Storage Scale	143
Red Hat OpenShift or Kubernetes	143
Notices	143
EU Data Act (Regulation 2023/2854)	143
Product documentation in guide format	145

Overview

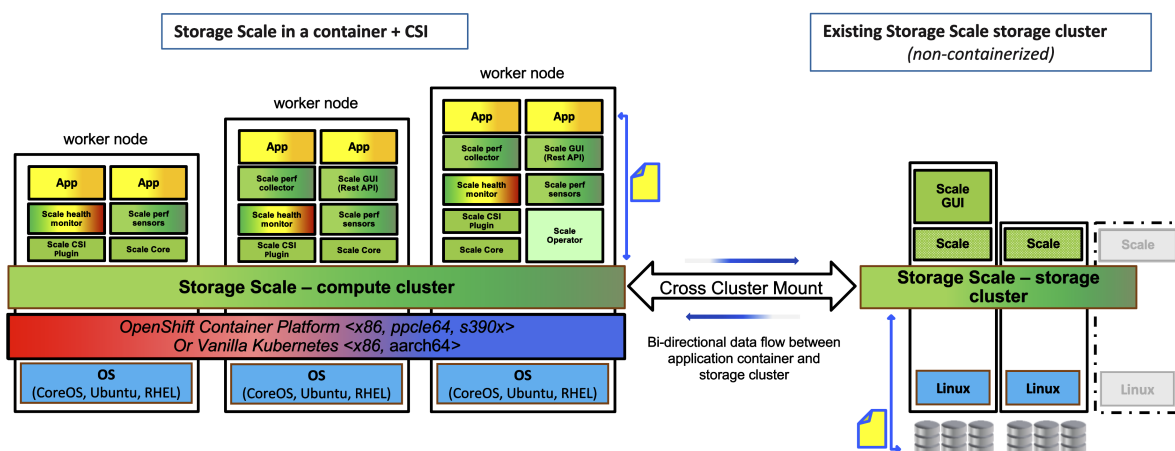
IBM Storage Scale container native is a containerized version of IBM Storage Scale.

IBM Storage Scale is a clustered file system that provides concurrent access to a single file system or set of file systems from multiple nodes. The nodes can be SAN attached, network attached, or a mixture of SAN attached and network attached. IBM Storage Scale enables high-performance access to this common set of data to support a scale-out solution or to provide a high availability platform. For more information about IBM Storage Scale features, see [Product overview](#) in IBM Storage Scale documentation.

IBM Storage Scale container native allows the deployment of IBM Storage Scale in a Red Hat OpenShift cluster. IBM Storage Scale container native deployment makes a persistent data store available by either IBM Storage Scale remote file system or local file system. This data store can be used by the applications through the IBM Storage Scale CSI driver by using Persistent Volumes (PVs). For more information, see [IBM Storage Scale Container Storage Interface Driver](#) in the IBM Storage Scale CSI documentation.

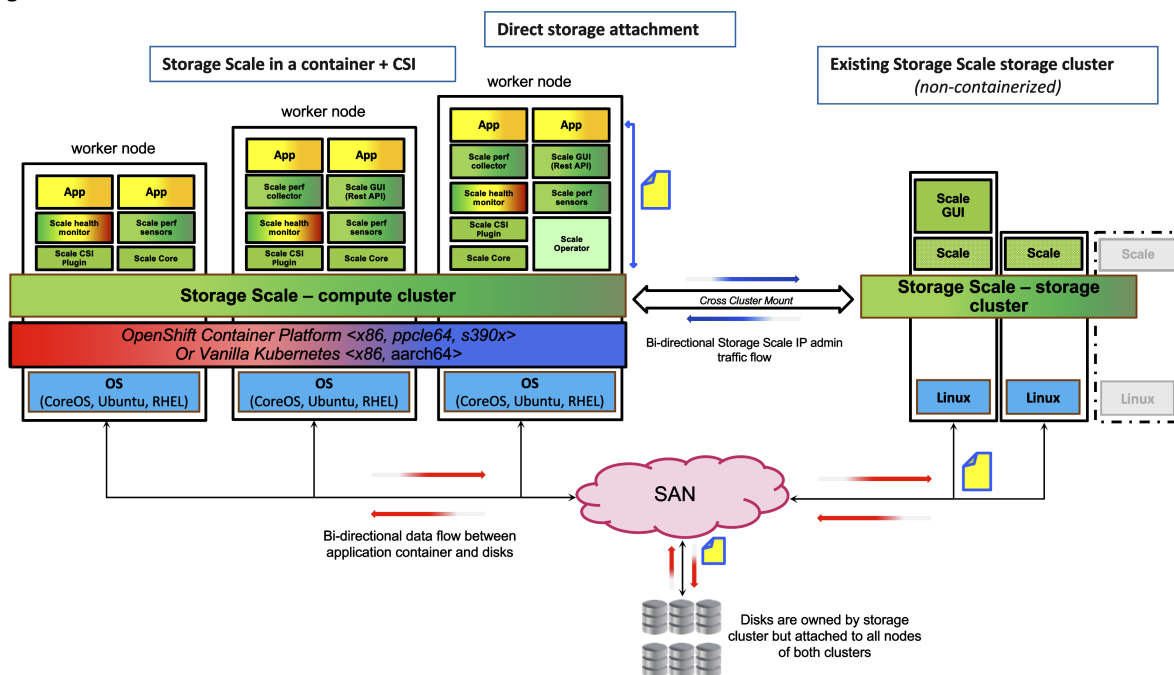
Remote mount use case

Figure 1. Remote mount



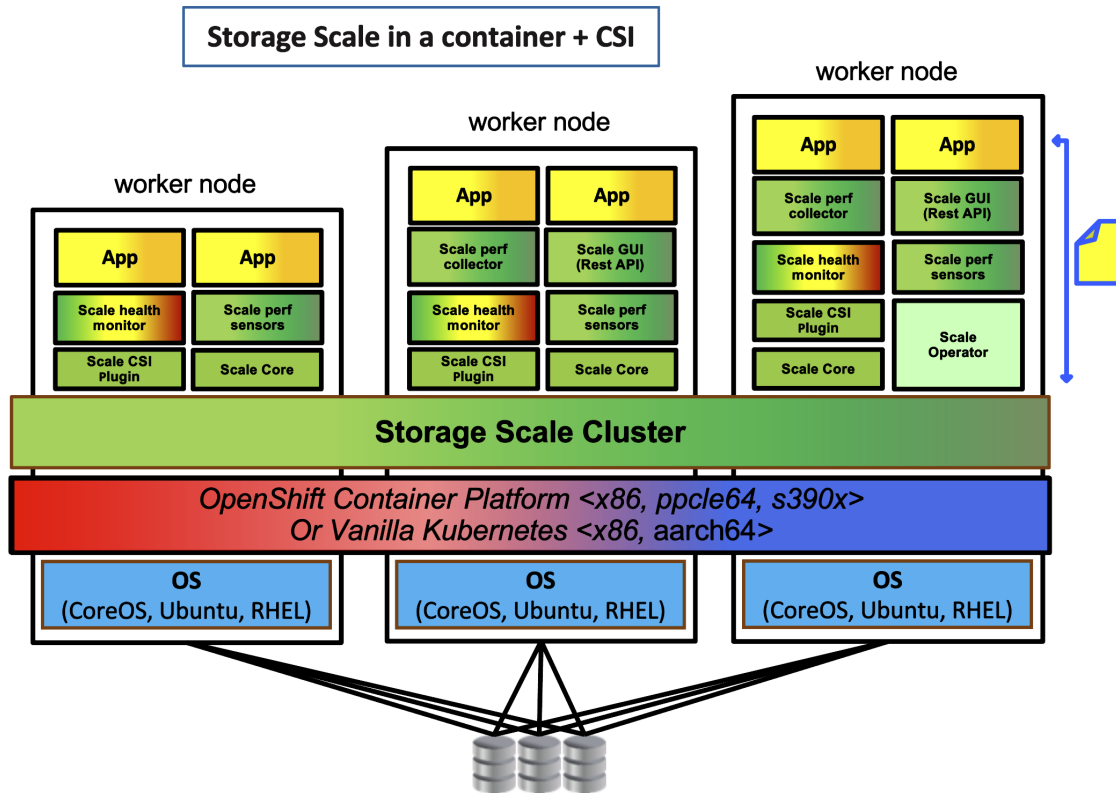
Direct attach use case

Figure 2. Direct attach



Local file system use case

Figure 3. Local file system



- [What's new?](#)
- [Supported features](#)
- [Limitations](#)
- [Technology Preview](#)

What's new?

The following enhancements are made in this release:

6.0.0.2

- Container Storage Interface (CSI) 3.0.2. For more information, see [Summary of changes](#) in IBM Storage Scale CSI Documentation
- Support for Red Hat OpenShift v4.21 with IBM Storage Scale container native v6.0.0.2. For more information, see [Red Hat OpenShift Software Requirements](#) for a full list of supported versions.
- Support for Kubernetes v1.34 with IBM Storage Scale container native v6.0.0.2. For more information, see [Vanilla Kubernetes Software Requirements](#)
- Technology Preview: IBM Storage Scale container native deployed in a Google Kubernetes Engine (GKE) added Kubernetes v1.34 and newer kernel levels.
 - COS milestone 113 [6.1.x] - kernel 6.1.90 - 6.1.155
 - COS milestone 117 [6.6.x] - kernel 6.6.72 - 6.6.123
 - COS milestone 121 [6.6.x] - kernel 6.6.74 - 6.6.122
 - For more information, see [Support for Google Kubernetes Engine \(GKE\)](#).

6.0.0.1

- Container Storage Interface (CSI) 3.0.1. For more information, see [Summary of changes](#) in IBM Storage Scale CSI Documentation
- Support for Red Hat 9.7
 - [Vanilla Kubernetes Software Requirements](#)

6.0.0.0

- Container Storage Interface (CSI) 3.0.0. For more information, see [Summary of changes](#) in IBM Storage Scale CSI Documentation
- Support for Red Hat OpenShift v4.20 with IBM Storage Scale container native v6.0.0.1. For more information, see [Red Hat OpenShift Software Requirements](#) for a full list of supported versions.
- Support for aarch64 (ARM64) architecture on Vanilla Kubernetes and Google Kubernetes Engine (GKE). For more information, use the following links for a full list of supported versions.
 - [Vanilla Kubernetes Software Requirements](#)
 - [Google Kubernetes Engine \(GKE\) Software Requirements](#)
- Support for deployment of NVIDIA MOFED pods with IBM Storage Scale container native. For more information, see [NicClusterPolicy CR](#)
- Support for GPU Direct Storage (GDS) with IBM Storage Scale container native. For more information, see [GPU Direct Storage](#)
- Removal of primary filesystem and fileset dependency. For more information, see [Upgrade Considerations for CSI Volumes](#)
- Migration support from an existing IBM Storage Scale CSI driver cluster to a new IBM Storage Scale Container Native cluster on Vanilla Kubernetes. For more information, see [Migration from IBM Storage Scale CSI](#)
- Technology Preview: IBM Storage Scale container native deployed in a Google Kubernetes Engine (GKE) v1.30, v1.31, v1.32, v1.33
 - COS milestone 113 [6.1.x] - kernel 6.1.77 - 6.1.144
 - COS milestone 117 [6.6.x] - kernel 6.6.72 - 6.6.97
 - COS milestone 121 [6.6.x] - kernel 6.6.74 - 6.6.97
 - For more information, see [Support for Google Kubernetes Engine \(GKE\)](#).

Supported features

The IBM Storage Scale container native operator automatically deploys and manages an IBM Storage Scale cluster in Red Hat OpenShift and Kubernetes environments.

The following table shows functions supported by IBM Storage Scale container native:

Table 1. IBM Storage Scale container native functions

Function	Red Hat OpenShift	Red Hat OpenShift Service on AWS (ROSA)	Vanilla Kubernetes	Google Kubernetes Engine (GKE) - technology preview
Remote Cluster File system	Yes	Yes	Yes	Yes
Shared Disk Local File system	Yes	No	Yes	No
AFM PV Caching	Yes	Yes	Yes	Yes
Encryption	Yes	No	Yes	No
Host Networking (default)	Yes	Yes	Yes	Yes
CNI Networking	Yes	Yes	Yes	Yes
InfiniBand RDMA	Yes	Yes	Yes	Yes
RDMA over Converged Ethernet (RoCE)	Yes	Yes	Yes	Yes
Integrated CSI ¹	Yes	Yes	Yes	Yes
Cluster monitoring	Yes	Yes	Yes	Yes
Call Home	Yes	Yes	Yes	Yes
Performance data collection	Yes	Yes	Yes	Yes
Grafana support	Yes	Yes	Yes	Yes
Rolling and parallel upgrade	Yes	Yes	Yes	Yes
Compression	Yes	Yes	Yes	Yes
Direct attach on remote mount	Yes	No	Yes	No
Topology aware quorum selection	Yes	Yes	Yes	Yes
3-node Kubernetes cluster	Yes	n/a	Yes	n/a
Infrastructure node utilization	Yes	Yes	n/a	n/a
IBM Storage Scale GUI for cluster monitoring	Yes	Yes	Yes	Yes
Air Gap installation	Yes	n/a	Yes	Yes
Architecture: x86_64	Yes	Yes	Yes	Yes
Architecture: ppc64le	Yes	No	No	No
Architecture: s390x	Yes	No	No	No
Architecture: aarch64	No	No	Yes	Yes ²
Architecture: mixed	Yes	No	Yes	Yes

The following table shows functions that are inherited from the remote mounted storage cluster:

Table 2. Function inherited from remote mount storage cluster

Function	Inherited from the storage cluster
File Audit Logging	Yes
ACLs: mmacl commands	Yes
Information Lifecycle Management (ILM): migration	Yes
Snapshots ³	Yes
Watch Folders	Yes

1. For a detailed list of functions supported by IBM Storage Scale CSI, see [Features covered](#).
2. IBM Storage Scale container native only supports Ubuntu on Google Kubernetes Engine (GKE) using the aarch64 architecture.
3. Function can be utilized on the storage cluster but administrators should make sure it does not overlap with CSI invoked functionality.

Limitations

- IBM Storage Scale container native supports
 - Remote mount of the file system
 - Local file system with shared disks
- Deployment of IBM Storage Scale pods on master nodes is not supported except for "compact" Red Hat OpenShift clusters.
- Single node IBM Storage Scale container native clusters are not supported.

Scalability constraints

Overall

Table 1. Maximum Capacity Specification

Description	Max supported
Number of worker nodes	128

Remote file system

Table 2. Maximum Capacity Specification for Remote File Systems

Description	Max supported
Number of remote clusters	4
Number of remote file systems	16

Local file system

Table 3. Maximum Capacity Specification for Local File Systems

Description	Max supported
Number of local file systems	3
Number of storage nodes	32
Number of disks per local file system	128

Technology preview

Technology preview features are not supported for use within production environments. Use with non-production workloads, in demo or proof of concept environments only. IBM production service level agreements (SLAs) are not supported. Technology preview features may not be functionally complete. The purpose of technology preview features is to give access to new and exciting technologies, enabling customers to test functionality and provide feedback during the development process. The existence of a technology preview feature does not mean a future release is guaranteed. Feedback is welcome and encouraged.

Current features in technology preview

Support for Google Kubernetes Engine (GKE)

During qualification a problem with upgrades on GKE was found. As a result, access to the Storage Scale filesystems might be lost and complex manual recovery necessary. Teams from IBM and Google are working on a resolution. As long as this resolution does not exist, production environments cannot be supported. Therefore deploying IBM Storage Scale container native in Google Kubernetes Engine (GKE) is a technology preview. For more information, see [Google Kubernetes Engine \(GKE\)](#) and [Google Kubernetes Engine \(GKE\) cluster](#).

Use the `ccloudkit` CLI to assist creating a remote storage cluster in GKE. This is required when configuring a [Remote file system](#) in IBM Storage Scale container native. For more information on `ccloudkit`, see [Configuring your GCP cloud account](#) in the IBM Storage Scale documentation.

Replication in local file system

The replication function of a local file system is technology preview. For more information, see [Configure replication](#).

Planning

Installing an IBM Storage Scale container native cluster consists of many steps and is critical to follow the sequence of the documentation.

- [Overview](#)
- [Hardware requirements](#)
- [Software requirements](#)
- [Deployment considerations](#)
- [Networking](#)
 - [Network and firewall requirements](#)
 - [Container Network Interface \(CNI\) configuration](#)
 - [NVIDIA Networking](#)
- [IBM Storage Scale container native and SELinux](#)
- [Roles and personas](#)
- [Container images](#)
- [Air-gapped installs](#)

Overview

These sections outline the general process during the installation of the IBM Storage Scale container native cluster. Each step is explained in detail in the chapters that follow.

Consider the following questions:

- What version of Red Hat OpenShift Container Platform or Vanilla Kubernetes is installed?
- What are the hardware requirements?
- Are all the necessary ports opened in your environment?
- Is the cluster in a restricted network environment?
- What is the minimum level of IBM Storage Scale installed on the storage cluster?

Preparations for deploying the IBM Storage Scale container native cluster

Before deploying the container native pods, some steps need to happen to prepare the Red Hat OpenShift or Vanilla Kubernetes cluster.

- Validate Red Hat OpenShift or Vanilla Kubernetes cluster has internet access to the YAML files in the IBM Storage Scale container native repository of GitHub: [IBM Storage Scale container native](#).
- If you are in a restricted network environment (air-gap), mirror the container images for IBM Storage Scale container native into a site-managed private image registry.

The YAML files provided inline in the installation steps need to be made available in the air-gap environment.

- For the Red Hat OpenShift cluster, apply and validate the Machine Config settings are updated on all nodes for IBM Storage Scale container native cluster.
- Obtain an entitlement key to access the container images in the IBM Cloud Container Registry. This entitlement key is used to create namespace pull secrets. For Red Hat OpenShift, optionally add an entry into the global pull secret.

Deploying the IBM Storage Scale container native cluster

To deploy a cluster, complete the following steps:

1. Create the IBM Storage Scale container native and IBM Storage Scale CSI operators by deploying the operator installer file: [install.yaml](#).
2. Download the sample [cluster.yaml](#) CR, make necessary change, and apply to the cluster.
 - Specify the IBM Storage Scale Edition in the license field.
 - Configure appropriate node selectors for the IBM Storage Scale container native deployment.
 - Enable the optional Grafana Bridge.
3. Download the sample [callhome.yaml](#) CR, make the necessary changes, and apply to the cluster.

Creating a file system

Remote Mounted file system

1. Download the sample [remotecluster.yaml](#) CR, make the necessary changes, and apply to the cluster.

Configure host aliases in the Cluster CR, or configure the proper DNS for your environment to allow for communication to the storage cluster.
2. Download the sample [filesystem.remote.yaml](#) CR, make the necessary changes, and apply to the cluster.
3. If accessing encrypted data on the storage cluster, download the sample [encryptionconfig.remote.yaml](#) CR, make necessary changes, and apply to the cluster.
4. Complete the storage cluster configuration.
 - a. Create a GUI user on the storage cluster with the **ContainerOperator** role.
 - b. Create a GUI user on the storage cluster with the **CsiAdmin** role.
 - c. Configure CSI prerequisites on the storage cluster.
5. Use the storage cluster GUI user credentials for **ContainerOperator** GUI user to create a secret in the **ibm-spectrum-scale** namespace.
6. Use the storage cluster GUI user credentials for **CsiAdmin** GUI user to create a secret in the **ibm-spectrum-scale-csi** namespace.
7. Create a storage class to create volumes to use with your container native cluster.

Local file system

1. Download the sample [filesystem.local.yaml](#) CR, make the necessary changes, and apply to the cluster.
2. Download the sample [localdisk.yaml](#) CR, make the necessary changes, and apply to the cluster
3. Create a storage class to create volumes to use with your container native cluster.

Hardware requirements

The following sections describe hardware requirements to consider to deploy an IBM Storage Scale container native.

Network

- All nodes in the compute cluster must be able to communicate with all nodes in the storage cluster.
- A minimum of 10 Gb network is needed but 40 - 100 Gb is recommended.

Worker node requirements

IBM Storage Scale container native supports many architectures and combinations. Confirm all the worker nodes are supported.

- [Red Hat OpenShift Software Requirements](#)
- [Vanilla Kubernetes Software Requirements](#)
- [Google Kubernetes Engine \(GKE\) Software Requirements](#)

Starting with IBM Storage Scale container native v5.1.9, the UBI 9 base image is used and it is based on RHEL 9. And as RHEL 9 no longer supports Power8, IBM Storage Scale container native also no longer supports Power8. For more information, see [Minimum IBM Power requirements](#).

IBM Storage Scale container native deploys several pods in the cluster. The following table shows the resource consumption of those pods.

Table 1. Hardware requirements

Pod	Where deployed	CPU request	Memory request	Storage	Notes
core (created with the k8s Node shortname)	Nodes that are labeled with nodeSelector in cluster CR	Minimum 1000mCPU. The sample CR sets 2000mCPU.	Minimum 4GiB (2 GiB on s390). The sample CR sets 4GiB for client role.	config in <code>/var</code> (~25 GiB)	This is the pod that provides the file system service for the node. It must be deployed on all nodes where PVs are accessed from application pods. The CPU and memory requests and limits can be customized in the cluster CR.
operator	Single Node	500mCPU	200 MiB		The controller runtime that manages all custom resources
gui	Two Nodes	630mCPU	1.4 GiB		The graphical user interface and ReST API
pmcollector	Two Nodes	120 mCPU	5.2GiB	Local PV/Dynamic provisioned PV for DB	The performance collector database
grafana-bridge	Single Node	100mCPU	1GiB		The bridge for accessing pmcollector from grafana

The shown values are requests. For more information, see [Kubernetes resource management](#). The limits are higher. This means that for CPU the pods might have bursts with more CPU usage at times where the CPU has free cycles. For memory the pods should not exceed their request significantly. By default the core pods take 25% of the worker node capacity. 25% might be oversized in many applications. For more information about configuring the requests for both CPU and memory, see [Cluster custom resource](#).

- * Allocating more resources to IBM Storage Scale results in better storage performance.
- * Allocating less allows more applications to be scheduled on a node.

For CPU, allocation can be reduced if the core fs pods consistently stay under the request. The CPU allocation can be monitored on the Red Hat OpenShift console. When going too low the file system daemon might starve on CPU cycles that destabilizes the whole cluster and can result in outages. For memory, there is no real monitoring, allocating more results in more data is cached which can boost performance. But this will be only seen indirectly by observing application performance.

The CPU request can be reduced under the 1000mCPU minimum. Your system might run fine with, for example, a 100mCPU. But, if a service ticket is opened for an issue that might be in any way that is related to this setting you will be asked to go up to 1000mCPU. The ticket is accepted only if the problem keeps showing up. Examples for related issues are, node expels, lag on PV creation in CSI, slow policy runs, bad performance, long servers, and so on. The Red Hat OpenShift console reports all worker nodes as overcommitted. The reason is that the CPU and memory limits of the pods add up to more than then total capacity of the node. This is normal and no reason for concern. Pods are scheduled based on their requests and the scheduler ensures that nodes will not be overcommitted in this regard. Higher limits allow pods to use resources that are free at the moment, but only the requested resources are guaranteed to them by Kubernetes. For more information about pod scheduling, see [Kubernetes resource management](#)

Pod memory request and limits

The following table shows the memory requests and limits configuration for each pod created.

Table 2. Pod memory requests and limits

Namespace	Pod name	Container	Memory request \	limit	CPU request \	limit
ibm-spectrum-scale	Grafana Bridge	grafanabridge	1000Mi \	4000Mi	100 m \	500 m
ibm-spectrum-scale	GUI	liberty	750Mi \	1000Mi	500 m \	2
ibm-spectrum-scale	GUI	postgres	400Mi \	1Gi	100 m \	1
ibm-spectrum-scale	GUI	sysmon	200Mi \	500Mi	20 m \	100 m
ibm-spectrum-scale	GUI	logs	20Mi \	60Mi	10 m \	100 m
ibm-spectrum-scale	PM Collector	pmcollector	5000Mi \	10000Mi	100 m \	500 m
ibm-spectrum-scale	PM Collector	sysmon	200Mi \	500Mi	20 m \	100 m
ibm-spectrum-scale	Core	gpfs	4Gi \	8Gi ¹	2 \	4 ²
ibm-spectrum-scale	Core	logs	20Mi \	60Mi	10 m \	100 m
ibm-spectrum-scale-operator	controller-manager	manager	200Mi \	400Mi	500 m \	1500 m
ibm-spectrum-scale-dns	CoreDNS	coredns	70Mi \	140Mi	50 m \	100 m

For more information, see [Requests and limits](#) in the Kubernetes documentation.

Local file system

The Red Hat OpenShift nodes can provide disks or volumes to be used as local storage for creating PVs. These disks or volumes serve as storage for a local file system. A disk or volume can be a single drive, a partition of a single drive, or a volume from a RAID controller. Disks must be attached to the Red Hat OpenShift nodes as shared disks or SANs, each disk must have a connection to at least 2 nodes.

The storage device that provides the disks must protect against losing data, for example by using replication, RAID or erasure coding.

VMware³ thin provisioned virtual disks are not supported. For more information, see the "Disk Questions" section in [IBM Storage Scale FAQ](#).

1. The memory limit is set to double of request memory by default. It can be set to another custom value into the **Cluster** custom resource. The new custom value must be at least twice as the memory request value. [↗](#)
2. The CPU limit is set to double of the CPUs of request by default. This can also be modified into the **Cluster** custom resource, but the new custom value must be at least twice as the CPU requested value. [↗](#)
3. VMware, the VMware logo, VMware Cloud Foundation, VMware Cloud Foundation Service, VMware vCenter Server, and VMware vSphere are registered trademarks or trademarks of VMware, Inc. or its subsidiaries in the United States and/or other jurisdictions. [↗](#)

Software requirements

Use the following tables to determine the software versions for each release.

Table 1. IBM Storage Scale versions

IBM Storage Scale Container Native	IBM Storage Scale CSI	Minimum IBM Storage Scale Remote Storage cluster level	File system version cannot be newer than
5.2.3.9	2.14.8	5.1.9.0	37.00
6.0.0.0	3.0.0	5.2.0.0	38.00
6.0.0.1	3.0.1	5.2.0.0	38.00
6.0.0.2	3.0.2	5.2.0.0	38.00

See the following tables for software requirements specific to different container platforms.

Table 2. Container platform specific software requirements

Container platform	Software requirements
Red Hat OpenShift	Software requirements for Red Hat OpenShift
Kubernetes	Software requirements for Kubernetes
Google Kubernetes Engine	Software requirements for Google Kubernetes Engine

It is allowed for the storage cluster to be at an earlier version than IBM Storage Scale container native client cluster. CSI functions are highly dependent upon the release, file system level, and version of IBM Storage Scale installed on the storage cluster. If the storage cluster is at an earlier version, some functions may not be available. For more information about the minimum levels required for specific CSI functions, see [Hardware and Software Requirements](#) in the IBM Storage Scale CSI documentation. For more information about compatibility and software matrix, see [Section 18.3](#) in the IBM Storage Scale FAQ documentation.

IBM Storage Scale container native doesn't support secure boot, the secure boot feature needs to be disabled for Red Hat OpenShift Container Platform nodes

IBM Storage Scale Container Storage Interface (CSI)

IBM Storage Scale Container Storage Interface (CSI) will be installed along with IBM Storage Scale container native.

Storage cluster

- The storage cluster must be at IBM Storage Scale v5.2.0.0 or later.
 - For cache volume support, the storage cluster must be at IBM Storage Scale v5.2.3.0 or later.
- To take advantage of all functions provided by IBM Storage Scale Container Storage Interface Driver (CSI), the storage cluster is recommended to be at the latest versions configured with the latest file system level format. If using earlier versions, functional restrictions may apply. For more information, see [Hardware and software requirements](#) in the IBM Storage Scale CSI documentation.

For more information, see [Upgrading multi-cluster environments](#) in IBM Storage Scale documentation.

- Determine whether the storage cluster is running a GUI high availability configuration, for example, having 2 or more GUI nodes installed.

On the storage cluster, issue the following command. If two or more GUI nodes are displayed, then the storage cluster is running a GUI high availability configuration.

```
/usr/lpp/mmfs/gui/cli/lsnode
```

For more information, see [Ensuring high availability of the GUI service](#) in IBM Storage Scale documentation.

- Enable the `--auto-inode-limit` parameter on the remotely mounted file system.

For more information about `auto-inode-limit` parameter, see [mmchfs command](#) in IBM Storage Scale documentation.

The `--auto-inode-limit` option is available only with file system format level 28.00 or later.

- Encrypted file systems are supported. Configure the EncryptionConfig custom resource with the necessary key client and key server information. For more information, see [Encryption](#).

Auxiliary helper applications

- `curl` is used to retrieve some files required for the IBM Storage Scale container native installation.
- `jq 1.5+` is used to help parse and format JSON output.

Deployment considerations

Before deployment, make sure that you are aware of the Red Hat OpenShift version, cluster network, persistent storage, and the IBM Storage Scale storage cluster considerations.

Red Hat OpenShift cluster considerations

The following list includes the Red Hat OpenShift cluster considerations:

- A minimum configuration of three master nodes and three worker nodes, with a maximum of 128 worker nodes is required.
- Deploying IBM Storage Scale pods on master nodes is not supported. An exception is in a "compact" cluster configuration. For more information, see [Master node configuration \(compact cluster\)](#).
- Single node Red Hat OpenShift clusters are not supported. Instead, access data on an IBM Storage Scale storage cluster through NFS.
- Red Hat Enterprise Linux™ CoreOS (RHCOS) restricts new file system mounts to the `/mnt` subtree. IBM Storage Scale mounts any file system under `/mnt` on the Red Hat OpenShift cluster regardless of the default mount point that is defined on the storage cluster.

Red Hat OpenShift cluster network considerations

The IBM Storage Scale container native comes with a collection of different pods. A subset of these pods can be considered regular pods that behave like typical application pods. Those pods are the operator, the GUI pods, and the performance data collector pods. The exception is what is referred to as the "core pods" as they provide the actual file system services. The core pods are not deployed by the Kubernetes scheduler through a regular DaemonSet. Instead, the IBM Storage Scale container native operator handles the management of those pods.

- The file system daemon inside the core pods requires a static IP address for communication between daemons on different nodes.
- All core pods must be able to communicate with each other through the chosen network.

There are two network configurations that can be employed: host network or container network interface (CNI) network. Only one network configuration can be chosen.

Host network

By default, the IBM Storage Scale pods use the host network. This is the simplest configuration but has some disadvantages:

- Using the host network breaks the network isolation that usually comes with containers. For example, any network port opened by IBM Storage Scale may conflict with a network port opened by another component on the host.
- Security features, like network policies, are not available for the host network.
- If the node has multiple network adapters, there is no way to select a different adapter. The host network always uses the network adapter that the worker node IP is assigned to.

Container Network Interface (CNI) network

As an alternative to host network, IBM Storage Scale can use a CNI network. There is more configuration effort to set up the CNI:

- In this configuration, core pods have an IP address on the usual Red Hat OpenShift SDN and another one on the CNI network.
 - Red Hat OpenShift SDN is used for communication with other pods.
 - CNI network is used for communication between file system daemons, both inter-cluster and with the storage cluster.
- If the node is equipped with high-speed network, the CNI interface should use the high-speed network.
 - This is the daemon network where the file system I/O runs on. High bandwidth and low latency are highly beneficial for performance.
- The CNI network will be used exclusively by IBM Storage Scale and eliminates the potential for port conflicts with other components.
- Security features like network policies work on **MACVLAN** CNIs.

- The DNS must be configured properly to allow the worker nodes the ability to resolve the storage cluster nodes.
 - For more information, see [Host aliases](#).

Advanced features of **SR-IOV** type CNIs, such as GPUdirect, are not yet supported.

For more information about configuring CNI with IBM Storage Scale container native, see [Container network interface \(CNI\) configuration](#).

Red Hat OpenShift cluster persistent storage considerations

The following list includes the Red Hat OpenShift cluster persistent storage considerations:

- The IBM Storage Scale pods use host path mounts to store IBM Storage Scale cluster metadata and various logs.
- The IBM Storage Scale container native operator creates two local PersistentVolumes (PVs) on two eligible worker nodes. At least 25 GB of free space must be available in the file system that contains the `/var` directory on all eligible worker nodes to avoid potential failures during the deployment. These PVs are created with the ReadWriteOnce (RWO) access mode.
- Both the host path mounts and local PVs are not automatically cleaned up when you delete the associated IBM Storage Scale container native cluster. You must manually clean these up. For more information about cleaning up the persistent storage, see [Cleaning up the worker nodes](#) and [Cleaning up IBM Storage Scale container native](#).
- IBM Storage Scale container native pmcollector supports the use of dynamically created PVs using specified storage class see [Pmcollector Volumes for Performance Data](#) but does not support pre-created PVs.

Considerations for enterprise grade image registry

The following list includes the considerations for enterprise grade image registry:

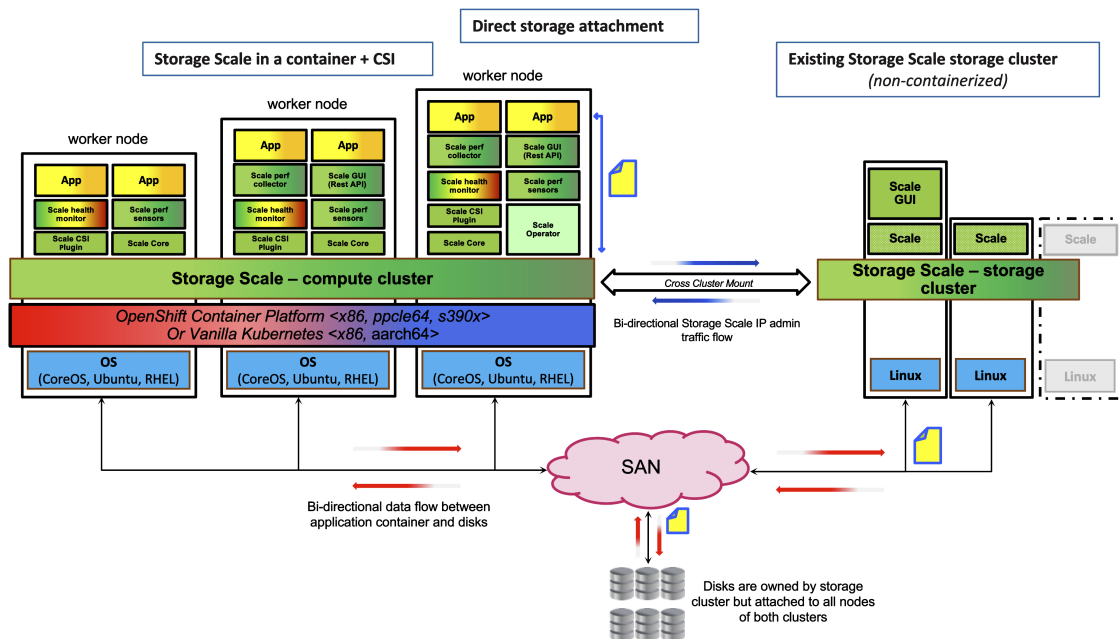
- In a restricted network environment where the Red Hat OpenShift Container Platform cluster cannot pull IBM Storage Scale images from the IBM Container Repository, images must be mirrored to a production grade enterprise image registry that the Red Hat OpenShift Container Platform cluster can access.
- In a restricted network environment, there must be a node that can communicate externally and also with the target Red Hat OpenShift Container Platform cluster.
- Any registry that is used for hosting the container images of IBM Storage Scale container native must not be accessible to external users. Also, it must be restricted to the service account used for IBM Storage Scale container native management. All users and machines that are accessing these container images must be authorized per the IBM Storage Scale license agreement.

Considerations for direct storage attachment

The following list includes the considerations for direct storage attachment:

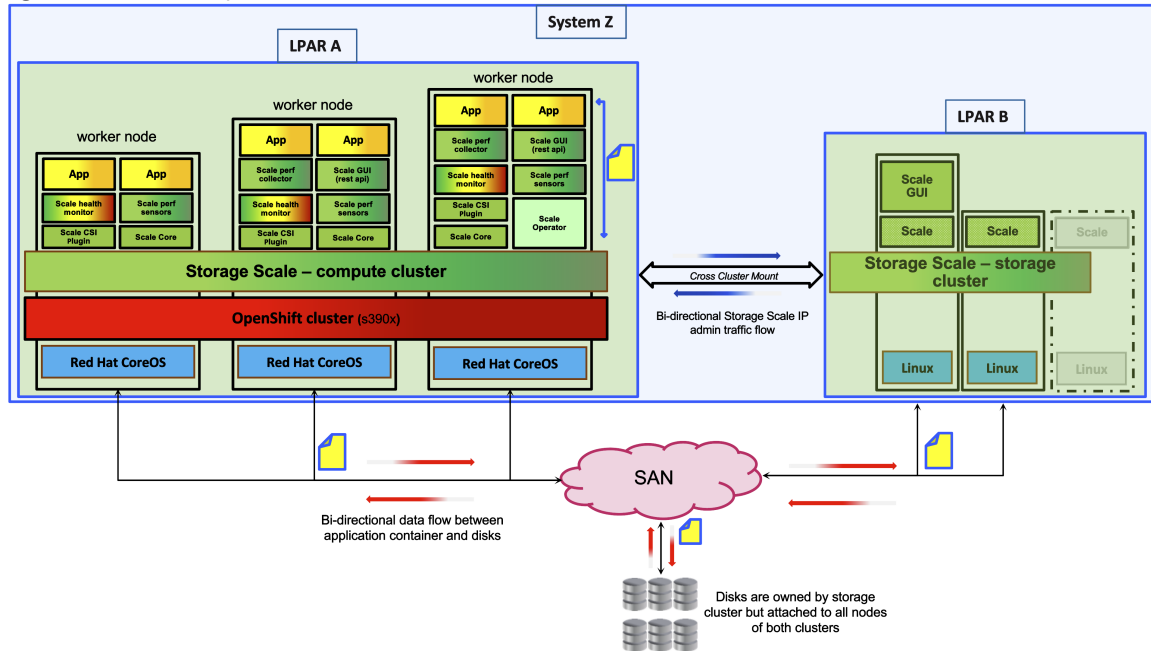
- Support for direct storage attachment on x86_64, ppc64le (Power), and s390x (IBM Z) servers. In direct storage attachment configuration, the worker nodes use the SAN fabric instead of the IBM Storage Scale NSD protocol for I/O traffic.
- If using x86_64 or ppc64le (Power) servers, it might be necessary to load multi-path drivers through Red Hat CoreOS before storage can be seen.

Figure 1. Direct attach config



- The virtualization layers of an IBM Z server allow the physical connection of the disks containing the IBM Storage Scale file system data to both the storage cluster and the IBM Storage Scale container native cluster.
- For more information about setting up a direct storage attachment, see [Attaching direct storage on IBM Z](#) in IBM Storage Scale documentation.

Figure 2. Direct attach SystemZ



GPU Direct Storage

GPUDirect Storage (GDS) is an NVIDIA technology that enables a direct data path for data transfers between storage devices and GPU memory, bypassing the CPU. This increases the system bandwidth, reduces data transfer latency, and lowers CPU utilization.

To enable GPU Direct storage on the OpenShift Container Platform with IBM Storage Scale container native, several manual steps are required to be run.

Prerequisites

- Red Hat OpenShift cluster must have NVIDIA InfiniBand or RoCE networking enabled. More information, see [Networking](#).
- IBM Storage Scale container native cluster must be created with RDMA networking enabled. For more information, see [NVIDIA Networking](#)
- NVIDIA GPU operator needs to be installed with RDMA enabled and GDS driver image. For more information, see [NVIDIA GPU Operator](#)
- GPUDirect Storage must be enabled on both the storage and client clusters.

Support matrix

Several software components are required for the GPU Direct Storage. For more details, see [GPU Direct Storage Software requirements](#).

Steps to enable GPU Direct Storage

1. Stop operator pod.

```
kubectl -n ibm-spectrum-scale-operator scale --replicas=0 deployment ibm-spectrum-scale-controller-manager
```

2. Get a core pod.

```
POD=$(kubectl get pods -n ibm-spectrum-scale -lapp.kubernetes.io/name=core -o jsonpath='{.items[0].metadata.name}')
```

3. List all file systems and unmount every file system listed. The unmounting of the file system fails as long as application pods that use CSI PVs are still running.

```
kubectl -n ibm-spectrum-scale exec $POD -it -- mmlsfs all
```

The sample command to unmount file system mounted at `/mnt/remote-sample` directory.

```
kubectl -n ibm-spectrum-scale exec $POD -it -- mmunmount /mnt/remote-sample -a
```

4. Stop the GPFS daemon on all nodes.

```
kubectl -n ibm-spectrum-scale exec $POD -it -- mmshutdown -a
```

5. Enable GPU Direct Storage.

```
kubectl -n ibm-spectrum-scale exec $POD -it -- mmchconfig verbsGPUDirectStorage=enable
```

6. Start the GPFS daemon on all nodes.

```
kubectl -n ibm-spectrum-scale exec $POD -it -- mmstartup -a
```

7. Can ensure active state of all GPFS cluster nodes.

```
kubectl -n ibm-spectrum-scale exec $POD -it -- mmgetstate -a
```

8. Start the operator pod. The operator pod mounts the all unmounted file systems.

```
kubectl -n ibm-spectrum-scale-operator scale --replicas=1 deployment ibm-spectrum-scale-controller-manager
```

Verify GPUDirect Storage configuration

Perform the following steps to check the GDS configuration:

1. Create a test pod to verify GDS configuration. Can ensure that the test pod image has CUDA and GDS tools such as `gdscheck` and `gdsio` packaged. For more information, see the blog post [Build RDMA GPU-Tools Container](#). Can ensure that the pod has GPFS mounted in it. Sample `gds` tools pod `yaml` is given.

```
apiVersion: v1
kind: Pod
metadata:
  name: gds-check-workload
  namespace: ibm-spectrum-scale
spec:
  hostNetwork: true
  containers:
  - image: <GDS-CONFIGURATION-VERIFICATION-IMAGE>
    securityContext:
      privileged: true
      add: [ "IPC_LOCK" ]
    imagePullPolicy: Always
    name: gds-check-workload
    resources:
      limits:
        nvidia.com/gpu: 1
      requests:
        nvidia.com/gpu: 1
    volumeMounts:
    - name: ibm-fs
      mountPath: /mnt/ibm
    - name: udev
      mountPath: /run/udev
    - name: kernel-config
      mountPath: /sys/kernel/config
    - name: sys
      mountPath: /sys
    - name: lib
      mountPath: /lib/modules
  volumes:
  - name: ibm-fs
    hostPath:
      path: <HOST-PATH-WHERE-GPFS-IS-MOUNTED>
  - name: udev
    hostPath:
      path: /run/udev
  - name: kernel-config
    hostPath:
      path: /sys/kernel/config
  - name: sys
    hostPath:
      path: /sys
  - name: lib
    hostPath:
      path: /lib/modules
```

2. Exec into the pod to configure CUDA.

```
oc -n ibm-spectrum-scale rsh gds-check-workload
```

3. Configure CUDA where GPFS daemon networking is RDMA over Converged Ethernet (RoCE) with hostNetwork enabled.

You need to update the following configuration settings in the CUDA file:

rdma_dev_addr_list:

Defines the RDMA devices to be used as a list of IP addresses. The IP addresses (RoCE) specified must be consistent with the values that are set for the verbsPorts parameter in the cluster custom resource.

allow_compat_mode:

Disable compatibility mode by setting allow_compat_mode to false. If compatibility mode is not disabled, even with incorrect GDS configuration, IOs pass.

1. Verify GDS configuration that uses GDSCHECK tool. Can ensure that **gdscheck** tool identifies **IBM Spectrum Scale** as supported combination inside the test pod.

```
#/usr/local/cuda-13.0/gds/tools/gdscheck -p | grep "IBM Spectrum Scale"
IBM Spectrum Scale : Supported
```

2. Do write and read operations on the mounted GPFS with gdsio tool.

Change parameter values to the gdsio command as per configuration on your setup.

```
```bash
% oc rsh -n ibm-spectrum-scale gds-check-workload
```
```

Sample write command using GPUD

```
```bash
/usr/local/cuda-13.0/gds/tools/gdsio -x 0 -I 1 -s 40G -i 4M -d 0 -w 32 -f /mnt/ibm/file.dat
```
```

Sample read command using GPUD

```
```bash
/usr/local/cuda-13.0/gds/tools/gdsio -x 0 -I 0 -s 40G -i 4M -d 0 -w 32 -f /mnt/ibm/file.dat
```
```

Networking

- [Network and firewall requirements](#)
- [Container Network Interface \(CNI\) configuration](#)
- [NVIDIA Networking](#)
 - [NVIDIA Network Operator](#)
 - [RDMA over InfiniBand](#)
 - [RDMA over Converged Ethernet \(RoCE\)](#)
 - [Networking Scenarios and Examples](#)
 - [RoCE deployment with hostNetwork=false](#)

Network and firewall requirements

By default, Kubernetes clusters are created where egress communication outside of the cluster is allowed. This abets attacks where malicious code is sideloaded from the internet and customers should restrict any internet access from within the Red Hat OpenShift Container Platform cluster only to necessary sites. For more information, see [Configuring your firewall](#).

The following list covers only the requirements of IBM Storage Scale container native and IBM Storage Scale CSI.

IBM Storage Scale uses several communication channels that need to be opened in the corresponding firewalls otherwise some or all functions might be broken. The channels can be categorized into three major groups:

1. Communication between pods inside a Kubernetes cluster.
 - Communication inside the Kubernetes cluster is subject to firewall rules on the worker nodes kernels and network policies that are defined.
2. Communication between Kubernetes cluster and storage cluster.

- Communication with the storage cluster is subject to the firewall that sits on the load-balancer, in front of the Kubernetes cluster.
3. Communication between Kubernetes cluster and servers on the internet.
- Communication with the internet is subject to the boundary firewall of the data center and the boundary firewall of the intranet.

Communication between pods

IBM Storage Scale container native core pods use two network interfaces, an "admin" interface and a "daemon" interface. The "admin" interface is used for monitoring and management traffic while the "daemon" interface is used for file system I/O traffic. In the default configuration, the pod is created with `hostNetwork: true` and both admin and daemon networks use the Kubernetes node internal network on the host and the host network namespace is exposed from inside the pod.

When using the default host networking, the networking requirements that are outlined must be satisfied and opened across the Kubernetes node internal network.

For better network security and network isolation, you may consider using Container Network Interface (CNI) to configure pod networking. If using CNI to configure networking, only the **Core Pod Daemon Network Requirements** need to be satisfied. The admin network will use the Kubernetes pod SDN Network. For more information, see [Container network interface \(CNI\) configuration](#).

If you plan to use NVIDIA networking solutions to deploy NVIDIA MOFED drivers, the recommendation is to use the default deployment with `hostNetwork=true` set on the core pods and not to deploy with CNI. For more information, see [NVIDIA Networking](#)

Core pod admin network requirements

The following network requirements must be met when `hostNetwork: true` on the pod specification. This is the default deployment method and the ports only need to be opened between nodes within the local IBM Storage Scale container native cluster.

Table 1. Admin network requirements

| Port number | Protocol | Use | Initiated direction |
|-------------|----------|--------------------|---|
| 12345 | TCP | Administrative SSH | All nodes to all nodes within the local cluster |

Core pod daemon network requirements

Those are identical to the requirements for communication with a storage cluster. For more information, see [Daemon network requirements](#) in the storage cluster section.

Communication with storage cluster

Daemon network requirements

The following list of ports and protocols are used to communicate between all nodes within the IBM Storage Scale container native cluster and between nodes in any configured remote storage clusters. Each node acts as a server that may initiate connections to any other node. Help ensure the list of ports and protocols are open for both inbound and outbound packet flows.

Table 2. Daemon network requirements

| Port number | Protocol | Use | Initiated direction |
|-----------------------------------|----------|--------------------|------------------------|
| 1191 | TCP | GPFS | All nodes to all nodes |
| -- | ICMP | GPFS | All nodes to all nodes |
| Configurable ephemeral port range | TCP | GPFS Policy Engine | All nodes to all nodes |

Ephemeral port ranges

Ephemeral ports are used by the GPFS Policy Engine, which is used by the CSI snapshot and compression features. When `tscCmdAllowRemoteConnections=yes` is configured in the Cluster CR, ephemeral port ranges are also used for communication and compatibility with remote clusters.

If ephemeral ports are configured on the remote storage cluster, help ensure that they are also configured in the Cluster CR for the IBM Storage Scale container native deployment.

For more information on how to set ephemeral port ranges, see [Ephemeral port range](#).

REST API access

Table 3. REST API network requirements

| Port number | Protocol | Use | Initiated direction |
|-------------|----------|--------------------------|--|
| 443 | TCP | Storage cluster REST API | Pod network to storage cluster GUI nodes |

Pod network encapsulates the requirement that IBM Storage Scale container native operator and IBM Storage Scale CSI operator and driver require the ability to reach the storage cluster GUI nodes REST API. These respective pods use the default pod network and schedule to any node selected.

Name resolution between IBM Storage Scale container native and storage cluster

The IBM Storage Scale container native core pods require name resolution to all storage cluster nodes, specifically the node's daemon node name. If these names cannot be configured in the environment's domain name service (DNS), then host aliases can be added in the cluster CR `.spec.daemon.hostAliases`. This configuration adds these host aliases to the internal DNS managed by IBM Storage Scale container native and applies only to name resolution performed by the IBM Storage Scale container native core pods.

Communication with the internet

Container registries

The container images for IBM Storage Scale container native containers are located on the IBM Cloud Registry.

To access the IBM Cloud Container Registry by using the domains `cp.icr.io` and `icr.io`, you must add the following hostnames to your firewall rules:

- `dd0.icr.io`
- `dd2.icr.io`
- `dd4.icr.io`
- `dd6.icr.io`

Users that are located in China must also allow the following hostnames:

- `dd1-icr.ibm-zh.com`
- `dd3-icr.ibm-zh.com`
- `dd5-icr.ibm-zh.com`
- `dd7-icr.ibm-zh.com`

Egress to these sites is required unless the cluster is configured in airgap mode. For more information, see [Disconnected installs](#).

Callhome

If callhome function is enabled, ingress and egress need to be open to `esupport.ibm.com (129.42.0.0/18)`.

For more information, see [Firewall recommendations for call home](#) in the IBM Storage Scale documentation.

Encryption

If encryption is configured on the file system, egress on the following ports is required for each service to function:

Table 4. Encryption network requirements

| Port number | Protocol | Use | Service |
|-------------|----------|---------------|---|
| 9083 | TCP | initial setup | WebSphere Application Server |
| 9443 | TCP | initial setup | Security Key Lifecycle Manager REST API |
| 5696 | TCP | key retrieval | Security Key Lifecycle Manager KMIP API |

For more information, see [Firewall recommendations for IBM Security Key Lifecycle Manager](#) in the IBM Storage Scale documentation.

Container network interface (CNI) configuration

IBM Storage Scale container native core pods use two network interfaces, an "admin" and a "daemon" interface. The "admin" interface is used for monitoring and management, while the "daemon" interface is used for file system I/O. In a default configuration, both networks use the Kubernetes node internal network that is exposed on the host, and the pods are given access to the host network namespace. (`hostNetwork=true`)

Configuring CNI provides advantages over the default configuration. The "admin" and "daemon" network interfaces become private to the IBM Storage Scale container native core pod. The "admin" interface moves to use the Kubernetes pod network, which includes security and isolation that is provided by Kubernetes and NetworkPolicies. The "daemon" interface is set up to be isolated, private, and high-speed.

If you plan to use NVIDIA networking solutions to deploy NVIDIA MOFED drivers, the recommendation is to use the default deployment with `hostNetwork=true` set on the core pods and not to deploy with CNI. For more information, see [NVIDIA Networking](#)

It is not possible to change between a "hostNetwork" and "CNI" configuration after the IBM Storage scale container native cluster is created. A change required a full cleanup and redeployment.

Considerations for CNI

To understand and configure CNI on Red Hat OpenShift, see [understanding multiple networks](#).

- If the node has only a single physical network attachment, then the network adapter needs to be shared between networks. There are several CNI flavors that allow this: **Bridge**, **IPVLAN**, and **MACVLAN**.
 - The **MACVLAN** plug-in supports network policies and should be the default choice.
- If the node has multiple physical network attachments and you want to dedicate one of the physical networks to IBM Storage Scale, select **host-adapter** CNI. It maps a physical network adapter into a pod, making it inaccessible to the host and other pods.
- **SR-IOV** surpasses the capabilities of **host-adapter**. Advanced features like RDMA, GPUdirect, and bondings of network ports are accessible via **SR-IOV** hardware network. For more information, see [About Single Root I/O Virtualization \(SR-IOV\) hardware networks](#).

Features: GPUdirect and bondings of network ports are not currently supported by IBM Storage Scale. Also **SR-IOV** allows to partition the hardware adapter and hand those partitions to different pods. Configuration is more complex compared to other CNIs and choice of supported network adapters is limited. As of today, IBM Storage Scale has not been tested with **SR-IOV**.

- The IP address mapping is required to be static. This can be achieved by setting up static IPs or by configuring DHCP static mapping. For remote mount of a file system from an IBM Storage Scale storage cluster, this network must be routed to the storage cluster's daemon network.

How to configure CNI networking with IBM Storage Scale container native

Attaching one or more CNI networks to the IBM Storage Scale container native core pods is done by using annotations on the Kubernetes Node object. By annotating the node with a valid annotation, the core pods are created with **hostNetwork=false**. The following two Node annotations are supported:

1. `scale.spectrum.ibm.com/daemon-network`
2. `scale.spectrum.ibm.com/additional-network`

The `scale.spectrum.ibm.com/daemon-network` is required.

daemon-network

This annotation specifies the name of the **NetworkAttachmentDefinition** to be used for the "daemon" interface. The CNI annotation value must conform to the format specified by `k8s.v1.cni.cncf.io/networks`.

The following example annotates the node with a static IP of `192.169.10.1` and uses the `macvlan-net1` network:

```
oc annotate node/worker0.example.com scale.spectrum.ibm.com/daemon-network='{ "name": "macvlan-net1", "ips": [ "192.169.10.1/24" ] }'
```

- `ips` field for `daemon-network` must be set with a static IP
- `mac` field is optional if you use DHCP IPAM (backed by a statically mapped DHCP). `ips` still must be set in addition to `mac`.

This might seem redundant, but the IBM Storage Scale container native uses `ips` to set up its own name resolution. This process is asynchronous and independent of the pod created. If `ips` was not set, then DHCP address would not be set until after pod creation.

When the core pod comes up, the following network is added and the IBM Storage Scale container native daemon interface will use this IP address:

```
3: net1@if7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether da:ec:38:ca:2a:77 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.10.1/24 brd 192.168.10.255 scope global net1
        valid_lft forever preferred_lft forever
    inet6 fe80::d8ec:38ff:feca:2a77/64 scope link
        valid_lft forever preferred_lft forever
```

If using NVIDIA networking with MOFED, the `daemon-network` annotation must not specify any **NetworkAttachmentDefinition** name that uses NVIDIA adapters on the host.

additional-network

This annotation specifies the name of the **NetworkAttachmentDefinition** to be used for any added network to attach into the core pods. The CNI annotation value must conform to the format specified by `k8s.v1.cni.cncf.io/networks`.

The following example annotates the `worker0` node with two more networks:

```
oc annotate node/worker0.example.com scale.spectrum.ibm.com/additional-networks='[{ "name": "macvlan-net2", "ips": [ "192.169.20.1/24" ] }, { "name": "macvlan-net3", "ips": [ "192.169.30.1/24" ] }]'
```

When the core pod comes up, the following networks are added:

```
4: net2@if7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether ee:04:33:41:2b:30 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.20.1/24 brd 192.168.20.255 scope global net2
        valid_lft forever preferred_lft forever
    inet6 fe80::ec04:33ff:fe41:2b30/64 scope link
        valid_lft forever preferred_lft forever
```

```

5: net3@if7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
   link/ether 62:f4:93:0a:a1:cd brd ff:ff:ff:ff:ff:ff link-netnsid 0
   inet 192.168.30.1/24 brd 192.168.30.255 scope global net3
       valid_lft forever preferred_lft forever
   inet6 fe80::60f4:93ff:fe0a:a1cd/64 scope link
       valid_lft forever preferred_lft forever

```

NVIDIA Networking

NVIDIA provides a network operator that assists with many configuration options when using their networking fabric. For information on installing the NVIDIA network operator, see [Getting Started with Red Hat OpenShift](#) in the NVIDIA documentation.

The following sections discuss NVIDIA network operator custom resources and configurations to be considered when using with IBM Storage Scale container native.

These NVIDIA CRs help to configure the networking interfaces when using

- [NVIDIA Network Operator](#)
 - [NicClusterPolicy CR](#)
 - [IPoIBNetwork CR](#)
 - [MacvlanNetwork CR](#)
 - [HostDeviceNetwork CR](#)
- [RDMA over InfiniBand](#)
- [RDMA over Converged Ethernet \(RoCE\)](#)
- [Networking Scenarios and Examples](#)
 - [RoCE deployment with hostNetwork=false](#)

NVIDIA Network Operator

NVIDIA network operator provides a few custom resources to help assist with the configuration of the networking. The following table identifies which CRs are required when in use with IBM Storage Scale container native.

Table 1. NVIDIA CR requirements with IBM Storage Scale container native

| Custom Resource | Required | Purpose with IBM Storage Scale container native |
|-------------------|---|--|
| NicClusterPolicy | Yes | define OFED and RDMA configuration |
| IPoIBNetwork | Optional, for CNI 1 | Helps set up networking for InfiniBand |
| MacvlanNetwork | Optional, for CNI ^[^tabletext] | Helps set up networking for RoCE |
| HostDeviceNetwork | Optional | Do not use with IBM Storage Scale container native |

See the following sections for more details on each of the custom resource and specific configuration examples.

- [NicClusterPolicy CR](#)
- [IPoIBNetwork CR](#)
- [MacvlanNetwork CR](#)
- [HostDeviceNetwork CR](#)

NicClusterPolicy CR

The NVIDIA **NicClusterPolicy** spec allows the user to configure many networking features supported by NVIDIA.

spec.ofedDriver

The following table shows the networking configuration options that are supported by IBM Storage scale container native for the core pod:

Table 1. IBM Storage Scale container native core pod networking with MOFED support

| Core pod network | Pod Host Network | Storage Scale daemon interface | Can deploy NVIDIA MOFED pods |
|-------------------|------------------|--|--|
| Host Network | true | Configured to use the Node Internal IP | Yes, must also apply buildgpl.nvidia.yaml |
| Using CNI plug-in | false | Not using TCP over NVIDIA networking adapter | Yes, must also apply buildgpl.nvidia.yaml |
| Using CNI plug-in | false | Using TCP over an NVIDIA networking adapter | No, must use OS provided OFED |

If you have chosen a scenario where you plan to deploy the NVIDIA MOFED pods, help ensure that you deploy the **buildgpl.nvidia.yaml** configmap in your IBM Storage Scale container native deployment. For more information, see [Building the kernel portability layer](#)

If you have chosen a scenario in the table where you must use the OS provided OFED, **delete** the **spec.ofedDriver** section of the NVIDIA **NicClusterPolicy** CR to avoid MOFED pods from being deployed.

The following example defines a resource that is called `nic-cluster-policy`:

```
kind: NicClusterPolicy
apiVersion: mellanox.com/v1alpha1
metadata:
  name: nic-cluster-policy
spec:
  ofedDriver:
    image: doca-driver
    imagePullSecrets: []
    livenessProbe:
      ...
  rdmaSharedDevicePlugin:
    config: |-
      {
        "configList": [
          {
            "resourceName": "rdma_ibs5f0",
            "rdmaHcaMax": 100,
            "selectors": {
              "vendors": ["15b3"],
              "ifNames": ["ibs5f0"]
            }
          }
        ]
      }
    image: k8s-rdma-shared-dev-plugin
    repository: ghcr.io/mellanox
    version: v1.5.1
```

The resource has the following properties:

- RDMA resource name is `rdma_ibs5f0`
- defines `rdmaHcaMax`: 100, the maximum number of RDMA Host Channel Adapter (HCA) devices that the system can support
- defines the interface name `ibs5f0`

For more information, see [NicClusterPolicy](#) in the NVIDIA documentation.

IPoIBNetwork CR

This custom resource is used to help configure a CNI network.

The **IPoIBNetwork** custom resource assists users in defining their IP over InfiniBand (IPoIB) network configuration. When applied, the NVIDIA network operator creates a **NetworkAttachmentDefinition** in the target **networkNamespace**.

The following example defines a resource that is called `ipoibnetwork-scale`:

```
apiVersion: mellanox.com/v1alpha1
kind: IPoIBNetwork
metadata:
  name: ipoibnetwork-scale
spec:
  ipam: |
    {
      "type": "whereabouts",
      "range": "192.168.0.0/24",
      "exclude": [
        "192.168.0.31/32",
        "192.168.0.32/32",
        "192.168.0.33/32"
      ]
    }
  master: ibs5f0
  networkNamespace: ibm-spectrum-scale
```

The resource has the following properties:

- allows the CNI network to take IP addresses from the network range: `192.168.0.0/24`
- exclude 3 IP addresses from that range
- the physical InfiniBand interface on the nodes where core pods are to be deployed is: `ibs5f0`
- the target namespace is `ibm-spectrum-scale`

For more information, see [IPoIBNetwork](#) in the NVIDIA documentation.

MacvlanNetwork CR

This custom resource is used to help configure a CNI network.

The **MacvlanNetwork** custom resource assists users in defining their network configuration resulting in a **NetworkAttachmentDefinition** created that uses CNI `type=macvlan`.

The following example defines a resource that is called `macvlannetwork-scale`:

```
apiVersion: mellanox.com/v1alpha1
kind: MacvlanNetwork
metadata:
  name: macvlannetwork-scale
spec:
  ipam: |
    {
      "type": "static"
    }
  master: ens5f1np1
  mode: bridge
  mtu: 9000
  networkNamespace: ibm-spectrum-scale
```

The resource has the following properties:

- the name of the CR is `macvlannetwork-scale`
- static IP addresses are provided by the admin in the network annotation
- the physical interface (ETHERNET MODE) on the nodes where core pods are to be deployed: `ens5f1np1`
- the target namespace is `ibm-spectrum-scale`
- The MTU is set to 9000

For more information, see [IPoIBNetwork](#) in the NVIDIA documentation.

HostDeviceNetwork CR

The **HostDeviceNetwork** CR from NVIDIA does not create the **NetworkAttachmentDefinition** configuration correctly and will cause issues. If `type=host-device` is preferred, manually define a **NetworkAttachmentDefinition**.

-
1. These custom resources help with creating **NetworkAttachmentDefinitions**. In Kubernetes, a **NetworkAttachmentDefinition** is needed to allow pods to access network interfaces beyond the default SDN network. [↗](#)

RDMA over InfiniBand

To create an IBM Storage Scale container native cluster that uses NVIDIA InfiniBand networking, start by downloading the sample cluster CR from [NVIDIA InfiniBand Cluster CR](#).

To enable RDMA, uncomment the RDMA-specific configuration from the sample CR and make the following changes:

- Change `verbsPort` to a value that matches your specific hardware configuration. This value matches the network interface name for the InfiniBand device. For example: `mlx5_2` or `ibs5f0`.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
metadata:
  name: ibm-spectrum-scale
spec:
  daemon:
    clusterProfile:
      # -----
      # RDMA specific configuration.
      # Change verbsPort to value based on your configuration. This value should match the
      # network interface name for the InfiniBand device. For example: "mlx5_2", "ibs5f0", etc.
      # -----
      verbsPorts: mlx5_2
      verbsRdma: enable
      verbsRdmaSend: "yes"
      ...
```

If you have an existing cluster already that uses NVIDIA InfiniBand networking, enable RDMA by adding the `verbs*` tuning parameters to your existing cluster resource configuration.

RDMA over Converged Ethernet (RoCE)

To create an IBM Storage Scale container native cluster that uses NVIDIA Ethernet networking, start by downloading the sample cluster CR from [NVIDIA InfiniBand Cluster CR](#).

To enable RDMA over Converged Ethernet (RoCE), uncomment the RDMA-specific configuration from the sample CR and make the following changes:

- Change `verbsPort` to a value that matches your specific hardware configuration.
 - If core pods are `hostNetwork=true`, these values match the names of the NVIDIA adapters that are returned from the command output: `mccmi getifconf`. For example: `ens1f0np0`.
 - If core pods are `hostNetwork=false`, these values match the interface name of the NVIDIA adapter that is attached to the pods by CNI. For example: `net1`
- `verbRdmaCm` must be set to "enable".

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
metadata:
  name: ibm-spectrum-scale
spec:
  daemon:
    clusterProfile:
      ...
      ...
      verbsPorts: net1
      verbsRdma: enable
      verbsRdmaSend: "yes"
      # Only enable `verbsRdmaCm` when deploying RoCE - RDMA over Converged Ethernet
      verbsRdmaCm: enable
      ...
```

If you have an existing cluster already that uses NVIDIA Ethernet networking, enable RDMA by adding the `verbs*` tuning parameters to your existing cluster resource configuration.

Networking Scenarios and Examples

The following example shows how to configure IBM Storage Scale container native that uses RDMA over Converged Ethernet (RoCE) networking with a CNI overlay network.

- [RoCE deployment with hostNetwork=false](#)

RoCE deployment with hostNetwork=false

The example that follows shows how to configure IBM Storage Scale container native that uses RDMA over Converged Ethernet (RoCE) networking along with a CNI `macvlan` overlay network, resulting in the core pods configured with `hostNetwork=false`. An IBM Storage Scale file system is remote-mounted to the container native cluster, providing the storage. The sample environment consists of:

- Red Hat OpenShift cluster at 4.18.19 with 6 nodes. (3 control-plane/masters and 3 workers)
- Mellanox Technologies MT28908 Family [ConnectX-6]: VPI adapter with 1 port that is configured to ETH mode
- IBM Elastic Storage System (ESS) 5000 v6.2.3 (IBM Storage Scale 5.2.3) providing the storage file system

The samples that are provided in this documentation are taken at a single point in time and might be out of date. Always refer to the product documentation from NVIDIA to pull the updated CRDs and image versions to match your running environment.

Overview

- Deploy NVIDIA Network Operator v25.4.0 and create the custom resources (CRs)
- Deploy NVIDIA GPU Operator v25.3.2 and create the custom resources (CRs)
- Install IBM Storage Scale container native v6.0.0.0
 - Deploy the operator
 - Create the custom resources (CRs)
 - Enable GPU Direct Storage
- Verification
 - Verify RDMA functions
 - Verify GPU Direct Storage functions

If you are not planning to deploy GPU Direct Storage (GDS) functions, skip the sections that discuss GDS.

NVIDIA Network Operator

1. Install the NVIDIA Network Operator per instructions from NVIDIA. For more information, see [Getting Started with Red Hat OpenShift](#) in the [NVIDIA Network Operator v25.4.0](#) documentation.

Verify that the operator is running:

```
$ oc get pods -n nvidia-network-operator
nvidia-network-operator-controller-manager-7b44ddf687-tncj4    1/1    Running    0    5m
```

With the NVIDIA network operator installed, start creating custom resources.

To use a CNI overlay network, there needs to be a **NetworkAttachmentDefinition** configured. The NVIDIA network operator can create and manage the **NetworkAttachmentDefinition** by the **MacvlanNetwork** custom resource.

2. Determine the physical adapter on the CoreOS host that is providing the RoCE connection. Inspect the worker node by use of a debug pod and check what Ethernet interfaces are available.

```
$ oc debug node/worker0.example.com
Starting pod/worker0examplecom-debug-wkx4k ...
To use host binaries, run `chroot /host`
sh-5.1# chroot /host
sh-5.1# ip addr show
```

The adapter that is used here on the system is **ens5f1np1**:

```
36: ens5f1np1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc mq state UP group default qlen 1000
    link/ether e8:eb:d3:6f:47:fb brd ff:ff:ff:ff:ff:ff
    altname enp161s0f1np1
    inet 192.168.1.31/24 brd 192.168.1.255 scope global noprefixroute ens5f1np1
        valid_lft forever preferred_lft forever
    inet6 fe80::eaeb:d3ff:fe6f:47fb/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

3. Use the following command to create a **MacvlanNetwork** definition:

Replace **ens5f1np1** with the device name that is found on your system.

```
cat << EOF | kubectl apply -f -
apiVersion: mellanox.com/v1alpha1
kind: MacvlanNetwork
metadata:
  name: macvlannetwork-ens5f1np1-scale
spec:
  ipam: |
    {
      "type": "static"
    }
  master: ens5f1np1
  mode: bridge
  mtu: 9000
  networkNamespace: ibm-spectrum-scale
EOF
```

The **metadata.name** for the **MacvlanNetwork** becomes the name of the network attachment definition that is created to the namespace specified in **spec.networkNamespace**. In this example, when the **ibm-spectrum-scale** namespace is available, the NVIDIA network operator creates a **network-attachment-definition** called **macvlannetwork-ens5f1np1-scale** in that namespace.

In the IPAM configuration, **type: "static"** is used because a static IP address is annotated to the Nodes and is used as the IBM Storage Scale daemon network IP.

4. Use the following command to create a **NicClusterPolicy** definition:

Replace **ens5f1np1** with the device name that is found on your system.

```
cat << EOF | kubectl apply -f -
apiVersion: mellanox.com/v1alpha1
kind: NicClusterPolicy
metadata:
  name: nic-cluster-policy
spec:
  ofedDriver:
    env:
      - name: UNLOAD_STORAGE_MODULES
        value: "true"
    forcePrecompiled: false
    image: doca-driver
    imagePullSecrets: []
```

```

livenessProbe:
  initialDelaySeconds: 30
  periodSeconds: 30
readinessProbe:
  initialDelaySeconds: 10
  periodSeconds: 30
repository: nvcr.io/nvidia/mellanox
startupProbe:
  initialDelaySeconds: 10
  periodSeconds: 20
terminationGracePeriodSeconds: 300
upgradePolicy:
  autoUpgrade: true
  drain:
    deleteEmptyDir: true
    enable: true
    force: true
    podSelector: ""
    timeoutSeconds: 300
    maxParallelUpgrades: 1
    safeLoad: false
  version: 25.04-0.6.1.0-2
rdmaSharedDevicePlugin:
  config: |-
    {
      "configList": [
        {
          "resourceName": "rdma_ens5f1np1",
          "rdmaHcaMax": 10,
          "selectors": {
            "vendors": ["15b3"],
            "ifNames": ["ens5f1np1"]
          }
        }
      ]
    }
  image: k8s-rdma-shared-dev-plugin
  imagePullSecrets: []
  repository: ghcr.io/mellanox
  version: sha256:1eb5ab7f4b6f0cf89c954cf7ac8eb2e6fb2290a4f5f57417136a5302f20b12c8
secondaryNetwork:
  ipoib:
    image: ipoib-cni
    imagePullSecrets: []
    repository: nvcr.io/nvidia/cloud-native
    version: v1.1.0
EOF

```

The `NicClusterPolicy` example contains image values that are provided by the v25.4.0 network operator and might be out of date.

NVIDIA GPU Operator

1. Install the NVIDIA GPU Operator per instructions from NVIDIA. For more information, see [Installing the NVIDIA GPU Operator on Red Hat OpenShift](#) in the [NVIDIA GPU Operator v25.3.2](#) documentation.

Verify that the operator is running:

```

$ oc get pods -n nvidia-gpu-operator

```

| NAME | READY | STATUS | RESTARTS | AGE |
|-------------------------------|-------|---------|----------|------|
| gpu-operator-595696d549-7pffb | 1/1 | Running | 0 | 2m2s |

2. With the NVIDIA GPU operator installed, create a ClusterPolicy CR `gpu-cluster-policy` and help ensure that the `spec.gds` fields are set to "enabled". Use the following command to create a `ClusterPolicy`:

```

cat << EOF | kubectl apply -f -
apiVersion: nvidia.com/v1
kind: ClusterPolicy
metadata:
  name: gpu-cluster-policy
spec:
  daemonsets:
    rollingUpdate:
      maxUnavailable: "1"
    updateStrategy: RollingUpdate
  dcgm:
    enabled: true
  dcgmExporter:
    config:
      name: ""
    enabled: true

```

```

    serviceMonitor:
      enabled: true
    devicePlugin:
      config:
        default: ""
        name: ""
      enabled: true
      mps:
        root: /run/nvidia/mps
    driver:
      certConfig:
        name: ""
      enabled: true
      kernelModuleConfig:
        name: ""
      kernelModuleType: open
      licensingConfig:
        configMapName: ""
        nlsEnabled: true
      rdma:
        enabled: true
      repoConfig:
        configMapName: ""
      upgradePolicy:
        autoUpgrade: true
      drain:
        deleteEmptyDir: false
        enable: false
        force: false
        timeoutSeconds: 300
        maxParallelUpgrades: 1
        maxUnavailable: 25%
      podDeletion:
        deleteEmptyDir: false
        force: false
        timeoutSeconds: 300
      waitForCompletion:
        timeoutSeconds: 0
      useNvidiaDriverCRD: false
      virtualTopology:
        config: ""
    gdrccopy:
      enabled: false
    gds:
      enabled: true
      image: nvidia-fs
      repository: nvcr.io/nvidia/cloud-native
      version: 2.26.6
    gfd:
      enabled: true
    mig:
      strategy: single
    migManager:
      enabled: true
    nodeStatusExporter:
      enabled: true
    operator:
      defaultRuntime: crio
      initContainer: {}
      runtimeClass: nvidia
      use_ocp_driver_toolkit: true
    sandboxDevicePlugin:
      enabled: true
    sandboxWorkloads:
      defaultWorkload: container
      enabled: false
    toolkit:
      enabled: true
      installDir: /usr/local/nvidia
    validator:
      plugin:
        env:
          - name: WITH_WORKLOAD
            value: "false"
    vfioManager:
      enabled: true
    vgpuDeviceManager:
      enabled: true
    vgpuManager:
      enabled: false
EOF

```

For more information, see [Create the ClusterPolicy instance](#).

3. Run a sample GPU application pod to help ensure that GPU on the hosts is being used.

```
cat << EOF | kubectl create -f -
apiVersion: v1
kind: Pod
metadata:
  name: cuda-vectoradd
spec:
  restartPolicy: OnFailure
  containers:
  - name: cuda-vectoradd
    image: "nvcr.io/nvidia/k8s/cuda-sample:vectoradd-cuda12.5.0-ubi8"
    resources:
      limits:
        nvidia.com/gpu: 1
EOF
```

When the pod is in **Completed** state, check the logs:

```
$ kubectl logs cuda-vectoradd
[Vector addition of 50000 elements]
Copy input data from the host memory to the CUDA device
CUDA kernel launch with 196 blocks of 256 threads
Copy output data from the CUDA device to the host memory
Test PASSED
Done
```

For more information, see [Running a sample GPU Application](#)

4. Run `nvidia-smi` to get information about the GPU installed in your system.

- get the pods that have `openshift.driver-toolkit=true`

```
kubectl get pods -n nvidia-gpu-operator -lopenshift.driver-toolkit=true
```

- exec into the first one to check the output of `nvidia-smi`

```
kubectl exec -it -n nvidia-gpu-operator $(kubectl get pods -n nvidia-gpu-operator -
lopenshift.driver-toolkit=true -o jsonpath='{.items[0].metadata.name}') -- nvidia-smi
```

For more information, see [Getting information about the GPU](#)

Install IBM Storage Scale container native

Before proceeding, complete all the prerequisite planning steps for the Storage Cluster and the Red Hat OpenShift Container Platform. For more information, see [Storage Cluster](#) and [Red Hat OpenShift Container Platform](#).

1. Install the IBM Storage Scale container native operator by using the following command:

```
kubectl apply -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-
native/v6.0.0.x/generated/scale/install.yaml
```

Now, the `ibm-spectrum-scale` namespace is created and the NVIDIA network operator creates a network attachment definition from the configuration set in the `MacvlanNetwork` CR.

```
$ oc get network-attachment-definition -n ibm-spectrum-scale
NAME                                AGE
macvlannetwork-ens5f1np1-scale     30s
```

For more information, see [Installing the IBM Storage Scale container native operator and cluster](#)

2. Annotate the nodes with the `scale.spectrum.ibm.com/daemon-network` annotation. This annotation contains the static IP and name of the CNI overlay network (`macvlannetwork-ens5f1np1-scale`) that was created.

```
kubectl annotate node worker0.example.com 'scale.spectrum.ibm.com/daemon-network={ "name":
"macvlannetwork-ens5f1np1-scale", "ips": [ "192.168.1.41/24" ] }' --overwrite
kubectl annotate node worker1.example.com 'scale.spectrum.ibm.com/daemon-network={ "name":
"macvlannetwork-ens5f1np1-scale", "ips": [ "192.168.1.42/24" ] }' --overwrite
kubectl annotate node worker2.example.com 'scale.spectrum.ibm.com/daemon-network={ "name":
"macvlannetwork-ens5f1np1-scale", "ips": [ "192.168.1.43/24" ] }' --overwrite
```

Check the annotation by using the following command:

```
oc describe node | egrep "Name:|daemon-network"
```

3. Apply the `buildgpl.nvidia.yaml` configmap. This configmap needs to be deployed when the NVIDIA network operator is installed to give awareness to the MOFED builds of the kernel modules.

```
kubectl apply -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/openshift/buildgpl.nvidia.yaml
```

For more information, see [Building the kernel portability layer](#)

4. Label the nodes where core pods are to be deployed by using the `scale.spectrum.ibm.com/daemon-selector=` label.

```
# Set all Kubernetes worker nodes to run a core pod
kubectl label node --node-role.kubernetes.io/worker= scale.spectrum.ibm.com/daemon-selector=
```

5. Download the sample NVIDIA Cluster CR containing RDMA-specific configuration options.

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/cluster/cluster.nvidia.yaml > cluster.yaml || echo "Failed to download Cluster sample CR"
```

- Edit the `cluster.yaml` and uncomment the RDMA `verbs*` specific configuration lines. Set the `verbsPorts` value to the network interface name that appears in the pod.

```
spec:
  daemon:
    clusterProfile:
      ...
      # -----
      # RDMA specific configuration.
      # Change verbsPort to value based on your configuration. This value should match the
      # network interface name for the InfiniBand device. For example: "mlx5_2", "ibs5f0", etc.
      # -----
      verbsPorts: net1
      # For mixed config scenario, uncomment this next line and delete the line above
      # verbsPorts: mlx5_3/1 mlx5_2/1
      verbsRdma: enable
      verbsRdmaSend: "yes"
      # Only enable `verbsRdmaCm` when deploying RoCE - RDMA over Converged Ethernet
      verbsRdmaCm: enable
```

If you do not know the CNI network name ahead of time, set `verbsPorts` to `net1` for now and allow the core pods to be deployed. After the pods are running, inspect the core pod annotation for the `k8s.v1.cni.cncf.io/network-status` and retrieve the interface name corresponding to the network attachment definition.

Because a MacvlanNetwork was created named `macvlannetwork-ens5flnp1-scale` with target namespace `ibm-spectrum-scale`, there is an entry for `ibm-spectrum-scale/macvlannetwork-ens5flnp1-scale` and the interface is `net1`.

```
$ oc get pods worker0 -o jsonpath='{.metadata.annotations.k8s.v1.cni.cncf.io/network-status}'
[[{"name": "ibm-spectrum-scale/macvlannetwork-ens5flnp1-scale",
  "interface": "net1",
  "ips": [
    "192.168.1.41"
  ],
  "mac": "42:21:3a:22:43:96",
  "dns": {}
}]
```

With the interface name, modify the `verbsPorts` in the cluster CR: `kubectl edit cluster ibm-spectrum-scale`. The operator updates the configuration.

- Accept the license by changing `accept:` to `true`:

```
spec:
  license:
    accept: true
```

- Apply the cluster CR:

```
kubectl apply -f cluster.yaml
```

Wait for the core pods to be created and go into `Running` state. If the core pods are stuck in `Init`, describe the pods to see what is causing the errors and resolve it.

For more information, see [Cluster](#).

6. After the core pods are in `Running` state, enable `verbsGPUDirectStorage` configuration by running the following steps:

- Go into a core pod

```
kubectl exec -it -n ibm-spectrum-scale $(kubectl get pods -n ibm-spectrum-scale -l app.kubernetes.io/name=core -o jsonpath='{.items[0].metadata.name}') -- sh
```

* Make the configuration change. This configuration change requires IBM Storage Scale being stopped.

*** Stop IBM Storage Scale**

```
```bash
mmshutdown -a
```
```

*** enable verbsGPUDirectStorage**

```
```bash
mmchconfig verbsGPUDirectStorage=enable
```
```

*** start IBM Storage Scale**

```
```bash
mmstartup -a
```
```

*** verify that the configuration is set**

```
```bash
mmlsconfig verbsGPUDirectStorage
```
```

There should not be any remote mounted file system or workload that uses storage while this configuration change is being run. If you have an existing IBM Storage Scale container native cluster that is deployed and looking to enable GDS functions, see [Enable GPU Direct Storage](../../planning/gds/gpu-direct-storage.html) for more details.

The configuration keyword `verbsGPUDirectStorage` must also be enabled on the storage cluster.

1. Apply the Remote Cluster CR

Establish a connection to the remote cluster GUI server. For more information, see [RemoteClusters](#)

2. Apply the Filesystem CR

Mount a file system from the IBM Storage Scale cluster. For more information, see [Remote file system](#)

You now have a running IBM Storage Scale container native cluster!

Verification

Verify RDMA

The following steps can help with some validation to help ensure that RDMA is configured correctly from the IBM Storage Scale container native perspective.

1. Go into a core pod

```
kubectl exec -it -n ibm-spectrum-scale $(kubectl get pods -n ibm-spectrum-scale -l app.kubernetes.io/name=core -o jsonpath='{.items[0].metadata.name}') -- sh
```

2. Check for successful messages that VERBS RDMA started.

```
sh-5.1# mmfsadm test verbs status
VERBS RDMA status: started
```

The messages in the GPFS log also show successful detection of the device or errors. If there are errors, they can help give hints on resolving the problems.

```
sh-5.1# grep -i RDMA /var/adm/ras/mmfs.log.latest | more
...
2025-10-14 22:35:57.569+0000: [I] VERBS RDMA parse verbsPorts net1
2025-10-14 22:35:57.569+0000: [I] VERBS RDMA Found IP interface net1 192.168.1.41/24
2025-10-14 22:35:57.569+0000: [I] VERBS RDMA device characteristics: IP interface net1, IB device
mlx5_3, IB port: 1
2025-10-14 22:35:57.574+0000: [I] VERBS RDMA parse success verbsPort net1 IP device mlx5_3 port
1 fabnum 0 sl 0 idx 0 interface
net1 ip 192.168.1.41
2025-10-14 22:35:57.606+0000: [I] VERBS RDMA open success verbsPort net1 device mlx5_3 port 1
fabnum 0 sl 0 interface net1 ip 1
92.168.1.41 lid 0 4x EDR ETHERNET
2025-10-14 22:35:57.608+0000: [I] VERBS RDMA WRITE flush enabled via RDMA_READ.
2025-10-14 22:35:57.608+0000: [I] VERBS RDMA started.
```

3. Look for RDMA connections between the nodes of the client cluster and the storage cluster in the `mmdiag --network` output:

```
sh-5.1# mmdiag --network
...
RDMA Connections between nodes:
Fabric 0 - Device mlx5_3 Port 1 Width 4x Speed EDR lid 0
hostname                               idx CM state VS buff RDMA_CT(ERR) RDMA_RCV_MB RDMA_SND_MB
VS_CT(ERR) VS_SND_MB VS_RCV_MB WAIT_CON_SLOT WAIT_NODE_SLOT
worker1                                0 Y RTS (Y)256 0 (0 ) 0 0 24
(0 ) 0 0 0 0
worker2                                0 Y RTS (Y)256 0 (0 ) 0 0 36
(0 ) 0 0 0 0
ems-eth-hs                             0 Y RTS (Y)256 0 (0 ) 0 0 12
(0 ) 0 0 0 0
ess-b-eth-hs                           0 Y RTS (Y)256 0 (0 ) 0 0 146
(0 ) 0 0 0 0
ess-a-eth-hs                           0 Y RTS (Y)256 0 (0 ) 0 0 23
(0 ) 0 0 0 0
```

Write some dummy data to the file system and check to see RDMA counter activity go up.

```
# The file system is mounted at /mnt/remote-sample
mkdir -p /mnt/remote-sample/junk

# write some dummy data to the file system
dd oflag=direct bs=4194304 count=1000 if=/dev/zero of=/mnt/remote-sample/junk/testfile
```

After the dd command completes, you will see that counters values go up on the storage nodes. In the following output, **ess-a-eth-hs** and **ess-b-eth-hs** increased under **VS_CT** counters.

```
sh-5.1# mmdiag --network
...
RDMA Connections between nodes:
Fabric 0 - Device mlx5_3 Port 1 Width 4x Speed EDR lid 0
hostname                               idx CM state VS buff RDMA_CT(ERR) RDMA_RCV_MB RDMA_SND_MB
VS_CT(ERR) VS_SND_MB VS_RCV_MB WAIT_CON_SLOT WAIT_NODE_SLOT
worker1                                0 Y RTS (Y)256 0 (0 ) 0 0 36
(0 ) 0 0 0 0
worker2                                0 Y RTS (Y)256 0 (0 ) 0 0 54
(0 ) 0 0 0 0
ems-eth-hs                             0 Y RTS (Y)256 0 (0 ) 0 0 34
(0 ) 0 0 0 0
ess-b-eth-hs                           0 Y RTS (Y)256 0 (0 ) 0 0 1250
(0 ) 0 0 0 0
ess-a-eth-hs                           0 Y RTS (Y)256 0 (0 ) 0 0 1578
(0 ) 4 0 0 0
```

Verify GPU Direct Storage

1. Help ensure that the **nvidia** kernel mods are loaded on the worker nodes.

```
$ oc debug node/worker0.example.com
Starting pod/worker0examplecom-debug-wkx4k ...
To use host binaries, run `chroot /host`
sh-5.1# chroot /host
sh-5.1# lsmod | grep nvidia
nvidia_fs                 327680 0
nvidia_peermem            24576 0
nvidia_modeset            1753088 0
nvidia_uvm                4087808 12
nvidia                    14381056 27 nvidia_uvm,nvidia_peermem,nvidia_fs,nvidia_modeset
video                     73728 1 nvidia_modeset
ib_uverbs                 221184 41 nvidia_peermem,rdma_ucm,mlx5_ib
drm                       741376 5 drm_kms_helper,drm_shmem_helper,nvidia,mgag200
```

2. Create a container image that holds GPU tools, such as **gdscheck** and **gdsio**.

There are no readily available containers images that package this tool, you need to create your own. For detailed information on how to do this, see the [Build RDMA GPU-Tools Container](#) blog post.

3. Deploy the test image into a pod that mounts the IBM Storage Scale file system.

In the **image:** section, complete the path to the GPU tools image that you created.

```
apiVersion: v1
kind: Pod
metadata:
  name: gds-check-workload
  namespace: ibm-spectrum-scale
spec:
  hostNetwork: true
```

```
containers:
- image: <FILL IN YOUR GDS TOOLS CONTAINER IMAGE>
  securityContext:
    privileged: true
    add: [ "IPC_LOCK" ]
  imagePullPolicy: Always
  name: gds-check-workload
  resources:
    limits:
      nvidia.com/gpu: 1
    requests:
      nvidia.com/gpu: 1
  volumeMounts:
    - name: ibm-fs
      mountPath: /mnt/ibm
    - name: udev
      mountPath: /run/udev
    - name: kernel-config
      mountPath: /sys/kernel/config
    - name: sys
      mountPath: /sys
    - name: lib
      mountPath: /lib/modules
  volumes:
    - name: ibm-fs
      hostPath:
        path: /mnt/remote-sample/
    - name: udev
      hostPath:
        path: /run/udev
    - name: kernel-config
      hostPath:
        path: /sys/kernel/config
    - name: sys
      hostPath:
        path: /sys
    - name: lib
      hostPath:
        path: /lib/modules
```

4. Exec into the test pod and modify configuration settings in `/etc/cufile.json`.

```
# exec into the test pod
kubectl -n ibm-spectrum-scale exec -it gds-check-workload -- sh
```

Help ensure that the following configuration values are set in `/etc/cufile.json`:

- Set `allow_compat_mode` to false
- Set `rdma_dev_addr_list` to the RDMA port IP address.
- Set `gpfs.gds_write_support` to true
- Set `gpfs.gds_async_support` to false as Asynchronous CUDA IO is not supported.

For information on RDMA port IP address, see [Configuring RDMA ports on the GPU client](#)

1. Exec into this test pod and run `gdscheck -p`, verify that IBM Spectrum Scale is supported:

```
# exec into the test pod
kubectl -n ibm-spectrum-scale exec -it gds-check-workload -- sh
```

Check that the following properties are set:

```
sh-5.1# /usr/local/cuda-13.0/gds/tools/gdscheck -p | egrep "IBM|gpfs|Up:"
IBM Spectrum Scale : Supported
--rdma_device_status : Up: 1 Down: 0
fs.gpfs.gds_write_support: true
fs.gpfs.gds_async_support: false
```

2. Exec into the test pod and run `gdsio` commands to check the performance.

```
# exec into the test pod
kubectl -n ibm-spectrum-scale exec -it gds-check-workload -- sh
```

Sample READ commands using GPUD, CPU_ONLY and CPU_GPU:

```
/usr/local/cuda-13.0/gds/tools/gdsio -f /mnt/ibm/junk/file1.dat -d 0 -s 32G -i 16M -x 0 -I 0 -w 1
/usr/local/cuda-13.0/gds/tools/gdsio -f /mnt/ibm/junk/file1.dat -d 0 -s 32G -i 16M -x 1 -I 0 -w 1
/usr/local/cuda-13.0/gds/tools/gdsio -f /mnt/ibm/junk/file1.dat -d 0 -s 32G -i 16M -x 2 -I 0 -w 1
```

Sample READ commands using GPUD, CPU_ONLY and CPU_GPU:

```
/usr/local/cuda-13.0/gds/tools/gdsio -f /mnt/ibm/junk/file1.dat -d 0 -s 32G -i 16M -x 0 -I 1 -w 1
/usr/local/cuda-13.0/gds/tools/gdsio -f /mnt/ibm/junk/file1.dat -d 0 -s 32G -i 16M -x 1 -I 1 -w 1
/usr/local/cuda-13.0/gds/tools/gdsio -f /mnt/ibm/junk/file1.dat -d 0 -s 32G -i 16M -x 2 -I 1 -w 1
```

IBM Storage Scale container native and SELinux

IBM Storage Scale container native offers a default for Container Storage Interface (CSI) volume attachment behavior for Security-Enhanced Linux (SELinux) labels. This default is introduced beginning with IBM Storage Scale container native 5.1.7.0 release.

What is SELinux?

SELinux, or Security-Enhanced Linux, defines access controls for the applications, processes, and files on a system. It uses security policies, which are a set of rules that tell SELinux what can or cannot be accessed, to enforce the access allowed by a policy.

For more information about SELinux, see [What is SELinux?](#) in Red Hat Documentation.

How is SELinux controlled in Kubernetes?

Container processes are started with the SELinux context in the `.spec.securityContext.seLinuxOptions` field, while CSI-based volumes have all their files labeled to match on pod attachment during container creation.

SELinux Multi-Category Security (MCS) is used in Red Hat OpenShift. By default, containers have their SELinux level set to values annotated on the namespace. The category defaults annotated on the namespace are assigned automatically by OpenShift to limit overlap.

Red Hat OpenShift is based on Kubernetes.

Problems with SELinux relabel on volume attach

When attaching CSI-based volumes to pods in Kubernetes the container-runtime SELinux relabels all files in the volume. This is problematic for shared filesystems that support SELinux, such as the IBM Storage Scale filesystem.

Relabeling on volume attachment moves the security control to the consumer of file volumes instead of the owner of the files. In classic shared filesystem environments, such as IBM Storage Scale, access is controlled by node administrators and file owners. In Kubernetes, volume access is isolated to a namespaced volume claim and controlled using Kubernetes role-based access control (RBAC). While in classic shared filesystem environments, such as IBM Storage Scale, access is controlled by node administrators and file owners. This Kubernetes behavior makes it very difficult to maintain access controls for volumes and files that are shared outside of Kubernetes. This is because Kubernetes ignores and overwrites any SELinux security isolation set by external administrators.

In addition, relabeling all files in a volume is a non-trivial operation, which may generate a large I/O load on the backend storage system. This introduces a performance and denial of service concern. Volumes with too many files, or shared volumes that are attached concurrently, may easily swamp the storage subsystem and cause cascading pod creation timeouts.

Upstream Kubernetes limitation for SELinux relabel

The SELinux relabel issue is not unique to IBM Storage Scale. Upstream Kubernetes does not give CSI volume driver implementations control of SELinux relabel on volume attachment.

Legacy in-tree volume drivers, such as the in-tree NFS volume driver, have the ability to control SELinux relabeling. However, legacy in-tree volume drivers are deprecated.

In Red Hat OpenShift, the container-runtime that performs the SELinux relabel will skip the relabeling if the container SELinux context is set to `spc_t`, which is the super privileged container. It is an uncontained type and may access any file on the system allowed by standard file permissions.

Default volume attachment behavior by IBM Storage Scale container native

To prevent Kubernetes from relabeling SELinux file labels, IBM Storage Scale container native by default will mount the filesystem with a container permissive context. This disallows the `security.selinux` label from being set, and all files inside the filesystem will be considered to have the context defined on the filesystem mount. By default that context would allow all containers running as `container_t` SELinux type to access files on the filesystem allowed by standard file permissions. By default, the mount context is set to `system_u:object_r:container_file_t:s0`.

Comparing SELinux relabel on attach and SELinux mount context

Security considerations

Setting a container permissive SELinux mount context or doing an SELinux relabel on volume attachment have similar access control within Kubernetes. Any container that can claim a volume may access its files.

However, if a process could escape a container or the host was able to be compromised, then any `container_t` constrained context would have access to the filesystem from a SELinux access perspective. Standard linux file permissions would still prevent access. This means containers that run within the "restricted" SecurityContextConstraint defined by OpenShift would not be able to access files in a volume that it did not explicitly share.

If a volume is shared with Kubernetes from an external application, then SELinux relabeling breaks SELinux security isolation managed externally. SELinux relabel on attach allows a volume consumer to ignore and overwrite SELinux labels, potentially exposing the files to other external systems. Setting the SELinux mount context will only allow the external SELinux labels to be ignored within the confines of the Kubernetes cluster.

Performance considerations

Since SELinux mount context disallows SELinux relabeling, volume attachments to pods do not generate extra I/O load and will attach faster than if SELinux relabeling is done.

If SELinux relabeling occurs, it must complete within a minute or the container creation will timeout and fail. Multiple relabel operations occurring concurrently are more likely to trigger timeout and failure. This cascading failure may only be exposed due to wider outages (node, cluster, lab), or maintenance.

External access considerations

External access refers to any components outside of the IBM Storage Scale container native cluster. This includes the storage cluster that is remotely mounted to the IBM Storage Scale container native cluster.

SELinux mount context means all new files are created without SELinux labels, and are considered `unlabeled_t`. This would include files created by application containers using IBM Storage Scale container native volumes. Generally, access to `unlabeled_t` files is only allowed by uncontained or more privileged process contexts. To access the unlabeled files, the external applications outside of Kubernetes must be given access to `unlabeled_t` via SELinux user policies, the files must be labeled manually, or the mount used by the application has a valid SELinux context configured.

The behavior without mount context, which does an SELinux relabel of all files, is similar in that the SELinux context of the container should be configured to match what is desired externally, or the external application should be granted access to files created by the container SELinux context.

How to change mount context

The mount context may be set on the Filesystem kind. This applies to all applications using volumes within the filesystem. For more details, run `kubectl explain fs.spec.selinuxOptions` command.

When changing the SELinux context, applications with ReadWriteMany volumes running on nodes with mixed SELinux contexts may experience "Permission Denied" errors. Applications running on nodes with differing SELinux contexts may not have access to each other's files during the duration of the SELinux context update.

Changing the `fs.spec.selinuxOptions` field of a Filesystem kind will cause pods to restart, and the restart of the pod will cause a reboot of the node itself. Ensure that the proper maintenance window and precautions, similar to upgrade, have been taken prior to changing this field.

```
spec:
  selinuxOptions:
    user: <user>
    role: <object>
    type: <type>
    level: <level>
```

Legacy relabel on attach behavior

Earlier behavior of relabel on attach is considered legacy and the support is limited for it. If this behavior is required, contact IBM support.

Related links

- [Red Hat solution - pods fail to start due to volumes with high volume counts in Red Hat OpenShift](#)
- [Kubernetes Enhancements 1710 - Speed up SELinux volume relabeling using mounts](#)
 - <https://github.com/kubernetes/enhancements/issues/1710>
- [Red Hat OpenShift - Understanding host and VM security](#)
- [Red Hat OpenShift and SELinux](#)

Roles and personas

Different roles, cluster roles, and levels of access are needed to deploy a fully functioning IBM Storage Scale container native cluster.

Personas

Red Hat OpenShift cluster administrator

A user with cluster administrator privileges needs to deploy the IBM Storage Scale container native cluster. Cluster admin privileges are needed in order to create higher privilege artifacts such as:

- Namespaces
- Custom Resource Definitions
- Cluster Roles and Cluster Role Bindings
- Security Context Constraints

By having the OpenShift cluster administrator execute the cluster deployment, the operator pod can be configured in a more restricted manner with minimal privilege.

IBM Storage Scale Storage cluster administrator

A user with privilege and access to configure the existing IBM Storage Scale storage cluster for remote access is also required for a successful deployment of an IBM Storage Scale container native cluster. Since the container native cluster utilizes remote mounts, the storage cluster admin must be able to execute commands against the storage cluster. For more information, see [IBM Storage Scale storage cluster](#).

Lab administrator

A user with access to customer infrastructure and network is required to ensure a successful IBM Storage Scale container native cluster deployment. All OpenShift nodes that comprise the IBM Storage Scale container native cluster must be able to communicate with the remote IBM Storage Scale storage cluster. This may require a Lab Administrator to tune the customer network firewall to allow such communications. For more information, see [Firewall recommendations](#).

Operator permissions

The IBM Storage Scale container native operator is a cluster-scoped operator. The operator watches all namespaces on the OpenShift cluster it is deployed into. Since the operator is cluster scoped, it requires access to cluster level resources to successfully deploy. Access to cluster level resources is handled through a cluster role that is deployed via RBAC YAML files. The cluster role is bound to the custom `ibm-spectrum-scale-operator` ServiceAccount, which the operator uses to create the IBM Storage Scale container native cluster.

To view the permissions of the operator, use the following query on a system that has a deployed container native operator.

```
kubectl describe clusterrole ibm-spectrum-scale-operator
```

This command will list out every resource the operator has access to and what it can do with them.

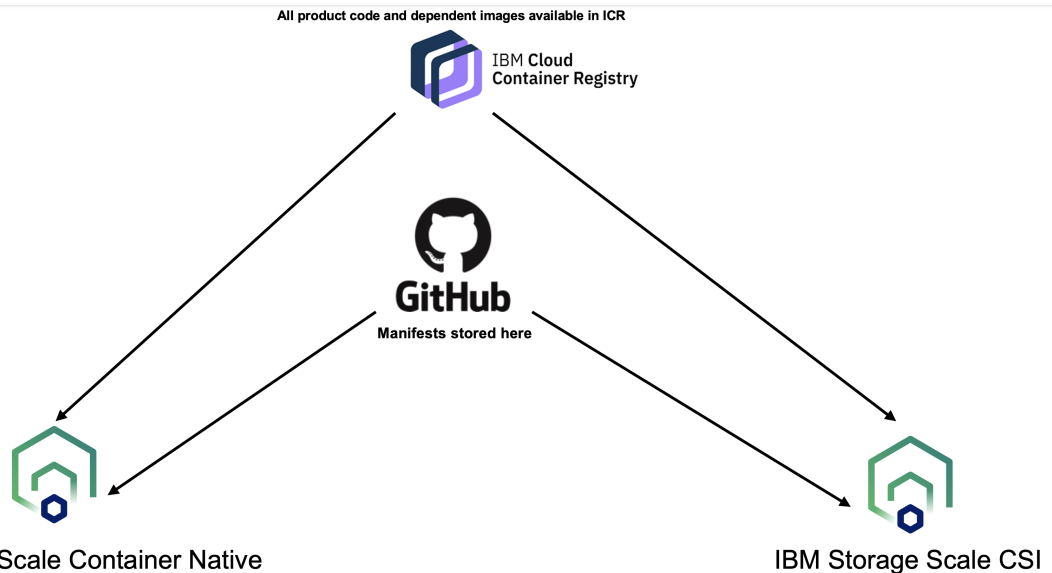
Roles

Once a IBM Storage Scale container native cluster is operational, users can authenticate to the IBM Storage Scale container native GUI via existing OCP roles. For more information, see [OpenShift users](#).

Container images

Obtaining container images are necessary for the successful deployment of IBM Storage Scale container native. All container images are sourced from the IBM Container Repository, ICR.

Figure 1. Dependent images available in ICR



Always use the latest fix pack release available.

For the detailed list of container images, see [Container image list for IBM Storage Scale container native](#) on GitHub.

Air-gapped installs

In production, it is common to have a cluster that does not have internet access. If your Kubernetes cluster is air-gapped (otherwise known as offline or disconnected), you can install an IBM Storage Scale container native by mirroring images. These tasks must be ran by a Kubernetes administrator.

A bastion host is a device that has access to both the public internet and the network-restricted environment where a local registry and Kubernetes clusters reside. Using the bastion host, you can mirror your images directly to the local registry.

You need to do the Air gap setup if the worker nodes are not able to access the repository due to network and firewall restrictions.

Prerequisites

- A production-grade Docker V2 registry that is available and accessible from the Kubernetes cluster nodes such as Quay Enterprise, JFrog Artifactory, or Docker Registry. The Red Hat OpenShift Internal Registry is not supported.
- A bastion node that:
 - Access to the public internet.
 - Access to the network-restricted environment where the local registry exists.
 - Access to the entitled registry: `icr.io:443`, `cp.icr.io:443`.
 - `skopeo` command is installed.
- Access to the Kubernetes cluster as a user with the `cluster-admin` role.

Do not use Red Hat OpenShift internal image registry as your local registry. The Red Hat OpenShift registry does not support multi-architecture images or path separators in the image name.

Before starting, export the following variables. They are used in the subsequent steps.

- export the internal registry domain

```
export INTERNAL_REGISTRY=example.io
```
- export the internal registry target path

```
export INTERNAL_REGISTRY_PATH=path/to/images
```

Mirror the images

Mirror the images by using `skopeo` to copy images from the IBM Container Repository, ICR, to an internal container image registry.

Complete the following steps from the bastion node:

1. Log in to the IBM Entitled Container Registry:

```
skopeo login cp.icr.io
```

2. Log in to the internal image registry:

```
#  
# assumes you have exported INTERNAL_REGISTRY  
#  
skopeo login $INTERNAL_REGISTRY
```

3. Use `skopeo copy` to copy the images from ICR to the internal image registry.

The image listing for each release is available on GitHub. See the [Container image list for IBM Storage Scale container native](#) in the GitHub repository.

Starting with IBM Storage Scale container native v5.2.3, the ICR path for the container images have been changed from `spectrum/scale` to `gpfs`.

Copy the air-gapped images from GitHub to a file called `images.txt`. You can then use the following helper script to create the `skopeo copy` commands to run:

```
#  
# assumes you have exported INTERNAL_REGISTRY  
#  
for image in `cat images.txt | grep -v \#`; do  
    if [[ $image == *cpopen* ]]; then  
        echo "skopeo copy --all docker://${image} docker://${INTERNAL_REGISTRY}/${image}#cp.icr.io/"  
    else  
        echo "skopeo copy --all docker://${image} docker://${INTERNAL_REGISTRY}/${image}#cp.icr.io/"  
    fi  
done
```

The helper script is provided as an example. Use at your own risk.

For more information, see [Skopeo Copy to the Rescue](#).

4. Log out of the IBM Container Registry:

```
skopeo logout cp.icr.io
```

5. Log out of your internal image registry:

```
#  
# assumes you have exported INTERNAL_REGISTRY  
#  
skopeo logout ${INTERNAL_REGISTRY}
```

Modify the installer yamls

1. Download installation manifest file

For Red Hat OpenShift:

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/install.yaml > install.yaml || echo "Failed to download install yaml"
```

For Kubernetes:

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/install-k8s.yaml > install.yaml || echo "Failed to download install yaml"
```

Ensure that you pick the correct curl command based on your environment. The source file is different but the downloaded files are renamed to `install.yaml`.

2. Use `sed` to replace production path to the internal image registry path

```
#  
# assumes you have exported INTERNAL_REGISTRY  
#  
sed -i s#cp.icr.io/cp#${INTERNAL_REGISTRY}#g install.yaml  
sed -i s#icr.io/cpopen#${INTERNAL_REGISTRY}#g install.yaml
```

3. Apply the installation manifest file to the cluster

```
kubectl apply -f install.yaml
```

Create namespace docker pull secrets

Create **ibm-entitlement-key** pull secret in the following namespaces with credentials to your internal registry.

- ibm-spectrum-scale
- ibm-spectrum-scale-operator
- ibm-spectrum-scale-dns
- ibm-spectrum-scale-csi

This is required to allow the images to pull successfully.

1. Export the entitlement key and user from your internal image registry.

```
export INTERNAL_REGISTRY=example.io
export ENTITLEMENT_USER=<REPLACE WITH USER OF INTERNAL IMAGE REGISTRY>
export ENTITLEMENT_KEY=<REPLACE WITH PASSWORD OF INTERNAL IMAGE REGISTRY>
```

2. Use the following helper script to create a docker-registry secret for each namespace:

```
for namespace in ibm-spectrum-scale ibm-spectrum-scale-operator ibm-spectrum-scale-dns ibm-spectrum-scale-csi; do
    kubectl create secret docker-registry ibm-entitlement-key -n ${namespace} \
        --docker-server=${INTERNAL_REGISTRY} \
        --docker-username=${ENTITLEMENT_USER} \
        --docker-password=${ENTITLEMENT_KEY}
done
```

3. Unset the export:

```
unset INTERNAL_REGISTRY
unset ENTITLEMENT_USER
unset ENTITLEMENT_KEY
```

Verify the deployment

After deploying the installation manifest files and creating the image pull secrets, the images should pull successfully and the operator pod will be in a running state.

```
# kubectl get pods -n ibm-spectrum-scale-operator
```

| NAME | READY | STATUS | RESTARTS | AGE |
|--|-------|---------|----------|-----|
| ibm-spectrum-scale-controller-manager-64d548f576-znrgc | 1/1 | Running | 0 | 37m |

Storage cluster

IBM Storage Scale container native supports remote mounting file systems in the following scenarios.

- [On-premise](#)
- [Cloud](#)

A storage cluster is not needed if only local file systems are used.

On-premise

Some additional tasks need to be performed on the IBM Storage Scale storage cluster that runs on-premise.

IBM Storage Scale storage cluster

The operators of IBM Storage Scale container native and IBM Storage Scale CSI interact with the storage cluster through ReST API (which is part of the GUI stack). To enable the operator to interact with storage cluster, user IDs need to be created on the storage cluster GUI. Specific roles are used to grant those user IDs only the operations that are needed to provide their functionality. In addition, some settings on the cluster and the filesystem are required for interoperability with CSI.

Creating container operator user

Complete the following steps in the shell of the GUI node of the storage cluster:

Storage cluster older than v5.2.2.x

1. To create container native operator GUI user, enter the following command:

```
/usr/lpp/mmfs/gui/cli/mkuser cnsa_storage_gui_user -p cnsa_storage_gui_password -g ContainerOperator
```

By default, user passwords expire after 90 days. If the security policy of your organization permits it, use the **-e 1** option on the **mkuser** command to create a user with a password that does not expire.

2. To create the CSI GUI user, enter the following commands:

```
/usr/lpp/mmfs/gui/cli/mkuser csi_storage_gui_user -p csi_storage_gui_password -g CsiAdmin
```

By default, user passwords expire after 90 days. If the security policy of your organization permits it, use the **-e 1** option on the **mkuser** command to create a user with a password that does not expire.

After creating the container operator and CSI users, there are two possible ways on how to change the passwords of these users at later times:

1. Let the operator automatically change the passwords on a regular basis. For more information, see [Automatic password rotation](#).
2. Manually update the passwords. For more information, see [Updating authentication to the storage cluster](#).

Storage cluster v5.2.3.x and newer

1. To create container native operator GUI user, enter the following command:

```
/usr/lpp/mmfs/gui/cli/mkuser cnsa_storage_gui_user -p cnsa_storage_gui_password -g ContainerOperator
```

By default, user passwords expire after 90 days. If the security policy of your organization permits it, use the **-e 1** option on the **mkuser** command to create a user with a password that does not expire.

After creating the container operator user, there are two possible ways on how to change the password of this user at later times:

1. Let the operator automatically change the password on a regular basis. For more information, see [Automatic password rotation](#).
2. Manually update the password. For more information, see [Updating authentication to the storage cluster](#).

Storage cluster configuration for Container Storage Interface (CSI)

Complete the following steps on the *storage cluster* to ensure that the IBM Storage Scale CSI driver can operate successfully:

1. Ensure that the perfilesset quota on the file systems that are used by IBM Storage Scale Container Storage Interface driver is set to **No**.

The IBM Storage Scale Container Storage Interface driver will create many filesets (one per PV). Tracking user and group quotas on a per-fileset basis will significantly increase the overhead of quota management. As a result, the file system performance can suffer.

```
mmfsfs fs1 --perfilesset-quota
```

2. Enter the following command to enable the Quota in the file systems:

The IBM Storage Scale Container Storage Interface driver translates capacity of persistent volumes to fileset quotas. For this to work, quotas are required to be enabled in the file system.

```
mmchfs fs1 -Q yes
```

Enter the following command to verify quota is enabled:

```
mmfsfs fs1 -Q
```

3. Enable the quota for root user by entering the following command:

On Kubernetes, the containers may run as root, so ensure that quotas are enforced for the root user as well.

```
mmchconfig enforceFilesetQuotaOnRoot=yes -i
```

4. Ensure that the **ignoreReplicationForQuota** parameter is set to "yes" by entering the following command:

In case file system is configured to have more than 1 data replica, Then this setting is required for IBM Storage Scale Container Storage Interface driver to configure the quota correctly.

```
mmchconfig ignoreReplicationForQuota=yes -i
```

5. Ensure that the **controlSetxattrImmutableSELinux** parameter is set to "yes" by entering the following command:

Kubernetes does not honor immutability of files/directories when setting SELinux labels. This creates issues, for example, with the immutable **.snapshot** directory.

```
mmchconfig controlSetxattrImmutableSELinux=yes -i
```

6. Enable **filesetdf** of the file system by entering the following command:

IBM Storage Scale Container Storage Interface driver will only be able to report free space on persistent volumes if **filesetdf** is set correctly.

```
mmchfs fs1 --filesetdf
```

7. IBM Storage Scale Container Storage Interface driver has no information about the number of inodes a persistent volume will consume. Therefore, corresponding independent filesets are created with default values for maxIndoes. To enable automatic expansion of the inode space so persistent volumes do not run out of inodes, enter the following command:

```
mmchfs fs1 --auto-inode-limit
```

The `--auto-inode-limit` option is available only with filesystem format level 28.00 or later. Enable `auto-inode-limit` as soon as the filesystem format level is updated to 28.00 or later. On older filesystem levels the administrator of the storage cluster needs to manually increase the inode limit when warnings for low inodes are raised by the health monitoring.

For more information about `auto-inode-limit` parameter, see [mmchfs command](#) in IBM Storage Scale documentation.

Configure cluster profile with `tscCmdAllowRemoteConnections`

Starting with version IBM Storage Scale and IBM Storage Scale container native 5.1.3, the `tscCmdAllowRemoteConnections` configuration is recommended to be set to `no`. If the storage cluster and all client clusters (including IBM Storage Scale container native) are at versions $\geq 5.1.3$, it is recommended to set this value to `no`. However, if any version is $< 5.1.3$, `tscCmdAllowRemoteConnections` must be set to `yes` on the storage cluster and client clusters to successfully communicate between the clusters.

Use the following table as a reference.

Table 1. `tscCmdAllowRemoteConnections` configuration

| Storage cluster version | IBM Storage Scale container native version | <code>tscCmdAllowRemoteConnections</code> |
|-------------------------|--|---|
| $< 5.1.3$ | $< 5.1.3.0$ | yes |
| $\geq 5.1.3$ | $< 5.1.3.0$ | yes |
| $\geq 5.1.3$ | $\geq 5.1.3.0$ | no |

- To change this value on the storage cluster, run: `mmhconfig tscCmdAllowRemoteConnections='yes|no'`.
- To change this value on the IBM Storage Scale container native cluster, set `tscCmdAllowRemoteConnections: yes|no` in the `clusterProfile` section of the cluster spec:

```
kind: Cluster
metadata:
  name: ibm-spectrum-scale
spec:
  ...
  daemon:
    ...
  clusterProfile:
    tscCmdAllowRemoteConnections: "yes"
```

For more information to configure the `clusterProfile` section of the cluster spec, see [Cluster profile](#).

For more information about all IBM Storage Scale services, see [Securing the IBM Storage Scale system using firewall](#) in IBM Storage Scale documentation.

Cloud

AWS storage cluster

IBM Storage Scale container native with Red Hat OpenShift Service on AWS (ROSA) provides a highly scalable and performant solution for containerized applications. You can use the IBM Storage Scale Storage cluster on public clouds (such as AWS) to store its persistent data. IBM Storage Scale storage clusters on public clouds can be deployed and managed with the `cloudkit` CLI. For more information, see [Cloudkit](#) in IBM Storage Scale documentation.

`cloudkit` is included in the IBM Storage Scale Self-Extracting (SE) package and enables users to deploy IBM Storage Scale clusters across multiple cloud providers (currently limited to AWS), providing deployment topology flexibility. `cloudkit` helps:

- Reduce the complexity and time that is needed to set up the storage cluster.
- Offers the ability to configure various cluster parameters, such as disk type, instance type, and network configurations.
- Offers the ability to integrate with other IBM Storage Scale features, such as data replication, data tiering, and data encryption.

Considerations

Consider the list:

- Understand the requirements in terms of performance, scalability, data availability, and data protection.
- Consider the structure of your storage, including the number of storage nodes, the file system size, and the number of replicas you want to maintain.
- ROSA and `cloudkit` both use Multi-Availability Zone (AZ) to ensure resiliency, but in Multi-AZ mode, the inter-AZ network limit constrains the throughput.
- Review the AWS quota limits to ensure that your existing Virtual Private Cloud (VPC) has enough free IP addresses to accommodate both ROSA and `cloudkit`. If you are planning to create a new VPC, you should plan the network to accommodate the IP requirements for both services.
- Users have the flexibility to start with either ROSA in new VPC mode and `cloudkit` in existing VPC mode (or `cloudkit` in new VPC mode and ROSA in existing VPC mode, depending on the requirements.
- For optimal performance, choose a storage cluster deployment within the same subnet that is in use by the ROSA, providing low latency and higher throughput.

The following tasks assume that the installation of IBM Storage Scale container native on AWS ROSA is complete.

Deploy storage cluster

Use the `cloudkit` command to deploy the storage cluster.

```
cloudkit create cluster
```

`cloudkit` provides various deployment profiles, including `Throughput-Performance-Scratch-Storage`, `Throughput-Performance-Persistent-Storage`, `Throughput-Advance-Persistent-Storage`, and `Balanced`.

If you are deploying to a Single AZ, it is advisable to use the `Throughput-Performance-Persistent-Storage` profile. For multi-AZ, use the `Balanced` profile.

Do not use the `Throughput-Performance-Scratch-Storage` profile for IBM Storage Scale container native workloads. This profile uses instance storage and results in data loss when storage nodes are shut down.

Details of the cluster can be viewed by using the describe option:

```
cloudkit describe cluster
```

Perform grant filesystem operation

`cloudkit` performs setting up security rules for data access and exchange between the ROSA worker security group and storage cluster. It performs the prerequisite configurations such as creating users in the `ContainerOperator` and `CsiAdmin` groups on the storage cluster.

Use the `cloudkit` command to simplify the data management across multiple IBM Storage Scale container native clusters.

```
cloudkit grant filesystem
```

Details of the file system access granted by `cloudkit` can be viewed by using the describe option:

```
cloudkit describe grant
```

Perform revoke filesystem operation

If the IBM Storage Scale container native is uninstalled, revoke the access to the file system by using the following command:

```
cloudkit revoke filesystem
```

Delete storage cluster

The following action cannot be undone and will destroy all scale instances and wipe the disks that are used for the file system. Take the appropriate data backups before proceeding, failure to do so will result in permanent loss of data.

Use the following `cloudkit` command to delete the storage cluster:

```
cloudkit delete cluster
```

Red Hat OpenShift Container Platform

Refer to the following sections to help prepare your Red Hat OpenShift environments.

- [Software requirements](#)
- [On-premise](#)
 - [Worker node configuration](#)
 - [Master node configuration \(compact cluster\)](#)
 - [Infrastructure node configuration](#)
- [Red Hat OpenShift service on AWS](#)

- [Support matrix for IBM Storage Scale container native and ROSA](#)
- [Red Hat OpenShift configuration on AWS](#)
- [Validating Red Hat OpenShift configuration](#)

Software requirements

Use the following table to determine the software requirement levels for Red Hat OpenShift for each release.

Table 1. Software Requirements for Red Hat OpenShift

| IBM Storage Scale Container Native | Architecture | OpenShift Container Platform level | Red Hat CoreOS |
|------------------------------------|------------------------|------------------------------------|------------------------------|
| 5.2.3.9 | x86_64, ppc64le, s390x | 4.16, 4.17, 4.18, 4.19, 4.20 | 4.16, 4.17, 4.18, 4.19, 4.20 |
| 6.0.0.0 | x86_64, ppc64le, s390x | 4.17, 4.18, 4.19, 4.20 | 4.17, 4.18, 4.19, 4.20 |
| 6.0.0.1 | x86_64, ppc64le, s390x | 4.17, 4.18, 4.19, 4.20 | 4.17, 4.18, 4.19, 4.20 |
| 6.0.0.2 | x86_64, ppc64le, s390x | 4.17, 4.18, 4.19, 4.20, 4.21 | 4.17, 4.18, 4.19, 4.20, 4.21 |

Major version upgrades that span multiple release levels may not be supported due to overlapping compatibility constraints with Red Hat OpenShift Container Platform (OCP) versions. Refer to the Software Requirements section corresponding to your cluster's operating system and platform for detailed compatibility guidance.

GPU Direct Storage Software requirements

Use the following table to determine the software requirement for GPU Direct Storage for each release.

Table 2. Software Requirements for GPU Direct Storage

| IBM Storage Scale Container Native | OpenShift Container Platform level | CUDA | NVIDIA Network Operator | NVIDIA GPU Operator | GDS Driver (nvidia-fs) | NVIDIA Driver | RDMA |
|------------------------------------|------------------------------------|------|-------------------------|---------------------|------------------------|---------------|------|
| 6.0.0.x | 4.18 | 13.0 | 25.4.0 | 25.3.2 | 2.26.6 | 580.82.07 | RoCE |

On-premise

Extra configurations need to be made to the Red Hat OpenShift cluster installation for IBM Storage Scale container native cluster to operate correctly. For more information on, see [Installing a cluster on any platform](#) in Red Hat OpenShift documentation.

These configuration tasks can also be handled during the Red Hat OpenShift Container Platform installation by adding day-1 kernel arguments. For more information, see [Customizing nodes](#) in Red Hat OpenShift documentation.

Validate the following tasks are complete ahead of installing the IBM Storage Scale container native code on an on-premises Red Hat OpenShift.

- [Worker node configuration](#)
- [Master node configuration \(compact cluster\)](#)
- [Infrastructure node configuration](#)

Worker node configuration

Applying Machine Config Operator (MCO) settings

The following **MachineConfiguration** is provided as a convenience to easily change the following configuration:

- Kernel development and kernel header packages

Install the kernel-related packages for IBM Storage Scale to successfully build its portability layer.

- Increase vmalloc kernel parameter

Modify the kernel parameters that are needed to operate properly with Red Hat CoreOS. This applies only to the IBM Storage Scale running on Linux™ on Z.

Apply the correct sample file based on your OpenShift Container Platform version and machine architecture.

- If you are running x86_64 or ppc64le, enter the following command:

```
kubectl apply -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/mco/mco.yaml
```

- If you are running s390x, enter the following command:

```
kubectl apply -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/mco/mco_s390x.yaml
```

Infrastructure node configuration

Similar to the configuration tasks on the workers nodes, Machine Configuration Operator (MCO) settings must be applied to the infrastructure nodes if you want to run IBM Storage Scale core pods.

If your infrastructure nodes share roles with the worker pool, indicated by **ROLES: infra,worker**, you can ignore the following instructions.

Apply the correct sample file based on your OpenShift Container Platform version and machine architecture.

- If you are running x86_64 or ppc64le, enter the following command:

```
curl https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/mco/mco.yaml | sed 's/worker/infra/g' | kubectl apply -f -
```

- If you are running s390x, enter the following command:

```
curl https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/mco/mco_s390x.yaml | sed 's/worker/infra/g' | kubectl apply -f -
```

Master node configuration (compact cluster)

You can deploy compact-3-node clusters with Red Hat OpenShift Container Platform 4.5 and later. For more information, see [Delivering a Three-node Architecture for Edge Deployments](#) in Red Hat Hybrid Cloud documentation.

In this configuration, verify that the system is sized correctly to operate smoothly. In compact clusters, the Kubernetes control plane, IBM Storage Scale container native pods, and user applications, all compete for the same resources in the cluster. High application loads can impact the control plane resources, causing the Red Hat OpenShift cluster to become unusable or unstable. etcd is sensitive to latency and might present as frequent leader elections, and other instabilities. IBM Storage Scale container native might also take down the file system if resources are constrained.

For more information, see [Recommended etcd practices](#) in Red Hat OpenShift documentation.

Schedulable control plane nodes

To allow pod placement for master nodes, also known as control plane nodes, make sure that they are configured as **schedulable**. By default, control plane nodes are not schedulable.

Verify **mastersSchedulable** is set to **true** by entering the following command:

```
kubectl get schedulers.config.openshift.io cluster -ojson | jq -r ".spec.mastersSchedulable"
```

If this value is not **true**, patch the cluster by entering the following command:

```
kubectl patch schedulers.config.openshift.io cluster --type='json' \
-p='[{"op": "replace", "path": "/spec/mastersSchedulable", "value": true}]'
```

For more information, see [Configuring control plane nodes as schedulable](#) in the Red Hat OpenShift documentation.

Applying Machine Config Operator (MCO) settings

Similar to the configuration tasks required on the workers nodes, these Machine Config Operator (MCO) settings must be applied to the master nodes in a compact-cluster environment.

Apply the correct sample file based on your OpenShift Container Platform version and machine architecture.

- If you are running x86_64 or ppc64le, enter the following command:

```
curl https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/mco/mco.yaml | sed 's/worker/master/g' | kubectl apply -f -
```

- If you are running s390x, enter the following command:

```
curl https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/mco/mco_s390x.yaml | sed 's/worker/master/g' | kubectl apply -f -
```

Red Hat OpenShift Service on AWS

Red Hat OpenShift Service on AWS (ROSA) is a fully managed service that provides an enterprise-ready Red Hat OpenShift cluster to run applications on AWS. It offers an easy and quick deployment of an operational Red Hat OpenShift cluster without the concerns to administer the underlying infrastructure. The service also provides a range of features to help improve the security and reliability of Red Hat OpenShift clusters, including automated backups and recovery, fine-grained access control, and integration with AWS security services.

For more information on ROSA, see [Red Hat OpenShift Service on AWS Documentation](#)[: external].

Refer to the following sections to know about the configuration requirements for Red Hat OpenShift on AWS.

- [Support matrix for IBM Storage Scale container native and ROSA](#)
- [Red Hat OpenShift configuration on AWS](#)

Support matrix for IBM Storage Scale container native and ROSA

The following table lists the support matrix for IBM Storage Scale container native and Red Hat Open Shift Service on AWS (ROSA).

Table 1. AWS support matrix

| ROSA CLI version | OCP version | Autoscaling workers | Spot instances |
|------------------|---|---------------------|----------------|
| 1.2.45+ | See supported OCP versions in Software requirements | No | No |

In case of issues related to IBM Storage Scale container native, contact IBM Support. For problems related to Red Hat OpenShift and AWS infrastructure (ROSA), contact AWS support.

Red Hat OpenShift configuration on AWS

Modify the Red Hat OpenShift Service on AWS (ROSA) platform for IBM Storage Scale container native to operate correctly.

Add Scale labels to the machine pool

- Modify the default machine pool

ROSA provides a Command-Line Interface (CLI) which can be obtained from <https://github.com/openshift/rosa/releases> for editing node labels. This is needed to customize the machine pool that is used for running IBM Storage Scale container native. When you install ROSA, it creates a machine pool named "Default" that includes nodes with roles **worker** and **worker,infra**.

To use these nodes for running IBM Storage Scale container native, you can modify the "Default" machine pool as follows:

```
rosa edit machinepool --cluster=<rosa-cluster-name> --replicas=3 --labels='node-role.kubernetes.io/scale=' , 'scale.spectrum.ibm.com/daemon-selector=' worker
```

It is recommended to configure a minimum of 3 replicas.

- If you prefer to isolate your workload to a custom machine pool, you can create one and assign labels to the nodes by using the following command:

```
rosa create machinepool -c <rosa-cluster-name> --name=<custom-pool-name> --labels node-role.kubernetes.io/scale=,scale.spectrum.ibm.com/daemon-selector= --replicas 3
```

It is recommended to configure a minimum of 3 replicas.

Create a service account for applying Machine Config Operator (MCO)

As a regular user, you are prevented from modifying Red Hat managed resources. Therefore, it is advised to create a service account for Machine Config Operator (MCO) by following the provided steps:

1. Create a service account for Machine Config Operator (MCO):

```
kubectl create serviceaccount scale-mco-sa
```

2. Add a role to service account. To add a "cluster-admin" role to a service account "scale-mco-sa", execute the following command:

```
oc adm policy add-cluster-role-to-user cluster-admin -z scale-mco-sa
```

3. To create a new token for the service account (scale-mco-sa) and store it in the MCO_SA_TOKEN environment variable, execute the following command:

```
export MCO_SA_TOKEN=$(kubectl create token scale-mco-sa)
```

4. Log in to OpenShift using the new token.

```
oc login --token=$MCO_SA_TOKEN
```

For information on how to use service account in applications, see [Using service accounts in applications](#) in the Red Hat OpenShift documentation.

Create a new Machine Config Pool

To separate out nodes with labels "scale" and "worker" into a new machine pool, execute the following command:

```
echo '
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: scale-mcp
spec:
  machineConfigSelector:
    matchExpressions:
      - {key: machineconfiguration.openshift.io/role, operator: In, values: [worker,scale]}
  nodeSelector:
    matchLabels:
      node-role.kubernetes.io/scale: ""
' | kubectl apply -f -
```

Applying Machine Config Operator (MCO) settings

Apply the following MCO setting to install the kernel related packages for IBM Storage Scale to successfully build its portability layer.

```
echo '
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: "scale"
  name: 00-worker-ibm-spectrum-scale-kernel-devel
spec:
  selector:
    matchLabels:
      machineconfiguration.openshift.io/role: "scale"
  config:
    ignition:
      version: 3.2.0
    extensions:
      - kernel-devel
' | kubectl apply -f -
```

To validate the kernel-devel packages have been installed onto the machines, see [Validate kernel packages](#).

Cleanup the service account

Once the Machine Config Operator task has been successfully completed, complete the following steps to clean up the service account:

1. Log out of the OpenShift cluster:

```
oc logout
```

1. Log in to the OpenShift cluster as a regular user:

```
oc login <OpenShift_URL> -u <regular_user> -p <regular_user_password>
```

2. Delete the service account that was created for performing MCO task:

```
kubectl delete sa scale-mco-sa
```

Authorize ROSA worker security group to allow IBM Storage Scale container native ports

Complete the following steps to authorize ROSA worker security group:

1. In your AWS Management Console, navigate to the "EC2 Dashboard" in the region where ROSA is installed and select the "Security Groups" option from the navigation pane. Locate the Security Group for ROSA worker security group id and export the following variable.

```
export AWS_ROSA_WORKER_SECURITY_GROUP=<security_group_id>
```

2. Use the following commands to Authorize Ingress Traffic to allow ports needed by IBM Storage Scale container native.

```
aws ec2 authorize-security-group-ingress --group-id ${AWS_ROSA_WORKER_SECURITY_GROUP} --protocol tcp --port 12345 --source-group ${AWS_ROSA_WORKER_SECURITY_GROUP}
aws ec2 authorize-security-group-ingress --group-id ${AWS_ROSA_WORKER_SECURITY_GROUP} --protocol tcp --port 1191 --source-group ${AWS_ROSA_WORKER_SECURITY_GROUP}
aws ec2 authorize-security-group-ingress --group-id ${AWS_ROSA_WORKER_SECURITY_GROUP} --protocol tcp --port 60000-61000 --source-group ${AWS_ROSA_WORKER_SECURITY_GROUP}
```

Validating Red Hat OpenShift configuration

Red Hat OpenShift machine configuration changes are handled by the Machine Config Operator (MCO). To see the status of any machine configuration ongoing, run the following command:

```
oc get MachineConfigPool
```

Validate kernel packages

Validate that the **kernel-devel** package is installed on the nodes that deploy core pods.

In the following example, the label **scale.spectrum.ibm.com/daemon-selector=** is used to select the nodes that run core pods. Replace this selector to match what is configured in your cluster.

```
oc get nodes -lscale.spectrum.ibm.com/daemon-selector= \
-ojsonpath="{range .items[*]}{.metadata.name}{'\n'}" | \
xargs -I{} oc debug node/{} -T -- chroot /host sh -c "rpm -q kernel-devel"
```

This command creates a debug pod for all nodes that match the label selector. Use it with discretion if you have a large system.

s390x specific validation

Perform the extra validation steps if deployed on the s390x architecture.

Validate that the **vmalloc** kernel parameter is applied on the Red Hat OpenShift Container Platform worker nodes by entering the following command:

```
oc get nodes -lscale.spectrum.ibm.com/daemon-selector= \
-ojsonpath="{range .items[*]}{.metadata.name}{'\n'}" | \
xargs -I{} oc debug node/{} -T -- cat /proc/cmdline
```

In the following example, the value **vmalloc=4096G** is seen in the output at the end:

```
# oc debug node/worker1.example.com -- cat /proc/cmdline
Starting pod/worker1examplecom-debug ...
To use host binaries, run `chroot /host`
rhcos.root=crypt_rootfs random.trust_cpu=on ignition.platform.id=metal rd.luks.options=discard
$ignition_firstboot
ostree=/ostree/boot.1/rhcos/51e4c768b7c3dcec3bb63b01b9de9e8741486bf00dd4ae4df2d1ff1f872efe2e/0 vmalloc=4096G
```

This command creates a debug pod for all nodes that match the label selector. Use it with discretion if you have a large system.

Validate secure boot

Validate that the **secure boot** is disabled on the nodes that deploy core pods.

In the following example, the label **scale.spectrum.ibm.com/daemon-selector=** is used to select the nodes that run core pods. Replace this selector to match what is configured in your cluster.

```
oc get nodes -lscale.spectrum.ibm.com/daemon-selector= \
-ojsonpath="{range .items[*]}{.metadata.name}{'\n'}" | \
```

```
xargs -I{} oc debug node/{} -T -- chroot /host sh -c "mokutil --sb-state"
```

If the **SecureBoot enabled** is seen in the output, disable the **secure boot** with the procedure according to your infrastructure type.

This command creates a debug pod for all nodes that match the label selector. Use it with discretion if you have a large system.

Kubernetes

Refer to the following sections to help prepare your Kubernetes environments.

- [Software requirements](#)
- [Configuration](#)
- [Migration from IBM Storage Scale CSI](#)

Software requirements

Use the following table to determine the software requirement levels for Kubernetes for each release.

Table 1. Software Requirements for Kubernetes

| IBM Storage Scale Container Native | Architecture | Kubernetes level | Red Hat Enterprise Linux (RHEL) OS level | Ubuntu OS level |
|------------------------------------|-----------------|------------------------------|--|-----------------|
| 5.2.3.9 | x86_64 | 1.29, 1.30, 1.31, 1.32, 1.33 | 8.10, 9.4, 9.5, 9.6, 9.7, 9.8 | 22.04, 24.04 |
| 6.0.0.0 | x86_64, aarch64 | 1.30, 1.31, 1.32, 1.33 | 8.10, 9.4, 9.5, 9.6 | 22.04, 24.04 |
| 6.0.0.1 | x86_64, aarch64 | 1.30, 1.31, 1.32, 1.33 | 8.10, 9.4, 9.5, 9.6, 9.7 | 22.04, 24.04 |
| 6.0.0.2 | x86_64, aarch64 | 1.30, 1.31, 1.32, 1.33, 1.34 | 8.10, 9.4, 9.5, 9.6, 9.7 | 22.04, 24.04 |

Configuration

Refer to the following sections to help prepare your Kubernetes environment.

Cert Manager

[cert-manager](#) is used on Kubernetes to replace the **Certificate management** functions provided by Red Hat OpenShift Container Platform.

Install cert manager:

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.17.4/cert-manager.yaml
```

IBM Storage Scale CSI Snapshotter

The IBM Storage Scale CSI snapshot feature depends on a volume snapshot controller and the custom resource definitions (CRDs) for volume snapshot. Both the volume snapshot controller and the CRDs are independent of any CSI driver. If your Kubernetes distribution does not bundle the snapshot controller, you can manually install these components.

For more information, see [Performing preinstallation tasks](#) in IBM Storage Scale CSI documentation.

Vanilla Kubernetes

In addition, vanilla Kubernetes requires the following configurations.

Red Hat configurations

The following are required when using Red Hat servers.

Kernel packages

To compile the IBM Storage Scale kernel portability layer successfully, kernel development packages and kernel headers need to be installed onto the nodes that will run a core pod. The kernel packages must match the running kernel (`uname -r`).

```
dnf install -y kernel-devel
```

Ubuntu configurations

The following are required when using Ubuntu servers.

kernel headers

To compile the IBM Storage Scale kernel portability layer successfully, kernel development packages and kernel headers need to be installed onto the nodes that will run a core pod. The kernel packages must match the running kernel (`uname -r`).

```
apt-get install -y linux-generic
```

build-essentials

`build-essential` package needs to be installed on each Ubuntu server.

```
apt-get install -y build-essential
```

Migration from IBM Storage Scale CSI

An existing [IBM Storage Scale Container Storage Interface Driver](#) cluster can be migrated into a new IBM Storage Scale container native cluster. The native installation of IBM Storage Scale CSI has to be removed from the K8s nodes and all persistent volumes and volume claims are transferred (using the migration script) into IBM Storage Scale container native. This is not an online migration and production downtime is required.

The following steps help guide you in the migration to IBM Storage Scale container native.

1. Verify IBM Storage Scale CSI Status.

- Ensure the IBM Storage Scale CSI driver is up and running.
- Check that all CSI-related pods are running in the IBM Storage Scale CSI namespace:

```
kubectl get pods -n ibm-spectrum-scale-csi-driver
```

2. Delete Workload Pods.

- Delete/remove all **application pods or workload pods** that are using PVCs/PVs created via the IBM Storage Scale CSI.
- Repeat this step in every namespace where such application pods exist.

Do **not** delete associated PVCs or PVs.

3. Remove IBM Storage Scale CSI and Local IBM Storage Scale.

- Remove the **IBM Storage Scale CSI**, see [Cleaning up IBM Storage Scale Container Storage Interface driver and Operator by using CLIs](#).
- Remove the **Local IBM Storage Scale**, see [Uninstalling IBM Storage Scale](#)

Do **not** remove any **Remote IBM Storage Scale**.

4. Install IBM Storage Scale container native.

- Deploy IBM Storage Scale container native [Installing the IBM Storage Scale container native operator and cluster](#).

5. Execute migration script.

- Refer the migration guide to get the details of the migration script, see [PV and PVC Migration Script – Standalone CSI to CNSA](#) for details.{: external}
- Use the command to download a copy of the migration script:

```
curl -L -O https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-csi/refs/heads/release-3.0/tools/volume_migration_scripts/migration_csi_to_cnsa.bash || echo "Failed to download migration script"
```

- Use the following command to start migration:

```
./migration_csi_to_cnsa.bash --new_path_prefix </var/mnt>
```

- Validate that the migration is successful. The migration summary(shown once the migration script finishes) should have only successful or skipped PVs. There shouldn't be any Failed PVs in the summary.

| | |
|---|--|
| Migration Summary: | |
| ----- | |
| Successful PVs: 1 | |
| PVC Name | PV Name |
| ----- | |
| ibm-spectrum-scale-pvc-clone-from-pvc-advanced
08b1-4a53-ae4d-6740e2ec9a89 | pvc-7981775f-
08b1-4a53-ae4d-6740e2ec9a89 |
| Skipped PVs (already migrated): 2 | |
| PVC Name | PV Name |
| ----- | |
| scale-advance-pvc
382c-4487-a215-91dd22a01d45 | pvc-dd8e015c-
382c-4487-a215-91dd22a01d45 |
| ibm-spectrum-scale-pvc-advance-from-snapshot
7a25-4017-9c8d-6469750a4772 | pvc-e920921e-
7a25-4017-9c8d-6469750a4772 |

6. Recreate Workload Pods.

- Once migration completes, all PVs and PVCs will be ready.
- Re-deploy application/workload pods, which will seamlessly attach to the migrated PVCs.

Google Kubernetes Engine (GKE)

IBM Storage Scale container native deployed in a Google Kubernetes Engine (GKE) is available as a technology preview. For more information, see [Support for Google Kubernetes Engine \(GKE\)](#).

Refer to the following sections to help prepare your Google Kubernetes Engine (GKE) environments.

- [Software requirements](#)
- [Configuration](#)

Software requirements

IBM Storage Scale container native deployed in a Google Kubernetes Engine (GKE) is available as a technology preview. For more information, see [Support for Google Kubernetes Engine \(GKE\)](#).

Use the following table to determine the software requirement levels for Google Kubernetes Engine (GKE) for each release.

Table 1. Software Requirements for Google Kubernetes Engine (GKE)

| IBM Storage Scale Container Native | Architecture | Kubernetes level | Google Kubernetes Engine (GKE) level | Kernel levels | Container-Optimized OS milestones |
|------------------------------------|------------------------------|------------------------|--------------------------------------|--|-----------------------------------|
| 5.2.3.9 | x86_64 | 1.29, 1.30, 1.31 | 1.30, 1.31 | 6.1.77, 6.1.90, 6.1.100, 6.1.112, 6.1.123, 6.6.72 | 113, 117 |
| 6.0.0.0 | x86_64, aarch64 ¹ | 1.31, 1.32, 1.33 | 1.31, 1.32, 1.33 | 6.1.77, 6.1.90, 6.1.100, 6.1.112, 6.1.123, 6.6.72 | 113, 117, 121 |
| 6.0.0.1 | x86_64, aarch64[^tablet ext] | 1.31, 1.32, 1.33 | 1.31, 1.32, 1.33 | 6.1.90, 6.1.114, 6.1.123, 6.1.144, 6.6.72, 6.6.97 | 113, 117, 121 |
| 6.0.0.2 | x86_64, aarch64[^tablet ext] | 1.31, 1.32, 1.33, 1.34 | 1.31, 1.32, 1.33, 1.34 | 6.1.90, 6.1.114, 6.1.123, 6.1.144, 6.1.155, 6.6.72, 6.6.74, 6.6.97, 6.6.122, 6.6.123 | 113, 117, 121 |

For more information about Container-Optimized OS milestones, see [Long Term Supported \(LTS\) milestones](#) in the Container-Optimized OS documentation.

1. IBM Storage Scale container native only supports Ubuntu on Google Kubernetes Engine (GKE) using the aarch64 architecture.²

Configuration

IBM Storage Scale container native deployed in a Google Kubernetes Engine (GKE) is available as a technology preview. For more information, see [Support for Google Kubernetes Engine \(GKE\)](#).

Refer to the following sections to help prepare your Google Kubernetes Engine (GKE) environment.

Cert Manager

Install a cert manager as described in [Cert Manager](#).

IBM Storage Scale CSI Snapshotter

Consider description in [IBM Storage Scale CSI Snapshotter](#).

Google Kubernetes Engine (GKE) node pools

By default IBM Storage Scale container native is deployed on all nodes of the GKE cluster. IBM Storage Scale container native can also be deployed on a subset of the GKE node pools. Use the [Node selector expressions](#) in the [Cluster](#) custom resource to select the node pools where to deploy IBM Storage Scale container native.

For example, use the following configuration to deploy on node pools `default-pool` and `second-pool`:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
...
spec:
  daemon:
    nodeSelectorExpressions:
      - key: cloud.google.com/gke-nodepool
        operator: In
        values:
          - default-pool
          - second-pool
```

Configure pmcollector volumes for Performance Data

By default, the pmcollector pods of IBM Storage Scale container native uses local persistent volumes to store performance data. In Google Kubernetes Engine (GKE), a storage class must be specified to create Persistent Volumes for pmcollector. For more information, see [Pmcollector Volumes for Performance Data](#).

Prepare for Google Kubernetes Engine (GKE) upgrade

To support [Upgrading Google Kubernetes Engine \(GKE\)](#), IBM Storage Scale container native must be deployed on nodes of at least two [Google Kubernetes Engine \(GKE\) node pools](#). Each GKE node pool must have at least 3 nodes.

Use the [Surge upgrades](#) strategy for GKE node pool upgrades. The [Blue-green upgrades](#) strategy is not supported on GKE node pools that have IBM Storage Scale container native deployed.

The IBM Storage Scale container native configuration must be adjusted based on the configuration of the GKE surge upgrade. For more information, see [Configuration for Google Kubernetes Engine \(GKE\)](#).

Installing the IBM Storage Scale container native operator and cluster

The installation of the IBM Storage Scale container native operator and cluster includes several procedures.

- [Labels and annotations](#)
- [Install](#)
- [Container Images](#)
- [Building the kernel portability layer](#)
- [Kubernetes resources](#)
 - [Cluster](#)
 - [Callhome](#)
 - [RemoteCluster](#)
 - [Creating secrets for storage cluster GUI users](#)
 - [Configuring Certificate Authority \(CA\) certificates](#)
 - [Filesystems](#)
 - [Encryption](#)

Labels and annotations

IBM Storage Scale container native assigns labels to worker nodes.

Designation labels

IBM Storage Scale container native automatically assigns designations to some worker nodes. You do not need to explicitly designate the worker nodes but if it is required then it can be done by using node labels.

The following mechanisms are supported to designate IBM Storage Scale container native nodes:

- [Automatic](#) (*Recommended*) - Allows the Operator to designate the nodes automatically.
- [Manual](#) (*Optional*) - Allows administrators to have more control of the placement of IBM Storage Scale node designations (like the quorum designation) to pods on specific worker nodes.

Manual labeling requires insight about IBM Storage Scale and must not be used by inexperienced administrators.

Automatic

If the user does not label any nodes as quorum nodes, the Operator automatically applies quorum annotations to a subset of the nodes in the cluster. The number of nodes to be annotated depends on the number of nodes in the cluster:

- If the number of nodes in the cluster definition is less than 4, all nodes are designated as quorum nodes.
- If the number of nodes in the cluster definition is between 4 and 9 inclusive, 3 nodes are designated as quorum nodes.
- If the number of nodes in the cluster definition is between 10 and 18 inclusive, 5 nodes are designated as quorum nodes.
- If the number of nodes in the cluster definition is greater than 18, 7 nodes are designated as quorum nodes.

Kubernetes zones are considered whether they are configured in the Red Hat OpenShift cluster. The operator selects quorum nodes across all zones. For example, if 3 zones and 3 quorum nodes are to be designated as quorum nodes, then one node of each zone is designated as quorum node. For more information, see [Kubernetes zones](#).

The automatic node designation works at initial cluster creation time only. When the cluster is created, the operator does not change node designations automatically (for example if nodes are added to the cluster). If node designations must be changed on an existing cluster, the [Manual](#) steps can be done, even if [Automatic](#) mode was used at cluster creation time.

Manual

Supported designation label values are **quorum** and **manager**. The nodes designated as **quorum** nodes also automatically assume the role of **manager**. If sufficient **quorum** nodes are designated, unlabeled nodes become client nodes within the cluster.

Quorum node designations can be changed on the IBM Storage Scale container native cluster by manually applying or removing node labels. Manual labeling requires insight about IBM Storage Scale and must not be done by inexperienced administrators.

IBM Storage Scale quorum designation

For more information about IBM Storage Scale quorum designation, see [Quorum](#) in IBM Storage Scale documentation. It is mandatory to configure an odd number of nodes, with 3, 5, or 7 nodes are the typical numbers used.

IBM Storage Scale manager designation

For more information about IBM Storage Scale manager designation, see [Special management functions](#) in IBM Storage Scale documentation.

Node labeling

To see the list of nodes in your cluster, enter the `kubectl get nodes` command:

```
# kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
master0.example.com                Ready    master   50d    v1.30.6
worker0.example.com                Ready    worker   50d    v1.30.6
worker1.example.com                Ready    worker   50d    v1.30.6
worker2.example.com                Ready    worker   50d    v1.30.6
```

The following labels can be applied to nodes in the Red Hat OpenShift cluster to dictate how the pods are deployed on the nodes that are designated:

```
scale.spectrum.ibm.com/designation=quorum
scale.spectrum.ibm.com/designation=manager
```

To apply a label to a node, enter the `kubectl label node <node name> scale.spectrum.ibm.com/designation=<designation>` command as follows:

```
kubectl label node worker0.example.com scale.spectrum.ibm.com/designation=quorum
```

To verify that the label was applied to the node, enter the `kubectl describe node <node name>` command as follows:

```
# kubectl describe node worker0.example.com
Name:                worker0.example.com
...
Labels:              ...
                    ...
                    scale.spectrum.ibm.com/designation=quorum
...
```

To remove a label from a node, enter the following command:

```
kubectl label node <node name> scale.spectrum.ibm.com/designation-
```

Install

If you are looking to upgrade an existing IBM Storage Scale container native cluster, see [Upgrading IBM Storage Scale container native](#).

The installation process for IBM Storage Scale container native begins with applying the `install.yaml` to create and define Kubernetes configuration across the following namespaces:

- `ibm-spectrum-scale-operator`
- `ibm-spectrum-scale-dns`
- `ibm-spectrum-scale-csi`
- `ibm-spectrum-scale`

Run the following command to install the operator into the target environment:

Red Hat OpenShift cluster

Ensure all prerequisite steps are completed before proceeding, see [Red Hat OpenShift Container Platform](#).

```
kubectl apply -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/install.yaml
```

Vanilla Kubernetes cluster

For a Google Kubernetes Engine (GKE) cluster, use the [Google Kubernetes Engine \(GKE\) cluster](#) section.

Ensure all prerequisite steps are completed before proceeding, see [Kubernetes](#).

```
kubectl apply -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/install-k8s.yaml
```

Google Kubernetes Engine (GKE) cluster

Ensure all prerequisite steps are completed before proceeding, see [Google Kubernetes Engine \(GKE\)](#).

IBM Storage Scale container native deployed in a Google Kubernetes Engine (GKE) is available as a technology preview. For more information, see [Support for Google Kubernetes Engine \(GKE\)](#).

```
kubectl apply -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/install-k8s.yaml
```

Verification

Validate that the following namespaces have been created by running the command:

```
kubectl get namespaces | grep ibm-spectrum-scale

$ kubectl get namespaces | grep ibm-spectrum-scale
ibm-spectrum-scale           Active    4s
ibm-spectrum-scale-csi       Active    27s
ibm-spectrum-scale-dns       Active    26s
ibm-spectrum-scale-operator  Active    26s
```

Validate that operator pod is running in the following namespace:

```
kubectl get pods -n ibm-spectrum-scale-operator
```

```
$ kubectl get pods -n ibm-spectrum-scale-operator
```

| NAME | READY | STATUS | RESTARTS | AGE |
|--|-------|---------|----------|-----|
| ibm-spectrum-scale-controller-manager-78df9cf866-jd89q | 1/1 | Running | 0 | 78s |

Container Images

Access to container images are necessary for the successful deployment of IBM Storage Scale container native. Container images are hosted on the IBM Cloud Container Registry, ICR.

IBM Cloud Container Registry Entitlement

Certain images, such as the operator image, can be pulled from ICR without the need for entitlement. Most other images require a valid credential. Obtain an entitlement key from [IBM container software library](#).

Image pull failures can occur due to an invalid entitlement key or a key belonging to an account that does not have a valid entitlement.

The credentials to IBM Cloud Container Registry use **cp** as the username and the entitlement key as the password.

Pull Secrets

In order to allow the Kubernetes deployments to pull entitled images to the container environment, docker pull secrets need to be created. The suggested method is to configure namespace image pull secrets over modifying the Red Hat OpenShift global pull secret.

Namespace Pull Secrets

Create the **ibm-entitlement-key** pull secret for each of the following namespaces:

- ibm-spectrum-scale
- ibm-spectrum-scale-operator
- ibm-spectrum-scale-dns
- ibm-spectrum-scale-csi
- Export the entitlement key obtained previously from [IBM container software library](#):

```
export ENTITLEMENT_KEY=<REPLACE WITH ICR ENTITLEMENT KEY>
```

- Use the following helper script to create a docker-registry secret for each namespace:

```
for namespace in ibm-spectrum-scale ibm-spectrum-scale-operator ibm-spectrum-scale-dns ibm-spectrum-scale-csi; do
  kubectl create secret docker-registry ibm-entitlement-key -n ${namespace} \
    --docker-server=cp.icr.io \
    --docker-username=cp \
    --docker-password=${ENTITLEMENT_KEY}
done
```

- Unset the export:

```
unset ENTITLEMENT_KEY
```

Global Pull Secrets (Red Hat Openshift ONLY)

Use namespace pull secret over Red Hat OpenShift global pull secret. With Kubernetes 1.28, missing image pull secrets that are defined in the service accounts trigger warnings on resource and pod events. For more information, see [Unable to retrieve some image pull secrets](#).

1. Create a base64 encoded string of the credentials used to access the image registry.

```
echo -n "cp:REPLACE_WITH_ENTITLEMENT_KEY" | base64 -w0
```

2. Create an **authority.json** to include the base64 encoded string of your credentials, the fixed username **cp**, and the entitlement key for the IBM Cloud Container Registry.

```
{
  "auth": "REPLACE_WITH_BASE64_ENCODED_KEY_FROM_PREVIOUS_STEP",
  "username": "cp",
  "password": "REPLACE_WITH_ENTITLEMENT_KEY"
}
```

3. Enter the following command to include the `authority.json` as a new authority in your `.dockerconfigjson` and store it as `temp_config.json`:

Using the IBM Cloud Container Registry as the authority, use `cp.icr.io` as the input key for the contents of `authority.json`.

```
oc get secret/pull-secret -n openshift-config -ojson | \
jq -r '.data[".dockerconfigjson"]' | \
base64 --decode | \
jq '.[]."cp.icr.io" += input' - authority.json > temp_config.json
```

This command requires `jq` version 1.5, or later.

- Verify that your authority credentials are appended at the end of the file:

```
# cat temp_config.json
{
  "auths": {
    ...
    ...
    ...
    "cp.icr.io": {
      "auth": "REPLACE_WITH_BASE64_ENCODED_KEY_FROM_PREVIOUS_STEP",
      "username": "cp",
      "password": "REPLACE_WITH_ENTITLEMENT_KEY"
    }
  }
}
```

4. Use the contents of the `temp_config.json` file, and apply the updated config to the Red Hat OpenShift cluster.

```
oc set data secret/pull-secret -n openshift-config --from-file=.dockerconfigjson=temp_config.json
```

To verify that your pull-secret is updated with your new authority, issue the following command and confirm that your authority is present:

```
oc get secret/pull-secret -n openshift-config -ojson | \
jq -r '.data[".dockerconfigjson"]' | \
base64 -d -
```

5. This update takes some time to roll out and image pulls to ICR will fail until the update is complete. Use the following command to check the progress:

```
oc get mcp
```

6. Enter the following command to remove the temporary files that were created:

```
rm authority.json temp_config.json
```

Building the kernel portability layer

The IBM Storage Scale kernel portability layer needs to be built against the Linux kernel on the OS. For custom environments, a script can be defined in the configmap `buildgpl` to be used to compile IBM Storage Scale kernel modules. Its execution occurs in the `mmbuildgpl` init container on the core pods.

- [Red Hat OpenShift Container Platform](#)
- [Kubernetes](#)
- [Google Kubernetes Engine \(GKE\)](#)

Red Hat OpenShift Container Platform

Red Hat OpenShift Container Platform does not need further action for `mmbuildgpl`. The previous section [Red Hat OpenShift Container Platform](#) discusses applying the machine config operator config that installs the `kernel-devel` package onto the CoreOS operating system that allows the `mmbuildgpl` command to run successfully in the core pod init containers.

However, if your environment is using the NVIDIA network operator to deploy MOFED pods, use the following custom `buildgpl`:

```
kubect1 create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/openshift/buildgpl.nvidia.yaml
```

Kubernetes

Red Hat

A custom **buildgpl** configmap needs to be created on Kubernetes environments on top of Red Hat servers. Run the following command to create the configmap:

```
kubect1 create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/redhat/buildgpl.yaml
```

However, if your environment is using the NVIDIA network operator to deploy MOFED pods, use the following custom **buildgpl**:

```
kubect1 create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/redhat/buildgpl.nvidia.yaml
```

Ubuntu

A custom **buildgpl** configmap needs to be created on Kubernetes environments on top of Ubuntu servers. Run the following command to create the configmap:

```
kubect1 create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/ubuntu/buildgpl.yaml
```

However, if your environment is using the NVIDIA network operator to deploy MOFED pods, use the following custom **buildgpl**:

```
kubect1 create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/ubuntu/buildgpl.nvidia.yaml
```

Google Kubernetes Engine (GKE)

IBM Storage Scale container native deployed in a Google Kubernetes Engine (GKE) is available as a technology preview. For more information, see [Support for Google Kubernetes Engine \(GKE\)](#).

A custom **buildgpl** configmap needs to be created on GKE environments. Run the following command to create the configmap:

```
kubect1 create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/gke/buildgpl.yaml
```

However, if your environment is using the NVIDIA network operator to deploy MOFED pods, use the following custom **buildgpl**:

```
kubect1 create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/gke/buildgpl.nvidia.yaml
```

Kubernetes resources

The following sections describe Kubernetes resources that need to be defined to the Red Hat OpenShift cluster to drive features of the IBM Storage Scale container native cluster.

The following table lists the Custom Resource Definitions (CRDs) managed by the operator:

Table 1. IBM Storage Scale container native cluster custom resources

| Resource | Short name | Description |
|-----------------------------|-------------------|--|
| cluster | gpfs | Set attributes for the IBM Storage Scale container native cluster. |
| callhome | none | Configures IBM Storage Scale callhome. |
| remoteclusters | remotegpfs | Provide configuration details to the IBM Storage Scale remote cluster. For more information, see Remote file system section. |
| filesystem | fs | Configure the file systems for the container native cluster. |
| localdisk | ld | Configure the disks or volumes to be used as storage for local file systems. |
| encryptionconfig | ec | Allows users to configure encryption. |
| cachevolumeoperation | cvo | Allows users to trigger download or eviction for cache volumes. |

Use the following sections to guide you to create the custom resources:

- [Cluster](#)
- [Callhome](#)
- [RemoteCluster](#)
 - [Creating secrets for storage cluster GUI users](#)
 - [Configuring Certificate Authority \(CA\) certificates](#)
- [File Systems](#)
 - [Remote file system](#)
 - [Local file system](#)
- [Encryption](#)
- [CacheVolumeOperation](#)

Cluster

A cluster definition is needed to declare the properties of the IBM Storage Scale container native cluster. The following steps describe creating a **Cluster** custom resource.

Download cluster CR sample

Download a sample **Cluster** custom resource (CR) hosted on the GitHub repository. Samples are provided as a starting point with some defaults already set in the configuration based on the target environment.

Use one of the following commands to download a copy of the sample.

Red Hat OpenShift Cluster CR

For a Red Hat OpenShift cluster:

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/cluster/cluster.yaml > cluster.yaml || echo "Failed to download Cluster sample CR"
```

Kubernetes Cluster CR

For a Kubernetes cluster:

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/cluster/cluster-k8s.yaml > cluster.yaml || echo "Failed to download Cluster sample CR"
```

Google Kubernetes Engine Cluster CR

IBM Storage Scale container native deployed in a Google Kubernetes Engine (GKE) is available as a technology preview. For more information, see [Support for Google Kubernetes Engine \(GKE\)](#).

For a Google Kubernetes Engine cluster:

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/cluster/cluster-gke.yaml > cluster.yaml || echo "Failed to download Cluster sample CR"
```

NVIDIA Networking CR

For a cluster where NVIDIA networking is being used, InfiniBand or RoCE, start with the **cluster.nvidia.yaml** file. This sample contains RDMA options that need to be configured before creating the resource.

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/cluster/cluster.nvidia.yaml > cluster.yaml || echo "Failed to download Cluster sample CR"
```

Apply the Cluster CR

1. Make other changes specific to your installation. For more information on the Cluster specification, see [Cluster spec](#).
2. Set the node labels to identify the nodes in which the IBM Storage Scale container native operator deploys a core pod to. The sample cluster CR defines the selector as **scale.spectrum.ibm.com/daemon-selector: ""**. If you want IBM Storage Scale core pods to run on the Red Hat OpenShift worker nodes, then apply the label to the Kubernetes worker group by using the following command:

```
kubectl label nodes -l node-role.kubernetes.io/worker= scale.spectrum.ibm.com/daemon-selector=
```

For Kubernetes clusters, the command would need a different group label or individual node names depending on the cluster labels for worker nodes. The following is an example of individual nodes when a common worker label is not set.

```
kubectl label nodes <node1> <node2> <node3>... scale.spectrum.ibm.com/daemon-selector=
```

3. Apply the cluster **yaml** by entering the following command:

```
kubectl apply -f cluster.yaml
```

If you apply the **Cluster** custom resource and do not see any IBM Storage Scale core pods, check the Events in the output of **kubectl describe cluster.scale** and **kubectl describe daemon -n ibm-spectrum-scale** for information about what might be causing the problem.

After the cluster resource is defined to Kubernetes, use **kubectl edit cluster ibm-spectrum-scale** modify properties of the resource.

Cluster spec

The following table describes the properties for the **Cluster** custom resource **spec**.

Table 1. Cluster property and description

| Property | Required | Default | Description |
|---------------------------------------|----------|---|---|
| license | Yes | Not accepted | The license section allows the user to accept the license and specify the edition of IBM Storage Scale. See the License section for more details. |
| license.accept | Yes | false | The license must be accepted. Read the license and change to "true" to accept. See License section for more details. |
| license.license | Yes | data-access | Specifies the IBM Storage Scale edition, "data-access" or "data-management". See the License section for more details. |
| daemon | Yes | N/A | daemon specifies the configuration of the IBM Storage Scale daemons. See the Daemon section for more details. |
| daemon.nodeSelector | No | scale.spectrum.ibm.com/daemon-selector | Allows user to set a nodeSelector to indicate which Red Hat OpenShift nodes to run IBM Storage Scale core pods. See the Daemon section for more details. |
| daemon.nodeSelectorExpressions | No | N/A | Can be used as alternative or in addition to daemon.nodeSelector to indicate which Kubernetes nodes to run IBM Storage Scale core pods. |
| grafanaBridge | No | Disabled | Specifies the configuration of Grafana bridge. See the Grafana bridge section for more details. |
| gui | No | N/A | Specifies extra configuration of the GUI. |
| pmcollector | No | N/A | Specifies extra configuration of the pmcollector. |
| networkPolicy | Yes | Enabled | The operator creates network policy rules in the IBM Storage Scale container native namespaces. The network policy is not necessary for a Kubernetes cluster. |
| update | No | N/A | Specifies the configuration of how the IBM Storage Scale container native core pods are updated. See the Update section for more details. |

License

The **license** section allows accepting and choosing the IBM Storage Scale edition to be deployed in the IBM Storage Scale container native cluster. Complete the following activities:

- Review the appropriate license documentation through the URL in the CR.
- Accept the license by specifying **true** in the **license.accept** field.
- Supply the edition used in the **license.license** field.

The sample CR defaults to **data-access** under the **license.license** field, indicating IBM Storage Scale Data Access Edition. If you need the IBM Storage Scale Data Management Edition, then change the value in **license.license** to **data-management**.

The **data-access** edition is sufficient if remote mounted file systems are used. To use local file systems, the **data-management** license is required.

Specifying an edition without proper entitlement results in image pull failures during deployment.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  license:
    accept: true
    license: data-access
```

Enter the **kubectl explain cluster.spec.license** command to view more details.

Demon

The daemon section in the cluster specification specifies configuration for the IBM Storage Scale core pods.

Cluster name override

If you have multiple container native clusters that are attempting to remote mount from a single IBM Storage Scale storage cluster, each client GPFS cluster must have a unique GPFS Cluster name. Depending on the configuration of your Red Hat OpenShift cluster, the default cluster might not be unique and cause failures to mount the file system.

The cluster name is created by taking **ibm-spectrum-scale** and adding it to the base domain of the Red Hat OpenShift cluster (**kubectl get dns cluster -ojson | jq -r '.spec.baseDomain'**). If this cluster name is not unique, use the **daemon.clusterNameOverride** field to override the generated name.

The `clusterNameOverride` field must be configured before cluster creation. This field cannot be changed after a cluster is created. Setting this field after a cluster is created requires a full cleanup and redeployment of the cluster.

If the following value in the `clusterNameOverride` field is used:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
...
spec:
  daemon:
    clusterNameOverride: ocp.c2.example.com
```

As a result, the container native cluster that is created has the following GPFS cluster name:

```
kubectl exec -n ibm-spectrum-scale \
$(kubectl get pods -lapp.kubernetes.io/name=core -n ibm-spectrum-scale -ojsonpath="
{.items[0].metadata.name}") \
-- mmlscluster
```

The output appears as shown:

```
mmlssh-4.4# mmlscluster

GPFS cluster information
=====
GPFS cluster name:      ocp.c2.example.com
...
...
```

Tolerations

The `spec.daemon.tolerations` section allows the user to configure extra tolerations to determine where IBM Storage Scale core pods can be scheduled. For more information on tolerations, see [Taints and Tolerations](#) in Kubernetes documentation.

The operator rolls out the configured Tolerations to the IBM Storage Scale core pods.

Enter the `kubectl explain cluster.spec.daemon.tolerations` command to view more details.

Node selectors

Use the `daemon.nodeSelector` section to configure a nodeSelector to determine where IBM Storage Scale pods can be deployed.

The Operator checks that a node has all defined labels present to deem a node eligible to deploy IBM Storage Scale pods. In the `Cluster` custom resource sample, the operator deploys IBM Storage Scale pods on nodes with the following label:

`scale.spectrum.ibm.com/daemon-selector:`

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  daemon:
    nodeSelector:
      scale.spectrum.ibm.com/daemon-selector: ""
```

Enter the `kubectl explain cluster.spec.daemon.nodeSelector` command to view more details.

If no node selector is specified, IBM Storage Scale pods are deployed on all cluster nodes.

Node selector expressions

The `daemon.nodeSelectorExpressions` can be used as alternative or in addition to `daemon.nodeSelector`. Enter the `kubectl explain cluster.spec.daemon.nodeSelectorExpressions` command to view more details.

In the following example, IBM Storage Scale pods are deployed on nodes with label `test=first` or `test=second`:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
...
spec:
  daemon:
    nodeSelectorExpressions:
    - key: test
      operator: In
      values:
      - first
      - second
```

Host aliases

It is recommended that proper DNS is configured in your environment.

The `daemon.hostAliases` section allows for user-defined entries to be added into the IBM Storage Scale CoreDNS service. The IBM Storage Scale CoreDNS service provides the name resolution for the IBM Storage Scale core pods.

If the core pods are unable to resolve the hostname of the servers in the storage cluster by DNS, their hostname and their IP addresses can be specified in the `hostAliases` as follows:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  daemon:
    hostAliases:
      - hostname: node1.example.com
        ip: 10.0.0.1
      - hostname: node2.example.com
        ip: 10.0.0.2
```

The IBM Storage Scale CoreDNS service handles name resolution for the core pods. For `RemoteCluster` CR, the hostname that is provided in the `remotecluster.spec.gui.host` field must be DNS resolvable and using host aliases is not a valid workaround.

Enter the `kubectl explain cluster.spec.daemon.hostAliases` command to view more details.

Cluster profile

The `daemon.clusterProfile` allows the user to set default IBM Storage Scale configuration parameters for the cluster at cluster creation time.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  daemon:
    ...
    clusterProfile:
      controlSetxattrImmutableSELinux: "yes"
      enforceFilesetQuotaOnRoot: "yes"
      ignorePrefetchLUNCount: "yes"
      initPrefetchBuffers: "128"
      maxblocksize: 16M
      prefetchPct: "25"
      prefetchTimeout: "30"
```

Changing the values in the `clusterProfile` is not supported and must be avoided unless advised by IBM Support. For the two exceptions where changing values in the `clusterProfile` is supported, see [Ephemeral port range](#) and [Ephemeral port ranges](#).

Enter the `kubectl explain cluster.spec.daemon.clusterProfile` command to view more details.

Ephemeral port range

If the storage cluster configures ephemeral port ranges, you need to set `tscCmdPortRange` on the container native cluster to match the range.

For example, if the storage cluster is configured to use port range of 60000-61000, set this value under the `clusterProfile` section in the `Cluster` custom resource.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  daemon:
    clusterProfile:
      ...
      tscCmdPortRange: "60000-61000"
```

Node Profiles

The `daemon.nodeProfiles` section allows user to set IBM Storage Scale configuration parameters on a group of nodes. There are a few configuration parameters, which can have different values on different nodes. The operator identifies node profile nodes that use the node label. If a parameter is specified in `daemon.nodeProfile` and also in [Cluster profile](#) or in [Roles](#), the value from nodeProfile is applied on selected nodes.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  daemon:
    clusterProfile:
      ...
```

```

...
nodeProfiles:
- name: dell147
  nodeSelector:
    matchLabels:
      machinetype: "dell14711"
  profile:
    verbsPorts: "mlx5_2"
- name: dell1188
  nodeSelector:
    matchLabels:
      machinetype: "dell18888"
  profile:
    verbsPorts: "mlx5_1"
    maxMBps: "4096"

```

The only supported parameters in nodeProfiles are **verbsPorts**, **maxMBps**, **numaMemoryInterleave**.

Roles

The **daemon.roles** section in the **Cluster** spec allows the user to fine-tune memory and CPU requests by using the **resources** object on the nodes that are part of specific IBM Storage Scale roles. For more information, see [Resource Management for Pods and Containers](#) in Kubernetes documentation.

client role

The default node role is **client** when the node is not labeled with an AFM role.

For the **client** role, the configuration recommendation for requests is 2 CPU and 4 GiB. On systems with many CPU cores, large memory, and/or high-speed network, the storage performance might increase with higher values. Therefore, encryption and compression of **PersistentVolumes** result in higher CPU load higher resource values can be beneficial. On smaller systems and/or applications with low I/O workloads, 1 CPU and 2 GiB can be set.

For the **AFM** role, the resource values are considerably higher and should be considered for proper performance. For more information, see [General recommendations for AFM gateway node configuration](#)

Low resource configurations might yield poor performance.

The resource properties of the IBM Storage Scale core pods are as follows:

- By default, role resources are set to 25% of the capacity of the nodes. The **daemon.roles.resources** parameter in cluster spec is identical to the **requests** parameter in the Kubernetes pod spec.
- By default, role limits are set to 50% of the capacity of the nodes.

All node resources must be homogeneous for the same role.

To set memory and CPU requests and limits for the **client** role, specify the request values under **daemon.roles.resources** and the limit values under **daemon.roles.limits** with **kubect1 edit cluster** command.

Limits must have at least the double value as the requests when specified. See the example (**kubect1 describe cluster**):

Changing these values on an existing cluster causes a rolling drain of nodes and pod restarts.

```

apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
...
  daemon:
    roles:
    - name: client
      resources:
        cpu: "2"
        memory: 4Gi
      limits:
        cpu: "4"
        memory: 8Gi

```

For s390x, the sample cluster custom resource ships with "4Gi" memory request. You must reduce the memory request to "2Gi" if the hardware does not have enough physical memory.

Enter the **kubect1 explain cluster.spec.daemon.roles** command to view more details.

Update

The **daemon.update** section specifies the configuration of how the IBM Storage Scale core pods are updated. By default the core pods are updated in a rolling node-by-node fashion. Therefore, in a cluster with many nodes the update takes a long time. To speed up, multiple core pods can be updated in parallel.

The `daemon.update.maxUnavailable` field allows update of multiple core pods in parallel. For example, `maxUnavailable: 3` causes that up to 3 nodes are cordoned and drained by the operator, and 3 core pods are updated in parallel.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  daemon:
    update:
      maxUnavailable: 3
```

Instead of a total number a percentage can be specified. For example, `maxUnavailable: 33%` causes that up to a third of all core pods are updated at the same time. In this case, all core pods are updated in at least 3 update cycles.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  daemon:
    update:
      maxUnavailable: 33%
```

The operator keeps the container native cluster operational during the update. At least 3 update cycles are needed because most quorum core pods must always be running to prevent downtime of the container native cluster. For example, with 7 quorum pods, first up to 3 quorum pods are updated. Then, the next 3 quorum pods are updated. Finally, the last quorum pod is updated. Therefore, it's typically not useful to specify `maxUnavailable: 100%` because the pods are still updated in three cycles: The first 3 quorum pods plus all nonquorum pods, then the next 3 quorum pods and the last quorum pod. Therefore, `maxUnavailable: 33%` is recommended as an optimal configuration when the update speed must be short.

In addition to quorum, other characteristics limit the number of core pods to be updated in parallel. As a result, it can happen that fewer pods than defined in `maxUnavailable` are updated in parallel.

The update can be stopped by specifying `paused: true`.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  daemon:
    update:
      maxUnavailable: 3
      paused: true
```

The `daemon.update.pools` section is used to define update pools. Use update pools to specify the update configuration for a subset of core pods. The core pods for the pool are selected with a node selector.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  daemon:
    update:
      maxUnavailable: 2
      pools:
        - name: fast
          nodeSelector:
            matchLabels:
              fast-update: ""
          maxUnavailable: 10
```

In the example, all core pods on nodes with label `fast-update: ""` are part of the `fast` update pool. Up to 10 core pods in this pool can be updated in parallel. For other pods (without node label `fast-update: ""`) the `maxUnavailable: 2` matters, which allows to update up to 2 pods in parallel. Each core pod must belong to maximum one update pool.

Using the `matchExpressions` is another way for selecting nodes for a pool.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  daemon:
    update:
      maxUnavailable: 1
      pools:
        - name: expressionSample
          nodeSelector:
            matchExpressions:
              - key: scale.spectrum.ibm.com/designation
                operator: DoesNotExist
              - key: topology.kubernetes.io/zone
```

```

operator: In
values:
- us-east1-a
- us-east1-b
maxUnavailable: 4

```

Only nodes that run an IBM Storage Scale pod are considered for update pools. Enter the `kubectl explain cluster.spec.daemon.update.pools.nodeSelector` command to view more details.

Check daemon status to verify whether the expected nodes are selected with the node selector. The nodes that do not belong to any update pool are listed with pool name `default`.

```

# kubectl describe daemon ibm-spectrum-scale -n ibm-spectrum-scale
...
Status:
...
Update:
  Pools:
    Name:      default
    Node Count: 3
    Nodes:     worker0.example.com, worker1.example.com, worker2.example.com
    Name:      example-pool
    Node Count: 5
    Nodes:     worker3.example.com, worker4.example.com, worker5.example.com, worker6.example.com,
worker7.example.com

```

Update pools with prefix `mcp/` in their name reference to an Red Hat OpenShift MachineConfigPool. In this case, the parameters `maxUnavailable`, `nodeSelector`, and `paused` are copied from the Red Hat OpenShift MachineConfigPool and the update pool configuration prevents these parameters.

```

apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
...
  daemon:
    update:
      pools:
        - name: "mcp/worker"

apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: worker
spec:
...
  maxUnavailable: 1
  nodeSelector:
    matchLabels:
      node-role.kubernetes.io/worker: ""
  paused: false

```

Parallel Update recommendations

The `maxUnavailable` parameter allows to enable parallel update. For more information, see [Update](#). When parallel update is enabled, it makes sense to reduce the `maxUnavailable` number for a subset of the nodes to increase the availability.

Quorum

By default, the operator updates as many quorum pods in parallel as possible if parallel update is enabled. With 5 quorum pods, then 2 quorum pods are updated in parallel and the remaining 3 quorum pods keep running. This causes a container native cluster downtime if one more quorum pod is going down unexpectedly during an update. If availability of the container native cluster is more important than update speed, create the following update pool for quorum pods that allows to update one quorum pod at any time.

```

apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
...
  daemon:
    update:
      maxUnavailable: 5
      pools:
        - name: quorum
          nodeSelector:
            matchLabels:
              scale.spectrum.ibm.com/designation: quorum
          maxUnavailable: 1

```

Configuration for Google Kubernetes Engine (GKE)

IBM Storage Scale container native deployed in a Google Kubernetes Engine (GKE) is available as a technology preview. For more information, see [Support for Google Kubernetes Engine \(GKE\)](#).

The Google Kubernetes Engine (GKE) cluster can be spread across multiple node pools. The node pools can be selected with [Node selector expressions](#) in the Cluster custom resource. The following example selects the nodes of the **default-pool** GKE node pool and the **second-pool** GKE node pool for running Scale pods:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
...
spec:
  daemon:
    nodeSelectorExpressions:
      - key: cloud.google.com/gke-nodepool
        operator: In
        values:
          - default-pool
          - second-pool
```

To support [Upgrading Google Kubernetes Engine \(GKE\)](#), IBM Storage Scale container native must be deployed on at least two GKE node pools.

If IBM Storage Scale container native is deployed in Google Kubernetes Engine (GKE), the surge upgrade strategy is used for GKE node pool upgrades. For more information, see [Surge upgrades](#). For each node pool, in GKE a corresponding update pool must be created. The sum of the **maxUnavailable** and the **maxSurge** parameters for the surge configuration of a GKE node pool should match to the **maxUnavailable** parameter of the update pool.

For example, in the following GKE surge upgrade configuration

- GKE node pool **default-pool** has **maxUnavailable=3** and **maxSurge=2**
- GKE node pool **second-pool** has **maxUnavailable=2**

the corresponding update pool configuration looks like

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  daemon:
    update:
      pools:
        - name: default-pool
          nodeSelector:
            matchLabels:
              cloud.google.com/gke-nodepool: default-pool
          maxUnavailable: 5
        - name: second-pool
          nodeSelector:
            matchLabels:
              cloud.google.com/gke-nodepool: second-pool
          maxUnavailable: 2
```

Local file system

Local file systems use disks that have a shared connection to many nodes. Even though the operator avoids that all connection nodes of a disk are updated in parallel, it makes sense to reduce the number of disk connection nodes. If availability of the local file systems and the I/O performance is more important than update speed, create an update pool for connection nodes. The following update pool causes that connection nodes are updated one at a time, if the shared disk connection nodes are labeled with **connection=true**.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  daemon:
    update:
      maxUnavailable: 5
      pools:
        - name: diskConnection
          nodeSelector:
            matchLabels:
              connection: true
          maxUnavailable: 1
```

For more information, see [Local file system](#).

Grafana bridge

The **grafanaBridge** section allows the user to enable the deployment of the IBM Storage Scale bridge for Grafana application. This tool allows the IBM Storage Scale container native users to perform performance monitoring for IBM Storage Scale devices, by using third-party applications

such as Grafana or Prometheus software. For more information, see [IBM Storage Scale bridge for Grafana](#).

Specify `grafanaBridge: {}` to enable Grafana Bridge:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  grafanaBridge: {}
```

Grafana bridge is optional

Enter the `kubectl explain cluster.spec.grafanabridge` command to view more details.

Prometheus

You can expose the IBM Storage Scale performance metrics to a Prometheus instance that can be configured by Red Hat OpenShift to monitor the project. Using this allows monitoring of both Red Hat OpenShift components and IBM Storage Scale container native resources in a single place. For an end-to-end guide on this configuration, see [Setup Red Hat OpenShift Monitoring Stack with IBM Storage Scale container native](#).

Set `enablePrometheusExporter` property to `true`

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  grafanaBridge:
    enablePrometheusExporter: true
```

Pmcollector Volumes for Performance Data

The `pmcollector` requires persistent volumes for storing performance data. Depending on setup, there are two modes for configuring these volumes:

1. **Local Persistent Volumes:** When no storage class is defined in `pmcollector` section, the default storage class `ibm-spectrum-scale-internal` is used and local volumes are created.
2. **Specific Storage Class for Persistent Volumes:** If a storage class is specified in `pmcollector` section, Persistent Volume Claims (PVCs) are created from this storage class. This is commonly used in cloud environments, where storage management is handled by the cloud provider.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  pmcollector:
    storageClass: "your-storage-class"
```

Enter the `kubectl explain cluster.spec.pmcollector` command to view more details.

Modifying `storageClass` field deletes existing PVCs and create new ones with the updated storage class. The outcome is the loss of the historical performance data.

Use a storage class (second option above) when IBM Storage Scale container native is deployed in Google Kubernetes Engine (GKE).

CSI Configuration

The `csi` section allows the user to define the configuration of the Container Storage Interface (CSI).

By default, the CSI pods are scheduled on worker nodes. The `csi.sidecar.nodeSelector` can be used to schedule the IBM Storage Scale Container Storage Interface (CSI) sidecar pods (external-attacher, external-provisioner, external-snapshotter, and external-resizer) onto specific set of nodes.

The CSI sidecar `nodeSelector` section allows specifying a [Kubernetes Node Selector](#). The `nodeSelector` must select at least two nodes to provide high availability of CSI sidecar pods.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  csi:
    sidecar:
      nodeSelector:
        key: value
```

Enter the `kubectl explain cluster.spec.csi` command to view more details.

CSI configuration is optional

Infrastructure nodes

The GUI pods, pmcollector pods, and [Grafana bridge](#) pods can be placed onto Red Hat OpenShift infrastructure nodes. At least two infrastructure nodes are needed because two replicas of GUI and pmcollector pods must run on different infrastructure nodes.

GUI, pmcollector, and Grafana bridge pods

The `gui`, `pmcollector` and `grafanaBridge` sections allow to specify a [Kubernetes Node Selector](#) and [Kubernetes Taints and Tolerations](#).

If the Red Hat OpenShift infrastructure nodes are labeled with `node-role.kubernetes.io/infra=""` and have the `node-role.kubernetes.io/infra:NoSchedule` and `node-role.kubernetes.io/infra:NoExecute` taints, add the following lines to run the `gui`, `pmcollector` and `grafanaBridge` sections:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  gui:
    nodeSelector:
      node-role.kubernetes.io/infra: ""
    tolerations:
      - effect: NoSchedule
        key: node-role.kubernetes.io/infra
        operator: Exists
      - effect: NoExecute
        key: node-role.kubernetes.io/infra
        operator: Exists
  pmcollector:
    nodeSelector:
      node-role.kubernetes.io/infra: ""
    tolerations:
      - effect: NoSchedule
        key: node-role.kubernetes.io/infra
        operator: Exists
      - effect: NoExecute
        key: node-role.kubernetes.io/infra
        operator: Exists
  grafanaBridge:
    nodeSelector:
      node-role.kubernetes.io/infra: ""
    tolerations:
      - effect: NoSchedule
        key: node-role.kubernetes.io/infra
        operator: Exists
      - effect: NoExecute
        key: node-role.kubernetes.io/infra
        operator: Exists
```

CSI sidecar pods

The `csi` section allow to specify a [Kubernetes Node Selector](#).

If the Red Hat OpenShift infrastructure nodes are labeled with `node-role.kubernetes.io/infra=""`, add the following lines to run CSI sidecar pods on infrastructure nodes:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  csi:
    sidecar:
      nodeSelector:
        node-role.kubernetes.io/infra: ""
```

The CSI sidecar pods tolerate the `NoExecute` and `NoSchedule` taint. Therefore, the sidecar pods can run on infrastructure nodes even if these taints are present.

If parallel updates (`maxUnavailable > 1`) are enabled along with placing CSI sidecar pods on infrastructure nodes, create an update pool for nodes with `maxUnavailable = 1`. This ensures that only one core pod is updated at a time, therefore CSI is operational on all other nodes where the core pods are running.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
spec:
  ...
  daemon:
    update:
      pools:
        - name: infra
```

```
nodeSelector:
  matchLabels:
    node-role.kubernetes.io/infra: ""
maxUnavailable: 1
```

IBM Storage Scale core pods

To use infrastructure nodes, it is required that IBM Storage Scale core pods also run on the infrastructure nodes.

- Label all worker nodes and infrastructure nodes with a common label. The sample **Cluster** custom resource defines the node selector: `scale.spectrum.ibm.com/daemon-selector: ""`. To use this selector, apply the following labels to worker and infrastructure nodes:

```
kubect1 label nodes --selector node-role.kubernetes.io/worker scale.spectrum.ibm.com/daemon-selector=""
```

```
kubect1 label nodes --selector node-role.kubernetes.io/infra scale.spectrum.ibm.com/daemon-selector=""
```

The IBM Storage Scale core pods tolerate the **NoExecute** and **NoSchedule** taint. Therefore, the core pods run on infrastructure nodes even if these taints are present.

Network policies

The `cluster.spec.networkPolicy` object indicates whether the container native operator creates and manages the network policy rules within the IBM Storage Scale container native namespaces. The default sample cluster CR enables this option: `networkPolicy: {}`.

Modifying the network policies that are owned by the Cluster controller gets overwritten by the operator.

Enabling and disabling network policy management

With IBM Storage Scale container native v5.1.9 or later, the sample cluster custom resource enables network policies by default.

If network policies are not created by the operator, use the following command to patch the **Cluster** spec:

```
kubect1 patch cluster.scale.spectrum.ibm.com ibm-spectrum-scale --type='json' \
-p='[{"op": "add", "path": "/spec/networkPolicy", "value":{}}]'
```

If you find that the network policy rules are interfering with functions of the container native cluster, disable the option in the **Cluster** spec with the following command:

```
kubect1 patch cluster.scale.spectrum.ibm.com ibm-spectrum-scale --type='json' \
-p='[{"op": "remove", "path": "/spec/networkPolicy"}]'
```

Delete the network policies with the following commands:

```
kubect1 delete networkpolicy -n ibm-spectrum-scale-operator --all
kubect1 delete networkpolicy -n ibm-spectrum-scale --all
kubect1 delete networkpolicy -n ibm-spectrum-scale-dns --all
```

Allowing communication between trusted namespaces

To allow ingress and egress communication to and from a trusted namespace within your Red Hat OpenShift cluster, label the namespace with the label: `scale.spectrum.ibm.com/networkpolicy=allow`.

To enable communication, add the label to the namespace:

```
kubect1 label namespace <TARGET_NAMESPACE> scale.spectrum.ibm.com/networkpolicy=allow
```

To disable communication, remove the label on the namespace:

```
kubect1 label namespace <TARGET_NAMESPACE> scale.spectrum.ibm.com/networkpolicy-
```

Enabling Network Policy for IBM Storage Scale Container Storage Interface pods

The following section describes how to create network policies that help secure communication in the IBM Storage Scale Container Storage Interface (CSI) namespace.

- Create the deny-all policy using the following command.

```
cat << EOF | kubect1 apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
  namespace: ibm-spectrum-scale-csi
spec:
  podSelector: {}
  policyTypes:
```

```
- Ingress
- Egress
EOF
```

2. Create a policy to allow dns traffic using the following command.

```
cat << EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns-traffic
  namespace: ibm-spectrum-scale-csi
spec:
  policyTypes:
  - Egress
  - Ingress
  egress:
  - ports:
    - port: 5353
      protocol: UDP
    - port: 5353
      protocol: TCP
  ingress:
  - ports:
    - port: 5353
      protocol: UDP
    - port: 5353
      protocol: TCP
EOF
```

3. Create a policy to allow all egress communication from IBM Storage Scale Container Storage Interface namespace.

```
cat << EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-all-egress
  namespace: ibm-spectrum-scale-csi
spec:
  podSelector: {}
  egress:
  - {}
  policyTypes:
  - Egress
EOF
```

1. Create a policy using the following steps that allow egress communication to the kubernetes API endpoints.

1. Create a `allow-kubernetes-endpoints.yaml` file using the following command.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-kubernetes-endpoints
  namespace: ibm-spectrum-scale-csi
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress:
  - to:
    - ipBlock:
        cidr: 10.0.0.1/32
      ports:
      - protocol: TCP
        port: 6443
    - to:
    - ipBlock:
        cidr: 10.0.0.2/32
      ports:
      - protocol: TCP
        port: 6443
    - to:
    - ipBlock:
        cidr: 10.0.0.3/32
      ports:
      - protocol: TCP
        port: 6443
```

2. Use the following command to get a list of the kubernetes ENDPOINTS specific to your cluster:

```
kubectl get endpoints kubernetes -n default
```

3. Edit the `allow-kubernetes-endpoints.yaml` file and replace the `cidr` value with the endpoint IP's obtained from your cluster.
4. Apply the network policy using the following command.

```
kubectl apply -f allow-kubernetes-endpoints.yaml
```

2. Create a policy to allow secure communication with the gui pods.

```
cat << EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-egress-gui-pods
  namespace: ibm-spectrum-scale-csi
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress:
  - to:
    - podSelector:
        matchLabels:
          app.kubernetes.io/name: gui
        namespaceSelector:
          matchLabels:
            app.kubernetes.io/instance: ibm-spectrum-scale
EOF
```

3. Create a policy using the following steps that allow egress communication with remote GUI.

1. Create a file called `allow-remote-gui.yaml` with the following content.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-remote-gui
  namespace: ibm-spectrum-scale-csi
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress:
  - to:
    - ipBlock:
        cidr: 10.0.0.1/32
      ports:
      - protocol: TCP
        port: 443
```

2. Edit the `allow-remote-gui.yaml` file and replace `cidr` with remote storage cluster GUI IP.
3. Apply the network policy using the following command.

```
kubectl apply -f allow-remote-gui.yaml
```

Cluster status

Status **Conditions** can be viewed as a snapshot of the current and most up-to-date status of a **Cluster**.

- The **Success** condition is set to **True** if the **Cluster** is successfully configured.

Callhome

To enable call home functionality, create a **CallHome** custom resource to the kubernetes cluster. The following steps describe creating a **Callhome** CR.

1. Download a copy of the sample [callhome.yaml](#) from the GitHub repository.

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/callhome/callhome.yaml > callhome.yaml || echo "Failed to download Callhome sample CR"
```

2. Edit the `callhome.yaml` file and make changes specific to your installation. For details on how to fill out the sections of the Callhome specification, see [Callhome spec](#).
3. After you have made your changes, apply the callhome yaml using the following command:

View the Callhome resources using the following command:

Once deployed, use the command `kubectl edit callhome -n ibm-spectrum-scale` to modify properties of the resource.

For more information, see [Understanding call home](#) in IBM Storage Scale documentation.

The following table describe the properties for **Callhome**:

| Property | Required | Default | Description |
|-------------------------------|---|---------|--|
| <code>companyEmail</code> | Yes | None | The address of the system administrator who can be contacted by the IBM Support. Usually this e-mail address is directed towards a group or task e-mail address. For example, <code>itsupport@mycompanyname.com</code> . |
| <code>companyName</code> | Yes | None | The company to which the contact person belongs. This name can consist of any alphanumeric characters and these non-alphanumeric characters are '-', '_', ':', ' ', '.', |
| <code>countryCode</code> | Yes | None | The two-letter capital letter country codes as defined in ISO 3166-1 alpha-2. |
| <code>customerID</code> | Yes | None | The customer ID of the system administrator who can be contacted by the IBM Support. This can consist of any alphanumeric characters and these non-alphanumeric characters are '-', '_', ' ', |
| <code>license.accept</code> | Yes | None | License must be accepted by the end user to enable Callhome. |
| <code>proxy</code> | No | None | If specified, defines a proxy server configuration. |
| <code>proxy.host</code> | Yes, if <code>proxy</code> is specified | None | The host of proxy server as hostname or IP address. |
| <code>proxy.port</code> | Yes, if <code>proxy</code> is specified | None | The port of proxy server. |
| <code>proxy.secretName</code> | Yes, if <code>proxy</code> is specified | None | The secret name of a basic authentication secret, which contains username and password for proxy server. |

License agreement

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Callhome
...
spec:
  ...
  license:
    accept: true
```

Under the **spec** for **Callhome**, enter your **companyName**, the **customerID** that IBM provided to you, the **companyEmail** and the **countryCode**.

Type

Proxy (optional)

```
kubectl create secret generic proxyServerSecret --from-literal=username='<proxy_username>' \
--from-literal=password='<proxy_password>' -n ibm-spectrum-scale
```

Then add your configuration into the CR:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Callhome
...
spec:
  ...
  proxy:
    host: proxyserver.example.com
    port: 443
    secretName: proxyServerSecret
```

Enter the `kubectl explain callhome` command to view more details.

Callhome status

Status **Conditions** can be viewed as a snapshot of the current and most up-to-date status of **Callhome**.

- The **Enabled** condition is set to **True** if **Callhome** functionality is enabled by accepting the license.
- The **Success** condition is set to **True** if **Callhome** configured successfully and is able to communicate with the IBM Callhome server.

RemoteClusters

To allow the IBM Storage Scale container native cluster to access remote IBM Storage Scale storage clusters, a **RemoteCluster** custom resource (CR) must be defined for each storage cluster. For more information to prepare the storage cluster, see [Storage cluster](#).

The following steps help guide you in creating a **RemoteCluster** resource.

1. To help with fields in the **RemoteCluster** custom resource specification, see:
 - [Creating secrets for storage cluster GUI users](#)
 - [Configuring Certificate Authority \(CA\) certificates](#)
2. Download a copy of the sample [remoteclasser.yaml](#) from the GitHub repository.

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-
native/v6.0.0.x/generated/scale/cr/remoteclasser/remoteclasser.yaml > remoteclasser.yaml || echo
"Failed to download RemoteCluster sample CR"
```

The sample `remoteclasser.yaml` uses `remoteclasser-sample` as the resource name. You can change this name. Each resource must have a unique `metadata.name` when multiple remote cluster resources are created.

3. Edit the `remoteclasser.yaml` file and change the fields that are specific to your installation. For details on the Remote Cluster specification, see [RemoteCluster spec](#).
4. Apply the resource by using the following command:

```
kubectl apply -f remoteclasser.yaml
```

5. View the remote cluster resources by using the following command:

```
kubectl get remoteclasser -n ibm-spectrum-scale
```

After the resource is defined to Kubernetes, use `kubectl edit remoteclasser <remoteclasser-name> -n ibm-spectrum-scale` to modify the properties of the resource.

RemoteCluster spec

The following table describes the properties for **RemoteCluster**:

Table 1. Remoteclasser property and description

| Property | Required | Default | Description |
|----------------------------|----------|---------|---|
| <code>metadata.name</code> | Yes | None | The name of the CR. This field is used to identify the remote storage cluster in the Filesystem CR. |
| <code>contactNodes</code> | No | None | This property is optional and provides a list of nodes from the storage cluster to be used as the remote cluster contact nodes. The names must be the daemon node names. If not specified, the operator uses all quorum nodes that are detected from the storage cluster. If the contact node names are not resolvable in local cluster, configure the 'hostAliases' with node name and IP in Cluster CR. |
| <code>gui</code> | Yes | None | It specifies the details for the IBM Storage Scale Remote Cluster GUI. |

| Property | Required | Default | Description |
|--|--|------------------------------------|--|
| <code>gui.cacert</code> | No | None | It specifies the name of the RootCA configmap. When using multiple GUI hosts, the CA certificate configMap must contain a bundle CA certificate with all individual certificates concatenated. For more information, see Configuring Certificate Authority (CA) certificates . |
| <code>gui.csiSecretName</code> | Field is required if storage cluster is older than 5.2.3.x | csi-remote-mount-storage-cluster-1 | It references the secret that contains the username and password of the CSI admin user in the <code>ibm-spectrum-scale-csi</code> namespace. This field is unused if storage cluster is 5.2.3 or newer. This field is DEPRECATED and will be removed in a future release. |
| <code>gui.host</code> | No | None | The host references a REST API endpoint on the remote cluster. This host must be resolvable by DNS. This field is DEPRECATED and will be removed in a future release. Use the 'hosts' field instead. |
| <code>gui.hosts</code> | No | None | The hosts references one or more REST API endpoints on the remote cluster. Up to 3 endpoints can be specified. The hosts must be resolvable by DNS. |
| <code>gui.insecureSkipVerify</code> | No | None | The parameter controls whether a client verifies the storage cluster's GUI certificate chain and hostname. If set to <code>true</code> , TLS is susceptible to machine-in-the-middle attacks. The default value is <code>false</code> . |
| <code>gui.port</code> | No | 443 | It specifies the port of the Remote Cluster. |
| <code>gui.scheme</code> | No | https | The default value is 'https'. No other value is supported. |
| <code>gui.secretName</code> | Yes | None | The name of the Kubernetes secret created during the storage cluster configuration. |
| <code>gui.passwordRotation</code> | No | None | Automatic password rotation is enabled if this object is defined. For more information, see Automatic password rotation . |
| <code>gui.passwordRotation.passwordChangeInterval</code> | No | 80 | Specifies the interval to automatically change the passwords, in days. For more information, see Automatic password rotation . |

You can define 1 or more **RemoteClusters** to the cluster, one for each Storage Cluster you want to mount file systems from.

Either `gui.host` or `gui.hosts` is mandatory in **Remotecluster** CR. Use `gui.hosts` instead of `gui.host`, as `gui.host` is deprecated and will get removed in a future release.

Contact Nodes

The `spec.contactNodes` field allows the administrator to specify the exact list of contact nodes from the storage cluster to be used in the remote cluster configuration. If none is provided, the operator picks all quorum nodes from querying the remote cluster REST API. The hostnames for the contact nodes need to be DNS resolvable within the core pods in the Red Hat OpenShift cluster. If DNS resolution does not work, you can specify the hostname and IP address mapping by using the `hostAlias` field in the Cluster Spec. For more information, see [hostAlias](#).

GUI

The `spec.gui` contains the properties that are needed to communicate with the IBM Storage Scale remote storage cluster.

- **cacert** - provide the name of the ConfigMap storing the CA Certificate for the Storage Cluster GUI. When using multiple GUI hosts for high availability, you must create a bundle CA certificate that contains all individual certificates of each GUI host concatenated together. For more information, see [Configuring Certificate Authority \(CA\) certificates](#).
- **insecuritySkipVerify** - controls whether a client verifies the server's certificate chain and hostname. Default to false.
- **host** - provide the Remote Cluster GUI host endpoint. This field is deprecated. Use **hosts** instead.
- **hosts** - provide the Remote Cluster GUI host endpoints. Multiple (up to 3) GUI host endpoints can be specified when the GUI is configured on multiple nodes of the remote storage cluster. Use this property to get high availability of the GUI.
- **secretName** - The name of the Kubernetes secret that contains the credentials for the ContainerOperator user on the storage cluster
- **csiSecretName** - The name of the Kubernetes secret that contains the credentials for the CsiAdmin user on the storage cluster. This property is unused if storage cluster is 5.2.3 or newer.

Deleting a **RemoteCluster** custom resource definition does not delete the access permission of the IBM Storage Scale container native cluster to the file systems on the remote storage cluster.

Enter the `kubectl explain remotecruster.spec` command to view more details.

Automatic password rotation

The IBM Storage Scale container native operator can automatically rotate the password of the ContainerOperator GUI user. As part of the password update, the operator also updates the password in the respective secret, which is specified by `secretName`. By default, the password rotation is disabled.

If storage cluster version is older than 5.2.3.0, it also rotates the password of the CsiAdmin GUI user. As part of the password update, the operator also updates the password in the respective secret, which is specified by `csiSecretName`.

If multiple container native clusters are accessing the same storage cluster, unique GUI users must be created for each container native cluster for automatic password rotation to be used. LDAP and Microsoft Active Directory are not supported.

[Creating secrets for storage cluster GUI users](#) describes how to create the secrets that contain the usernames and passwords for the storage cluster GUI users. Once created, the operator can automatically change the passwords of these users regularly.

To enable password rotation, add the `gui.passwordRotation: {}` section in the RemoteCluster CR spec:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: RemoteCluster
metadata:
  name: ...
spec:
  gui:
    ...
    passwordRotation: {}
```

If password rotation is enabled, the password change interval is 80 days, by default. To change the interval to a different value, add the `passwordChangeInterval` parameter to use a different change interval. For example, to configure password rotation every 60 days:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: RemoteCluster
metadata:
  name: ...
spec:
  ...
  passwordRotation:
    passwordChangeInterval: "60"
```

If automatic password rotation is enabled, the `PasswordRotation` status condition is displayed and the `Last Password Change` field indicates when the password has been changed last time. A `PasswordChanged` event and status is created when the operator has changed the password successfully.

```
$ kubectl describe remoteccluster
...
Status:
  Conditions:
    Last Transition Time: 2024-01-22T12:02:57Z
    Message: The remote cluster has been configured successfully.
    Reason: AuthCreated
    Status: True
    Type: Ready
    Last Transition Time: 2023-12-14T05:25:23Z
    Message: The passwords for GUI users cnsa_storage_gui_user and csi_storage_gui_user have
been changed successfully in remote cluster.
    Reason: PasswordChanged
    Status: True
    Type: PasswordRotation
    Last Password Change: 2023-12-14T05:25:23Z
    Local Key SHA Digest: aa...3422
    Remote Key SHA Digest: ee...4766
  Events:
    Type Reason Age From Message
    ---
    Normal PasswordChanged 44m RemoteClusterUser The passwords for GUI users cnsa_storage_gui_user and
csi_storage_gui_user have been changed successfully in remote cluster.
```

Password policy

The GUI has a password policy that can be seen by running `/usr/lpp/mmfs/gui/cli/lspasswordpolicy` on the remote cluster GUI node. By default the policy looks like this:

```
sh-4.4$ /usr/lpp/mmfs/gui/cli/lspasswordpolicy
Reject user-name = true
Minimum upper case characters = 0
Maximum password age = 90
Minimum password length = 6
Minimum different characters with respect to old password = 1
Remember old passwords = 3
Minimum digits = 0
Minimum password age = 0
Maximum number of repeat characters = 0
Minimum lower case characters = 0
Minimum special case characters = 0
EFSSG1000I The command completed successfully.
```

The password expiration in the storage cluster policy "Maximum password age" must be a longer period than the `spec.gui.passwordRotation.passwordChangeInterval` in the remote cluster CR. {.attention}

The password policy that is configured in the storage cluster GUI must fulfil the following rules:

Table 2. Password policy requirements

| Policy parameter | Supported values |
|---|---|
| Reject user-name | true, false |
| Minimum upper case characters | 0-10 |
| Maximum password age | must be higher than <code>passwordChangeInterval</code> (or higher than the default value 80) |
| Minimum password length | 1-40 |
| Minimum different characters with respect to old password | any value allowed |
| Remember old passwords | any value allowed |
| Minimum digits | 0-10 |
| Minimum password age | must be lower than <code>passwordChangeInterval</code> (or lower than the default value 80) |
| Maximum number of repeated characters | |
| Minimum lowercase characters | 0-10 |
| Minimum special case characters | 0-10 |

At the time when operator is in the process of rotating the password, cluster-specific CSI operations like dynamic PV creation or snapshot create can take more time than usual (up to 8 minutes). After this period, CSI operations will be back to their usual timing. The 8-minute time can be reduced, see [Failed login attempts timeout \(403 error\)](#).

One-time password change

In certain scenarios, a one-time password change can be necessary to synchronize secrets between the `ibm-spectrum-scale` and `ibm-spectrum-scale-csi` namespaces. This can help resolve desynchronization issues, particularly when GUI users encounter **403 Forbidden** errors.

This capability can be used to perform a password update for security or operational reasons, without waiting for the regular rotation cycle.

After the one-time password change, the time to change password gets restarted to the one defined in `remotecluster passwordChangeInterval` field.

To initiate a one-time password change, add the `scale.spectrum.ibm.com/one-time-password-change=` annotation to the remote cluster:

```
oc annotate remotecluster <name> scale.spectrum.ibm.com/one-time-password-change=
```

This triggers the password update process while maintaining a log entry in the status field, consistent with the current behavior for standard password rotation. The annotation is then removed when the process is complete.

RemoteCluster status

Status **Conditions** can be viewed as a snapshot of the current and most up-to-date status of a `Remotecluster` instance.

- The `Ready` condition is set to `True` if the `Remotecluster` credentials are established.

Examples

Additional remote clusters

It is possible to mount other file systems that are served by multiple remote clusters. For each remote cluster, a separate resource must be defined into the Kubernetes cluster.

If applying the sample `remotecluster.yaml` CR from the examples, you would have a single resource defined:

```
$ kubectl get remotecluster -n ibm-spectrum-scale
NAME          READY  AGE
remotecluster-sample  True   25h
```

Duplicate the sample `remotecluster` CR and make changes to reflect your second remote storage cluster. The `metadata.name` must be changed to be unique, in this example, we use `xyz-storagecluster`. Create the resource and verify that it is defined:

```
$ kubectl get remotecluster -n ibm-spectrum-scale
NAME          READY  AGE
remotecluster-sample  True   25h
xyz-storagecluster    True   10m
```

Creating secrets for storage cluster GUI users

Create a secret on the Red Hat OpenShift cluster that holds the credentials for GUI users who are defined on the IBM Storage Scale Storage cluster. The secret is used by the operator to communicate with the storage cluster to configure the remote mount.

The username and password that is specified for the secrets must match the GUI user that was created on the storage cluster [Creating container operator user](#).

Storage cluster v5.2.3.0 and newer

A new secret must be added for each storage cluster being configured.

1. Create a secret for the **ContainerOperator** GUI user defined on the storage cluster.

To create a secret named **cnsa-remote-mount-storage-cluster-1** in the **ibm-spectrum-scale** namespace, enter the following command:

```
kubectl create secret generic cnsa-remote-mount-storage-cluster-1 --from-literal=username='cnsa_storage_gui_user' \
--from-literal=password='cnsa_storage_gui_password' -n ibm-spectrum-scale
```

2. Label the secret, enter the following command:

```
kubectl label secret cnsa-remote-mount-storage-cluster-1 -n ibm-spectrum-scale product=ibm-spectrum-scale
```

Storage cluster older than v5.2.3.0

Two new secrets must be added for each storage cluster being configured.

1. Create a secret for the **ContainerOperator** GUI user defined on the storage cluster.

To create a secret named **cnsa-remote-mount-storage-cluster-1** in the **ibm-spectrum-scale** namespace, enter the following command:

```
kubectl create secret generic cnsa-remote-mount-storage-cluster-1 --from-literal=username='cnsa_storage_gui_user' \
--from-literal=password='cnsa_storage_gui_password' -n ibm-spectrum-scale
```

2. Create a secret for the **CsiAdmin** GUI user defined on the storage cluster.

To create the secret named **csi-remote-mount-storage-cluster-1** in the **ibm-spectrum-scale-csi** namespace, enter the following command:

```
kubectl create secret generic csi-remote-mount-storage-cluster-1 --from-literal=username='csi_storage_gui_user' \
--from-literal=password='csi_storage_gui_password' -n ibm-spectrum-scale-csi
```

3. Label the secret, enter the following command:

```
kubectl label secret cnsa-remote-mount-storage-cluster-1 -n ibm-spectrum-scale product=ibm-spectrum-scale
```

When the passwords on the storage cluster for these users change, the credentials in the secrets must be updated. For instructions to update the secrets, see [Updating user secrets for the storage cluster on Red Hat OpenShift](#).

Optionally, automatic password rotation for these users can be enabled. If enabled, the operator frequently changes the user passwords on the storage cluster and also update the two secrets. For more information, see [Automatic password rotation](#).

Configuring Certificate Authority (CA) certificates

IBM Storage Scale container native uses Transport Layer Security (TLS) verification to guarantee secure HTTPS communication with the storage cluster GUI. It verifies the server's certificate chain and host name.

Configure a security protocol

A security protocol must be configured for use with IBM Storage Scale container native in one of three different ways.

Option 1 - CA certificate ConfigMap

A ConfigMap containing the CA certificate of the storage cluster GUI must be created to allow the IBM Storage Scale container native operator to perform TLS verification. CA certificate data can exist in base64 encoded or decoded forms.

When using multiple GUI hosts for high availability, you must create a bundle CA certificate that contains all the individual certificates of each GUI host concatenated together. The single bundle certificate is then used to create the ConfigMap.

In the following example, we create a ConfigMap from `storage-cluster-1.crt` file. This file contains the storage cluster CA certificate data in decoded form. The decoded form must appear as shown:

```
# cat storage-cluster-1.crt
-----BEGIN CERTIFICATE-----
MIIDZDC.....
.....n/J9OJFdoXs=
-----END CERTIFICATE-----
```

For multiple GUI hosts, the certificate file should contain all certificates concatenated:

```
# cat storage-cluster-1.crt (bundle for multiple GUI hosts)
-----BEGIN CERTIFICATE-----
MIIDZDC.....gui-host-1.crt_content.....
.....n/J9OJFdoCs=
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
MIIDZDC.....gui-host-2.crt_content.....
.....n/J9OJFdoBs=
-----END CERTIFICATE-----
```

Create the ConfigMap with one of the following commands:

```
kubectl create configmap cacert-storage-cluster-1 --from-file=storage-cluster-1.crt=storage-cluster-1.crt -n
ibm-spectrum-scale
```

By default, the storage cluster GUI self-signs a certificate that can be used in lieu of a CA certificate. This certificate can be obtained and used to create the `cacert` ConfigMap by entering the following command. Replace the `<gui host>` with the hostname of the storage cluster GUI.

For a single GUI host:

```
kubectl create configmap cacert-storage-cluster-1 --from-literal=storage-cluster-1.crt="$(openssl s_client -
showcerts -connect <gui host>:443 </dev/null 2>/dev/null | openssl x509 -outform PEM)" -n ibm-spectrum-scale
```

For multiple GUI hosts, retrieve each certificate and create a bundle:

```
# Retrieve certificates from each GUI host
openssl s_client -showcerts -connect <gui-host-1>:443 </dev/null 2>/dev/null | openssl x509 -outform PEM >
gui-host-1.crt
openssl s_client -showcerts -connect <gui-host-2>:443 </dev/null 2>/dev/null | openssl x509 -outform PEM >
gui-host-2.crt

# Create a bundle certificate by concatenating all certificates
cat gui-host-1.crt gui-host-2.crt > storage-cluster-1.crt

# Create the ConfigMap from the bundle certificate
kubectl create configmap cacert-storage-cluster-1 --from-file=storage-cluster-1.crt=storage-cluster-1.crt -n
ibm-spectrum-scale
```

Option 2 - Storage cluster uses the OpenShift Container Platform CA or a Red Hat default CA

IBM Storage Scale container native automatically includes the OpenShift Container Platform CA and the default Red Hat CA bundle for storage cluster GUI communication. If the storage cluster uses the OpenShift Container Platform CA or a Red Hat trusted CA, a ConfigMap, as described in Option 1, does **not** need to be created for the CA certificate and the `cacert` field should be deleted from the RemoteCluster Custom Resource. For more information, see [RemoteClusters](#).

Option 3 - Skip verification

Storage cluster verification may be skipped if desired, however, TLS is susceptible to machine-in-the-middle attacks. To skip verification, the `insecureSkipVerify` option must be set to `true`, when configuring the RemoteCluster Custom Resource. For more information, see [RemoteClusters](#).

Storage cluster verification

Events are posted onto the `RemoteCluster` resource if configuration is missing. For example, if secrets and ConfigMaps are missing, you may see events similar to the following sample:

```
$ kubectl describe remoteccluster.scale remoteccluster-sample
...
Events:
  Type      Reason      Age      From      Message
  ----      -
  Warning   ...         ...         ...         ...
```

| | | | | |
|---------|-----------------|-------------------|---------------|--|
| Warning | RemoteConnError | 6m3s | RemoteCluster | Secret "cnsa-remote-mount-storage-cluster-1" not found |
| Warning | RemoteConnError | 3s (x6 over 5m3s) | RemoteCluster | ConfigMap "cacert-storage-cluster-1" not found |

Filesystem

The **Filesystem** custom resource supports the following file systems:

- [Remote file systems](#) are mounted from a remote IBM Storage Scale storage cluster.
- [Local file systems](#) use local disks or volumes of Kubernetes nodes as the storage.

Remote file system

To configure a file system in the IBM Storage Scale container native cluster, a **Filesystem** custom resource (CR) must be defined for each file system you want mounted.

An IBM Storage Scale remote storage cluster serves the file system. Create a **RemoteCluster** custom resource for that storage cluster before proceeding. For more information, see [RemoteCluster](#).

Download Remote file system CR sample

Samples are provided as a starting point with some defaults already set in the configuration based on the target environment. Use one of the following commands to download a copy of the sample.

Use caution to select the correct curl command based on your environment. The source files are different but the downloaded files are all named **filesystem.remote.yaml**.

Red Hat OpenShift

Use the command to download a copy of the sample for Red Hat OpenShift:

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/filesystem/filesystem.remote.yaml > filesystem.remote.yaml || echo "Failed to download Filesystem sample CR"
```

Kubernetes and Google Kubernetes Engine (GKE)

IBM Storage Scale container native deployed in a Google Kubernetes Engine (GKE) is available as a technology preview. For more information, see [Support for Google Kubernetes Engine \(GKE\)](#).

Use the command to download a copy of the sample for Kubernetes or Google Kubernetes Engine (GKE):

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/filesystem/filesystem.remote-kubernetes.yaml > filesystem.remote.yaml || echo "Failed to download Filesystem sample CR"
```

Apply the Remote file system resource

1. Edit the **filesystem.remote.yaml** file and change the fields that are specific to your installation. For more information, see [Filesystem Spec](#).

2. Use the following command to apply the changed yaml:

```
kubectl apply -f filesystem.remote.yaml
```

3. View the **Filesystem** resources with the following command:

```
kubectl get filesystem -n ibm-spectrum-scale
```

Use the command **kubectl edit filesystem <filesystem-name> -n ibm-spectrum-scale** to modify the properties of the resource.

Create a **EncryptionConfig** custom resource to encrypt the remote mounted file system for the IBM Storage Scale container native cluster. For more information, see [EncryptionConfig](#).

Filesystem spec

The following table describes the properties for **Filesystem**:

Table 1. Filesystem property and description

| Property | Required | Default | Description |
|-----------------------------|--|---------|---|
| <code>metadata.name</code> | Yes | None | The name of the CR. |
| <code>remote</code> | No | None | If specified, describes the file system to be a remote mounted file system. |
| <code>remote.fs</code> | Yes, if <code>remote</code> is specified | None | It is the name of the file system to mount, served by the remote cluster. |
| <code>remote.cluster</code> | Yes, if <code>remote</code> is specified | None | It is the name of the <code>RemoteCluster</code> resource. |

You can define 1 or more `Filesystem` custom resources, one for each file system that you want to mount from the storage cluster.

All file systems are mounted under `/mnt`. The mount path cannot be changed.

remote

The `spec.remote` section defines the remote file system properties and consists of two fields:

- `remote.cluster`: This field specifies the name of the `RemoteCluster` CR that is defined that is serving the file system.
- `remote.fs`: This field specifies the file system name on the remote storage cluster that is mounted into the container native cluster.

In the following example:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Filesystem
...
spec:
  remote:
    cluster: remoteclass-sample
    fs: fs1
```

The file system `fs1` provided by the remote cluster, which is defined in `remoteclass-sample` is made available in the container.

If a `Filesystem` custom resource is deleted, the file system configuration from the IBM Storage Scale Cluster **is not** deleted.

Enter the `kubectl explain filesystem.spec.remote` command to view more details.

File system status

Status `Conditions` can be viewed as a snapshot of the current and most up-to-date status of a `Filesystem` instance.

- The `Success` condition is set to `True` if the `Filesystem` is created and mounted.

Examples

More file systems

To create more file systems in the container native cluster, define a custom resource for each file system and apply it to the Kubernetes cluster.

Following the prior examples result in a single file system resource:

```
$ kubectl get filesystem -n ibm-spectrum-scale
NAME          ESTABLISHED  AGE
remote-sample  True         25h
```

1. To create another file system that is served by the **same** remote cluster, `remoteclass-sample`, define a yaml as follows:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Filesystem
metadata:
  labels:
    app.kubernetes.io/instance: ibm-spectrum-scale
    app.kubernetes.io/name: cluster
  name: c1-fs2
  namespace: ibm-spectrum-scale
spec:
  remote:
    cluster: remoteclass-sample
    fs: fs2
```

When applied, the file system `fs2` hosted by the remote cluster `remoteclass-sample` is made available to the container native cluster.

Displaying the `Filesystem` custom resources shows the following output:

```
$ kubectl get filesystem -n ibm-spectrum-scale -o wide
NAME          ESTABLISHED  REMOTE CLUSTER  MAINTENANCE MODE  AGE
remote-sample  True         remoteclass-sample  not supported      25h
c1-fs2        True         remoteclass-sample  not supported      10m
```

2. To create another file system that is served by a **different** remote cluster than `remoteccluster-sample-2`, define a yaml as follows:

As a prerequisite a RemoteCluster resource that is called `remoteccluster-sample-2` must exist. For more information, see [RemoteClusters](#).

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Filesystem
metadata:
  labels:
    app.kubernetes.io/instance: ibm-spectrum-scale
    app.kubernetes.io/name: cluster
  name: xyz-fs2
  namespace: ibm-spectrum-scale
spec:
  remote:
    cluster: remoteccluster-sample-2
    fs: fs2
```

When applied, the file system `fs2` hosted by the remote cluster `remoteccluster-sample-2` is made available to the container native cluster.

Displaying the `Filesystem` custom resources shows the following output:

```
$ kubectl get filesystem -n ibm-spectrum-scale -o wide
NAME           ESTABLISHED  REMOTE CLUSTER      MAINTENANCE MODE  AGE
remote-sample  True         remoteccluster-sample not supported      25h
cl-fs2         True         remoteccluster-sample not supported      20m
xyz-fs2        True         remoteccluster-sample-2 not supported      10m
```

Local file system

Creating a local file system consists of the following steps:

1. Specify device names.
2. Label nodes that have a connection to the shared disks.
3. Configure file system replication (optional).
4. Create a LocalDisk resource for each disk to use for the file system.
5. Create a Filesystem resource that references to the LocalDisk resources.

Specify device names

Skip this step if the used disks have one of the device names `sd*`, `hd*`, `vd*`, `scini*`, `pmem*`, `nvm*`, `dm-*`, `vpath*`, `dasd*`, or `emcpower*`.

If disks with different device names are used, the devices must be specified in the [Cluster](#) custom resource.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
...
spec:
  daemon:
    nsdDevicesConfig:
      localDevicePaths:
        - devicePath: /dev/test*
          deviceType: generic
```

Table 1. Cluster property and descriptions for nsdDevicesConfig options

| Property | Required | Default | Description |
|---|----------|----------------------|---|
| <code>cluster.spec.daemon.nsdDevicesConfig.localDevicePaths.devicePath</code> | Yes | None | The device name of the disks. The device name also allows wildcards. |
| <code>cluster.spec.daemon.nsdDevicesConfig.localDevicePaths.deviceType</code> | No | <code>generic</code> | The type of the device. The type is usually <code>generic</code> . Contact IBM if device type <code>generic</code> does not work. |

Configure replication (technology preview)

The replication feature is available as a technology preview. Technology preview features are not supported for use within production environments. Use with non-production workloads, in demo or proof of concept environments only. IBM production service level agreements

(SLAs) are not supported. Technology preview features might not be functionally complete. The purpose of technology preview features is to give access to new and exciting technologies, enabling customers to test functionality and provide feedback during the development process. The existence of a technology preview feature does not mean a future release is guaranteed. Feedback is welcome and encouraged.

Optionally the file system data can be replicated between disks by using 2-way or 3-way replication. The disks must be grouped in failure groups. The IBM Storage Scale file system writes the data block replicas to disks with different failure group numbers.

A failure group number can be specified per local disk. For more information, see [LocalDisk spec](#). All disks with the same number belong to the same failure group. The total disk capacity of all failure groups must be equal (with a small difference are tolerated). At least 2 failure groups are required for 2-way replication. At least 3 failure groups are required for 3-way replication. If more failure groups are created, IBM Storage Scale writes replicas to disks of different failure groups to evenly fill the disks.

Create LocalDisk resources

A **LocalDisk** custom resource must be created for each disk or volume that is used to provide storage for a local file system.

The following steps help guide you in creating a **LocalDisk** resource.

1. Download a copy of the sample [localdisk.yaml](#) from the GitHub repository.

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/localdisk/localdisk.yaml > localdisk.yaml || echo "Failed to download LocalDisk sample CR"
```

The sample **localdisk.yaml** provides a list of sample disks that must be changed. Each **LocalDisk** resource must have a unique **metadata.name**.

2. Edit the **localdisk.yaml** file and change the fields that are specific to your installation. For more information, see [LocalDisk Spec](#).
3. Apply the resource by using the following command:

```
kubectl apply -f localdisk.yaml
```

4. View the localdisk resources by using the following command:

```
kubectl get localdisk -n ibm-spectrum-scale
```

LocalDisk spec

The following table describes the properties for **LocalDisk**:

Table 2. LocalDisk property and description

| Property | Required | Default | Description |
|-------------------------------|----------|---------|---|
| metadata.name | Yes | None | The name of the CR. |
| device | Yes | None | The device path of the disk on the specified node (see node property) at creation time. After successful creation of the local disk, this parameter is no longer used. Note: Device paths of disks can change after a node reboot. |
| node | Yes | None | The Kubernetes node where the specified device exists at creation time. |
| nodeConnectionSelector | No | None | This node selector selects the nodes, which are expected to have a physical connection to the disk. If not specified, by default all nodes are expected to have a physical connection to the disk. At local disk creation time, the operator verifies whether the disk has a connection to the selected nodes. Only nodes that are selected by the cluster.spec.daemon.nodeSelector node selector are considered. For more information on cluster.spec.daemon.nodeSelector , see Cluster spec . |
| failureGroup | No | None | The failure group number is only needed if the file system that uses this disk is configured for 2-way or 3-way replication. The replicas of blocks are written to disks with different failure group numbers. Even though the failure group must be a positive number higher or equal than zero, the number must be specified in quotation marks like "2". This parameter cannot be changed anymore as soon as the disk is used by a file system. For more information, see Configure replication (technology preview) . |
| skipVerify | No | False | This property controls whether a device is verified before it is used as a Storage Scale disk. The verification checks if the device is already in use. Set skipVerify to true only if you know what you do. |
| existingDataSkipVerify | No | False | This property is deprecated and cannot be used anymore. Use skipVerify instead. |
| thinDiskType | No | false | The space reclaim disk type of IBM Storage Scale disks. |

In the following example:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: LocalDisk
metadata:
```

```

    name: disk0
    namespace: ibm-spectrum-scale
spec:
  node: worker0.example.com
  device: /dev/sdb
  nodeConnectionSelector:
    matchExpressions:
      - key: kubernetes.io/hostname
        operator: In
        values:
          - worker0.example.com
          - worker1.example.com
          - worker2.example.com
---
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: LocalDisk
metadata:
  name: disk1
  namespace: ibm-spectrum-scale
spec:
  node: worker0.example.com
  device: /dev/sdc
  nodeConnectionSelector:
    matchExpressions:
      - key: kubernetes.io/hostname
        operator: In
        values:
          - worker0.example.com
          - worker1.example.com
          - worker2.example.com
---
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: LocalDisk
metadata:
  name: disk2
  namespace: ibm-spectrum-scale
spec:
  node: worker0.example.com
  device: /dev/sdd
  nodeConnectionSelector:
    matchExpressions:
      - key: kubernetes.io/hostname
        operator: In
        values:
          - worker0.example.com
          - worker1.example.com
          - worker2.example.com

```

Three local disks with devices `/dev/sdb`, `/dev/sdc`, and `/dev/sdd` on the `worker0.example.com` Red Hat OpenShift node are created. All three disks are expected to have a connection to the `worker0.example.com`, `worker1.example.com`, and `worker2.example.com` nodes. For each disk, the connection to the `worker0.example.com`, `worker1.example.com`, and `worker2.example.com` nodes is verified after the local disk is created.

After a Red Hat OpenShift node is rebooted, the disks might have different `device` on the node. The different device is not a problem because the `device` parameter only matters at local disk creation time.

Enter the `kubectl explain localdisk.spec` command to view more details.

LocalDisk status

LocalDisk type

The **Type** in the local disk status indicates how the disk is connected to the nodes.

Table 3. LocalDisk types

| Type | Description |
|------------------|--|
| unshared | The disk is connected to the node specified in <code>spec.node</code> . The <code>spec.nodeConnectionSelector</code> parameter is not specified. |
| shared | This disk is connected to all nodes that are selected by <code>spec.nodeConnectionSelector</code> . If <code>spec.nodeConnectionSelector</code> is not specified, the disk is connected to all nodes that are selected to run a core pod. |
| partially-shared | The disk is connected to a subset of nodes that are selected by <code>spec.nodeConnectionSelector</code> . If <code>spec.nodeConnectionSelector</code> is not specified, the disk is connected to a subset of the nodes that are selected to run a core pod. |

Only file systems with type `shared` are supported. After the LocalDisk resources are created, they must all display the `shared` type.

```

kubectl get localdisk -n ibm-spectrum-scale
NAME      TYPE      READY   USED   AVAILABLE   FILESYSTEM   SIZE   AGE

```

| | | | | | | |
|-------|--------|------|-------|-------|---------|-----|
| disk0 | shared | True | False | False | 1.4 TiB | 80m |
| disk1 | shared | True | False | False | 1.4 TiB | 80m |
| disk2 | shared | True | False | False | 1.4 TiB | 80m |

If a disk has type `unshared` or `partially-shared` in status, the disk is not connected to all nodes selected by `spec.nodeConnectionSelector` or to all nodes that are selected to run a core pod. In this case, the connection of the disk must be fixed or `spec.nodeConnectionSelector` must be corrected before it can be used for the local file system.

If the file system must be replicated, use the `-o wide` option to display the failure group numbers.

```
$ kubectl get localdisk -o wide -n ibm-spectrum-scale
```

| NAME | TYPE | READY | USED | AVAILABLE | FILESYSTEM | POOL | FAILUREGROUP | SIZE | AGE |
|-------|--------|-------|-------|-----------|------------|------|--------------|---------|-----|
| disk0 | shared | True | False | False | | | 0 | 1.4 TiB | 80m |
| disk1 | shared | True | False | False | | | 1 | 1.4 TiB | 80m |
| disk2 | shared | True | False | False | | | 2 | 1.4 TiB | 80m |

LocalDisk status conditions

The status **Conditions** can be viewed as a snapshot of the current and most up-to-date status of a **LocalDisk** instance.

- The **Ready** condition is set to **True** if the **LocalDisk** successfully created a disk within the IBM Storage Scale cluster and the disk is ready to be used by a local file system.
- The **Used** condition is set to **True** if the disk is used within a local file system.
- The **Available** condition is set to **True** if the disk within the IBM Storage Scale cluster is operational. The condition has **Unknown** status if the disk is not used by a local file system.

Create Filesystem resource

To configure a local file system in the IBM Storage Scale container native cluster, a **Filesystem** custom resource (CR) must be defined for each local file system.

A **LocalDisk** custom resource must be created for each disk or volume before proceeding. For more information, see [Create LocalDisk resources](#).

Samples are provided as a starting point with some defaults already set in the configuration based on the target environment. Use one of the following commands to download a copy of the sample.

Use caution to select the correct curl command based on your environment. The source files are different but the downloaded files are all named `filesystem.local.yaml`.

Red Hat OpenShift

Use the command to download a copy of the sample for Red Hat OpenShift:

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/filesystem/filesystem.local.yaml > filesystem.local.yaml || echo "Failed to download Filesystem sample CR"
```

Kubernetes and Google Kubernetes Engine (GKE)

IBM Storage Scale container native deployed in a Google Kubernetes Engine (GKE) is available as a technology preview. For more information, see [Support for Google Kubernetes Engine \(GKE\)](#).

Use the command to download a copy of the sample for Kubernetes or Google Kubernetes Engine (GKE):

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/filesystem/filesystem.local-kubernetes.yaml > filesystem.local.yaml || echo "Failed to download Filesystem sample CR"
```

Apply the local file system resource

1. Edit the `filesystem.local.yaml` file to change the fields that are specific to your installation. For more information, see [Filesystem Spec](#).

2. After you complete the changes, use the following command to apply the yaml:

```
kubectl apply -f filesystem.local.yaml
```

3. Use the following command to view the **Filesystem** resources:

```
kubectl get filesystem -n ibm-spectrum-scale -o wide
```

To view the detailed properties of a local file system named `local-sample`:

```
kubectl describe filesystem local-sample -n ibm-spectrum-scale
```

Filesystem spec

The following table describes the properties for **Filesystem**:

Table 4. LocalDisk property and description

| Property | Required | Default | Description |
|--------------------------------|----------|-----------|--|
| <code>metadata.name</code> | Yes | None | The name of the CR. |
| <code>local</code> | No | None | If specified, describes the file system to be a local file system. |
| <code>local.type</code> | Yes | None | The type of the file system. Only shared is supported. |
| <code>local.replication</code> | Yes | None | Replication causes the number of replicas to create for each data/metadata block that is written to the file system. Specify 1-way to not replicate the file system data. Specify 2-way or 3-way to replicate data. |
| <code>local.pools</code> | Yes | None | List of file system pools. |
| <code>local.pools.name</code> | Yes | None | It is the name of the pool. One pool with name system is mandatory (default pool). |
| <code>local.pools.disks</code> | Yes | None | The names of LocalDisk resources that provide storage to this local file system pool. |
| <code>local.blockSize</code> | No | 4M | A block size of 4 MiB provides good sequential performance, makes efficient use of disk space, and provides good performance for small files. It works well for the widest variety of workloads. |

You can define 1 or more **Filesystem** custom resources, each file system must use its own local disks.

In the following example:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Filesystem
  name: local-sample
  namespace: ibm-spectrum-scale
spec:
  local:
    type: shared
    replication: 1-way
    pools:
      - disks:
          - disk0
          - disk1
          - disk2
        name: system
```

The local file system **local-sample** uses the disks that are represented by the **disk0**, **disk1**, and **disk2** local disk resources.

Enter the **kubectl explain filesystem.spec.local** command to view more details.

Consider the [Local file system hardware requirements](#).

Filesystem status

Status **Conditions** can be viewed as a snapshot of the current and most up-to-date status of a **Filesystem** instance. The **Success** condition is set to **True** if the file system is created and mounted.

Enter the **kubectl explain filesystem.status.pools** command to view more details.

Encryption

IBM Storage Scale container native supports remote mount of an encrypted filesystem. Encryption is managed through use of encryption keys stored on key server.

The following key servers are supported:

- IBM Security Guardium Key Lifecycle Manager (SKLM)

EncryptionConfig spec

The following table describe the properties for **EncryptionConfig**:

Table 1. EncryptionConfig property and description

| Property | Required | Default | Description |
|----------|----------|---------|-------------|
|----------|----------|---------|-------------|

| Property | Required | Default | Description |
|----------------------------|----------|---------|---|
| <code>metadata.name</code> | Yes | None | The name of the CR. |
| <code>server</code> | Yes | None | The key server host name or IP in which encryption keys are stored. |
| <code>backupServers</code> | No | None | The backup key servers configured for high availability. This field is optional. |
| <code>port</code> | No | None | It can be used to override the default port for the key server. |
| <code>cacert</code> | No | None | The ConfigMap storing CA and endpoint certificates used while adding/renewing key server certificate chain. |
| <code>secret</code> | Yes | None | The name of the basic-auth secret containing the username and password to the key server. |
| <code>tenant</code> | Yes | None | The tenant name on the key server that contains encryption keys. This has to be the same tenant name that is used to store the encryption keys of the remote storage file system. |
| <code>client</code> | Yes | None | The key client to communicate with the key Server. |
| <code>remoteRKM</code> | Yes | None | The RKM ID from the storage cluster corresponding to given key server and tenant. |

Deleting a **EncryptionConfig** custom resource does **not delete** the encryption configuration from the IBM Storage Scale container native cluster.

Updating `client` and `tenant` is not recommended as it causes loss of master encryption keys for that tenant.

For more information, enter the `kubectl explain encryptionconfig.spec` command.

EncryptionConfig status

Status **Conditions** can be viewed as a snapshot of the current and most up-to-date status of a **EncryptionConfig** instance.

- The **Success** condition is set to **True** if the **EncryptionConfig** is successfully configured.

Examples

To give IBM Storage Scale container native access to the encryption keyserver, an **EncryptionConfig** custom resource must be created. The configuration must add the same key server and tenant as configured on storage cluster hosting the filesystem. You can define more than one **EncryptionConfig** custom resource.

For more information, see [Encryption](#) in IBM Storage Scale documentation.

Pre-requisites

- Create a secret containing the administrator username and password credentials to the key server.

```
kubectl create secret generic keyserver-credentials -n ibm-spectrum-scale \
--from-literal=username=<keyserver_admin_name> \
--from-literal=password=<keyserver_admin_password>
```

- If using CA certificates, create the **ConfigMap** holding the CA certificate chain.
 - Obtain CA certificates and endpoint/server certificates. Separate the root certificate and the intermediate certificates into the following `.cert` files:
 - Root certificate: `root.cert`
 - Server or Endpoint certificate: `endpoint.cert`
 - Intermediate certificates: `intermediate<numeric_index>.cert`
 - Create the ConfigMap with the following command:

```
kubectl create ConfigMap sample-ca-cert \
--from-file=/path/to/root.cert \
--from-file=/path/to/intermediate1.cert \
--from-file=/path/to/intermediate2.cert \
--from-file=/path/to/intermediate3.cert \
--from-file=/path/to/endpoint.cert \
-n ibm-spectrum-scale
```

- Encryption details from storage cluster, specifically, **tenant** and **RKMID**.

If the core pods are unable to resolve the IP address of the IBM SKLM server, you can add **hostAliases** entries in the **Cluster** custom resource. For more information, see [Cluster](#).

Configure EncryptionConfig custom resource

The following steps describe creating a **EncryptionConfig** CR.

1. Download a copy of the sample [encryptionconfig.remote.yaml](#) from the GitHub repository.

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/encryptionconfig/encryptionconfig.remote.yaml > encryptionconfig.remote.yaml || echo "Failed to download EncryptionConfig sample CR"
```

2. Make changes specific to your installation. For more information about the **EncryptionConfig** specification, see [EncryptionConfig spec](#).

- Replace **keyserver.example.com** with your keyserver hostname
- Replace **keyserver1.example.com**, **keyserver2.example.com**, etc with your backup keyserver hostnames
- Replace **sampleTenant** with your tenant name
- Replace **sampleClient** with your client name
- Replace **sampleRKM** with your RKMID
- If using self-signed certificates, comment out the **cacert** field in the spec

3. Apply the cluster yaml using the following command:

```
kubectl apply -f encryptionconfig.remote.yaml
```

Once deployed, use the command **kubectl edit encryptionconfig <encryptionconfig-name> -n ibm-spectrum-scale** to modify properties of the resource.

IBM Storage Scale container native supports cache volumes to cache the contents of the remote object storage bucket. For more information, see [Caching data from object storage](#). And a **CacheVolumeOperation** CR is used to trigger download or eviction operation for a cache volume.

CacheVolumeOperation Spec

Table 1. CacheVolumeOperation property and description

| Property | Required | Default | Description |
|---------------|----------|---------|---|
| cacheVolume | Yes | None | The name of the CacheVolume CR. |
| action | Yes | None | The type of the operation to trigger Download or Evict |
| contentType | Yes | None | The type of the contents to consider for an operation, data, metadata or both. The possible values are DataAndMetadata , DataOnly , or MetadataOnly . \n For the action Download , the allowed values are DataAndMetadata and MetadataOnly . \n For the action Evict , the allowed values are DataAndMetadata and DataOnly . |
| contentFilter | No | None | The list of the contents to download from a bucket for a Download operation or the list of the contents to evict from a cache volume for an Eviction operation. \n Skipping the field contentFilter considers all the contents for an operation, and alternatively you can specify a "*" in the contentFilter list to explicitly mention considering all the contents for an operation. |
| ttlDuration | No | 24h | ttlDuration specifies the time duration to wait before automatically deleting a CacheVolumeOperation CR after successful completion. It can be specified in hours (for example 5h) or in days (for example 2d). |

Download

When a CacheVolume CR is created for a cache volume, you can download the contents of an object storage bucket by creating a CacheVolumeOperation CR. For more information on how to create a cache volume, see [Creating cache volumes](#).

The following steps describe creating a sample CacheVolumeOperation CR for a download operation.

1. Download a copy of the sample **cachevolumeoperation.download.yaml** from the GitHub repository.

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/cachevolumeoperation/cachevolumeoperation.download.yaml > cachevolumeoperation.download.yaml || echo "Failed to download CacheVolumeOperation sample CR"
```

1. Edit the sample **cachevolumeoperation.download.yaml** file and change the fields specific to your download operation. For more information about the fields, see [CacheVolumeOperation Spec](#). Make sure that the CacheVolumeOperation CR uses the same namespace as the CacheVolume CR.

To download all the contents of an object storage bucket, remove the **contentFilter** altogether from the sample CacheVolumeOperation CR.

You can choose to specify object names or folder names in the **contentFilter** list for download.

If the cache volume size is less than the data to download from a bucket, the data at the cache volume is overwritten. The data gets cached again on demand when an application accesses it.

1. Apply the resource by using the following command:

```
oc apply -f cachevolumeoperation.download.yaml
```

1. A successful download CacheVolumeOperation looks like this:

```
# kubectl get cvo cvo-sample -n ns1
```

| NAME | ACTION | CACHEVOLUME | STATUS | REASON | AGE |
|------------|----------|--|--------|--------------------|-----|
| cvo-sample | Download | pvc-ald91618-1c9d-4773-92a5-1492f6b0736e | True | DownloadSuccessful | 33s |

Eviction

To make a space available in the cache volume for the necessary data, you can evict the current cache volume contents by creating a CacheVolumeOperation CR.

The following steps describe creating a CacheVolumeOperation CR for an eviction operation.

1. Download a copy of the sample `cachevolumeoperation.evict.yaml` from the GitHub repository.

```
curl -fs https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/cr/cachevolumeoperation/cachevolumeoperation.evict.yaml > cachevolumeoperation.evict.yaml || echo "Failed to download CacheVolumeOperation sample CR"
```

1. Edit the sample `cachevolumeoperation.evict.yaml` file and change the fields specific to your eviction operation. For more information about the fields, see [CacheVolumeOperation Spec](#). Make sure that the CacheVolumeOperation CR uses the same namespace as the CacheVolume CR.

To evict all the contents of a cache volume, remove the `contentFilter` altogether from the sample CacheVolumeOperation CR.

1. Apply the resource by using the following command:

```
oc apply -f cachevolumeoperation.evict.yaml
```

1. A successful eviction CacheVolumeOperation looks like this:

```
# kubectl get cvo cvo-sample -n ns1
```

| NAME | ACTION | CACHEVOLUME | STATUS | REASON | AGE |
|------------|----------|--|--------|--------------------|-----|
| cvo-sample | Eviction | pvc-ald91618-1c9d-4773-92a5-1492f6b0736e | True | EvictionSuccessful | 33s |

Validating installation

The following sections will help validate the installation.

- [Verifying the IBM Storage Scale container native cluster](#)
- [Status and events](#)

Verifying the IBM Storage Scale container native cluster

Verify whether the deployment of the IBM Storage Scale container native cluster is done correctly.

Complete the following steps:

For more information, see [Debugging IBM Storage Scale deployment](#).

1. Verify that the Operator has created the cluster by checking the pods.

```
kubectl get pods -n ibm-spectrum-scale
```

A sample output is shown:

```
# kubectl get pods -n ibm-spectrum-scale
```

| NAME | READY | STATUS | RESTARTS | AGE |
|----------------------------------|-------|---------|----------|-------|
| ibm-spectrum-scale-gui-0 | 4/4 | Running | 0 | 5m45s |
| ibm-spectrum-scale-gui-1 | 4/4 | Running | 0 | 2m9s |
| ibm-spectrum-scale-pmcollector-0 | 2/2 | Running | 0 | 5m15s |
| ibm-spectrum-scale-pmcollector-1 | 2/2 | Running | 0 | 4m11s |
| worker0 | 2/2 | Running | 0 | 5m43s |
| worker1 | 2/2 | Running | 0 | 5m43s |
| worker3 | 2/2 | Running | 0 | 5m45s |

The following list includes considerations about the IBM Storage Scale cluster creation and its pods:

- The cluster takes some time to create.

- One core pod per node gets created on nodes matching the **nodeSelector**.
- Core pods can take several minutes to move to Running status.
- GUI pods do not achieve the Running status until all the core pods are in a Running status.
- Two GUI pods are created, where the second is created after the first is moved to Running status.
- Two pmcollector pods are created, where the second is created after the first is moved to Running status.
- Resulting cluster should have one core pod per node as specified by the **nodeSelector**, two GUI pods, and two pmcollector pods.

2. Verify that the IBM Storage Scale cluster is created correctly:

a) Enter the **mmclscluster** command:

```
kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core \
-ojsonpath="{.items[0].metadata.name}" -n ibm-spectrum-scale) \
-c gpfs -n ibm-spectrum-scale -- mmclscluster
```

The output from the command should show that an IBM Storage Scale cluster has been created, and all nodes as specified by the **nodeSelector** are present.

```
GPFS cluster information
=====
GPFS cluster name:      ibm-spectrum-scale.mycluster.example.com
GPFS cluster id:       835278197609441888
GPFS UID domain:       ibm-spectrum-scale.mycluster.example.com
Remote shell command:  /usr/bin/ssh
Remote file copy command: /usr/bin/scp
Repository type:       CCR
```

| Node Designation | Daemon node name | IP address | Admin node name |
|---|--|--------------|-----------------|
| 1 | worker2.daemon.ibm-spectrum-scale.stg.mycluster.example.com. | 172.29.0.145 | |
| worker2.admin.ibm-spectrum-scale.stg.mycluster.example.com. | quorum-manager-perfmon | | |
| 2 | worker1.daemon.ibm-spectrum-scale.stg.mycluster.example.com. | 172.29.0.146 | |
| worker1.admin.ibm-spectrum-scale.stg.mycluster.example.com. | quorum-manager-perfmon | | |
| 3 | worker3.daemon.ibm-spectrum-scale.stg.mycluster.example.com. | 172.29.0.148 | |
| worker3.admin.ibm-spectrum-scale.stg.mycluster.example.com. | quorum-manager-perfmon | | |

b) Enter the **mmgetstate** command:

```
kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core \
-ojsonpath="{.items[0].metadata.name}" -n ibm-spectrum-scale) \
-c gpfs -n ibm-spectrum-scale -- mmgetstate -a
```

The output from the command should show that the **GPFS state** for all nodes are listed as **active**.

| Node number | Node name | GPFS state |
|-------------|-----------|------------|
| 1 | worker0 | active |
| 2 | worker1 | active |
| 3 | worker3 | active |

3. Verify that the Remote Cluster authentication is successfully created.

a. Get a list of the remote clusters.

```
kubectl get remotecluster.scale -n ibm-spectrum-scale
```

b. Inspect the remote clusters and ensure that the value for **READY** is **True**.

Example:

```
# kubectl get remotecluster.scale -n ibm-spectrum-scale
NAME                READY  AGE
remotecluster-sample True   30h
```

4. Verify that the storage cluster file system is configured:

a. Get a list of the file systems:

```
kubectl get filesystem.scale -n ibm-spectrum-scale
```

b. Inspect the file systems and ensure that the value for **ESTABLISHED** is **True**.

```
$ kubectl get filesystem.scale -n ibm-spectrum-scale
NAME                ESTABLISHED  AGE
remote-sample       True         30h
```

5. Manually verify that the file system is mounted using the **mm1smount** command.

```
kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core \
-ojsonpath="{.items[0].metadata.name}" -n ibm-spectrum-scale) \
-c gpfs -n ibm-spectrum-scale -- mmlsmount remote-sample -L
```

Example output:

```
File system remote-sample (gpfs1.local:fs1) is mounted on ...
...
172.29.0.148 worker3.daemon.ibm-spectrum-scale.stg.mycluster.example.com. ibm-spectrum-
scale.mycluster.example.com
172.29.0.146 worker1.daemon.ibm-spectrum-scale.stg.mycluster.example.com. ibm-spectrum-
scale.mycluster.example.com
172.29.0.145 worker2.daemon.ibm-spectrum-scale.stg.mycluster.example.com. ibm-spectrum-
scale.mycluster.example.com
```

6. Verify that there are no problems reported in the operator status and events. For more information, see [Status and events](#).

7. Verify that the CSI pods are up and running.

```
kubectl get pods -n ibm-spectrum-scale-csi
```

8. Verify that the Core DNS pods are up and running. There will be at least one Core DNS pod per core pod.

```
kubectl get pods -n ibm-spectrum-scale-dns
```

Status and events

The custom resource (CR) objects contain helpful information which can be retrieved by entering the `kubectl describe` command. For each object, a `Status` attribute provides the last observed state of the resource. In the retrieved information, a log of recent `Events` pertaining to the resource is also shown. This information can be helpful to check the desired state of the resource or when debugging with the IBM Storage Scale container native cluster. For more information, see [Application Introspection and Debugging](#) in Kubernetes documentation.

The `kubectl describe <CR> -n ibm-spectrum-scale` command is used to view the `status` and `events` of the custom resources, such as `cluster`, `daemon`, `filesystem`, `remoteclass`, `callhome` and others.

The `Status` can be seen in the `Conditions` section:

```
$ kubectl describe callhome.scale -n ibm-spectrum-scale
...
Status:
  Conditions:
    Last Transition Time: 2021-08-31T12:54:05Z
    Message: Callhome is enabled.
    Reason: Enabled
    Status: True
    Type: Enabled
    Last Transition Time: 2021-08-31T12:54:07Z
    Message: Successfully tested connection to the IBM Callhome Server.
    Reason: TestPassed
    Status: True
    Type: Success
    Mode: test
...
```

A `Condition` has the following fields:

- `Type`: Type of condition.
- `Status`: Status of the condition, one of `True`, `False` or `Unknown`.
- `Reason`: The reason contains a programmatic identifier indicating the reason for the condition's last transition.
- `Message`: Message is a human readable message indicating details about the transition.
- `Last Transition Time`: This is the last time the condition transitioned from one status to another (For example, from `False` to `True`).

The `Events` section of `kubectl describe` output lists the `Events`:

```
$ kubectl describe callhome.scale -n ibm-spectrum-scale
...
Events:
  Type    Reason      Age   From    Message
  ----    -
  Normal  NodeUpdate  44m   Callhome  Callhome was enabled on 0 nodes before, but now it's enabled on all 5
nodes.
  Normal  Configured  44m   Callhome  Successfully updated callhome configuration. Customer=IBM,
CustomerID=123456, Email=sroth@de.ibm.de, Country=DE, Type=test
  Normal  Enabled     44m   Callhome  Callhome has been enabled.
```

Enter the `kubectl get crd | grep ibm` command to see a full list of CRs that can be checked for status and events with the `kubectl describe` command.

- The *Events* disappear after they are created.
- The *Status* and *Events* shown in the documentation are examples and may look slightly different on your system.

Using IBM Storage Scale GUI

The following section describes how to utilize the IBM Storage Scale container native cluster GUI.

- [GUI Access](#)
- [GUI User Management](#)

GUI Access

For Red Hat OpenShift cluster, complete the following steps to access the IBM Storage Scale GUI:

1. Run the following command to get the URL for the GUI.

```
echo -e "https://`kubectl get route -n ibm-spectrum-scale ibm-spectrum-scale-gui -ojson | jq -r .spec.host`"
```

Example:

```
$ echo -e "https://`kubectl get route -n ibm-spectrum-scale ibm-spectrum-scale-gui -ojson | jq -r .spec.host`"  
https://ibm-spectrum-scale-gui-ibm-spectrum-scale.apps.example.ibm.com
```

You can open the IBM Storage Scale GUI in your browser by using this URL.

2. Click **Sign in** and you are redirected to the Red Hat OpenShift log-in page.
3. Authenticate by using your OpenShift Container Platform user credentials. On success, you are redirected to the IBM Storage Scale GUI.

GUI User Management

Red Hat OpenShift roles are mapped to two IBM Storage Scale GUI user groups. The following table provides the role and privilege mapping:

Table 1. Roles and privileges

| | Roles | | | Privileges | |
|-----------------------------------|-------------|------|----------------------------|----------------------------|-------------------------------|
| OpenShift Container Platform role | GUI role | View | Download snap ¹ | Manage events ² | Test connection for call home |
| cluster-admin | Maintenance | Yes | Yes | Yes | Yes |
| ibm-spectrum-scale-maintenance | Maintenance | Yes | Yes | Yes | Yes |
| ibm-spectrum-scale-monitor | Monitor | Yes | No | No | No |

Add a role to a user or a group

Users who are created on the Red Hat OpenShift Container Platform having a role of `ibm-spectrum-scale-maintenance` or `ibm-spectrum-scale-monitor` can log in to the IBM Storage Scale container native GUI through Single Sign On using the OAuth implementation.

```
oc adm policy add-cluster-role-to-user <role> <user>  
oc adm policy add-cluster-role-to-group <role> <group>
```

1. Ability to download master and non-master snaps.[↗](#)
2. Ability to mark events as resolved, hiding resolved tips and notifications.[↗](#)

Upgrading IBM Storage Scale container native

When upgrading an IBM Storage Scale container native cluster that is less than v5.1.5.0, first upgrade to v5.1.5.0. For more information, see [Supported upgrade paths](#).

IBM Storage Scale container native upgrade consists of the following topics:

- [Supported upgrade paths](#)
- [Upgrade considerations for CSI volumes](#)
- [Upgrade](#)
- [Verification](#)
- [Post upgrade tasks](#)

Supported upgrade paths

Use the following table to understand the supported upgrade paths for IBM Storage Scale container native.

| Upgrade from | To: 5.1.9.x | To: 5.2.0.x | To: 5.2.1.x | To: 5.2.2.x | To: 5.2.3.x | To: 6.0.0.x | |
|--------------|-------------|-------------|-------------|-------------|-------------|-------------|--|
| 5.1.9.x | Yes | Yes | No | No | No | No | |
| 5.2.0.x | - | Yes | Yes | Yes | No | No | |
| 5.2.1.x | - | - | Yes | Yes | Yes | No | |
| 5.2.2.x | - | - | - | Yes | Yes | Yes | |
| 5.2.3.x | - | - | - | - | Yes | Yes | |
| 6.0.0.x | - | - | - | - | - | Yes | |

When upgrading an IBM Storage Scale container native cluster that is less than v5.1.5.0, first upgrade to v5.1.5.0.

Major version upgrades that span multiple release levels may not be supported due to overlapping compatibility constraints with Red Hat OpenShift Container Platform (OCP) versions. Refer to the Software Requirements section corresponding to your cluster's operating system and platform for detailed compatibility guidance.

Upgrade Considerations for CSI Volumes

In releases prior to v6.0.0.0, the IBM Storage Scale Container Storage Interface driver used to create PVCs under primary filesystem by referring CSI Custom Resource (CR)

Existing Behaviour of the primary filesystem

Earlier to v6.0.0 the primary filesystem and fileset was used to create soft links that points to the actual volume directories.

As a result, the primary filesystem and fileset was required to remain present, even if the persistent volumes themselves belonged to other filesystems.

Disabling the use of primary filesystem

Starting with v6.0.0.0, dependency on the primary filesystem has been removed as an enhancement feature.

New volumes created on v6.0.0 are placed directly in the target filesystem, without soft links in the primary fileset.

This simplifies the storage layout and eliminates the need to maintain a dedicated primary filesystem and fileset for new workloads.

Migration of Existing Volumes

For volumes created before v6.0.0.0, migration steps are available to move them to the new storage layout.

Migration of volumes is optional

1. If volumes are not migrated, they will continue to function but the primary filesystem and fileset must remain available.
2. If volumes are migrated, the dependency on the primary filesystem and fileset is eliminated.

Prerequisites for migration of volumes

Before running the migration script, ensure the following prerequisites are met

1. Delete any existing workloads or application pods attached to the PVCs that will be migrated

Do not delete associated PVCs or PVs. {: .attention}

2. The script should be run in the context of the cluster where IBM Storage Scale container native is deployed
3. Install jq for JSON parsing and manipulation of Kubernetes API responses

Use the following command to verify installation

```
jq --version
```

Migration Script Usage

This script helps migrate existing PersistentVolumes (PVs) in a IBM Storage Scale container native environment that were originally created when the primary filesystem & fileset was used. It updates PVs to use the actual fileset mount path on IBM Storage Scale container native worker nodes after the primary filesystem has been removed, ensuring workloads continue to access their data seamlessly.

Refer the migration guide to get the details of the migration script, see [PV Migration Script – IBM Storage Scale container native \(Primary filesystem → Actual Fileset Mount\)](#) for details. {: external}

1. Use the below command to download a copy of the migration script

```
curl -L -O https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-csi/refs/heads/release-3.0/tools/volume_migration_scripts/migration_cnsa_primary_removal.bash || echo "Failed to download migration script"
```

2. Execute the following command to start migration:

```
./migration_cnsa_primary_removal.bash --new_path_prefix </var/mnt>
```

--new_path_prefix specifies the base mount point for all IBM Storage Scale filesystems on CNSA worker nodes (e.g., /var/mnt, /mnt).

3. Validate that the migration is successful. The migration summary (shown once the migration script finishes) should have only successful or skipped PVs as below. There shouldn't be any Failed PVs in the summary.

```
Migration Summary:
-----
Successful PVs: 1
PVC Name | PV Name
-----
ibm-spectrum-scale-pvc-clone-from-pvc-advanced | pvc-7981775f-08b1-4a53-ae4d-6740e2ec9a89
Skipped PVs (already migrated): 2
PVC Name | PV Name
-----
scale-advance-pvc | pvc-dd8e015c-382c-4487-a215-91dd22a01d45
ibm-spectrum-scale-pvc-advance-from-snapshot | pvc-e920921e-7a25-4017-9c8d-6469750a4772
```

1. Once migration completes, all PVs and PVCs will be ready. Re-deploy application/workload pods, which will seamlessly attach to the migrated PVCs.

Upgrade

The following section describes how to upgrade the IBM Storage Scale container native cluster.

While an upgrade is in progress, do not perform the following:

- Do not make changes to the Cluster custom resource.
- Do not attempt to add a node to the cluster.
- Do not run other upgrades in parallel, like upgrading the Kubernetes nodes.

During an upgrade, the IBM Storage Scale operator orchestrates the upgrade procedure. Each node is:

- Cordoned (tainted unschedulable)
- Drained (pods are safely evicted and rescheduled to other available nodes)
- Rebooted, if necessary

- Uncordoned (returning it to normal service)

After the node is schedulable, IBM Storage Scale container native and IBM Storage Scale Container Storage Interface (CSI) pods will start. Applications may fail to attach storage until the system is started.

Parallel upgrade

By default the core pods are updated in a rolling node-by-node fashion. Therefore, in a cluster with many nodes the upgrade takes a long time.

To speed up, multiple core pods can be upgraded in parallel. For more information, see [Update](#) and [Parallel update recommendations](#).

Pause upgrade

An ongoing upgrade can be temporary paused by using the **paused** parameter in the Cluster custom resource. For more information, see [Update](#) or run `kubectl explain cluster.spec.daemon.update.paused` and `kubectl explain cluster.spec.daemon.update.pools.paused`.

Prerequisites

- All the core pods need to be in running status.

Use the following command to check the status of the core pods:

```
kubectl get daemons ibm-spectrum-scale -n ibm-spectrum-scale -ojson | jq -r '.status.podsStatus'
```

Help ensure that there are no pods in any of the following states:

- starting
- terminating
- unknown
- waitingForDelete

In the following example, the output shows 1 pod in "waitingForDelete", so the upgrade should **not** be done at this time.

```
$ kubectl get daemons ibm-spectrum-scale -n ibm-spectrum-scale -ojson | jq -r '.status.podsStatus'
{
  "running": "4",
  "starting": "0",
  "terminating": "0",
  "unknown": "0",
  "waitingForDelete": "1"
}
```

Apply the upgrade

If you have an air-gapped installation that is deployed, mirror the images to your internal Container Registry before proceeding. For the list of images, see the [Air gapped](#) section in the GitHub repository.

Red Hat OpenShift

1. The imagePullSecret **ibm-spectrum-scale-registrykey** in the service accounts have been deprecated. Delete the service accounts before applying the **install.yaml** as they will be created again.

```
for namespace in ibm-spectrum-scale ibm-spectrum-scale-operator ibm-spectrum-scale-dns; do
  kubectl delete sa --all -n ${namespace}
done
```

2. Configure namespace pull secrets.

Starting with Red Hat OpenShift 4.15 (Kubernetes 1.28), warning messages appear when secrets are referenced in a service account but are not created in the namespace. Configuring ICR entitlement using the Red Hat OpenShift global pull secret results in messages appearing in the events: [Unable to retrieve some image pull secrets](#).

For instructions on creating the namespace pull secrets, see [Namespace Pull Secrets](#).

For major version upgrades, consider taking this opportunity to switch to namespace pull secrets before initializing the upgrade, allowing the operator to orchestrate all pod restarts.

After namespace secrets are created, the pods need to restart for the warning messages to quiesce. For core pods only, use the following annotation to have the operator orchestrate the pod restarts, preserving quorum: `kubectl annotate pod -lapp.kubernetes.io/name=core scale.spectrum.ibm.com/pending=delete`. For other pods, you must delete them manually.

```
for name in gui pmcollector; do
  oc delete pod -l app.kubernetes.io/name=${name} -n ibm-spectrum-scale
done

for namespace in ibm-spectrum-scale-csi ibm-spectrum-scale-dns ibm-spectrum-scale-operator; do
  oc delete pod --all -n ${namespace}
done
```

3. If you did not change the operator memory limit for cache volumes, as mentioned in [Increasing the operator memory limit](#), apply the new manifests as they are.

```
kubectl apply -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/install.yaml
```

If you changed the operator memory limit for cache volumes, follow the instructions in [Updating the operator memory limit and applying manifests](#).

Kubernetes or Google Kubernetes Engine

If you did not change the operator memory limit for cache volumes, as mentioned in [Increasing the operator memory limit](#).

1. Apply the mmbuildgpl configmap in case changes for support of new kernels are needed. For more information, see [buildgpl configmap](#).

- For Red Hat Kubernetes environments:

```
kubectl create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/redhat/buildgpl.yaml
```

If environment is using the NVIDIA network operator to deploy MOFED pods, use the following custom **buildgpl**:

```
kubectl create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/redhat/buildgpl.nvidia.yaml
```

- For Ubuntu Kubernetes environments:

```
kubectl create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/ubuntu/buildgpl.yaml
```

If environment is using the NVIDIA network operator to deploy MOFED pods, use the following custom **buildgpl**:

```
kubectl create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/ubuntu/buildgpl.nvidia.yaml
```

2. Apply the new manifests as they are.

```
kubectl apply -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/install-k8s.yaml
```

If you changed the operator memory limit for cache volumes, follow the instructions in [Updating the operator memory limit and applying manifests](#).

Updating the operator memory limit and applying manifests

If you changed the operator memory limit for cache volumes, as mentioned in [Increasing the operator memory limit](#), download the new manifests and update the operator memory limit to the required value.

1. Get the current operator memory limit with the following command:

```
kubectl get deployment ibm-spectrum-scale-controller-manager -n ibm-spectrum-scale-operator -o jsonpath="{.spec.template.spec.containers[0].resources.limits.memory}"
```

2. Apply the mmbuildgpl configmap in case changes for support of new kernels are needed. For more information, see [buildgpl configmap](#).

- For Red Hat Kubernetes environments:

```
kubectl create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/redhat/buildgpl.yaml
```

If environment is using the NVIDIA network operator to deploy MOFED pods, use the following custom **buildgpl**:

```
kubectl create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/redhat/buildgpl.nvidia.yaml
```

- For Ubuntu Kubernetes environments:

```
kubectl create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/ubuntu/buildgpl.yaml
```

If environment is using the NVIDIA network operator to deploy MOFED pods, use the following custom **buildgpl**:

```
kubectl create -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/ubuntu/buildgpl.nvidia.yaml
```

3. Download the new manifests.

- For Red Hat OpenShift:

```
wget https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/install.yaml
```

- For Kubernetes or Google Kubernetes Engine:

```
wget https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-native/v6.0.0.x/generated/scale/install-k8s.yaml -O install.yaml
```

4. In the downloaded manifests, update the **resources.limits.memory** to the required memory limit in the **containers** section of operator deployment **ibm-spectrum-scale-controller-manager**.

5. Apply the updated manifests.

```
kubectl apply -f install.yaml
```

Verification

After deploying new installer manifest, the IBM Storage Scale container native operator starts to drive the upgrade process and will take some time to complete as the new images are rolled out to the cluster.

Validate the **app.kubernetes.io/version** on the operator deployment matches the new level:

```
kubectl get deployment ibm-spectrum-scale-controller-manager \
-n ibm-spectrum-scale-operator -ojson | jq -r .metadata.labels
```

To check the progress of the pod restarts and node reboots, query the daemon CR using the following command:

```
kubectl describe daemon ibm-spectrum-scale -n ibm-spectrum-scale
```

Information is available under the "Status Details" and "Events" sections.

Code version updated

The version details are listed under **.status.versions** in the Daemon CR and is updated as the pods roll. The following command shows the versions that core pods currently have on them. Wait until all the pods are reporting the same new version.

```
kubectl get daemon ibm-spectrum-scale -n ibm-spectrum-scale -ojson | jq -r .status.versions
```

After the versions are all reporting the same, proceed to the [Post upgrade tasks](#) section of the documentation.

Post upgrade tasks

The following section lists optional tasks for the administrator to consider when upgrading the IBM Storage Scale container native product to the new code levels.

Enable Network Policies (Red Hat Openshift cluster ONLY)

Network Policies is not supported on Kubernetes and GKE cluster

Starting at version v5.1.9, the operator can create network policies rules across the managed namespaces. If upgrading from an earlier version, the network policy configuration is not modified on upgrade. Use the following command to check the network policy configuration:

```
kubectl get cluster ibm-spectrum-scale -ojson | jq -r .spec.networkPolicy
```

If the response is **{}**, network policy is enabled, you are done.

If the response is **null** and you want to enable the operator to create network policy rules, patch the cluster by using the following command:

```
kubectl patch cluster.scale.spectrum.ibm.com ibm-spectrum-scale --type='json' \
-p='[{"op": "add", "path": "/spec/networkPolicy", "value": {}}]'
```

For more information, see [Network policies](#).

Approve the new cluster release level

It is suggested to first use the IBM Storage Scale container native cluster with the upgraded code before permanently setting the cluster to the new level.

For more information, see [File system format changes between versions of IBM Storage Scale](#) in the IBM Storage Scale documentation.

When a release level change is detected after an upgrade, the operator will create an **UpgradeApproval** resource. When you are ready to enable the new functions and lock in the new level, complete the following steps:

Upgrades within the .x version of a release typically do not have an upgrade approval.

1. Check to see whether any cluster upgrade approvals are present that require action.

An upgrade approval that shows nothing under the **COMPLETED** field are ones that require some action.

```
kubectl get upgradeapproval -n ibm-spectrum-scale
```

If an upgrade approval does not appear, check the Daemon CR status to validate that all pods are on the new version by using: `kubectl get daemon -n ibm-spectrum-scale -ojson | jq -r .items[].status.versions`

2. Check the `minReleaseLevel` of the cluster:

```
kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core -ojsonpath="{.items[0].metadata.name}" -n ibm-spectrum-scale) -c gpfs -n ibm-spectrum-scale -- mmlsconfig release
```

3. To approve the upgrade approval job, run the following command:

```
kubectl patch upgradeapproval <upgradeapproval-name> -n ibm-spectrum-scale --type='json' -p='[{"op": "replace", "path": "/spec/approved", "value": true}]'
```

Full Cluster Example:

```
# Check for any upgrade approvals for TYPE=cluster
$ kubectl get upgradeapproval -n ibm-spectrum-scale
NAME          TYPE      FILESYSTEM  LAST SCHEDULE TIME  LAST SUCCESSFUL TIME  RUNNING  COMPLETED
upgrade-rmlp4  cluster

# check the daemon status for the versions deployed on each core pod
$ kubectl get daemon -n ibm-spectrum-scale -ojson | jq .items[].status.versions
[
  {
    "count": "3",
    "version": "5.2.3.0"
  }
]

# check the current cluster release version
$ kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core -ojsonpath="{.items[0].metadata.name}" -n ibm-spectrum-scale) -c gpfs -n ibm-spectrum-scale -- mmlsconfig release
minReleaseLevel 5.2.3.0

# patch the upgrade approval
$ kubectl patch upgradeapproval upgrade-rmlp4 -n ibm-spectrum-scale \
> --type='json' -p='[{"op": "replace", "path": "/spec/approved", "value": true}]'
upgradeapproval.scale.spectrum.ibm.com/upgrade-rmlp4 patched

# query the upgrade approval to see it running
$ kubectl get upgradeapproval -n ibm-spectrum-scale
NAME          TYPE      FILESYSTEM  LAST SCHEDULE TIME  LAST SUCCESSFUL TIME  RUNNING
COMPLETED
upgrade-rmlp4  cluster          23s                               ibm-spectrum-
scale/worker2/gpfs/upgradeCluster_p6Jhz9

# upgrade approval job completed
$ kubectl get upgradeapproval -n ibm-spectrum-scale
NAME          TYPE      FILESYSTEM  LAST SCHEDULE TIME  LAST SUCCESSFUL TIME  RUNNING  COMPLETED
upgrade-rmlp4  cluster          38s                               6s                               Successful

# verify that the cluster release level has been updated
$ kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core -ojsonpath="{.items[0].metadata.name}" -n ibm-spectrum-scale) -c gpfs -n ibm-spectrum-scale -- mmlsconfig release
minReleaseLevel 6.0.0.0
```

For more information, see [Completing the upgrade to a new level of IBM Storage Scale](#) in IBM Storage Scale documentation.

Approve the new local filesystem version

It is suggested to first use the IBM Storage Scale container native filesystem version with the upgraded code before permanently setting the filesystem to the new version.

For more information, see [File system version changes between versions of IBM Storage Scale](#) in the IBM Storage Scale documentation.

When a release level change is detected after an upgrade, the operator will create an **UpgradeApproval** resource. When you are ready to enable the new functions and lock in the new level, complete the following steps:

Upgrades within the .x version of a release typically do not have an upgrade approval.

1. Check to see whether any filesystem upgrade approvals are present that require action.

An upgrade approval that shows nothing under the **COMPLETED** field are ones that require some action.

```
kubectl get upgradeapproval -n ibm-spectrum-scale
```

If a filesystem upgrade approval does not appear, the cluster upgrade approval must be done first: [Approve the new cluster release level](#)

2. Check the **value** of the filesystem:

```
kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core -ojsonpath="{.items[0].metadata.name}" -n ibm-spectrum-scale) -c gpfs -n ibm-spectrum-scale -- mmlsfs local-sample -V
```

3. To approve the upgrade approval job, run the following command:

```
kubectl patch upgradeapproval <upgradeapproval-name> -n ibm-spectrum-scale --type='json' -p='[{"op": "replace", "path": "/spec/approved", "value": true}]'
```

Full Filesystem Example:

```
# Check for any upgrade approvals for TYPE=filesystem
$ kubectl get upgradeapproval -n ibm-spectrum-scale
```

| NAME | TYPE | FILESYSTEM | LAST SCHEDULE TIME | LAST SUCCESSFUL TIME | RUNNING | COMPLETED |
|---------------|------------|--------------|--------------------|----------------------|---------|------------|
| upgrade-4xck4 | filesystem | local-sample | | | | |
| upgrade-dtqlz | cluster | | 46s | 15s | | Successful |

```
# check the current filesystem version
$ kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core -ojsonpath="{.items[0].metadata.name}" -n ibm-spectrum-scale) -c gpfs -n ibm-spectrum-scale -- mmlsfs local-sample -V
```

| flag | value | description |
|------|-----------------|---------------------|
| -V | 37.00 (5.2.3.0) | File system version |

```
# patch the upgrade approval
$ kubectl patch upgradeapproval upgrade-4xck4 -n ibm-spectrum-scale --type='json' -p='[{"op": "replace", "path": "/spec/approved", "value": true}]'
upgradeapproval.scale.spectrum.ibm.com/upgrade-4xck4 patched

# query the upgrade approval to see it running
$ kubectl get upgradeapproval -n ibm-spectrum-scale
```

| NAME | TYPE | FILESYSTEM | LAST SCHEDULE TIME | LAST SUCCESSFUL TIME | RUNNING |
|---|------------|--------------|--------------------|----------------------|---------------|
| COMPLETED | | | | | |
| upgrade-4xck4 | filesystem | local-sample | 10s | | ibm-spectrum- |
| scale/worker0/gpfs/upgradeFilesystem_QrMRbW | | | | | |
| upgrade-dtqlz | cluster | | 7m42s | 1m11s | |
| Successful | | | | | |

```
# upgrade approval job completed
$ kubectl get upgradeapproval -n ibm-spectrum-scale
```

| NAME | TYPE | FILESYSTEM | LAST SCHEDULE TIME | LAST SUCCESSFUL TIME | RUNNING | COMPLETED |
|---------------|------------|--------------|--------------------|----------------------|---------|------------|
| upgrade-4xck4 | filesystem | local-sample | 31s | 0s | | Successful |
| upgrade-dtqlz | cluster | | 8m3s | 1m32s | | Successful |

```
# verify that the filesystem version has been updated
$ kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core -ojsonpath="{.items[0].metadata.name}" -n ibm-spectrum-scale) -c gpfs -n ibm-spectrum-scale -- mmlsfs local-sample -V
```

| flag | value | description |
|------|-----------------|------------------------------|
| -V | 38.00 (6.0.0.0) | Current file system version |
| | 37.00 (5.2.3.0) | Original file system version |

For more information, see [Completing the upgrade to a new level of IBM Storage Scale](#) in IBM Storage Scale documentation.

Remote storage cluster considerations

If the storage cluster is being mounted by other GPFS client clusters that are running a prior version of code, performing the following step results in these client clusters being unable to mount file systems from this storage cluster. See [Software requirements](#) for minimum storage cluster level support.

The storage cluster supports earlier versions than IBM Storage Scale container native client cluster. CSI functions are highly dependent upon the release, file system level, and version of IBM Storage Scale installed on the storage cluster. If the storage cluster is at an earlier version, some functions may not be available. For more information about the minimum levels required for specific CSI functions, see [Hardware and Software Requirements](#) in the IBM Storage Scale CSI documentation. For more information about compatibility and software matrix, see [Section 18.3](#) in the IBM Storage Scale FAQ documentation.

1. Run the following command for each file system to upgrade to the latest metadata format available on the storage cluster:

This step is optional but recommended for enabling the functions that are provided at the latest levels of code.

```
mmchfs <Filesystem> -V full
```

Auto inode limit

1. Enable `auto-inode-limit` of the file system.

The `--auto-inode-limit` option is available only at file system format level of 28.00 or later. Enable this option when the file system is updated to 28.00 or later.

```
mmchfs <Filesystem> --auto-inode-limit
```

Remove deprecated resources

The following section provides steps to clean up deprecated resources.

Run the following commands to clean up deprecated resources, if they exist.

```
kubectl delete ClusterRoleBinding ibm-spectrum-scale-dns
kubectl delete ClusterRole ibm-spectrum-scale-dns
```

Remove deprecated CSI user secrets

The CSI user secrets are no longer needed if the storage clusters are running IBM Storage Scale 5.2.3.0 or newer.

Run the following steps to delete the deprecated CSI user secrets if the storage clusters are running IBM Storage Scale 5.2.3.0 or newer:

1. To list the names of CSI user secrets, run this command:

```
kubectl get remoteccluster -n ibm-spectrum-scale -o json | grep csiSecretName
```

2. Delete the listed secrets in the `ibm-spectrum-scale-csi` namespace, for example:

```
kubectl delete secret csi-remote-mount-storage-cluster-1 -n ibm-spectrum-scale-csi
```

3. Delete the `csiSecretName` parameter from the spec of the RemoteCluster custom resources.

```
kubectl edit remoteccluster -n ibm-spectrum-scale
```

For more information, see [Storage cluster v5.2.3.0 and newer](#).

Upgrading Google Kubernetes Engine (GKE)

IBM Storage Scale container native deployed in a Google Kubernetes Engine (GKE) is available as a technology preview. For more information, see [Support for Google Kubernetes Engine \(GKE\)](#).

Before the GKE nodes can be upgraded, the IBM Storage Scale container native configuration must be prepared. To support GKE upgrades, IBM Storage Scale container native must be deployed in a cluster with at least two GKE node pools. To upgrade GKE, the nodes of the first GKE node pool are upgraded before the next GKE node pool is upgraded. For more information, see [Prepare for Google Kubernetes Engine \(GKE\) upgrade](#). Avoid running CNSA and GKE upgrades at the same time to maintain cluster availability.

The following steps are performed to upgrade the nodes in a sample GKE cluster with node pools `default-pool` and `second-pool`:

1. Verify whether IBM Storage Scale container native is **active** on all nodes:

```
kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core \
-ojsonpath="{.items[0].metadata.name}" -n ibm-spectrum-scale) \
-c gpfs -n ibm-spectrum-scale -- mmgetstate -a
```

The output from the command must show that the **GPFS state** for all nodes are listed as **active**.

| Node number | Node name | GPFS state |
|-------------|-------------------------------------|------------|
| 1 | gke-test-default-pool-7123b6b0-tsv8 | active |

| | | |
|----|-------------------------------------|--------|
| 2 | gke-test-second-pool-f25776e0-k8hs | active |
| 3 | gke-test-second-pool-f25776e0-z6rj | active |
| 4 | gke-test-default-pool-7123b6b0-0g6m | active |
| 5 | gke-test-default-pool-7123b6b0-n8p3 | active |
| 6 | gke-test-default-pool-7123b6b0-7p7j | active |
| 7 | gke-test-second-pool-f25776e0-pxbg | active |
| 8 | gke-test-second-pool-f25776e0-gjlg | active |
| 9 | gke-test-second-pool-f25776e0-q063 | active |
| 10 | gke-test-second-pool-f25776e0-0qrg | active |
| 11 | gke-test-second-pool-f25776e0-q8cs | active |
| 12 | gke-test-default-pool-7123b6b0-6bp1 | active |
| 13 | gke-test-default-pool-7123b6b0-bccw | active |
| 14 | gke-test-default-pool-7123b6b0-c269 | active |
| 15 | gke-test-default-pool-7123b6b0-nfqd | active |
| 16 | gke-test-second-pool-f25776e0-18s7 | active |
| 17 | gke-test-second-pool-f25776e0-cn7 | active |
| 18 | gke-test-second-pool-f25776e0-182g | active |
| 19 | gke-test-default-pool-7123b6b0-1278 | active |
| 20 | gke-test-default-pool-7123b6b0-t4q0 | active |

- The `default-pool` node pool is upgraded first. The `scale.spectrum.ibm.com/designation=quorum` label from nodes in `default-pool` node pool must be moved to nodes in `second-pool`. The overall number of labeled nodes must stay equal.

Query the total number of quorum nodes:

```
kubectl get daemon ibm-spectrum-scale -n ibm-spectrum-scale -ojson | jq -r '.status.quorumPods.total'
```

Remember this **overall number of quorum nodes**, it is needed in later steps. Let's assume that we have 7 quorum nodes in this example.

- Remove the `scale.spectrum.ibm.com/designation=quorum` label from nodes in the `default-pool`:

```
kubectl label node -l cloud.google.com/gke-nodepool=default-pool scale.spectrum.ibm.com/designation-
```

- Query the nodes with the `scale.spectrum.ibm.com/designation=quorum` label from the `second-pool`:

```
kubectl get node -l cloud.google.com/gke-nodepool=second-pool, scale.spectrum.ibm.com/designation=quorum
```

The output lists the quorum nodes of the `second-pool` node pool.

| NAME | STATUS | ROLES | AGE | VERSION |
|------------------------------------|--------|--------|-----|----------------------|
| gke-test-second-pool-f25776e0-k8hs | Ready | <none> | 21d | v1.31.10-gke.1067000 |
| gke-test-second-pool-f25776e0-z6rj | Ready | <none> | 21d | v1.31.10-gke.1067000 |
| gke-test-second-pool-f25776e0-pxbg | Ready | <none> | 21d | v1.31.10-gke.1067000 |

- Label extra nodes of `second-pool` with `scale.spectrum.ibm.com/designation=quorum` until the number of nodes with `scale.spectrum.ibm.com/designation=quorum` label is the same as remembered in **overall number of quorum nodes** in a prior step.

In this example it results in 7 nodes of `second-pool` are labeled with `scale.spectrum.ibm.com/designation=quorum`:

```
# kubectl get node -l cloud.google.com/gke-nodepool=second-pool, scale.spectrum.ibm.com/designation=quorum
NAME                                STATUS    ROLES    AGE    VERSION
gke-test-second-pool-f25776e0-0qrg Ready     <none>   49m    v1.31.10-gke.1067000
gke-test-second-pool-f25776e0-18s7 Ready     <none>   49m    v1.31.10-gke.1067000
gke-test-second-pool-f25776e0-cn7  Ready     <none>   49m    v1.31.10-gke.1067000
gke-test-second-pool-f25776e0-gjlg Ready     <none>   49m    v1.31.10-gke.1067000
gke-test-second-pool-f25776e0-k8hs Ready     <none>   49m    v1.31.10-gke.1067000
gke-test-second-pool-f25776e0-pxbg Ready     <none>   49m    v1.31.10-gke.1067000
gke-test-second-pool-f25776e0-z6rj Ready     <none>   49m    v1.31.10-gke.1067000
```

- Verify whether quorum was successfully assigned to labeled nodes of `second-pool`.

```
kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core \
-ojsonpath="{.items[0].metadata.name}" -n ibm-spectrum-scale) -c gpfs \
-n ibm-spectrum-scale -- mmlscluster | grep quorum
```

The result lists the same nodes that are labeled:

```
2  gke-test-second-pool-f25776e0-k8hs.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.109
gke-test-second-pool-f25776e0-k8hs.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-perfmon
3  gke-test-second-pool-f25776e0-z6rj.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.106
gke-test-second-pool-f25776e0-z6rj.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-perfmon
7  gke-test-second-pool-f25776e0-pxbg.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.108
gke-test-second-pool-f25776e0-pxbg.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-perfmon
8  gke-test-second-pool-f25776e0-gjlg.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.32
gke-test-second-pool-f25776e0-gjlg.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-perfmon
10 gke-test-second-pool-f25776e0-0qrg.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.112
gke-test-second-pool-f25776e0-0qrg.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-perfmon
16 gke-test-second-pool-f25776e0-18s7.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.110
```

```
gke-test-second-pool-f25776e0-18s7.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-perfmon
17 gke-test-second-pool-f25776e0-cnx7.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.107
gke-test-second-pool-f25776e0-cnx7.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-perfmon
```

7. Perform GKE upgrade of node pool **default-pool**.

8. After the GKE upgrade is completed, it takes a while until the core pods are running on all nodes of **default-pool**. Wait until an IBM Storage Scale container native core pod is running on all nodes of node pool **default-pool**:

```
for node in $(kubectl get nodes -l cloud.google.com/gke-nodepool=default-pool \
-ojsonpath='{.items[*].metadata.name}'); do kubectl get pod -lapp.kubernetes.io/name=core \
-n ibm-spectrum-scale --field-selector spec.nodeName=$node; done
```

The output from the command must show that the **STATUS** for all pods are listed as **Running**.

| NAME | READY | STATUS | RESTARTS | AGE |
|-------------------------------------|-------|---------|----------|-----|
| gke-test-default-pool-7123b6b0-0g6m | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-default-pool-7123b6b0-1278 | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-default-pool-7123b6b0-6bp1 | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-default-pool-7123b6b0-7p7j | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-default-pool-7123b6b0-bccw | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-default-pool-7123b6b0-c269 | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-default-pool-7123b6b0-n8p3 | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-default-pool-7123b6b0-nfqd | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-default-pool-7123b6b0-t4q0 | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-default-pool-7123b6b0-tsv8 | 2/2 | Running | 0 | 21d |

9. Verify whether IBM Storage Scale container native is **active** on all nodes:

```
kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core \
-ojsonpath='{.items[0].metadata.name}') -n ibm-spectrum-scale \
-c gpfs -n ibm-spectrum-scale -- mmgetstate -a
```

In the next steps, the quorum labels are moved from nodes in the **second-pool** to nodes in the **default-pool**.

10. Label 7 nodes (this number is remembered in **overall number of quorum nodes** in a prior step) of **default-pool** with **scale.spectrum.ibm.com/designation=quorum**:

```
# kubectl get node -l cloud.google.com/gke-nodepool=default-
pool,scale.spectrum.ibm.com/designation=quorum
```

| NAME | STATUS | ROLES | AGE | VERSION |
|-------------------------------------|--------|--------|-----|----------------------|
| gke-test-default-pool-7123b6b0-0g6m | Ready | <none> | 49m | v1.31.10-gke.1067000 |
| gke-test-default-pool-7123b6b0-1278 | Ready | <none> | 49m | v1.31.10-gke.1067000 |
| gke-test-default-pool-7123b6b0-6bp1 | Ready | <none> | 49m | v1.31.10-gke.1067000 |
| gke-test-default-pool-7123b6b0-7p7j | Ready | <none> | 49m | v1.31.10-gke.1067000 |
| gke-test-default-pool-7123b6b0-bccw | Ready | <none> | 49m | v1.31.10-gke.1067000 |
| gke-test-default-pool-7123b6b0-c269 | Ready | <none> | 49m | v1.31.10-gke.1067000 |
| gke-test-default-pool-7123b6b0-n8p3 | Ready | <none> | 49m | v1.31.10-gke.1067000 |

11. Remove the **scale.spectrum.ibm.com/designation=quorum** label from nodes in the **second-pool**:

```
kubectl label node -l cloud.google.com/gke-nodepool=second-pool scale.spectrum.ibm.com/designation-
```

12. Verify whether quorum was successfully assigned to labeled nodes of **default-pool**.

```
kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core \
-ojsonpath='{.items[0].metadata.name}') -n ibm-spectrum-scale -c gpfs \
-n ibm-spectrum-scale -- mmlscluster | grep quorum
```

The result lists the same nodes that are labeled:

```
4 gke-test-default-pool-7123b6b0-0g6m.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.109
gke-test-default-pool-7123b6b0-0g6m.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-
perfmon
5 gke-test-default-pool-7123b6b0-n8p3.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.107
gke-test-default-pool-7123b6b0-n8p3.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-
perfmon
6 gke-test-default-pool-7123b6b0-7p7j.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.32
gke-test-default-pool-7123b6b0-7p7j.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-
perfmon
12 gke-test-default-pool-7123b6b0-6bp1.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.108
gke-test-default-pool-7123b6b0-6bp1.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-
perfmon
```

```

13 gke-test-default-pool-7123b6b0-bccw.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.112
gke-test-default-pool-7123b6b0-bccw.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-
perfmon
14 gke-test-default-pool-7123b6b0-c269.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.110
gke-test-default-pool-7123b6b0-c269.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-
perfmon
19 gke-test-default-pool-7123b6b0-1278.daemon.ibm-spectrum-scale.stg.cluster.local. 10.128.0.106
gke-test-default-pool-7123b6b0-1278.admin.ibm-spectrum-scale.stg.cluster.local. quorum-manager-
perfmon

```

13. Perform GKE upgrade of node pool `second-pool`.

14. After the GKE upgrade is completed, it takes a while until the core pods are running on all nodes of `second-pool`. Wait until an IBM Storage Scale container native core pod is running on all nodes of node pool `second-pool`:

```

for node in $(kubectl get nodes -l cloud.google.com/gke-nodepool=second-pool \
-ojsonpath='{.items[*].metadata.name}'); do kubectl get pod -lapp.kubernetes.io/name=core \
-n ibm-spectrum-scale --field-selector spec.nodeName=$node; done

```

The output from the command must show that the **STATUS** for all pods are listed as **Running**.

| NAME | READY | STATUS | RESTARTS | AGE |
|------------------------------------|-------|---------|----------|-----|
| gke-test-second-pool-f25776e0-0qrg | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-second-pool-f25776e0-18s7 | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-second-pool-f25776e0-cnx7 | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-second-pool-f25776e0-gjlq | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-second-pool-f25776e0-k8hs | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-second-pool-f25776e0-182g | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-second-pool-f25776e0-pxbg | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-second-pool-f25776e0-q063 | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-second-pool-f25776e0-q8cs | 2/2 | Running | 0 | 21d |
| NAME | READY | STATUS | RESTARTS | AGE |
| gke-test-second-pool-f25776e0-z6rj | 2/2 | Running | 0 | 21d |

15. Verify whether IBM Storage Scale container native is **active** on all nodes:

```

kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core \
-ojsonpath='{.items[0].metadata.name}') -n ibm-spectrum-scale \
-c gpfs -n ibm-spectrum-scale -- mmgetstate -a

```

This procedure also works in a similar way for more than 2 node pools.

Configuring IBM Storage Scale Container Storage Interface (CSI) driver

Use the following sections to help with deploying IBM Storage Scale CSI with IBM Storage Scale container native:

- [Configuring storage class to use CSI driver](#)
- [Managed CSI fields](#)

Configuring storage class to use CSI driver

Storage class is used for creating lightweight volumes and fileset based volumes.

Lightweight (directory) based volumes

A Storage class example for creating directory (lightweight) based volumes is provided.

Adjust the parameters according to your environment.

```

# cat storageClass_Lightweight.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ibm-spectrum-scale-csi-lt

```

```
provisioner: spectrumscale.csi.ibm.com
parameters:
  volBackendFs: "fs1"
  volDirBasePath: "pvfileset/lwdir" # relative path from filesystem mount point for creating lightweight
volume
reclaimPolicy: Delete
```

Enter the command to create the lightweight storage class:

```
kubectl create -f storageClass_Lightweight.yaml
```

Fileset based volumes

A Storage class example for creating fileset-based volumes is provided.

Adjust the parameters according to your environment.

```
# cat storageClass_fileset.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ibm-spectrum-scale-csi-fileset
provisioner: spectrumscale.csi.ibm.com
parameters:
  volBackendFs: fs1
reclaimPolicy: Delete
```

A sample fileset-based Storage Class is created that uses the given file system as the `volBackendFs`. It can be used to create other storage classes with a remote file system. Enter the `kubectl get storageclass -oyaml > storageClass_fileset.yaml` command to create a copy of this storage class. Then configure parameters as needed and create the configured storage class by using the following command:

```
kubectl create -f storageClass_fileset.yaml
```

For more information, see [Storage class](#) in the IBM Storage Scale CSI documentation.

CSI Snapshot

The IBM Storage Scale CSI snapshot feature depends on a volume snapshot controller and the custom resource definitions (CRDs) for volume snapshot. Both the volume snapshot controller and the CRDs are independent of any CSI driver. If your Kubernetes distribution does not bundle the snapshot controller, you can manually install these components.

For more information, see [Performing preinstallation tasks](#) in IBM Storage Scale CSI documentation.

Managed CSI fields

The CSI controller creates the CSI Custom Resource (CR), and manages some fields. If you change the managed fields, the controller overrides those fields. If needed, you can change the fields other than the managed fields.

Managed fields

The following fields are populated with default values by the CSI controller. Though any new values are accepted, any values that are manually removed are repopulated upon the next controller reconciliation cycle.

Table 1. Managed fields description

| Field | Default Value |
|--------------------------|---|
| clusters | Two entries are created by default (local and remote clusters). |
| clusters.id | Local Cluster ID or Cluster ID of Remote cluster. |
| clusters.secrets | ibm-spectrum-scale-gui-user for local cluster entry, and the secret that is specified in remote cluster CR for the remote cluster entry. |
| clusters.secureSSLMode | false |
| clusters.restApi.guiHost | The GUI route for local cluster entry and one or more GUI hosts that are specified in the remote cluster CR for the remote cluster entry. |
| tolerations | NoSchedule , NoExecute , and CriticalAddonsOnly |
| localScaleCluster | Cluster ID of the IBM Storage Scale cluster where Kubernetes or Red Hat OpenShift is running |
| attacherNodeSelector | scale=true |

| Field | Default Value |
|-------------------------|---------------|
| provisionerNodeSelector | scale=true |
| pluginNodeSelector | scale=true |
| snapshotterNodeSelector | scale=true |

Editing the CSI CR

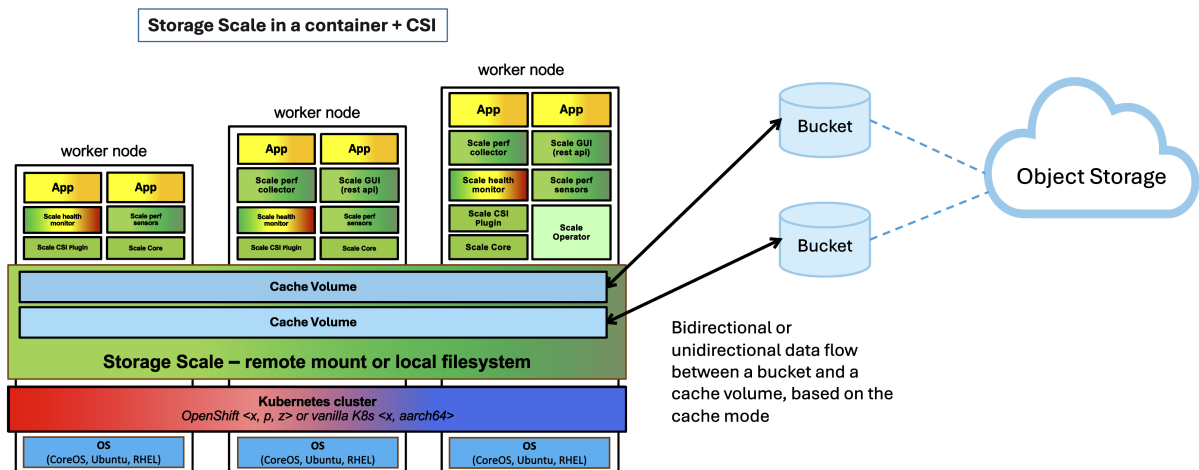
To edit the CSI CR, enter this command and update the needed field:

```
kubectl edit csiscaleoperator -n ibm-spectrum-scale-csi
```

Caching data from object storage

The cache volumes allow caching data and metadata from a remote object storage bucket locally, enabling local accelerated access. This feature reduces the latency that is associated with the repeated access to an object storage bucket, allowing frequently accessed bucket contents to be retrieved more quickly from local cache.

Figure 1. Cache volumes with IBM Storage Scale container native



The following are more details about caching:

- [Caching modes](#)
- [Kubernetes resources](#)
- [Storage class for cache volume](#)
- [Creating cache volumes](#)
- [Tuning cache configuration parameters](#)

Caching modes

Based on how an application needs to read from or write to an object storage bucket, the following are different caching modes:

- **ReadOnly:** Use this mode when an application needs to only read from an object storage bucket.
- **Exclusive:** Use this mode when an application needs to both read from and write to an object storage bucket. Make sure that only a single application writes to the object storage bucket in this case.
- **Parallel:** Use this mode when multiple applications need to both read from and write to an object storage bucket.
- **Detached:** Use this mode when an application needs to cache from an object storage bucket and modify the local cached copy without writing the changes back to the bucket.

Kubernetes resources

Table 1. Kubernetes resources for cache volumes

| Kubernetes Resource | Parameters |
|-----------------------------|---|
| StorageClass | A storage class with specific parameters for cache volumes. For more information, see Storage class for cache volume . |
| Secret | A secret with the object storage details such as an object storage endpoint, a bucket name, a bucket access key and a bucket secret key. For more information, see a secret example in Creating cache volumes . |
| PersistentVolumeClaim (PVC) | A PVC with a storage class for cache volumes. When a PVC is created with a storage class parameter volumeType="cache" , and the parameter cacheMode is not specified in the storage class then a default cache mode is selected based on the volume access mode that is specified in the PVC. By default, for the volume access mode ReadOnlyMany (ROX) the readonly cache mode is used, and for the other volume access modes, the parallel cache mode is used. |
| CacheVolume CR | A CacheVolume CR is created automatically when a cache volume is created. The CacheVolume CR can be used to see the details of the object storage bucket and other status information. |
| CacheVolumeOperation CR | To trigger a download or eviction operation on a cache volume, you must create CacheVolumeOperation CR. For more information, see CacheVolumeOperation . |

Storage class for cache volumes

The following fields come under the parameters section of the storage class for cache volumes. The parameters section is mandatory in the storage class.

Table 1. Storage class fields and description for cache volumes

| Field name | Required | Default | Description |
|--|----------|--|---|
| volBackendFs | Yes | None | The name of the file system under which the fileset is created. When a file system is remotely mounted on an IBM Storage Scale container native cluster, it is the file system name on the IBM Storage Scale container native cluster. |
| volumeType | Yes | None | volumeType : "cache" indicates that the storage class is for cache volumes. |
| csi.storage.k8s.io/provisioner-secret-name | Yes | None | csi.storage.k8s.io/provisioner-secret-name indicates the secret name, which holds the information about an object storage bucket. It is recommended to use the value \${pvc.name}-secret for this parameter. |
| csi.storage.k8s.io/provisioner-secret-namespace | Yes | None | csi.storage.k8s.io/provisioner-secret-namespace indicates the secret name, which holds the information about an object storage bucket. It is recommended to use the value \${pvc.namespace} for this parameter. |
| cacheMode | No | The default is selected based on access mode in PersistentVolumeClaim. For more information, see PersistentVolumeClaim in Kubernetes resources . | cacheMode indicates the caching mode for cache volumes. The supported values for cacheMode are readonly , exclusive , parallel , and detached . For more information, see Caching modes . |
| uid | No | 0 (root) | The user ID for the cache fileset. |
| gid | No | 0 (root) | The group ID for the cache fileset. |
| shared | No | false | Use shared: "true" , if you have pods with non-root users that are using PersistentVolumeClaims with ReadWriteMany (RWX) access mode. |

Creating cache volumes

A cache volume is a volume that caches data and metadata from an object storage bucket for local accelerated access. A special type of cache fileset is created on IBM Storage Scale for caching data and metadata.

Prerequisite for creating cache volumes

Remote mount

Use the following steps if you are using IBM Storage Scale container native with a remote mount.

1. Enable gateway nodes with the following command from the remote IBM Storage Scale cluster:

```
mmchnode -N <Scale node> --gateway
```

1. Update a parameter **afmObjKeyExpiration** to 300 seconds with the following command from the remote IBM Storage Scale cluster:

```
mmchconfig afmObjKeyExpiration=300 -i
```

You can verify the **afmObjKeyExpiration** value with the following command:

```
mmlsconfig afmObjKeyExpiration
```

Local file system

Use the following steps if you are using IBM Storage Scale container native with a local file system.

For the AFM role, the resource values are considerably higher and must be considered for proper performance. For more information, see [General recommendations for AFM gateway node configuration](#){: external}

1. Enable gateway nodes with the following command from the Red Hat OpenShift cluster:

```
kubectl label node <OpenShift node> scale.spectrum.ibm.com/role=afm
```

1. Update a parameter **afmObjKeyExpiration** to 300 seconds in the cluster CR.

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
...
spec:
  daemon:
    clusterProfile:
      afmObjKeyExpiration: "300"
```

You can verify the **afmObjKeyExpiration** value from any core pod with the following command:

```
oc rsh -n ibm-spectrum-scale worker0 mmlsconfig afmObjKeyExpiration
```

It is recommended to configure at least 2 gateway nodes.

For IBM Storage Scale container native with remote file system, the remote IBM Storage Scale level must be at least v5.2.3.x to use cache volumes for object storage.

Creating cache volumes

The following example flow shows how you can create a cache volume.

1. Create a storage class with the required parameters. For more information on the storage class parameters, see [Storage class for cache volume](#). For example:

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: ibm-scale-cache-s3
provisioner: spectrumscale.csi.ibm.com
parameters:
  volBackendFs: "fs0"
  volumeType: "cache"
  csi.storage.k8s.io/provisioner-secret-name: ${pvc.name}-secret
  csi.storage.k8s.io/provisioner-secret-namespace: ${pvc.namespace}
reclaimPolicy: Delete
allowVolumeExpansion: true
```

1. Create a secret with the details of the object storage bucket. For more information, see Secret in [Kubernetes resources](#)

```
kubectl create secret generic <PVC name>-secret -n <PVC namespace> \
--from-literal=endpoint=<object storage endpoint> --from-literal=bucket=<bucket name> \
--from-literal=accesskey=<bucket access key> --from-literal=secretkey=<bucket secret key>
```

For example:

```
kubectl create secret generic cache-s3-pvc1-secret -n ns1 \
--from-literal=endpoint=https://s3.us-east-1.amazonaws.com --from-literal=bucket=bucket1 \
--from-literal=accesskey=<your bucket access key> --from-literal=secretkey=<your bucket secret key>
```

It is recommended to use the secret name in the format of **<PVC name>-secret** and secret namespace same as the PVC namespace.

If the bucket keys are changed later in the object storage, update the secret with the new keys, and the cache volume uses the updated keys. Use this command to update the secret with the new keys: **kubectl patch secret <secret name> -n <secret namespace> -p**

```
'{"data":{"accesskey":"'$(echo -n '<new accesskey>' | base64)'"',"secretkey":"'$(echo -n '<new secretkey>' | base64)'"}}'
```

1. Create a PersistentVolumeClaim (PVC). For example:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: cache-s3-pvc1
  namespace: ns1
spec:
  storageClassName: ibm-scale-cache-s3
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 2Gi
```

1. When a PVC is in the bound state, for example:

```
# kubectl get pvc cache-s3-pvc1 -n ns1
```

| NAME | STATUS | VOLUME | CAPACITY | ACCESS MODES | STORAGECLASS |
|---------------|---------|--|----------|--------------|--------------|
| cache-s3-pvc1 | Bound | pvc-ald91618-1c9d-4773-92a5-1492f6b0736e | 2Gi | RWX | ibm-scale- |
| cache-s3 | <unset> | 5m39s | | | |

Then a CacheVolume CR is created automatically with the name same as the PersistentVolume name in the PVC namespace. For example, a successful CacheVolume CR looks like this:

```
# kubectl get cv pvc-ald91618-1c9d-4773-92a5-1492f6b0736e -n ns1
```

| NAME | CACHEMODE | HEALTHSTATUS | AGE |
|--|-----------|--------------|-------|
| pvc-ald91618-1c9d-4773-92a5-1492f6b0736e | parallel | True | 5m40s |

For a local file system with IBM Storage Scale container native level 5.2.3.x, or a remote mounted file system with a Storage Scale level 5.2.3.x, the command `ls -lsh` for a cache volume path, shows the size as 0 block for uncached objects and the objects with size less than the block size of the file system.

For a local file system with IBM Storage Scale container native level 6.0.0.x or more, or a remote mounted file system with a Storage Scale level 6.0.0.x or more, the command `ls -lsh` for a cache volume path, shows the size as 512 bytes (which is the size of a stat block), for uncached objects and the objects with size less than 512 bytes.

Increasing the operator memory limit

Creating cache PVCs in multiple namespaces, results in an increase in the memory consumption by the operator pod. In the IBM test environment, for cache PVCs in 100 different namespaces, the operator pod used up to **1300Mi** memory. So, you can change the memory limit of the operator pod as needed. An example command to set the memory limit of **1300Mi** to the operator pod:

```
kubectl set resources deployment ibm-spectrum-scale-controller-manager -n ibm-spectrum-scale-operator --limits memory=1300Mi
```

Downloading and evicting data

The contents of an object storage bucket are cached on demand. That means when an application accesses an object, it is fetched into the cache volume. But alternatively, you can also trigger the download operation manually in advance before an application accesses the objects. Similarly, to make the space available for the necessary data in a cache volume, you can choose to evict the contents of a cache volume. To trigger the download or eviction of the contents of a cache volume, create a **CacheVolumeOperation** CR. For more information, see [CacheVolumeOperation](#).

If **CacheType=ReadOnly** is used, the **CacheVolumeOperation** CR will show a FAILED status when performing download or eviction operation, but should have completed successfully. Manually verify the size. This will be resolved in a future release

Use a temporary directory for a cache volume

An application pod that uses a cache volume, which also writes data back to the object storage bucket, might not always want to write some of the data back to the bucket. For example, an application pod might create some temporary files that are deleted later, which are not necessary to be synced to the bucket. In such case, you can configure a temporary directory in your application pod to use a pre-created default directory `.cachevolumetmp` from the cache volume that can be mounted as a **subPath** to a necessary directory inside the pod. The following example **volumeMounts** entry shows an application pod that uses a directory `/tmpdir`, contents of which are not synced to the object storage bucket:

```
apiVersion: v1
kind: Pod
spec:
  ...
  volumeMounts:
    - mountPath: /tmpdir
```

```
name: <Name of the volume from spec.volumes>
subPath: .cachevolumetmp
```

The temporary directory is not applicable for the cache mode `readonly`.

Tuning cache configuration parameters

For the most users, the default cache configuration works fine. But if you want to further fine-tune the cache configuration parameters, you can do that through `VolumeAttributesClass`. For more information, see [VolumeAttributesClass](#).

For the Red Hat OpenShift platform, tuning parameters for caching is a technology preview feature, due to the `VolumeAttributesClass` support statement from Red Hat OpenShift. For more information, see [Understanding feature gates](#).

Which cache parameters can be tuned

The cache configuration parameters, which can be tuned with a `VolumeAttributesClass` are `afmNumFlushThreads`, `afmPrefetchThreshold`, `afmObjectFastReaddir`, `afmFileOpenRefreshInterval`, and `afmObjectSyncOpenFiles`. You can find more details about these parameters in the IBM Storage Scale [mmchfileset command](#).

Prerequisite for tuning cache configuration parameters

- Make sure that you are on Red Hat OpenShift 4.18 (or Kubernetes 1.31).
- Make sure that the feature gate for `VolumeAttributesClass` is enabled for your platform.

How to tune cache configuration parameters

You can tune the cache configuration parameters with the following steps:

1. Create a `VolumeAttributesClass` with the necessary parameters and values. For example:

```
apiVersion: storage.k8s.io/v1beta1
kind: VolumeAttributesClass
metadata:
  name: vac-cache1
driverName: spectrumscale.csi.ibm.com
parameters:
  afmNumFlushThreads: "10"
  afmPrefetchThreshold: "5"
```

1. Use the `VolumeAttributesClass` name in the `PersistentVolumeClaim` (PVC) spec. You can add the `VolumeAttributesClass` name in a PVC spec before creating a PVC or you can choose to update the PVC spec after it is created and in bound state. For example:

```
apiVersion: v1
kind: PersistentVolumeClaim
spec:
  ...
  volumeAttributesClassName: vac-cache1
```

Maintenance for a deployed cluster

The maintenance of a deployed IBM Storage Scale container native cluster includes certain procedures.

- [Shutting down a cluster](#)
- [IBM Storage Scale container native cluster and node maintenance](#)
- [Nodes](#)
 - [Draining nodes](#)
 - [Adding new nodes to an existing cluster](#)
 - [Deleting nodes from a cluster](#)
- [Starting the cluster after shutdown](#)
- [IBM Storage Scale storage cluster](#)
- [File systems](#)
 - [Local file systems](#)
 - [Add disks](#)
 - [Delete disks](#)
 - [Maintenance mode](#)

Shutting down a cluster

Before you begin the maintenance procedure, the IBM Storage Scale container native cluster must be shut down to avoid any issues.

For more information, see [On the nodes running CSI sidecars](#) in the IBM Storage Scale CSI documentation.

Complete the following steps to shut down a cluster:

1. Stop the running operator pod by setting the `replicas` in the deployment to 0.

```
kubectl scale deployment ibm-spectrum-scale-controller-manager -n ibm-spectrum-scale-operator --replicas=0
```

2. Enter the following command to remove the CSI label:

```
kubectl label node --all scale-
```

3. Enter the following command to delete the running core pods:

```
kubectl delete pods -lapp.kubernetes.io/name=core -n ibm-spectrum-scale
```

IBM Storage Scale container native cluster and node maintenance

The following section provides information on changing the IBM Storage Scale cluster configuration.

Limitations

While a configuration change is in progress, do not perform the following actions:

- Do not upgrade IBM Storage Scale container native.
- Do not attempt to add a node to the cluster.
- Do not upgrade Red Hat OpenShift.

Updating existing cluster configuration

The `Cluster` resource controls IBM Storage Scale cluster configuration.

To edit the configuration for an existing cluster:

```
kubectl edit cluster.scale
```

When a core pod configuration requires an update, the IBM Storage Scale operator cordons, drains, reboots (if necessary), and uncordons the node, which is performed one node at a time. When the drain is complete, the core pod is updated with the new configuration.

If the node was previously cordoned before the update, the operator does not uncordon the node.

Updating cluster configuration by upgrade

An upgrade of IBM Storage Scale container native is also considered an update of the cluster configuration, as the upgrade likely introduces changes to pod specification, for example update of container images.

For more information about how IBM Storage Scale Operator orchestrates an upgrade, see [Upgrading IBM Storage Scale container native](#).

Nodes

The following sections discuss maintenance actions on Kubernetes nodes.

- [Draining nodes](#)
- [Adding new nodes to an existing cluster](#)
- [Deleting nodes from a cluster](#)

Draining nodes

When an administrator performs maintenance on a node involving a drain, the IBM Storage Scale operator will intercept the drain call. The operator intercepts this request from the Kubernetes scheduler in order to evict the application pods that use the CSI PVs before allowing the deletion of the core pod. This allows the application to shutdown gracefully before storage access on the node is disrupted. On Red Hat OpenShift, this includes intercepting request from the Machine Config Operator that involve drain.

If the operator is not running, the interception and drain will fail.

While maintenance is in progress, do not perform the following actions:

- Do not update IBM Storage Scale container native configuration.
- Do not upgrade IBM Storage Scale container native.
- Do not attempt to add a node or delete a node to the cluster.

In addition, it is possible to have a deadlock due to the applications that are waiting to be safely evicted from one or more nodes under maintenance. The deadlock can occur if too many nodes are under maintenance concurrently, and the application workload gets disrupted if further action is taken. In these cases, the administrator should safely evict and reschedule impacted applications.

For more information about troubleshooting cluster maintenance issues, see [Identifying applications preventing cluster maintenance](#).

Node maintenance

When an administrator needs to perform manual maintenance on a node, it should be cordoned and safely drained of pods. Therefore, the application is rescheduled to other available nodes, and the IBM Storage Scale operator is notified of the node maintenance needed.

1. Drain the node that needs maintenance. The drain command also cordons the node.

On Red Hat OpenShift:

```
oc adm drain <node name>
```

On Kubernetes:

```
kubectl drain <node name>
```

2. When the drain completes (without error), maintenance can be performed on the node.
3. When maintenance is complete, uncordon the node to allow it to resume normal operation and workload scheduling.

On Red Hat OpenShift:

```
oc adm uncordon <node name>
```

On Kubernetes:

```
kubectl uncordon <node name>
```

Adding nodes to an existing cluster

Adding nodes

To add a node:

- Ensure the node that belongs to the existing Machine Config Pool configured to include the kernel-level extensions. This was configured as a prerequisite to IBM Storage Scale container native installation, and defaults to Red Hat OpenShift **worker** nodes. For more information, see [Red Hat OpenShift Container Platform](#).
- Ensure the node has a node selector that matches the existing cluster's node selector. For more information, see [Node selectors](#).

When the node selector is recognized by the IBM Storage Scale Operator, a pod will be created on the new node and will go to running state within a few minutes.

Post node addition tasks

Verify node addition

Check the progress of the creation of the new pod by entering the following command:

```
kubectl get pods -n ibm-spectrum-scale
```

Ensure the new pod is ready by entering the following command:

```
kubect1 exec -n ibm-spectrum-scale \
$(kubect1 get pods -lapp.kubernetes.io/name=core -n ibm-spectrum-scale -ojsonpath="
{.items[0].metadata.name}") \
-- mmgetstate -a
```

The output appears as shown:

| Node number | Node name | GPFS state |
|-------------|-----------|-------------|
| 1 | worker1 | active |
| 2 | new node | arbitrating |
| 3 | worker0 | active |

Once the pod has finished arbitrating and enters the active state, CSI will automatically be configured for use by the pod by the IBM Storage Scale and CSI Operators. Once CSI has completed configuration, then the newly added node can now be used for running applications.

Add quorum nodes (optionally)

When adding nodes it may make sense to select additional quorum nodes. For more information, see [Labels and annotations](#).

Deleting nodes from a cluster

Introduction

The following actions are automatically performed when a node is deleted:

- Delete the core pod
- Delete the node from the IBM Storage Scale cluster
- Move quorum designation to another node if deleted node is a quorum node
- Move GUI pod to another node if deleted node runs a GUI pod
- Move pmcollector pod to another node if deleted node runs a pmcollector pod and pmcollector uses a storage class. The storage class is used to create nonlocal persistent volumes. For more information, see [Pmcollector Volumes for Performance Data](#).

The following actions are not automatically performed when a node is deleted and manual actions are required:

- pmcollector pod must be manually moved to other node if deleted node runs a pmcollector pod and pmcollector uses a local persistent volume. For more information, see [Pmcollector Volumes for Performance Data](#) and [Move Pmcollector pod with local persistent volume](#).
- If IBM Storage Scale container native is removed from a Red Hat OpenShift node, this node needs to be manually cleaned up. Cleaning up the node is especially important, if it is intended to ever use the node for IBM Storage Scale container native again. For more information, see [Cleanup Red Hat OpenShift host](#).

It is recommended to use a storage class to create nonlocal persistent volumes for pmcollector if you plan to frequently delete and add nodes. For more information, see [Pmcollector Volumes for Performance Data](#).

Deleting a node that runs a pmcollector with local persistent volume results in partial data loss of historic performance data. It is recommended to avoid deleting nodes that run a pmcollector pod if you have the choice.

The cluster cannot get smaller than the number of quorum nodes that are currently defined. It is possible to undefine quorum nodes if there are more than three defined. For more information, see [Quorum Check](#).

When a node that has a connection to a local disk of a local filesystem is deleted, ensure that each local disk has at least two, but better three connection nodes left. For more information, see [Create LocalDisk resources](#).

Pre node deletion tasks

Quorum check

Run the following command to query the number of quorum nodes:

```
kubect1 exec -c gpfs -n ibm-spectrum-scale \
$(kubect1 get pods -lapp.kubernetes.io/name=core -n ibm-spectrum-scale -ojsonpath="
{.items[0].metadata.name}") \
-- mmlscluster | grep quorum | wc -l
```

If the number of quorum nodes is higher than the number of remaining nodes after deletion, either delete fewer nodes or reduce the number of quorum nodes. For more information on how to assign quorum nodes manually, see [Designation Labels - Manual](#).

Pmcollector check

Skip the chapter if the pmcollector pods use a storage class for creating nonlocal persistent volumes. For more information, see [Pmcollector Volumes for Performance Data](#).

If a pmcollector pod with a local persistent volume is scheduled on one of the nodes to delete, it must be manually moved to another node after the node was deleted.

Check whether a pmcollector pod is running on one of the nodes to delete:

```
kubect1 get pod -l app.kubernetes.io/name=pmcollector -o wide
```

The output appears as shown:

| NAME | READY | STATUS | RESTARTS | AGE | IP | NODE |
|----------------------------------|-----------------|---------|----------|------|--------------|---------------------|
| NOMINATED NODE | READINESS GATES | | | | | |
| ibm-spectrum-scale-pmcollector-0 | 2/2 | Running | 0 | 116m | 10.131.0.123 | worker1.example.com |
| <none> | <none> | | | | | |
| ibm-spectrum-scale-pmcollector-1 | 2/2 | Running | 0 | 115m | 10.131.2.92 | worker0.example.com |
| <none> | <none> | | | | | |

Remember the pmcollector pod names that run on nodes to delete. After the nodes are deleted, go to [Move Pmcollector pod with local persistent volume](#).

Local disk checks

Skip the chapter if you are not using local filesystems. For more information, see [Local file system](#).

Local disk connection node check

List the connection nodes of all local disks:

```
for localdisk in `kubect1 get localdisks -n ibm-spectrum-scale -ojson | jq -r .items[].metadata.name`; do
    echo -e "\n=== $localdisk ==="
    kubect1 get localdisk $localdisk -n ibm-spectrum-scale -ojson | jq -r .status.nodeConnections
done

=== disk0 ===
worker0.example.com, worker1.example.com, worker2.example.com

=== disk1 ===
worker0.example.com, worker1.example.com, worker2.example.com, worker3.example.com, worker4.example.com

=== disk2 ===
worker0.example.com, worker1.example.com, worker2.example.com, worker3.example.com, worker4.example.com
```

A deleted node is also deleted from the list of connection nodes of local disks. For safety reasons, it is recommended that each local disk has at least three connection nodes. Deleting a node that causes a local disk with less than two connection nodes must be avoided. A local disk also works with only one connection to a node. But as soon the Storage Scale core pod on the node is deleted, the local filesystem loses access to the local disk. This happens at least during next upgrade where core pods are deleted.

Deleting nodes

Select one of the following scenarios for deleting nodes:

1. [Delete IBM Storage Scale container native from Red Hat OpenShift nodes](#)
2. [Delete Red Hat OpenShift nodes](#)
3. [Delete Google Kubernetes Engine node pool](#)

Delete IBM Storage Scale container native from Red Hat OpenShift nodes

IBM Storage Scale container native runs on nodes that are selected by a node selector that is defined in the Cluster custom resource, see [Node selectors](#). To remove IBM Storage Scale container native from a node, the node must no longer be selected by the node selector.

The following command removes IBM Storage Scale container native from the node if the default node selector is used:

```
kubect1 label node <openshift_node> scale.spectrum.ibm.com/daemon-selector- -n ibm-spectrum-scale
```

The IBM Storage Scale operator cordons and drains the node, deletes the IBM Storage Scale container native core pod from the node, and finally uncordons the node.

Modifying the [Node selectors](#) in the cluster custom resource results in cordoning and draining all nodes and restarting all IBM Storage Scale container native core pods in a rolling manner. To avoid the restart of all core pods, remove labels from nodes instead of editing the [Node selectors](#) in the cluster custom resource.

Delete Red Hat OpenShift nodes

Cordon and drain the nodes before the Red Hat OpenShift nodes are deleted.

As soon as Red Hat OpenShift nodes get deleted, the IBM Storage Scale container native operator deletes the corresponding IBM Storage Scale container native nodes as well.

Delete Google Kubernetes Engine node pool

Before deleting node pools, see [Deleting nodes from a cluster](#) to review all the considerations for deleting individual nodes.

If deleting a node pool, GKE will delete all core pods. GKE will not give CNSA any chance to perform proper shut down actions. For example, CNSA does not have the chance to move the quorum designation from the nodes of the deleting node pool to another node pool. This causes a quorum loss in the Scale cluster which make it unusable.

The following actions are required before deleting a node pool. In this example the node pool `first-pool` should be deleted.

1. Ensure that the remaining node count is larger or equal than the total quorum count. Query the total quorum count via the following command:

```
# kubectl get daemon -n ibm-spectrum-scale -o json | jq -r '.items[].status.quorumPods.total'
7
```

In this example, the total quorum count is 7. The node count of remaining node pools must be at least 7. If the `second-pool` has less than 7 nodes, then increase the node count of this node pool before continuing with the next step.

2. Change the size of the GKE node pool that you want to delete to 0.
3. Wait until the node pool does not have any nodes any more. This takes a while.

```
# kubectl get node -l cloud.google.com/gke-nodepool=first-pool
No resources found
```

4. Delete the GKE node pool.

Post node deletion tasks

Verify node deletion

A successful node deletion is indicated by a `NodeDeleted` event on the Daemon custom resource:

```
$ kubectl describe daemon -n ibm-spectrum-scale
```

```
...
Events:
  Type      Reason          Age      From      Message
  ----      -
  Normal    NodeDeleted     14s      NodeDeletion  Node worker4.example.com
deleted from Scale cluster
```

Move Pmcollector pod with local persistent volume

Skip the chapter if the pmcollector pods use a storage class for creating nonlocal persistent volumes. For more information, see [Pmcollector Volumes for Performance Data](#).

Skip the chapter if no pmcollector pods were scheduled on the deleted nodes. For more information, see [Pmcollector check](#).

Move the pmcollector pod that was scheduled on a deleted node to another node:

1. Find the persistent volume claim that is used by the pmcollector.

```
kubectl get pods -l app.kubernetes.io/name=pmcollector -n ibm-spectrum-scale -o=json | \
jq -c '.items[] | {name: .metadata.name, claimName: .spec | select( has ("volumes") ).volumes[] | \
select( has ("persistentVolumeClaim") ).persistentVolumeClaim.claimName }'
```

The output appears as shown and lists the pmcollector pod names with related the persistent volume claim names:

```
{"name": "ibm-spectrum-scale-pmcollector-0", "claimName": "datadir-ibm-spectrum-scale-pmcollector-0"}
{"name": "ibm-spectrum-scale-pmcollector-1", "claimName": "datadir-ibm-spectrum-scale-pmcollector-1"}
```

Remember the name of the persistent volume claim name for later.

2. Stop all pmcollector pods

```
kubectl scale statefulset ibm-spectrum-scale-pmcollector -n ibm-spectrum-scale --replicas=0
```

3. Find the persistent volume that relates to the persistent volume claim.

```
kubectl get pv | grep <persistent_volume_claim>
```

The output appears as shown:

```
kubectl get pv | grep datadir-ibm-spectrum-scale-pmcollector-0
worker1.example.com-pmcollector 25Gi RWO Retain Bound ibm-spectrum-
scale/datadir-ibm-spectrum-scale-pmcollector-0 ibm-spectrum-scale-internal 3h22m
```

In this example, `worker1.example.com-pmcollector` is the related persistent volume. Remember the persistent volume name for later.

4. Delete the selected persistent volume claim.

```
kubectl delete pvc <persistent_volume_claim> -n ibm-spectrum-scale
```

5. Delete the selected persistent volume.

```
kubectl delete pv <persistent_volume>
```

6. Start the pmcollector pods

```
kubectl scale statefulset ibm-spectrum-scale-pmcollector -n ibm-spectrum-scale --replicas=2
```

7. Wait until both pmcollector pods are running.

```
kubectl get pod -l app.kubernetes.io/name=pmcollector -o wide -n ibm-spectrum-scale
```

The pmcollector pod that used to run on the deleted node is now scheduled on another node. The output appears as shown:

```
kubectl get pod -l app.kubernetes.io/name=pmcollector -o wide -n ibm-spectrum-scale
```

| NAME | READY | STATUS | RESTARTS | AGE | IP | NODE |
|----------------------------------|--------|---------|----------|------|--------------|------|
| NOMINATED NODE READINESS GATES | | | | | | |
| ibm-spectrum-scale-pmcollector-0 | 2/2 | Running | 0 | 118s | 10.131.0.123 | |
| worker2.example.com | <none> | <none> | | | | |
| ibm-spectrum-scale-pmcollector-1 | 2/2 | Running | 0 | 74s | 10.131.2.92 | |
| worker0.example.com | <none> | <none> | | | | |

Cleanup Red Hat OpenShift host

IBM Storage Scale container native creates persistent directories and files on the Red Hat OpenShift hosts where the core pods are deployed. These persistent files must be deleted from the host if IBM Storage Scale container native is removed. In addition, remove any node labels that are created by the IBM Storage Scale container native operator. For more information, see [Cleanup Red Hat OpenShift nodes](#).

Starting the cluster after shutdown

If the IBM Storage Scale cluster was shut down, start the cluster by using the following steps:

Ensure that the worker nodes are in Ready state before restart of the IBM Storage Scale cluster by entering the `kubectl get nodes` command. If any of the worker nodes are in a state other than Ready, the IBM Storage Scale cluster fails to restore.

1. Scale up the operator pods by setting the `replicas` in the deployment to 1.

```
kubectl scale deployment ibm-spectrum-scale-controller-manager -n ibm-spectrum-scale-operator --replicas=1
```

After the operator pod comes back up, the core pods are rescheduled and the default CSI label is reapplied.

IBM Storage Scale storage cluster

The following sections describe maintenance tasks for the remote IBM Storage Scale storage cluster.

- [Updating remote mount access key on storage cluster](#)
- [Updating authentication to the storage cluster](#)

Updating remote mount access key on storage cluster

An access key is used in the IBM Storage Scale storage cluster to grant file system access to the IBM Storage Scale container native cluster. The access key can be changed by following these steps:

1. Identify the `RemoteCluster` resource that you intend to change the access key.

```
kubectl get remoteccluster -n ibm-spectrum-scale
```

The `NAME` of the `RemoteCluster` resource is used in the following steps as `<remote-cluster>`.

1. On the IBM Storage Scale storage cluster, generate a new access key by entering the following command:

```
mmauth genkey new
```

Now, the storage cluster temporarily has two access keys.

2. Begin the update of the new generated key from the IBM Storage Scale container native cluster by labeling the **RemoteCluster** resource:

```
kubect1 label remotec1uster <remote-cluster> scale.spectrum.ibm.com/update= -n ibm-spectrum-scale
```

3. Check the success by watching status and events of the remote cluster resource:

```
kubect1 describe remotec1uster <remote-cluster> -n ibm-spectrum-scale
```

4. Verify that all your client clusters remote mounting file systems from this storage cluster is using the new key.
5. Commit the new access key by using the following command:

```
mmauth genkey commit
```

When the new key is committed, the old key is removed.

For more information, see [mmauth command](#).

Updating authentication to the storage cluster

The two possible ways on how to change the passwords of the GUI users on the storage cluster are:

1. Let the operator automatically change the passwords. For more information, see [Automatic password rotation](#).
2. Manually update the passwords.

Use the following documentation as a guide for the second option to manually update the authorization on the remote storage cluster. This procedure involves two parts:

1. [Updating user passwords on the storage cluster](#)
2. [Updating user secrets for the storage cluster on Red Hat OpenShift](#)

Updating user passwords on the storage cluster

To change the user passwords on the storage cluster, run the following actions on the storage cluster. For more information about users, see [Creating Container Operator and CSI users](#).

The administrator can change the password by using the following commands:

1. To change the container native operator GUI user, enter the following command on the storage cluster:

```
/usr/lpp/mmfs/gui/cli/chuser cnsa_storage_gui_user -p new_cnsa_storage_gui_password
```

2. To change the CSI GUI user, enter the following commands on the storage cluster:

```
/usr/lpp/mmfs/gui/cli/chuser csi_storage_gui_user -p new_csi_storage_gui_password
```

The **chuser** command needs to be run on a node where the GUI is installed in the storage cluster.

Updating user secrets for the storage cluster on Red Hat OpenShift

When user passwords on the remote IBM Storage Scale storage cluster changes, the corresponding Kubernetes secrets with the credentials must be updated for the operator to maintain communication. For more information about these secrets, see [Creating secrets for storage cluster GUI users](#).

1. To update the secret named **cnsa-remote-mount-storage-cluster-1** in the **ibm-spectrum-scale** namespace, enter the following command:

```
kubect1 patch secret cnsa-remote-mount-storage-cluster-1 -n ibm-spectrum-scale \
-p="{\"data\":{\"password\": \"`echo -n 'new_cnsa_storage_gui_password' | base64`\"}}"
```

Verify that the secret is updated:

```
echo -n `kubect1 get secret cnsa-remote-mount-storage-cluster-1 -n ibm-spectrum-scale \
-ojson | jq -r .data.password` | base64 -d && echo
```

2. To update the secret named **csi-remote-mount-storage-cluster-1** in the **ibm-spectrum-scale-csi** namespace, enter the following command:

```
kubectl patch secret csi-remote-mount-storage-cluster-1 -n ibm-spectrum-scale-csi \
-p="{\"data\":{\"password\": \"`echo -n 'new_csi_storage_gui_password' | base64`\"}}\""
```

Verify that the secret is updated:

```
echo -n `kubectl get secret csi-remote-mount-storage-cluster-1 -n ibm-spectrum-scale-csi \
-ojson | jq -r .data.password` | base64 -d && echo
```

File systems

The following sections discuss maintenance actions on file systems.

- [Local file systems](#)
 - [Add disks](#)
 - [Delete disks](#)
 - [Maintenance mode](#)

Local file systems

Add disks

Disks can be added to an existing local file system by adding disks to the `spec.local.pools.disks` parameter of the `Filesystem` custom resource, or by adding a pool in `spec.local.pools`. For more information, see [Filesystem spec](#).

Disks cannot be added if the [maintenance mode](#) is enabled, or if a disk deletion process is already running.

In the following example, a new disk must be added to an existing file system:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Filesystem
name: local-sample
namespace: ibm-spectrum-scale
spec:
  local:
    type: shared
    replication: 1-way
    pools:
      - disks:
        - disk0
        - disk1
        - disk2
        name: system
```

1. Create a `LocalDisk` custom resource with name `disk3`. For more information, see [Create LocalDisk resources](#).
2. Verify that the new `LocalDisk` displays `TYPE` as `shared` and `READY` as `True`:

```
$ kubectl get localdisk disk3 -n ibm-spectrum-scale
```

| NAME | TYPE | READY | USED | AVAILABLE | FILESYSTEM | SIZE | AGE |
|-------|--------|-------|-------|-----------|------------|---------|-----|
| disk3 | shared | True | False | False | | 1.4 TiB | 32s |

3. Add the name of the new `LocalDisk` custom resource to the list of disks of selected pool:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Filesystem
name: local-sample
namespace: ibm-spectrum-scale
spec:
  local:
    type: shared
    replication: 1-way
    pools:
      - disks:
        - disk0
        - disk1
        - disk2
        - disk3
        name: system
```

4. Verify that the disk was successfully added by seeing a `DiskAdded` event for the filesystem

```
$ kubectl describe filesystem local-sample -n ibm-spectrum-scale
...
Events:
  Type      Reason      Age   From      Message
  ----      -
  Normal    DiskAdded   13s   Filesystem Added disk disk3 to filesystem.
```

Rebalance disks

After disks are added, the file system is unbalanced. The added disks are empty and the older disks have data. Optionally, the file system can be rebalanced to improve performance. Rebalancing attempts to distribute file blocks evenly across the disks of the file system.

Rebalancing is an I/O intensive and time-consuming operation and is important only for file systems with large files that are mostly invariant. In many cases, normal file updates and creations do rebalance a file system over time without the cost of a complete rebalancing.

Disks cannot be rebalanced if the [maintenance mode](#) is enabled.

To check whether the file system is unbalanced, enter the following command (replace `local-sample` with the name of the file system). A balanced file system has similar usage percentages across all the disks where an unbalance file system will have very different disk utilization values.

```
kubectl exec -n ibm-spectrum-scale \
$(kubectl get pods -lapp.kubernetes.io/name=core -n ibm-spectrum-scale -ojsonpath="{.items[0].metadata.name}") \
-- mmdf local-sample
```

To rebalance the file system:

1. Create a `RestripeFsJob` object:

```
apiVersion: scale.spectrum.ibm.com/v1alpha1
kind: RestripeFSJob
metadata:
  name: rebalance-sample
  namespace: ibm-spectrum-scale
spec:
  filesystem: local-sample
  mode: rebalance
  maxRetryFailed: "0"
```

Specify the file system name in `filesystem`. The `mode` must be `rebalance`. `maxRetryFailed: "0"` means that the job does not automatically retry after failure.

2. Check the progress of the rebalancing

```
# kubectl get restripefsjob
NAME                FILESYSTEM      LAST SCHEDULE TIME   LAST SUCCESSFUL TIME   RUNNING   COMPLETED
rebalance-sample    local-sample     64s                  33s                    Successf
```

Delete disks

Disks can be deleted from an existing local file system by deleting disks from the `spec.local.pools.disks` parameter of the Filesystem custom resource, or by deleting a pool in `spec.local.pools`. Deleting disks is a long-lasting action because the existing data must be moved from disks to remove to free space of remaining disks. For more information, see [Filesystem spec](#).

Disks cannot be removed if the [maintenance mode](#) is enabled, or if a disk deletion process is already running.

In the following example, the disk `disk2` must be deleted from an existing file system:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Filesystem
name: local-sample
namespace: ibm-spectrum-scale
spec:
  local:
    type: shared
    replication: 1-way
    pools:
      - disks:
          - disk0
          - disk1
          - disk2
        name: system
```

1. Delete the name of the `LocalDisk` custom resource from the list of disks of selected pool:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Filesystem
name: local-sample
```

```

namespace: ibm-spectrum-scale
spec:
  local:
    type: shared
    replication: 1-way
    pools:
    - disks:
      - disk0
      - disk1
      name: system

```

Within a pool, the used capacity of disks to remove must not exceed the free capacity of remaining disks. If the disk deletion requested results in the free pool capacity calculated to be less than 10%, the change to the file system spec will fail.

2. Ensure that the disk is deleted successfully before any filesystem or cluster actions are performed.

- The disk deletion is still ongoing, indicated by a task shown under the RUNNING column:

```

$ oc get diskjob
NAME                                FILESYSTEM  LAST SCHEDULE TIME  LAST SUCCESSFUL TIME  RUNNING
COMPLETED
delete-disks-shared-fs  shared-fs    11s
scale/worker1/gpfs/diskDeleteAction_ABr13y

```

- The disk deletion completed successfully:

```

$ oc get diskjob
NAME                                FILESYSTEM  LAST SCHEDULE TIME  LAST SUCCESSFUL TIME  RUNNING
COMPLETED
delete-disks-shared-fs  shared-fs    101s
Successful

```

Depending on the data capacity to move from disks to delete to other disks, the deletion can take a long time.

Maintenance Mode

Maintenance mode must be activated when performing disruptive actions on the filesystem or disks. For more information, see [File system maintenance mode](#).

Enable Maintenance Mode

The following preconditions must be fulfilled before maintenance mode is enabled:

- All application pods that use the CSI persistent volumes are stopped
- Verify that no disk jobs are running (for example an ongoing disk deletion):

```
kubectl get diskjob -n ibm-spectrum-scale
```

- Verify that no compression jobs are running:

```
kubectl get compressionjob -n ibm-spectrum-scale
```

The maintenance mode for a local file system is enabled by adding the `scale.spectrum.ibm.com/maintenanceMode=true` label to the `Filesystem` custom resource:

```
kubectl label filesystem local-sample -n ibm-spectrum-scale scale.spectrum.ibm.com/maintenanceMode=true
```

Disable Maintenance Mode

Remove the `scale.spectrum.ibm.com/maintenanceMode` label from the `Filesystem` custom resource to disable the maintenance mode.

```
kubectl label filesystem local-sample -n ibm-spectrum-scale scale.spectrum.ibm.com/maintenanceMode-
```

Alternatively the label value can be set to `false`:

```
kubectl label filesystem local-sample scale.spectrum.ibm.com/maintenanceMode=false
```

Cleanup

The following procedures outline the steps to cleanup various pieces of the IBM Storage Scale container native solution. To delete the cluster and uninstall the product, run through each of the following sections.

- [Removing applications](#)
- [Cleanup Kubernetes resources](#)

- [Remote file systems](#)
- [Remote clusters](#)
- [Local file systems](#)
- [Cluster](#)
- [Cleanup IBM Storage Scale CSI](#)
- [Cleanup IBM Storage Scale container native](#)
- [Cleanup Kubernetes nodes](#)
 - [Cleanup Red Hat OpenShift nodes](#)
 - [Cleanup Kubernetes nodes](#)
- [Cleanup storage cluster](#)
- [Cleanup on AWS ROSA](#)

Removing applications

If there are applications that are accessing the storage filesystem through IBM Storage Scale CSI, they must be stopped before continuing.

1. Stop all the applications that are accessing storage via the IBM Storage Scale CSI driver.
2. Delete all the Persistent Volume Claims (PVCs) and Persistent Volumes (PVs) provisioned by IBM Storage Scale CSI Driver.

Cleanup Kubernetes resources

The following sections describe how to clean up Kubernetes custom resources that are defined in your IBM Storage Scale container native cluster.

- [Remote file systems](#)
- [Remote clusters](#)
- [Local file systems](#)
- [Cluster](#)

Remote file systems

After a **Filesystem** custom resource of a remote file system is deleted, the operator does not delete the remote mount file system configuration on the IBM Storage Scale container native cluster.

Make sure that no applications use Persistent Volumes that the file system serves.

In this example, the sample **Filesystem** is used:

```
kind: Filesystem
metadata:
  ...
  name: remote-sample
spec:
  remote:
    cluster: remoteccluster-sample
    fs: fs1
```

Complete the following steps:

1. Enter the following command to delete the file system from Red Hat OpenShift.
2. Log in to a core pod by using the following command and remove the file system from IBM Storage Scale.

```
kubectl delete filesystem remote-sample -n ibm-spectrum-scale

kubectl -n ibm-spectrum-scale exec -it \
$(kubectl get pods -n ibm-spectrum-scale -lapp.kubernetes.io/name=core -o
jsonpath='{.items[0].metadata.name}') -- sh
```

- Unmount the file system on all the container native pods.
- Delete the remote file system.

```
mmremotefs delete remote-sample
```

3. You can delete the remote cluster definition if the remote storage cluster is only configured to mount and serve the single **remote-sample** file system. Otherwise, the other file systems must be deleted by using the same process that is mentioned in the prior step.

- Find the remote clusters.

```
mmremoteccluster show all
```

- Delete the remote cluster that serves the remote file system. For example, to delete a remote cluster named **gpfs.storage**.

```
mmremoteccluster delete gpfs.storage
```

Remote clusters

Deleting a **RemoteCluster** custom resource **does not** result in the operator deleting the access permission of the IBM Storage Scale container native cluster to the file systems on the remote storage cluster. The **RemoteCluster** controller only handles creating the access permissions.

Before removing the remote cluster credentials, ensure that no file systems are using this credential. For information on removing file system resources, see [Filesystem](#).

For this example, the sample **RemoteCluster** is used:

```
kind: RemoteCluster
metadata:
  name: remoteccluster-sample
spec:
  ...
```

Perform the following steps:

1. Delete the **RemoteCluster** definition from OpenShift by entering the following command:

```
kubectl delete remoteccluster remoteccluster-sample -n ibm-spectrum-scale
```

2. Delete the secure credentials on the storage cluster. For more information, see [Cleaning up on the storage cluster](#) page.

Local file systems

File system

When a local file system custom resource is deleted, the operator deletes the actual file system running in the IBM Storage Scale container native cluster. All file system data will be erased.

Make sure that no Persistent Volumes use storage of this file system.

Complete the following steps:

1. To prevent deletion of a file system by mistake, a label must be added to the **Filesystem** custom resource to confirm deletion. Enter the following command to label the file system for deletion.

```
kubectl label filesystem local-sample -n ibm-spectrum-scale scale.spectrum.ibm.com/allowDelete=
```

2. Enter the following command to delete the file system from Red Hat OpenShift and erase the data.

```
kubectl delete filesystem local-sample -n ibm-spectrum-scale
```

Local disks

The disks can be either re-used for another filesystem, or deleted if they are no longer needed.

The delete the disks, complete the following steps:

1. The disk to delete must not be used by a file system. A **LocalDisk** custom resource indicates that it is not used by a file system with value **False** in the **USED** column.

```
$ kubectl get localdisk disk0 -n ibm-spectrum-scale
```

| | NAME | TYPE | READY | USED | AVAILABLE | FILESYSTEM | SIZE | AGE |
|-------|--------|------|-------|------|-----------|--------------|---------|-----|
| disk0 | shared | True | False | True | | | 3.5 TiB | 16h |
| disk1 | shared | True | True | True | | local-sample | 3.5 TiB | 16h |

2. If the disk is not used by any file system, enter the following command to delete it.

```
kubect1 delete localdisk disk0 -n ibm-spectrum-scale
```

3. If all local disks are deleted, enter the following command to delete the related node labels:

```
kubect1 label node --all scale.spectrum.ibm.com/nsdFailureGroup-  
scale.spectrum.ibm.com/nsdFailureGroupMappingType-
```

Cluster

Complete the following steps to delete a cluster:

1. Record the GPFS cluster name before deleting the cluster.

```
kubect1 get daemon -n ibm-spectrum-scale -o json | jq -r '.items[].status.clusterName'
```

This GPFS cluster name is required in subsequent steps when removing the authorization on the storage cluster.

2. Enter the following command to delete the IBM Storage Scale cluster.

```
kubect1 delete cluster.scale ibm-spectrum-scale
```

Cleanup IBM Storage Scale CSI

For steps to remove CSI from your cluster, see [Cleaning up IBM Storage Scale Container Storage Interface driver and Operator by using CLIs](#).

Cleanup IBM Storage Scale container native

Complete the following steps:

1. Change to a default namespace:

On Red Hat OpenShift:

```
oc project default
```

On Kubernetes:

```
kubect1 config set-context --current --namespace default
```

2. Enter the following command to uninstall the operator, Kubernetes objects, namespaces, and more.

Red Hat OpenShift Cluster CR

For a Red Hat OpenShift cluster:

```
kubect1 delete -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-  
native/v6.0.0.x/generated/scale/install.yaml --ignore-not-found=true
```

Enter the following command to delete the Security Context Constraints (SCC)

```
oc delete scc ibm-spectrum-scale-privileged spectrum-scale-csiaccess
```

Kubernetes Cluster CR

For a Kubernetes cluster:

```
kubect1 delete -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-  
native/v6.0.0.x/generated/scale/install-k8s.yaml --ignore-not-found=true
```

3. Enter the following command to clean up the performance monitoring and IBM Storage Scale CSI artifacts.

a. Enter the following command to list the PVs with claim of `datadir-ibm-spectrum-scale-scale-pmcollector`. Two PVs are returned.

```
kubect1 get pv -lapp.kubernetes.io/instance=ibm-spectrum-scale,app.kubernetes.io/name=pmcollector  
kubect1 delete pv -lapp.kubernetes.io/instance=ibm-spectrum-scale,app.kubernetes.io/name=pmcollector
```

b. Enter the following command to delete the Storage Classes created by performance monitoring and IBM Storage Scale CSI artifacts:

```
kubectl delete sc -lapp.kubernetes.io/instance=ibm-spectrum-scale,app.kubernetes.io/name=pmcollector
kubectl delete sc ibm-spectrum-scale-sample
```

Cleanup Kubernetes nodes

IBM Storage Scale container native uses hostPath volume mounts creating persistent directories and files on each of the Kubernetes nodes where core pods are deployed. To fully remove the installation of the cluster, these directories and files need to be deleted.

- [Cleanup Red Hat OpenShift nodes](#)
- [Cleanup Kubernetes nodes](#)

Cleanup Red Hat OpenShift nodes

Complete the following steps to remove the persistent files on the Red Hat OpenShift nodes.

1. Switch to the default project.

```
oc project default
```

2. Enter the following command to list the nodes.

```
kubectl get nodes -o jsonpath="{range .items[*]}{.metadata.name}{'\n'}"
```

Use the `nodeSelector` to reduce the set of nodes to cleanup, or run cleanup against all the nodes.

3. From the list of nodes, use the `oc debug node` command to create a debug pod accessing the host and allowing for the ability to remove the kernel modules and the host path volume-mounted directories that are used by IBM Storage Scale container native:

```
oc debug node/<openshift_node> -T -- chroot /host sh -c "rm -rf /var/mmfs; rm -rf /var/adm/ras; rmmmod tracedev mmfs26 mmfslinux;"
```

Example:

```
oc debug node/worker0.example.com -T -- chroot /host sh -c "rm -rf /var/mmfs; rm -rf /var/adm/ras; rmmmod tracedev mmfs26 mmfslinux;"
Starting pod/worker0examplecom-debug ...
To use host binaries, run `chroot /host`
Removing debug pod ...
```

4. Check that none of the artifacts remain by entering the following validation command:

```
oc debug node/<openshift_node> -T -- chroot /host sh -c "ls /var/mmfs; ls /var/adm/ras; rmmmod tracedev mmfs26 mmfslinux;"
```

Example:

```
oc debug node/worker0.example.com -T -- chroot /host sh -c "ls /var/mmfs; ls /var/adm/ras; rmmmod tracedev mmfs26 mmfslinux;"
Starting pod/worker0examplecom-debug ...
To use host binaries, run `chroot /host`
ls: cannot access '/var/mmfs': No such file or directory
ls: cannot access '/var/adm/ras': No such file or directory
rmmmod: ERROR: Module tracedev is not currently loaded
rmmmod: ERROR: Module mmfs26 is not currently loaded
rmmmod: ERROR: Module mmfslinux is not currently loaded
Removing debug pod ...
error: non-zero exit code from debug container
```

5. Remove any node labels that are created by the IBM Storage Scale container native operator:

```
# clean up all 'scale.spectrum.ibm.com' labels on the nodes
kubectl get nodes -ojson | jq -r '.items[].metadata.labels' | \
grep scale.spectrum.ibm.com | sort | cut -d: -f1 | \
tr -d "\"" | uniq | xargs -I{} kubectl label nodes --all {}-

# clean up 'scale' label on the nodes
kubectl label node --all scale-
```

Cleanup the k8s nodes

Complete the following steps to remove the persistent files on the Kubernetes nodes.

1. Switch to the default project.

```
kubectl config set-context --current --namespace default
```

2. Enter the following command to list the nodes.

```
kubectl get nodes -o jsonpath="{range .items[*]}{.metadata.name}{'\n'}"
```

3. From the list of nodes, access them directly via SSH. Remove directories that are used by IBM Storage Scale container native:

```
ssh <k8s nodes> "rm -rf /var/mmfs; rm -rf /var/adm/ras;rmmod tracedev mmfs26 mmfslinux;"
```

If nodes cannot be accessed via SSH, use following command to create a temporary debug pod and remove the resources.

```
kubectl debug node/<k8s node> -it --image=docker.io/busybox -- chroot /host sh -c "rm -rf /var/mmfs; rm -rf /var/adm/ras; rmmod tracedev mmfs26 mmfslinux;"
```

```
kubectl get pods --field-selector spec.nodeName=<k8s node> -o custom-columns=":metadata.name" | grep 'node-debugger' | xargs kubectl delete pod
```

4. Remove any node labels that are created by the IBM Storage Scale container native operator:

Clean up all 'scale.spectrum.ibm.com' labels on the nodes

```
kubectl get nodes -ojson | jq -r '.items[].metadata.labels' | \
grep scale.spectrum.ibm.com | sort | cut -d: -f1 | tr -d "\"" | uniq | \
xargs -I{} kubectl label nodes --all {}-
```

Clean up the 'scale' label on the nodes

```
kubectl label node --all scale-
```

Cleanup storage cluster

If your storage cluster is on AWS ROSA, skip this section and see [Revoke filesystem access from the storage cluster](#).

Delete the access permission that is granted to the IBM Storage Scale client cluster for mounting the remote file system by running the following steps on the storage cluster.

1. Enter the following command to query the name of the GPFS clusters that have established authorization to this storage cluster.

```
mmauth show all
```

2. If the name of your GPFS cluster is `ibm-spectrum-scale.clustername.example.com`, run the following command to remove the client cluster authorization.

```
mmauth delete ibm-spectrum-scale.clustername.example.com
```

Cleanup on AWS ROSA

This section provides cleanup procedures that needs to be done on an IBM Storage Scale container native deployment on Red Hat OpenShift Service on AWS (ROSA).

Cleaning up the ROSA worker security group

Complete the following steps to clean up ROSA worker security group:

1. In your AWS Management Console, navigate to the "EC2 Dashboard" in the region where ROSA is installed and select the "Security Groups" option from the navigation pane. Locate the Security Group for ROSA (worker security group id in case of ocp version < 4.16.x and node security group id in case of ocp version ≥ 4.16.x). Export the following variable.

```
export AWS_ROSA_WORKER_SECURITY_GROUP=<security_group_id>
```

2. Use the following commands to revoke the IBM Storage Scale container native ingress traffic rules.

```
aws ec2 revoke-security-group-ingress --group-id ${AWS_ROSA_WORKER_SECURITY_GROUP} --protocol tcp --
port 12345 --source-group ${AWS_ROSA_WORKER_SECURITY_GROUP}
aws ec2 revoke-security-group-ingress --group-id ${AWS_ROSA_WORKER_SECURITY_GROUP} --protocol tcp --
port 1191 --source-group ${AWS_ROSA_WORKER_SECURITY_GROUP}
```

```
aws ec2 revoke-security-group-ingress --group-id ${AWS_ROSA_WORKER_SECURITY_GROUP} --protocol tcp --port 60000-61000 --source-group ${AWS_ROSA_WORKER_SECURITY_GROUP}
```

Revoke filesystem access from the storage cluster

Before revoking access from the storage cluster, ensure that you have fully cleaned up the IBM Storage Scale container native cluster and no longer have any applications utilizing the filesystems.

After IBM Storage Scale container native has been uninstalled, use the following command to revoke the filesystem access:

```
cloudkit revoke filesystem
```

The following block shows an example of what you can expect to see when revoking the filesystem:

```
# ./cloudkit revoke filesystem
I: Logging at /root/scale-cloudkit/logs/cloudkit-16-2-2023_0-16-37.log
? Cloud platform name: AWS
=====
|                                     ! Note !                                     |
|                                                                              |
| Revoke cluster involves unmount of filesystem. Make sure the I/O is      |
| stopped and data is flushed before proceeding further.                    |
|                                                                              |
=====
? Select remotemount name: scale-strg-clr-rosa-scale
? IBM Storage Scale Container Native cluster name: ibm-spectrum-scale.stg.rosa-scale.example.com
? Storage cluster management GUI username: administrator
? Storage cluster management GUI password: *****
? Connectivity method to cloud: JumpHost
? Bastion/JumpHost instance login username: ec2-user
? Bastion/JumpHost instance public ip address: xxx.xxx.xxx.195
? Bastion/JumpHost SSH private key file path (will be used only for configuration): /root/bastion_pvt_key
I: Obtaining IBM Storage Scale storage cluster definition.
I: Initiating remote mount revoke configuration.
100% |██████████████████████████████████████████████████████████████████████|
(10/10, 1 it/min)
I: IBM Storage Scale cluster configuration completed.
I: Updating storage security group '<sg-storage-hash>' to revoke traffic from OCP security group '<sg-ocp-group-hash>'.
I: Access to IBM Storage Scale cluster 'scale-strg-clr' has been revoked.
```

Monitoring

The IBM Storage Scale container native cluster is monitored by sending the health status and events between its pods.

- [System monitor and Kubernetes readiness probe](#)
- [Viewing and analyzing the performance data with the IBM Storage Scale bridge for Grafana](#)

System monitor and Kubernetes readiness probe

The scale-monitor sidecar container has the following objectives:

- Runs the containermon service which is monitoring the service (GUI, pmcollector) in the same pod.
- Provides a readiness probe API (HTTPS).
- Sends the health status and events back to the core pod on the same worker node.
- Core pod is forwarding the events to GUI or mmhealth.
- Provides an API for call home data collection.
- Has several debug tools installed and can be used for problem determination.

For more information, see [Container probes](#) in Kubernetes documentation.

If the monitoring status is HEALTHY, the probe returns success 200. When the "unreadyOnFailed" option is enabled in containermon.conf (default=true), any FAILED state causes the probe to return 500. When a critical event occurred which has the "container_unready=True" flag, the probe returns 501. When the service faces an issue, for example, no service found, it returns 502.

Viewing and analyzing the performance data with the IBM Storage Scale bridge for Grafana

IBM Storage Scale has built-in performance monitoring tool that collects metrics from various GPFS components. These metrics can provide you with a status overview and trends of the key performance indicators. You can view and analyze the collected performance data with Grafana, a third-party visualization software.

For using Grafana, you need a running Grafana instance and the IBM Storage Scale performance monitoring bridge for Grafana deployed on your IBM Storage Scale container native cluster. For more information, see [IBM Storage Scale bridge for grafana](#) repository in GitHub.

The IBM Storage Scale bridge for Grafana is an open source tool, available for free usage on IBM Storage Scale devices. It translates the IBM Storage Scale metadata and performance data collected by the IBM Storage Scale performance monitoring tool to query requests acceptable by the Grafana-integrated openTSDB plugin.

The IBM Storage Scale performance monitoring bridge for Grafana can be deployed automatically through the operator. For more information, see [Grafana bridge](#).

For more information about setting up a Grafana instance for monitoring the IBM Storage Scale container native cluster, see [Setup Grafana for monitoring a IBM Storage Scale container native cluster in a k8s OCP environment](#) in GitHub documentation.

Support

There may be occasions where you need to troubleshoot or open a support case to IBM for further assistance.

- [Known issues](#)
- [Troubleshooting](#)
- [Gathering data about your cluster](#)

Known Issues

The following sections describe issues and solutions.

- [Pod Issues](#)
 - [A GUI pod is stuck in 3/4 running containers with multiple restarts](#)
 - [Error: daemon and kernel extension do not match](#)
 - [Error: could not insert module, required key not available](#)
 - [MountVolume.SetUp failed for volume "ssh-keys"](#)
 - [Unable to retrieve some image pull secrets](#)
 - [GUI or Grafana bridge pods fails to start, no data returned from pmcollector to frontend applications](#)
 - [pmcollector pod is in pending state during OpenShift Container Platform upgrade or reboot](#)
 - [pmsensors that shows null after failure of pmcollector node](#)
 - [pmcollector pods are not in Running state](#)
 - [pid limits set higher than podPidLimits, but not being honored](#)
 - [Upgrade failed due to PodsUnsafeToDelete: Unknown safety with reason](#)
 - [Core pods restart constantly because mount parameters have changed](#)
- [Cluster Issues](#)
 - [NumaMemoryInterleave is not set to yes](#)
- [Remote cluster Issues](#)
 - [RestError: failed to get storage cluster information. errmsg: 401 Unauthorized GET or PasswordsOutOfSync](#)
 - [Failed to establish remote cluster connection when cacert ConfigMap does not exist](#)
 - [Adding a Remote Cluster to an existing IBM Storage Scale container native cluster taking a long time to appear](#)
- [File system issues](#)
 - [Remote file systems are defined but not mounted on all nodes](#)
 - [Remote file systems unable to mount successfully](#)
- [Cache volume issues](#)
 - [Empty cache mode and health status in CacheVolume CR](#)
- [Upgrade Issues](#)
 - [tls errors found in the operator log](#)
- [Red Hat OpenShift Node issues](#)
 - [Adding a node fails - the node appears to already belong to a GPFS cluster](#)
- [Red Hat OpenShift Service on AWS](#)
 - [Error: InvalidGroup.NotFound](#)
 - [CSI pods in CrashLoopBackOff](#)

Pod Issues

- [A GUI pod is stuck in 3/4 running containers with multiple restarts](#)
- [Error: daemon and kernel extension do not match](#)
- [Error: could not insert module, required key not available](#)
- [MountVolume.Setup failed for volume "ssh-keys"](#)
- [Unable to retrieve some image pull secrets](#)
- [GUI or Grafana bridge pods fails to start, no data returned from pmcollector to front end applications](#)
- [pmcollector pod is in pending state during OpenShift Container Platform upgrade or reboot](#)
- [pmsensors that shows null after failure of pmcollector node](#)
- [pmcollector pods are not in Running state](#)
- [pid_limits set higher than podPidLimits, but not being honored](#)
- [Upgrade failed due to PodsUnsafeToDelete: Unknown safety with reason](#)
- [Core pods restart constantly because mount parameters have changed](#)

A GUI pod is stuck in 3/4 running containers with multiple restarts

During an upgrade or a rolling pod update, the Liberty container in a GUI pod has multiple restarts but does not recover.

```
kubectl get pod -n ibm-spectrum-scale | grep gui
```

| NAME | READY | STATUS | RESTARTS | AGE |
|--------------------------|-------|---------|-----------------|-------|
| ibm-spectrum-scale-gui-0 | 3/4 | Running | 705 (4m18s ago) | 3d |
| ibm-spectrum-scale-gui-1 | 4/4 | Running | 1 (17h ago) | 2d16h |

Manually delete the pod to fix the issue:

1. Identify the problematic GUI pod:

```
kubectl get pod -n ibm-spectrum-scale | grep gui
```

```
ibm-spectrum-scale-gui-0          3/4      Running    705 (4m18s ago)    3d
```

2. Delete the GUI pod:

```
kubectl delete pod -n ibm-spectrum-scale <gui_pod_name>
```

Example output:

```
kubectl delete pod -n ibm-spectrum-scale ibm-spectrum-scale-gui-0
pod "ibm-spectrum-scale-gui-0" deleted
```

1. Verify that the GUI pod recovers with 4/4 containers in READY.

```
kubectl delete pod -n ibm-spectrum-scale <gui_pod_name>
```

```
ibm-spectrum-scale-gui-0          4/4      Running    0                4m
ibm-spectrum-scale-gui-1          4/4      Running    0                16m
```

1. Verify that the GUI pods are at the correct level.

```
for pod in `kubectl get pods -lapp.kubernetes.io/name=gui -n ibm-spectrum-scale -ojson | jq -r
.items[].metadata.name`; do
    echo -e "\n==== $pod ===="
    kubectl logs $pod -n ibm-spectrum-scale -c liberty | grep 'GPFS GUI'
done

==== ibm-spectrum-scale-gui-0 ====
GPFS GUI Version:5.2.3-0
GPFS GUI Build Date: 20241119-1908

==== ibm-spectrum-scale-gui-1 ====
GPFS GUI Version:5.2.3-0
GPFS GUI Build Date: 20241119-1908
```

Error: daemon and kernel extension do not match

This error occurs when an unintentional upgrade of IBM Storage Scale container native happens and the issue presents itself as the **GPFS state** is **down**. The following error can be found in `/var/adm/ras/mmfs.log.latest` on the core pods.

Error: daemon and kernel extension do not match

To prevent this issue, follow proper upgrade procedures as the kernel module cannot be unloaded when a file system is in use.

To resolve this problem, restart the node or follow procedures to remove application workloads and use the following command to unload the kernel module. For more information, see [Removing applications](#).

```
rmmod tracedev mmfs26 mmfslinux
```

Error: could not insert module, required key not available

This error occurs when **secure boot** is enabled on Red Hat OpenShift nodes and the issue presents itself as the **GPFS state** is **down** by entering the following command:

```
kubectl exec -n ibm-spectrum-scale \
$(kubectl get pods -lapp.kubernetes.io/name=core -n ibm-spectrum-scale -ojsonpath="{.items[0].metadata.name}") \
-- mmgetstate -a
```

The output appears as shown:

| Node number | Node name | GPFS state |
|-------------|-----------|------------|
| 1 | worker0 | active |
| 2 | worker1 | down |
| 3 | worker2 | active |

The following error can be found in `/var/adm/ras/mmfs.log.latest` on the core pods that have GPFS state **down**.

```
ERROR: could not insert module /lib/modules/4.18.0-372.53.1.el8_6.x86_64/extra/tracedev.ko: Required key not available
```

To verify the **secure boot** state and resolve the problem, see [Validate secure boot](#)

MountVolume.SetUp failed for volume ssh-keys

```
Warning FailedMount 83m (x5 over 83m) kubelet, worker-0.example.ibm.com MountVolume.SetUp failed for volume "ssh-keys" : secret "ibm-spectrum-scale-ssh-key-secret" not found
```

Check the pod creation and secret creation times. It is common that the **ssh-key-secret** is created after the deployment of the core pods and the pods cannot find the secret as it does not exist yet.

The message can be misleading as it takes time for the operator to create all the resources. This error is transient and will resolve itself after the secret is present.

If the core pods are not in **Running** state and the secret is created and present for some time, deleting the core pods resolves the issue. This action causes the pods to be re-created and mounts the secret successfully.

Unable to retrieve some image pull secrets

Starting with Red Hat OpenShift 4.15 (kubernetes 1.28), warning messages appear when secrets are referenced in service account but are not created in the namespace. Configuring ICR entitlement by using the Red Hat OpenShift global pull secret result in these messages appearing in the events.

```
"Unable to retrieve some image pull secrets (...); attempting to pull the image may not succeed."
```

- If the warning is for **ibm-entitlement-key**

Resolve this warning by defining namespace pull secrets. For more information, see [Namespace Pull Secrets](#)

- If the warning is for **ibm-spectrum-scale-registrykey**

This secret is deprecated in IBM Storage Scale container native v5.2.2.0. To remove the warning messages the secret name needs to be removed from the service accounts.

Run the following script to clean up the service accounts:

```
for ns in ibm-spectrum-scale ibm-spectrum-scale-operator ibm-spectrum-scale-dns; do
  for sa in $(kubectl get serviceaccount -n $ns -ojson | jq -r '.items[].metadata.name' | grep ibm-); do
    kubectl get serviceaccount $sa -n $ns -o json | \
    jq '(.imagePullSecrets | map(select(.name != "ibm-spectrum-scale-registrykey")))) as $newSecrets | \
    .imagePullSecrets = $newSecrets' | \
    kubectl apply -f -
  done
done
```

After namespace secrets are created, the pods need to restart for the warning messages to quiesce. For core pods only, use the following annotation to have the operator orchestrate the pod restarts, preserving quorum: `kubectl annotate pod -lapp.kubernetes.io/name=core scale.spectrum.ibm.com/pending=delete`. For other pods, you must delete them manually.

GUI or Grafana bridge pods fails to start, no data returned from pmcollector to front end applications

There exists an issue where no data is returned to front end applications that are actively consuming performance metrics from IBM Storage Scale pmcollector. This also has a signature of Grafana Bridge pod failing to start. If this is experienced, apply the following workaround.

1. Check NodeNetworkConfigurationPolicy's to determine which network interfaces are configured for a node network.

- List the NodeNetworkConfigurationPolicies

```
kubectl get nnce
```

Example:

```
# kubectl get nnce
NAME                                     STATUS
compute-0.mycluster.example.com.bond1-ru5-policy  SuccessfullyConfigured
compute-1.mycluster.example.com.bond1-ru6-policy  SuccessfullyConfigured
compute-2.mycluster.example.com.bond1-ru7-policy  SuccessfullyConfigured
control-0.mycluster.example.com.bond1-ru2-policy  SuccessfullyConfigured
control-1.mycluster.example.com.bond1-ru3-policy  SuccessfullyConfigured
control-2.mycluster.example.com.bond1-ru4-policy  SuccessfullyConfigured
```

- Describe the NodeNetworkConfigurationPolicy to identify the network interface being used.

Example:

```
# kubectl describe nnce compute-0.mycluster.example.com.bond1-ru5-policy | grep Name
Name:          compute-0.mycluster.example.com.bond1-ru5-policy
Namespace:
Name:          bond1-ru5-policy
  Name:        bond1
  Name:        bond1.3201
```

In this particular example, the bond interfaces are configured for the node network traffic.

2. Change the Performance Data Collection rules to limit the discovery of the Network adapters to only to the configured interfaces.

- Stop the sensors activities on all Core nodes

```
kubectl get pods -lapp.kubernetes.io/name=core -n ibm-spectrum-scale \
-ojsonpath="{range .items[*]}{.metadata.name}{'\n'}" | \
xargs -I{} kubectl exec {} -n ibm-spectrum-scale -c gpfs -- \
kill $(pgrep -fx '/opt/IBM/zimon/sbin/pmsensors -E /opt/IBM/zimon -C /etc/scale-pmsensors-
configuration/ZIMonSensors.cfg -R /run/perfmon')
```

- Review the current filter settings for the Network sensor in the Performance Data Collection rules. These are stored in the `ibm-spectrum-scale-pmsensors-config` configmap.

```
kubectl describe cm ibm-spectrum-scale-pmsensors-config -n ibm-spectrum-scale | grep filter | grep
netdev
```

Example output:

```
# kubectl describe cm ibm-spectrum-scale-pmsensors-config -n ibm-spectrum-scale | grep filter |
grep netdev
filter = "netdev_name=veth.*|docker.*|flannel.*|cali.*|cbr.*"
```

The `filter =` output is used for exclusion logic.

- Edit the `ibm-spectrum-scale-pmsensors-config` configmap with the following command:

```
kubectl edit ibm-spectrum-scale-pmsensors-config -n ibm-spectrum-scale
```

Replace the substring `netdev_name=veth.*|docker.*|flannel.*|cali.*|cbr.*` with `netdev_name=^(?!bond).*`

Bond interface is being used in this example. Replace the `bond` with the respective adapter name that is used by the customer's network interface.

- Verify that the `ibm-spectrum-scale-pmsensors-config` configmap now reflects the wanted adapter.

```
kubectl describe cm ibm-spectrum-scale-pmsensors-config -n ibm-spectrum-scale | grep filter | grep
netdev
```

3. Clean up the metadata keys in the pmcollector database not related to the configured node network interfaces. Remote the shell into each pmcollector pod and issue the following commands.

```
kubectl -n ibm-spectrum-scale exec -c pmcollector -it \
$(kubectl get pods -lapp.kubernetes.io/name=pmcollector -o jsonpath='{.items[0].metadata.name}') -- sh

echo "delete key .*|Network|[a-f0-9]{15}|.*" | /opt/IBM/zimon/zc 0

echo "topo -c -d 6" | /opt/IBM/zimon/zc 0 | grep Network | cut -d'|' -f2-3 | sort | uniq -c | sort -n |
tail -50
```

Then, exit the container.

Example:

```
# kubectl -n ibm-spectrum-scale exec -c pmcollector -it \
$(kubectl get pods -lapp.kubernetes.io/name=pmcollector -o jsonpath='{.items[0].metadata.name}') -- sh
sh-4.4$ echo "delete key .*|Network|[a-f0-9]{15}|.*)" | /opt/IBM/zimon/zc 0
sh-4.4$ echo "topo -c -d 6" | /opt/IBM/zimon/zc 0 | grep Network | cut -d'|' -f2-3 | sort | uniq -c |
sort -n | tail -50
    96 Network|bond0
    96 Network|bond1
    96 Network|bond1.3201
    96 Network|lo
sh-4.4$ exit

# kubectl -n ibm-spectrum-scale exec -c pmcollector -it \
$(kubectl get pods -lapp.kubernetes.io/name=pmcollector -o jsonpath='{.items[0].metadata.name}') -- sh
sh-4.4$ echo "delete key .*|Network|[a-f0-9]{15}|.*)" | /opt/IBM/zimon/zc 0
sh-4.4$ echo "topo -c -d 6" | /opt/IBM/zimon/zc 0 | grep Network | cut -d'|' -f2-3 | sort | uniq -c |
sort -n | tail -50
    96 Network|bond0
    96 Network|bond1
    96 Network|bond1.3201
    96 Network|lo
sh-4.4$ exit
```

4. Start the sensors jobs on all Core nodes

```
kubectl get pods -lapp.kubernetes.io/name=core -n ibm-spectrum-scale \
-ojsonpath="{range .items[*]}{.metadata.name}{'\n'}" | \
xargs -I{} kubectl exec {} -n ibm-spectrum-scale -c gpfs -- \
/opt/IBM/zimon/sbin/pmsensors -E /opt/IBM/zimon -C /etc/scale-pmsensors-configuration/ZIMonSensors.cfg
-R /run/perfmon
```

5. Delete the pmcollector and Grafana bridge pods to update the configuration changes.

```
kubectl delete pod -lapp.kubernetes.io/instance=ibm-spectrum-scale,app.kubernetes.io/name=pmcollector
kubectl delete pod -lapp.kubernetes.io/instance=ibm-spectrum-scale,app.kubernetes.io/name=grafanabridge
```

After some time, the pmcollector and Grafana bridge pods are redeployed by the ibm-spectrum-scale-operator.

pmcollector pod is in pending state during OpenShift Container Platform upgrade or reboot

Events:

| Type | Reason | Age | From | Message |
|---------|------------------|-----------------------|-------------------|--|
| Warning | FailedScheduling | 65s (x202 over 4h43m) | default-scheduler | 0/6 nodes are available: 1 node(s) were unschedulable, 2 node(s) had volume node affinity conflict, 3 node(s) had taint {node-role.kubernetes.io/master:}, that the pod didn't tolerate. |

This issue is caused by a problem during OpenShift Container Platform Upgrade or when a worker node has not been reset to schedulable after reboot. The pmcollector remains in a **Pending** state until the pod itself and its respective Persistent Volume can be bound to a worker node.

```
# kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
master0.example.com                Ready    master   5d18h   v1.18.3+2fbd7c7
master1.example.com                Ready    master   5d18h   v1.18.3+2fbd7c7
master2.example.com                Ready    master   5d18h   v1.18.3+2fbd7c7
worker0.example.com                Ready    worker   5d18h   v1.17.1+45f8ddb
worker1.example.com                Ready,SchedulingDisabled  worker   5d18h   v1.17.1+45f8ddb
worker2.example.com                Ready    worker   5d18h   v1.17.1+45f8ddb
```

If the Persistent Volume has **Node Affinity** to the host that has **SchedulingDisabled**, the pmcollector pod remains in **Pending** state until the node associated with the PV becomes schedulable.

```
# kubectl describe pv worker1.example.com-pv
Name:                                worker1.example.com-pv
Labels:                              app=scale-pmcollector
Annotations:                          pv.kubernetes.io/bound-by-controller: yes
Finalizers:                          [kubernetes.io/pv-protection]
StorageClass:                        ibm-spectrum-scale-internal
Status:                              Bound
Claim:                               example/datadir-ibm-spectrum-scale-pmcollector-1
Reclaim Policy:                      Delete
Access Modes:                        RWO
VolumeMode:                          Filesystem
Capacity:                             25Gi
```

```
Node Affinity:
Required Terms:
Term 0:      kubernetes.io/hostname in [worker1.example.com]
Message:
Source:
Type: LocalVolume (a persistent volume backed by local storage on a node)
Path: /var/mmfs/pmcollector
```

If the issue was with OpenShift Container Platform upgrade, fixing the upgrade issue should resolve the pending pod.

If the issue is due to worker node in `SchedulingDisabled` state and not due to a failed OpenShift Container Platform upgrade, re-enable scheduling for the worker with the `oc adm uncordon` command.

pmsensors that shows null after failure of pmcollector node

If a node that is running the pmcollector pod is drained, when the node is uncordoned, the pmcollector pods get new IPs assigned. This leads to the pmsensors process issue. It displays the following message:

```
Connection to scale-pmcollector-0.scale-pmcollector successfully established.
```

But an error is reported:

```
Error on socket to scale-pmcollector-0.scale-pmcollector: No route to host (113)
```

See `/var/log/zimon/ZIMonSensors.log`. This issue can also be seen on the pmcollector pod:

```
# echo "get metrics cpu_user bucket_size 5 last 10" | /opt/IBM/zimon/zc 0
1:      worker1
2:      worker2
Row Timestamp      cpu_user
1  2020-11-16 05:27:25    null
2  2020-11-16 05:27:30    null
3  2020-11-16 05:27:35    null
4  2020-11-16 05:27:40    null
5  2020-11-16 05:27:45    null
6  2020-11-16 05:27:50    null
7  2020-11-16 05:27:55    null
8  2020-11-16 05:28:00    null
9  2020-11-16 05:28:05    null
10 2020-11-16 05:28:10    null
```

If the scale-pmcollector pods get their IP addresses changed, the pmsensors process needs to be stopped and restarted manually on all scale-core pods, to get the performance metrics collection resumed.

To stop the pmsensor process, run these commands on all the ibm-spectrum-scale-core pods. The `PMSENSORPID` variable holds the results of the `kubectl exec` command. If this variable is empty, then no process is running, and you do not need to enter the following command to stop the process.

```
PMSENSORPID=`kubectl exec <ibm-spectrum-scale-core> -n ibm-spectrum-scale -- pgrep -fx
'/opt/IBM/zimon/sbin/pmsensors -C /etc/scale-pmsensors-configuration/ZIMonSensors.cfg -R /run/perfmon'`
echo $PMSENSORPID
kubectl exec <scale-pod> -n ibm-spectrum-scale -- kill $PMSENSORPID
```

To start the service again, enter this command on all the scale pods.

```
kubectl exec <scale-pod> -n ibm-spectrum-scale -- /opt/IBM/zimon/sbin/pmsensors -C /etc/scale-pmsensors-
configuration/ZIMonSensors.cfg -R /run/perfmon
```

pmcollector pods are not in Running state

Reasons to fix the pmcollector pods:

- Pods cannot be rescheduled on previously scheduled nodes. The pmcollector nodes were deleted or unrecoverable.
- The pmcollector pods are not updated after updates to the pmcollector's statefulset toleration.

If there is something wrong with the node that is assigned to run the pmcollector pod, perform the actions in [move pmcollector pods to another nodes](#) to attempt to recover.

1. Verify that the pmcollector pods restarted successfully:

- Verify that the operator created the persistent volumes and are Bound.

```
kubectl get pv | grep pmcollector
```

- Verify that the persistent volume claims are Bound.

```
kubectl get pvc -o wide -n ibm-spectrum-scale | grep pmcollector
```

- Verify that the pmcollector pods are running all containers (2/2) on the intended nodes.

```
kubectl get pods -o wide -n ibm-spectrum-scale | grep pmcollector
```

pid_limits set higher than podPidLimits, but not being honored

With Red Hat OpenShift Container Platform 4.11, certain CRI-O fields that are introduced before the support were in kubelet have been deprecated. One of those deprecated fields is `pids_limit` that was configured in the `ContainerRuntimeConfig` CR. For more information, see that CRI-O should deprecate log size max and pids limit options](<https://issues.redhat.com/browse/OCPNODE-878>): external}.

If you had applied a custom MCO configuration with a `pids_limit` value higher than 4096, the container limits are restricted by the default `podPidsLimit` value in `kubelet.conf`. This default is set to 4096 on OpenShift Container Platform 4.11 and later. In order to increase this value, do the following:

It is highly recommended that you are at IBM Storage Scale container native v5.1.5 or higher before changing `MachineConfig` as the IBM Storage Scale container native operator will orchestrate the updates to `MachineConfig` as an attempt to keep the IBM Storage Scale cluster operational.

1. Define the `podPidsLimit` in the `KubeletConfig` custom resource.

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: 01-worker-ibm-spectrum-scale-increase-pid-limit
spec:
  machineConfigPoolSelector:
    matchLabels:
      pools.operator.machineconfiguration.openshift.io/worker: ''
  kubeletConfig:
    podPidsLimit: 8192
```

2. Delete the IBM Storage Scale container native `ContainerRuntimeConfig` resource in order to set the default value for `ContainerRuntime` to 0, effective unlimited:

```
kubectl delete ContainerRuntimeConfig 01-worker-ibm-spectrum-scale-increase-pid-limit
```

Upgrade failed due to PodsUnsafeToDelete: Unknown safety with reason

During an upgrade, not all the pods are updated due to following event message and `GPFS state` of the core pod in the `unknown` state.

```
Events:
  Type      Reason             Age          From          Message
  ----      -
  Normal    PodWaitDelete      39m          Daemon        3 pods will restart because images of containers
config, gpfs, logs, mmbuildgpl are updated. The nodes will be rebooted if the kernel modules can not be
unloaded properly.
  Normal    PodsUnsafeToDelete 7m27s (x3 over 38m) Daemon        3 pods cannot be deleted (worker1, worker2,
worker3). Unknown safety with reason: could not get daemon
```

```
# kubectl exec worker1 -c gpfs -n ibm-spectrum-scale -it -- mmgetstate -a
```

| Node number | Node name | GPFS state |
|-------------|-----------|------------|
| 1 | worker1 | active |
| 2 | worker2 | unknown |
| 3 | worker3 | unknown |

This error can be caused by the nameservers in the `/etc/resolv.conf` and `coredns` service `ClusterIP` address is mismatched.

Example:

```
# kubectl exec worker1 -c gpfs -n ibm-spectrum-scale -it -- cat /etc/resolv.conf
2025-05-30T21:10:17.374074Z: search ibm-spectrum-scale.svc.cluster.local svc.cluster.local cluster.local
fyre.ibm.com svl.ibm.com
nameserver 10.233.41.31
options ndots:5

# kubectl get svc -n ibm-spectrum-scale-dns
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
coredns   ClusterIP   10.233.40.131 <none>         53/UDP,53/TCP    3dlh
```

In the example, the IP in `/etc/resolv.conf` for the nameserver is 10.233.41.31 and does not match the `coredns` service IP of 10.233.40.131. Because of this mismatch, the core pod is unable to communicate with the nameserver to resolve hostnames. To fix this issue, delete the core pod. The operator recreates the core pod and it should have the correct `coredns` service IP in `/etc/resolv.conf`.

Core pods restart constantly because mount parameters have changed

If core pods show events like **"Error: services have not yet been read at least once, cannot construct envvars"** or the core pods keep restarting because of **mount parameters have changed** make sure SELinux is disabled in the Filesystem, and the Network Policy is disabled in the Cluster CR.

1. Filesystem sample:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Filesystem
metadata:
  name: local-sample
spec:
  selinuxOptions: {}
```

2. Cluster sample:

```
apiVersion: scale.spectrum.ibm.com/v1beta1
kind: Cluster
metadata:
  labels:
    app.kubernetes.io/instance: ibm-spectrum-scale
    app.kubernetes.io/name: cluster
  name: ibm-spectrum-scale
  namespace: ibm-spectrum-scale
spec:
  daemon:
    ...
# -----
# networkPolicy enabled by default
# -----
# networkPolicy: {}
~
```

Cluster Issues

- [NumaMemoryInterleave is not set to yes](#)

NumaMemoryInterleave is not set to yes

NumaMemoryInterleave is set to **yes** by default on IBM Storage Scale beginning with version 5.2.0.

IBM Storage Scale container native enabled this functionality beginning with version 5.2.2.0. If you upgraded from any version up to v5.2.2.0, the upgrade process does not modify the value of **NumaMemoryInterleave**. If you find this value is set to **no** and you want to enable it, you will need to set this configuration value to **true** in the cluster CR.

Run the following command to set **numaMemoryInterleave: yes**:

```
kubectl patch cluster.scale ibm-spectrum-scale --type='merge' --patch '{"spec":{"daemon":{"clusterProfile":{"numaMemoryInterleave":"yes"}}}}'
```

Remote cluster Issues

- [RestError: failed to get storage cluster information. errmsg: 401 Unauthorized GET or PasswordsOutOfSync](#)
- [Failed to establish remote cluster connection when cacert ConfigMap does not exist](#)
- [Adding a Remote Cluster to an existing IBM Storage Scale container native cluster taking a long time to appear](#)

RestError: failed to get storage cluster information. errmsg: 401 Unauthorized GET or PasswordsOutOfSync

An **PasswordsOutOfSync** event is shown in **RemoteCluster** CR by using **kubectl describe remoteccluster.scale**. Or events that show errors indicating **401 Unauthorized** are shown in **RemoteCluster** CR or other CRs (like Filesystem CR or Cluster CR). Also, CSI shows **401** and **403** errors. The IBM Storage Scale GUI REST credentials for storage clusters are stored in Kubernetes secrets. The **PasswordsOutOfSync** event or the **401 Unauthorized** errors indicate that the credentials that are provided by the Kubernetes secrets do not match the GUI or CSI user credentials in the storage cluster. For more information, see [Storage cluster](#).

The following lists a few possible root causes:

- The GUI or CSI user was never created as described in the procedure for creating operator user and group. For more information, see [Creating Container Operator and CSI users](#).
- The GUI or CSI user password has expired in the storage cluster and must be changed.
- The GUI or CSI user password is changed in the storage cluster.
- The GUI or CSI user is deleted in the storage cluster.
- Automatic password rotation failed.

Complete the following steps to solve this problem:

1. Get the names of the user credential secrets by entering `kubectl describe remoteclasscluster.scale -n ibm-spectrum-scale` command and looking for **Secret Name** (stores credentials for ContainerOperator user) and for **Csi Secret Name** (stores credentials for CsiAdmin user):

```
...
Spec:
  Contact Nodes:
    storagecluster1node1
    storagecluster1node2
  Gui:
    Cacert:          cacert-storage-cluster-1
    Csi Secret Name: csi-remote-mount-storage-cluster-1
    Host:            guihost.example.com
    Insecure Skip Verify: false
    Port:            443
    Secret Name:     cnsa-remote-mount-storage-cluster-1
...
```

2. If the error is displayed for the ContainerOperator user, the password can be synchronized by:
3. Issuing the following command in the shell of the GUI node of the storage cluster:

```
/usr/lpp/mmfs/gui/cli/chuser cnsa_storage_gui_user -p new_cnsa_storage_gui_password
```

4. Updating the password in the related secret:

```
kubectl patch secret cnsa-remote-mount-storage-cluster-1 -n ibm-spectrum-scale -p="{\"data\": {\"password\": \"`echo -n 'new_cnsa_storage_gui_password' | base64`\"}}\""
```

5. If the error is displayed for the CsiAdmin user, the password can be synchronized by
6. Issuing the following command in the shell of the GUI node of the storage cluster:

```
/usr/lpp/mmfs/gui/cli/chuser csi_storage_gui_user -p new_csi_storage_gui_password
```

7. Updating the password in the related secret:

```
kubectl patch secret csi-remote-mount-storage-cluster-1 -n ibm-spectrum-scale-csi -p="{\"data\": {\"password\": \"`echo -n 'new_csi_storage_gui_password' | base64`\"}}\""
```

Failed to establish remote cluster connection when cacert ConfigMap does not exist

When describing the remote cluster objects, you might see an error: **Error: ConfigMap "cacert-storage-cluster-1" not found.**

This issue is caused by not configuring TLS verification of CA certificates for the remote storage GUI.

For more information and resolution options, see [Configuring Certificate Authority \(CA\) certificates](#).

Adding a Remote Cluster to an existing IBM Storage Scale container native cluster taking a long time to appear

When adding a RemoteCluster CR after initial installation of an IBM Storage Scale container native, it can take some time for the operator to propagate this information to the CSI CR.

To resolve this, manually trigger a reconcile of the operator by deleting the operator pod and allowing it to be re-created.

```
kubectl delete pod -nibm-spectrum-scale-operator -lapp.kubernetes.io/name=operator
```

Once the operator reconciles, it updates the CSI CR with the new RemoteCluster CR.

File system issues

- [Remote file systems are defined but not mounted on all nodes](#)
- [Remote file systems unable to mount successfully](#)

Remote file systems are defined but not mounted on all nodes

If the RemoteMount controller shows that the target storage cluster file system is established, but the remote file system is not mounted on all the nodes in the ibm-spectrum-scale-core pods, run the following command to mount the file system manually from one of the scale-core pods:

```
# Replace FILESYSTEM with the name of your filesystem
FILESYSTEM="fs1"
kubectl exec $(kubectl get pods -lapp=ibm-spectrum-scale-core -ojsonpath="{.items[0].metadata.name}") --
mmount $FILESYSTEM -a
```

Remote file systems unable to mount successfully

Mismatch file system version

On the Filesystem CR, if you see the following event:

```
Events:
  Type      Reason      Age      From      Message
  ----      -
  Warning    MountError   6s (x2 over 19s)  Filesystem   Failed to mount filesystem. Filesystem format level on
the remote filesystem remote-sample, may be at a higher format level than this version of container native
scale.
```

It means that the remote cluster file system has a newer version than the IBM Storage Scale container native cluster. To fix this issue, upgrade the IBM Storage Scale container native cluster to the same file system level or higher as on the remote cluster.

For more information on supported versions, see [Software requirements](#).

Cache volume issues

- [Empty cache mode and health status in CacheVolume CR](#)

Empty cache mode and health status in CacheVolume CR

If you see a **CacheVolume** CR with the empty fields **CACHEMODE** and **HEALTHSTATUS**, by using a command `kubectl get cachevolume`:

```
# kubectl get cachevolume pvc-a5ab1d48-1dc5-4e61-80d0-ba56267eaa65 -n ns1
NAME                                CACHEMODE  HEALTHSTATUS  AGE
pvc-a5ab1d48-1dc5-4e61-80d0-ba56267eaa65    
12m
```

Run the commands to get these fields populated for the **CacheVolume** CR:

1. Delete the affected **CacheVolume** resource:

```
kubectl delete cachevolume <CacheVolume name> -n <namespace>
```

1. Restart of the operator:

```
kubectl rollout restart deployment ibm-spectrum-scale-controller-manager -n ibm-spectrum-scale-operator
```

After the operator is restarted, a new **CacheVolume** resource is created with the fields populated.

Upgrade Issues

- [tls errors found in the operator log](#)

tls errors found in the operator log

After upgrading the IBM Storage Scale container native, you can see many messages in the operator log similar to the following:

```
http: TLS handshake error from <IP>:<PORT> remote error: tls: bad certificate
```

Verify that the Webhooks have a `caBundle` injected for each defined `path`: You can use these commands to check this:

- To check the `MutatingWebhookConfiguration`:

```
kubectl get MutatingWebhookConfiguration ibm-spectrum-scale-mutating-webhook-configuration -oyaml |  
egrep "caBundle:|path:"
```

- To check the `ValidatingWebhookConfiguration`:

```
kubectl get ValidatingWebhookConfiguration ibm-spectrum-scale-validating-webhook-configuration -oyaml  
| egrep "caBundle:|path:"
```

For example, if running the command against the `ibm-spectrum-scale-mutating-webhook-configuration`, you can see an output similar to the following:

```
$ kubectl get MutatingWebhookConfiguration ibm-spectrum-scale-mutating-webhook-configuration -oyaml | egrep  
"caBundle:|path:"  
caBundle: <SOME CA BUNDLE DEFINED, LONG KEY>  
path: /mutate-scale-spectrum-ibm-com-v1beta1-cluster  
path: /mutate-scale-spectrum-ibm-com-v1beta1-filessystem
```

The output shows one `caBundle`: entry and two `path`: entries, meaning for the second path: `/mutate-scale-spectrum-ibm-com-v1beta1-filessystem`, a valid `caBundle` was NOT injected. This is caused by a bug in the Red Hat OpenShift service CA operator, [OCPBUGS-8512](#).

If this has occurred in your cluster, resolve the problem by using the following steps:

1. Delete the `MutatingWebhookConfiguration` and `ValidatingWebhookConfiguration`.

```
kubectl delete MutatingWebhookConfiguration ibm-spectrum-scale-mutating-webhook-configuration  
kubectl delete ValidatingWebhookConfiguration ibm-spectrum-scale-validating-webhook-configuration
```

2. Reapply the manifest for IBM Storage Scale container native.

```
kubectl apply -f https://raw.githubusercontent.com/IBM/ibm-spectrum-scale-container-  
native/v6.0.0.x/generated/scale/install.yaml
```

3. Running the webhook queries again should result in each endpoint having a `caBundle` injected in each path:

```
caBundle: <SOME CA BUNDLE DEFINED, LONG KEY>  
path: /mutate-scale-spectrum-ibm-com-v1beta1-cluster  
caBundle: <SOME CA BUNDLE DEFINED, LONG KEY>  
path: /mutate-scale-spectrum-ibm-com-v1beta1-filessystem
```

Red Hat OpenShift Node issues

- [Adding a node fails - the node appears to already belong to a GPFS cluster](#)

Adding a node fails - the node appears to already belong to a GPFS cluster

When adding a worker node into Red Hat OpenShift, and using the `nodeSelector` of `node-role.kubernetes.io/worker` in the Cluster CR, the IBM Storage Scale Container native operator deploys a core pod to the newly added node and attempt to add this node into the GPFS cluster. There can be a situation where the core pod is in "Init: 1/2" state with no sign of recovery.

The operator log contains entries matching `ERROR Failed to add node` and `mmaddnode` failing with the reason:

```
The node appears to already belong to a GPFS cluster.
```

To recover from this scenario, use the following steps:

1. Create a debug pod to the node where the pod is failing to start and delete the GPFS metadata.

```
oc debug node/<openshift_worker_node> -T -- chroot /host sh -c "rm -rf /var/mmfs; rm -rf /var/adm/ras"
```

Example:

```
oc debug node/worker0.example.com -T -- chroot /host sh -c "rm -rf /var/mmfs; rm -rf /var/adm/ras"  
Starting pod/worker0examplecom-debug ...  
To use host binaries, run `chroot /host`  
Removing debug pod ...
```

2. Delete the core pod. If the core pod is called `worker3`, run: `kubectl delete pod worker3 -n ibm-spectrum-scale`.

3. The operator should reconcile and attempt to create the pod again and succeed.

Red Hat OpenShift Service on AWS

- [Error: InvalidGroup.NotFound](#)
- [Error: CSI pods in CrashLoopBackOff](#)

Error: InvalidGroup.NotFound

The following error is seen in the Cloudkit logs:

```
{"level": "debug", "ts": "2024-02-19T03:11:56.724-0500", "caller": "commonhelpers/common_helpers.go:436", "msg": "...InvalidGroup.NotFound: You have specified two resources that belong to different networks."}
```

The jump host security group or the cloud-vm installer node security group belongs to different VPC than the VPC opted for IBM Storage Scale cluster deployment.

Cloudkit does not support VPC peering. It is required to use the cloud-vm installer node VPC for IBM Storage Scale deployment. If your installer node is in a different VPC, it needs to be migrated to the VPC where you want to deploy the IBM Storage Scale cluster.

Error: CSI pods in CrashLoopBackOff

This issue occurs during IBM Storage Scale container native installation on AWS ROSA where you may see some pods in `CrashLoopBackOff`:

```
oc get pod -n ibm-spectrum-scale-csi
```

| NAME | READY | STATUS | RESTARTS | AGE |
|---|-------|------------------|----------------|-------|
| ibm-spectrum-scale-csi-4s8mv | 1/3 | CrashLoopBackOff | 77 (3m47s ago) | 3h40m |
| ibm-spectrum-scale-csi-attacher-68c6bc8686-ds5qz | 1/1 | Running | 3 (9h ago) | 9h |
| ibm-spectrum-scale-csi-attacher-68c6bc8686-rtkht | 0/1 | CrashLoopBackOff | 195 (104s ago) | 9h |
| ibm-spectrum-scale-csi-hph92 | 3/3 | Running | 0 | 9h |
| ibm-spectrum-scale-csi-provisioner-76fcfdff4f-8kt8d | 1/1 | Running | 3 (9h ago) | 9h |
| ibm-spectrum-scale-csi-resizer-f9865c99d-sx5rg | 1/1 | Running | 3 (9h ago) | 9h |
| ibm-spectrum-scale-csi-snapshotter-5f99797d8-gmnv | 1/1 | Running | 3 (9h ago) | 9h |
| ibm-spectrum-scale-csi-vhxff | 3/3 | Running | 1 (9h ago) | 9h |

Steps to be followed to fix this issue:

1. Checking the logs of the CSI driver pod will show errors when trying to reach a REST endpoint.

```
# kubectl logs ibm-spectrum-scale-csi-4s8mv -n ibm-spectrum-scale-csi
...
I0505 16:51:00.674374    1 gpfs.go:248] [a367a3a6-a4cc-4404-a0eb-dea8c35a13ec] Initialize IBM
Storage Scale CSI driver
E0505 16:52:00.678377    1 rest_v2.go:1459] [a367a3a6-a4cc-4404-a0eb-dea8c35a13ec] rest_v2 doHTTP:
Error in connecting to GUI endpoint https://ibm-spectrum-scale-gui-0.ibm-spectrum-scale-gui.ibm-
spectrum-scale:47443/: Get "https://ibm-spectrum-scale-gui-0.ibm-spectrum-scale-gui.ibm-spectrum-
scale:47443/scalemgmt/v2/cluster": context deadline exceeded (Client.Timeout exceeded while awaiting
headers), checking next endpoint
```

2. Check whether the error is caused from unauthorized GUI requests. For more information, see [CSI pods stuck in CrashLoopBackOff \(Unauthorized GET request\)](#).
3. Verify that network policies are enabled by default, if the issue persists then it might be due to underlying network issues to make connection with GUI pods. For more information, see [Enabling and disabling network policy management](#).

For information on a similar known issue, see [Connections between host-network and normal pods failing after creating NetworkPolicy](#) in the Red Hat knowledgebase.

Troubleshooting

Use the following sections to help troubleshoot and debug specific issues with IBM Storage Scale container native deployment.

- [Troubleshooting operator](#)
- [Troubleshooting deployment](#)
- [Troubleshooting custom resources](#)
 - [Failed to register remote storage cluster on local GUI REST-API](#)

- [Failed login attempts timeout \(403 error\)](#)
- [Troubleshooting IBM Storage Scale Container Storage Interface \(CSI\)](#)
- [Troubleshooting PVCs](#)
- [Troubleshooting Pods](#)
- [Troubleshooting Red Hat OpenShift](#)
 - [GUI mounts not getting refreshed](#)
 - [Recovery after Red Hat OpenShift node failure](#)
- [Troubleshooting cluster maintenance](#)

Troubleshooting operator

- [operator pod is not successfully deployed](#)
- [operator pod shows container restarts](#)
- [operator pod is terminated with OOMKilled](#)

operator pod is not successfully deployed

No operator pod exists in the `ibm-spectrum-scale-operator` namespace.

- Verify that all worker nodes in the Red Hat OpenShift Container Platform cluster are in a **Ready** state. Nodes that are **NotReady** might be preventing the operator pod from being scheduled.

```
kubectl get nodes
```

```
# kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
master0.example.com                Ready     master   65d   v1.18.3+6c42de8
master1.example.com                Ready     master   65d   v1.18.3+6c42de8
master2.example.com                Ready     master   65d   v1.18.3+6c42de8
worker0.example.com                NotReady  worker   65d   v1.18.3+6c42de8
worker1.example.com                NotReady  worker   65d   v1.18.3+6c42de8
worker2.example.com                NotReady  worker   65d   v1.18.3+6c42de8
```

- Inspect the operator namespace and look for details that might point to problems.
 - Check the operator deployment: `kubectl get deployment -n ibm-spectrum-scale-operator`

Describe the deployment for more details:

```
kubectl describe deployment ibm-spectrum-scale-controller-manager -n ibm-spectrum-scale-operator
```

- Check the operator replicaset: `kubectl get replicaset -n ibm-spectrum-scale-operator`

Describe the replicaset for more details:

```
kubectl describe replicaset $(kubectl get replicaset -n ibm-spectrum-scale-operator \
-ojson | jq -r .items[0].metadata.name) -n ibm-spectrum-scale-operator
```

operator pod shows container restarts

Kubernetes keeps the logs of the current container and the previous container.

Look at the previous container logs and look for any issues that might be causing the container to restart.

```
kubectl logs -p $(kubectl get pods -n ibm-spectrum-scale-operator -ojson | \
jq -r .items[0].metadata.name) -n ibm-spectrum-scale-operator
```

operator pod is terminated with OOMKilled

The operator default memory limit is set too low for the cluster configuration and needs to be adjusted to a higher value.

1. Edit the operator deployment:

```
oc edit deploy -n ibm-spectrum-scale-operator ibm-spectrum-scale-controller-manager
```

1. Increase the operator deployment memory limit. This example has 1Gi but can vary depending on the environment.

```
spec:
  containers:
  - args:
    resources:
```

```
limits:
  cpu: 1500m
  memory: 1Gi
```

2. Save and quit (:wq)
3. Verify the deployment restarts the operator pod.

```
oc get pod -n ibm-spectrum-scale-operator
```

| NAME | READY | STATUS | RESTARTS | AGE |
|---|-------|---------|----------|-----|
| ibm-spectrum-scale-controller-manager-dfbb9c87d-xt9qd | 1/1 | Running | 0 | 23s |

Troubleshooting deployment

- [no endpoints available for service "ibm-spectrum-scale-webhook-service"](#)
- [core pods are stuck in init](#)
- [core, GUI, or collector pods are in ErrImgPull or ImagePullBackOff state](#)
- [core, GUI, or collector pods are not up](#)
- [core, GUI, or collector pods show container restarts](#)
- [all pods are running but the GPFS cluster is stuck in the "arbitrating" state](#)
- [remote mount file system not getting configured or mounted](#)

no endpoints available for service "ibm-spectrum-scale-webhook-service"

When creating or editing IBM Storage Scale container native custom resources, it is possible that validating or mutating webhooks fails when the operator pod is unavailable. If you receive an error: **no endpoints for service "ibm-spectrum-scale-webhook-service"**, check the state of the operator pod.

Example error:

```
# kubectl apply -f remotecuster.yaml
remotecuster.scale.spectrum.ibm.com/remotecuster-sample unchanged
Error from server (InternalError): error when creating "remotecuster.yaml": Internal error occurred: failed calling webhook "mcluster.scale.spectrum.ibm.com": failed to call webhook: Post "https://ibm-spectrum-scale-webhook-service.ibm-spectrum-scale-operator.svc:443/mutate-scale-spectrum-ibm-com-v1beta1-cluster?timeout=10s": no endpoints available for service "ibm-spectrum-scale-webhook-service"
```

Checking the operator pod, the STATUS is **ImagePullBackOff**:

```
# kubectl get pods -n ibm-spectrum-scale-operator
```

| NAME | READY | STATUS | RESTARTS | AGE |
|--|-------|------------------|----------------|-----|
| pod/ibm-spectrum-scale-controller-manager-64bb4798df-rrj4j | 0/1 | ImagePullBackOff | 10 (4m14s ago) | 34m |

In the example, the operator is unable to pull the image due to errors in the pull credentials. To remedy the **no endpoints available** issue, the operator problems must be resolved first, then retry the original commands.

core pods are stuck in init

If for some reason, the IBM Storage Scale container native cluster fails to create, the core pods on the worker nodes get stuck in the Init container.

```
# kubectl get pods
```

| NAME | READY | STATUS | RESTARTS | AGE |
|---------|-------|----------|----------|-----|
| ... | | | | |
| worker0 | 2/2 | Init:1/2 | 0 | 2h |
| worker1 | 2/2 | Init:1/2 | 0 | 2h |
| worker2 | 2/2 | Init:1/2 | 0 | 2h |
| worker3 | 2/2 | Init:1/2 | 0 | 2h |

The following steps might fix the problem without having to clean up the cluster.

1. An application might be blocking the CNSA pods from starting up properly. Check what other pods are running on the node.

```
oc get pod -A -owide | grep <node_name>
```

2. The coredns namespace does not have a secret for pulling images. See [Namespace Pull Secrets](#)

```
oc get secret -nibm-spectrum-scale-dns ibm-entitlement-key
```

3. The memory and cpu values that are specified in the client role are insufficient. See [Cluster](#). Search for **client role**.

4. The `hostAliases` might be incorrect. Verify and validate that the name and IP address values are accurate. See [Cluster](#). Search for `hostAliases`.
5. Lastly, the problem cannot be recovered and requires manual clean up. See [Cleanup IBM Storage Scale container native](#) and [Cleanup Red Hat OpenShift nodes](#). For more information on redeploying, see [Installing the IBM Storage Scale container native operator and cluster](#).

core, GUI, or collector pods are in ErrImgPull or ImagePullBackOff state

When viewing `kubectl get pods -n ibm-spectrum-scale`, if any of the pods are in `ErrImgPull` or `ImagePullBackOff` state, use `kubectl describe pod <podname>` to get more details on the pod and look for any errors that may be happening.

```
kubectl describe pod <pod-name> -n ibm-spectrum-scale
```

core, GUI, or collector pods are not up

- If the pods are not deployed in the `ibm-spectrum-scale` namespace, or the cluster is not created, examine operator pod logs:

```
kubectl logs $(kubectl get pods -n ibm-spectrum-scale-operator -ojson | jq -r ".items[0].metadata.name") -n ibm-spectrum-scale-operator
```

core, GUI, or collector pods show container restarts

- Kubernetes keeps the logs of the current container and the previous container. Look at the previous container's logs for any clues by using the following command:

```
kubectl logs -p <scale pod> -n ibm-spectrum-scale
```

all pods are running but the GPFS cluster is stuck in the "arbitrating" state

If the cluster is stuck in an arbitrating state:

- Check the output of `mmlscluster`.

```
kubectl exec $(kubectl get pods -lapp.kubernetes.io/name=core -n ibm-spectrum-scale -o json | jq -r ".items[0].metadata.name") -- mmlscluster
```

- Check the GPFS logs.

```
kubectl logs $(kubectl get pods -lapp.kubernetes.io/name=core -n ibm-spectrum-scale -o json | jq -r ".items[0].metadata.name") -c logs | grep mmfs.log.latest
```

remote mount file system not getting configured or mounted

- Check the `RemoteCluster` objects and the `Filesystem` objects. The `Filesystem` controller waits until the `RemoteCluster` object is Ready before attempting to configure the remote mount file system. Describe the objects and check `Status` or `Events` for any reasons for failures.

- Remote Clusters

```
kubectl get remotecluster.scale -n ibm-spectrum-scale
kubectl describe remotecluster.scale <name> -n ibm-spectrum-scale
```

* File system

```
```bash
kubectl get filesystem.scale -n ibm-spectrum-scale
kubectl describe filesystem.scale <name> -n ibm-spectrum-scale
```
```

Check the ``Status`` and ``Events`` for any reasons of failure.

If nothing, check the operator logs for any errors:

```
```bash
kubectl logs $(kubectl get pods -n ibm-spectrum-scale-operator -ojson | jq -r ".items[0].metadata.name") -n ibm-spectrum-scale-operator
```
```

- Enter the `mmnetverify` command to verify the network between the clusters. For more information, see [mmnetverify command](#) in IBM Storage Scale documentation.

Troubleshooting custom resources

Troubleshooting remote cluster

- [Failed to register remote storage cluster on local GUI REST-API](#)
- [Failed login attempts timeout \(403 error\)](#)

RemoteCluster

- [Failed to register remote storage cluster on local GUI REST-API](#)
- [Failed login attempts timeout \(403 error\)](#)

Failed to register remote storage cluster on local GUI REST-API

The remote cluster resource is not READY and describing the resource shows the status: **Failed to register remote storage cluster on local GUI REST-API. Check the operator logs for more details.**

This error might be caused from the inability to resolve the hostname of the contact nodes on the storage cluster from the core pods. Check the operator logs and look for a failure on `mmremoteccluster add` command.

```
mmremoteccluster: Incorrect node node1.example.com specified for command.  
mmremoteccluster: Command failed. Examine previous error messages to determine cause.
```

To resolve, double check that you are able to resolve the hostnames of the contact nodes by using `nslookup`. If you are unable to fix the site DNS, you can also resolve this using `hostAliases` field in the Cluster spec to add entries into the IBM Storage Scale CoreDNS host file. For more information, see [Host aliases](#).

Failed login attempts timeout (403 error)

The GUI dashboard has a **Failed login attempts timeout** parameter that can be used to reduce the extended time that is taken by CSI operations when automatic password rotation is activated for **remoteccluster** GUI.

The indication of extended time is seen when CSI related operation like volume creation shows a failure event with message string code **Internal desc = '403: Forbidden GET request'**. This is a temporary error and only impact CSI operations, which are in flight or submitted during the password rotation process. The error is resolved when Kubernetes reattempt the operation again but it can take more time (up to 8 min) to complete the operation. This timing can be further reduced by changing **Failed login attempts timeout** to **1 min** from default **5 min**, which help to reduce the extended time by 4 minutes.

Steps to modify the parameter:

1. Log in to the GUI dashboard with [Security Administrator Access](#).
2. Go to Services -> GUI -> Preferences.
3. Change **Failed login attempts timeout** to **1 min**.

Troubleshooting IBM Storage Scale Container Storage Interface (CSI)

- [CSI pods stuck in CrashLoopBackOff \(Unauthorized GET request\)](#)
- [CSI CR is never created](#)

CSI pods stuck in CrashLoopBackOff (Unauthorized GET request)

The following output shows an example of the CSI pods in **CrashLoopBackOff**.

```
# kubectl get pods  
NAME                                READY   STATUS             RESTARTS   AGE  
ibm-spectrum-scale-csi-95661        1/2     CrashLoopBackOff    9           26m  
ibm-spectrum-scale-csi-attacher-0    1/1     Running             0           85m  
ibm-spectrum-scale-csi-klr7x         1/2     CrashLoopBackOff    9           26m  
ibm-spectrum-scale-csi-provisioner-0 1/1     Running             0           85m  
ibm-spectrum-scale-csi-xlxkl        1/2     CrashLoopBackOff    9           26m
```

The logs of the CSI pods might reveal what is causing the problems.

```
# kubectl logs ibm-spectrum-scale-csi-95661 -c ibm-spectrum-scale-csi
```

```
...
```

```
I1218 17:27:33.875884      1 http_utils.go:60] http_utils FormatURL. url: https://ibm-spectrum-scale-gui-ibm-spectrum-scale.apps.example.com:443/
I1218 17:27:33.875894      1 rest_v2.go:586] rest_v2 doHTTP. endpoint: https://ibm-spectrum-scale-gui-ibm-spectrum-scale.apps.example.com:443/scalemgmt/v2/cluster, method: GET, param: <nil>
I1218 17:27:33.875900      1 http_utils.go:74] http_utils HttpExecuteUserAuth. type: GET, url: https://ibm-spectrum-scale-gui-ibm-spectrum-scale.apps.example.com:443/scalemgmt/v2/cluster, user: csi-cnsa-gui-user
```

- Check that the `csi-cnsa-gui-user` role was created.

```
# kubectl exec ibm-spectrum-scale-gui-0 -n ibm-spectrum-scale -- /usr/lpp/mmfs/gui/cli/luser
Defaulting container name to liberty.
Use 'kubectl describe pod/ibm-spectrum-scale-gui-0 -n ibm-spectrum-scale' to see all of the
containers in this pod.
  Name                               Long name Password status Group names                               Failed login attempts Target Feedback
Date
ContainerOperator                    active                               ContainerOperator 0
EFSSG1000I The command completed successfully.
```

In this case, the `csi-cnsa-gui-user` role was not created. To resolve the issue, enter the following command to create the GUI user:

```
# kubectl exec -c liberty ibm-spectrum-scale-gui-0 -n ibm-spectrum-scale --
/usr/lpp/mmfs/gui/cli/mkuser csi-cnsa-gui-user -p csi-cnsa-gui-password -g CsiAdmin
EFSSG0019I The user csi-cnsa-gui-user has been successfully created.
EFSSG1000I The command completed successfully.
```

- Check that the `csi-remote-mount-storage-cluster-1` secret was created with correct credentials.

```
# kubectl get secrets csi-remote-mount-storage-cluster-1 -n ibm-spectrum-scale-csi -
ojsonpath='{.data.username}' | base64 --decode
csi-cnsa-gui-user

# kubectl get secrets csi-remote-mount-storage-cluster-1 -n ibm-spectrum-scale-csi -
ojsonpath='{.data.password}' | base64 --decode
this-is-a-bad-password
```

In this case, the `csi-remote-mount-storage-cluster-1` secret was created without the correct password. To resolve the issue, enter the following command to delete the secret and re-create it with correct values:

```
# kubectl delete secrets csi-remote-mount-storage-cluster-1 -n ibm-spectrum-scale-csi
secret "csi-remote-mount-storage-cluster-1" deleted

# kubectl create secret generic csi-remote-mount-storage-cluster-1 --from-literal=username=csi-cnsa-
gui-user --from-literal=password=csi-cnsa-gui-password -n ibm-spectrum-scale-csi
secret/csi-remote-mount-storage-cluster-1 created

# kubectl label secret csi-remote-mount-storage-cluster-1 product=ibm-spectrum-scale-csi -n ibm-
spectrum-scale-csi
secret/csi-remote-mount-storage-cluster-1 labeled
```

CSI CR is never created

When all the core pods are running and the IBM Storage Scale container native cluster appears to be in a good state, the CSI CR is created automatically. In some error paths this does not happen and causes the driver pods to not be scheduled.

```
# kubectl get pods -n ibm-spectrum-scale-csi
No resources found in ibm-spectrum-scale-csi namespace.
```

- Check that all the GUI pods are up and running.

```
# kubectl get pods -n ibm-spectrum-scale
NAME                               READY   STATUS    RESTARTS   AGE
ibm-spectrum-scale-gui-0           4/4     Running   0           3m58s
ibm-spectrum-scale-gui-1           4/4     Running   0           95s
ibm-spectrum-scale-pmcollector-0    2/2     Running   0           3m59s
worker0                            2/2     Running   0           3m59s
worker1                            2/2     Running   0           3m58s
worker2                            2/2     Running   0           3m58s
```

All GUI pods must be up and running before the CSI CR is created. Each pod can take a few minutes for all containers in the pod to enter the `Running` state.

- Check that the daemon status has a nonempty cluster ID.

```
kubectl describe daemon -n ibm-spectrum-scale
```

Find the status section and validate that the `Cluster ID` field exists and is not empty.

Status:
Cluster ID: 3004252500454687654
Cluster Name: example.cluster.com

If those fields are missing, then the IBM Storage Scale container native cluster is experiencing an issue. Check the operator logs for more information.

Troubleshooting PVCs

- [PVC creation issue: PVC binding in pending state](#)

PVC creation issue: PVC binding in pending state

If there are problems when PVCs are created:

- Check whether multiple GUIs are installed at the remote storage cluster.
- SSH to one of the GUI nodes and run the following command:

```
mmfsfileset <filesystem name>
```

- Check whether file set is unlinked.

If multiple GUIs are installed, then contact IBM support for further guidance.

Troubleshooting Pods

- [Move pmcollector pods to other nodes](#)
- [Move pmcollector pod to another node](#)

The pmcollector pods are scheduled on nodes that host Persistent Volumes. To allow the pod to be rescheduled to a different node, it is necessary to clean up the associated Persistent Volume (PV) and Persistent Volume Claim (PVC) being used by the pod.

Move pmcollector pods to other nodes

1. Stop the pmcollector pods by scaling the statefulset down to 0:

```
kubectl scale statefulset.apps/ibm-spectrum-scale-pmcollector -n ibm-spectrum-scale --replicas=0
```

2. Delete the pmcollector's Persistent Volume Claims and Persistent Volumes:

```
kubectl delete pvc -lapp.kubernetes.io/instance=ibm-spectrum-scale,app.kubernetes.io/name=pmcollector -n ibm-spectrum-scale
kubectl delete pv -lapp.kubernetes.io/instance=ibm-spectrum-scale,app.kubernetes.io/name=pmcollector -n ibm-spectrum-scale
```

3. Start the pmcollector pods by scaling up the statefulset to 2:

```
kubectl scale statefulset.apps/ibm-spectrum-scale-pmcollector -n ibm-spectrum-scale --replicas=2
```

4. Restart the operator deployment to create a new pmcollector pod on a different node:

```
kubectl rollout restart deployment/ibm-spectrum-scale-controller-manager -n ibm-spectrum-scale-operator
```

Move pmcollector pod to another node

To change an individual pmcollector pod to a different node, use these steps:

Delete Persistent Volume Claim (PVC) attached to the pmcollector pod

1. Identify the PVC needed to delete, in this case, the pod that needs reschedule is the pmcollector-0.

```
kubectl get pvc -n ibm-spectrum-scale -lapp.kubernetes.io/name=pmcollector
```

Example output:

```
$ kubectl get pvc -n ibm-spectrum-scale -lapp.kubernetes.io/name=pmcollector
NAME                                STATUS    VOLUME
```

| CAPACITY | ACCESS MODES | STORAGECLASS | | VOLUMEATTRIBUTESCLASS | AGE |
|--|--------------|---|---------|-----------------------|-----|
| datadir-ibm-spectrum-scale-pmcollector-0 | Bound | worker3.example.com.vmlocal-pmcollector | | | |
| 25Gi | RWO | ibm-spectrum-scale-internal | <unset> | | 20d |
| datadir-ibm-spectrum-scale-pmcollector-1 | Bound | worker5.example.com.vmlocal-pmcollector | | | |
| 25Gi | RWO | ibm-spectrum-scale-internal | <unset> | | 20d |

The PVC needed to delete is `datadir-ibm-spectrum-scale-pmcollector-0`.

2. Delete the PVC:

```
kubectl delete pvc <pvc_name> -n ibm-spectrum-scale
```

Example output:

```
$ kubectl delete pvc datadir-ibm-spectrum-scale-pmcollector-0 -n ibm-spectrum-scale
persistentvolumeclaim "datadir-ibm-spectrum-scale-pmcollector-0" deleted
```

Delete the PV that was being used by the pmcollector pod:

1. Identify the PV needed to delete, it is the one that is claimed by `pmcollector-0`.

```
kubectl get pv -n ibm-spectrum-scale
```

Example output:

```
$ kubectl get pv -n ibm-spectrum-scale
```

| NAME | CAPACITY | ACCESS MODES | RECLAIM POLICY | STATUS | CLAIM |
|---|-----------------------|-----------------------------|----------------|---------|-------|
| STORAGECLASS | VOLUMEATTRIBUTESCLASS | REASON | AGE | | |
| worker3.example.com.vmlocal-pmcollector | 25Gi | RWO | Retain | Bound | |
| ibm-spectrum-scale/datadir-ibm-spectrum-scale-pmcollector-1 | | ibm-spectrum-scale-internal | | <unset> | |
| 20d | | | | | |
| worker5.example.com.vmlocal-pmcollector | 25Gi | RWO | Retain | Bound | |
| ibm-spectrum-scale/datadir-ibm-spectrum-scale-pmcollector-0 | | ibm-spectrum-scale-internal | | <unset> | |

In this case, it is the `worker3.example.com.vmlocal-pmcollector` one.

2. Delete the PV:

```
kubectl delete pv <pv_name> -n ibm-spectrum-scale
```

Example output:

```
kubectl delete pv worker3.example.com.vmlocal-pmcollector -n ibm-spectrum-scale
persistentvolume "worker3.example.com.vmlocal-pmcollector" deleted
```

3. Delete the pmcollector pod.

```
kubectl delete pod <pv_name> -n ibm-spectrum-scale
```

Example output:

```
kubectl delete pod ibm-spectrum-scale-pmcollector-0 -n ibm-spectrum-scale
pod "ibm-spectrum-scale-pmcollector-0" deleted
```

4. Restart the operator deployment to create a new `pmcollector` pod on a different node:

```
kubectl rollout restart deployment/ibm-spectrum-scale-controller-manager -n ibm-spectrum-scale-operator
```

Troubleshooting Red Hat OpenShift

- [GUI mounts not getting refreshed](#)
- [Recovery of IBM Storage Scale container native cluster after Red Hat OpenShift node failure](#)

GUI mounts not getting refreshed

During upgrading Red Hat OpenShift, a problem can occur where the GUI mount is not being refreshed when multiple Red Hat OpenShift clusters are remote mounting the same file system.

To resolve this issue, unmount the file system from the other Red Hat OpenShift clusters.

```
# /usr/lpp/mmfs/gui/cli/runtask FILESYSTEM MOUNT
err: Batch entry 3 INSERT INTO FSCC.FILESYSTEM MOUNTS (CLUSTER_ID, DEVICENAME, HOST_NAME, MOUNT_MODE,
LAST_UPDATE) VALUES ('5228226002706731921', 'fs1', 'worker1.example.com', 'RW', '2021-07-28
```

```
19:06:15.111000+00':timestamp) was aborted: ERROR: duplicate key value violates unique constraint
"filesystem_mounts_pk"
Detail: Key (host_name, cluster_id, devicename)=(worker1.example.com, 5228226002706731921 , fs1) already
exists. Call getNextException to see other errors in the batch.
EFSSG1150C Running specified task was unsuccessful.
# /usr/lpp/mmfs/gui/cli/runtask FILESYSTEM_MOUNT
EFSSG1000I The command completed successfully.
# exit
exit
```

Recovery of cluster after Red Hat OpenShift node failure

IBM Storage Scale container native operator does not have support for the removal of a node. There have been cases where a node fails to recover after the node suffers an outage, for example, after a Red Hat OpenShift update.

- [Before you begin](#)
- [Collect configuration before servicing the OpenShift Container Platform cluster](#)
- [Delete one Scale node from IBM Storage Scale container native cluster](#)
- [Add new Red Hat OpenShift node for use in existing IBM Storage Scale container native cluster](#)
- [Validate the IBM Storage Scale container native cluster after removal and replacement of Red Hat OpenShift node](#)

Before you begin

Before the maintenance of the failed node, ensure that the node selectors, quorum and manager designations, and network configuration annotations are fully understood. These configurations are applied in subsequent steps after removal of the node to ensure parity of the cluster.

- Log the node selector labels (required for the Scale core pods) that are being used by the Cluster CR.
- Understand if the node to be removed is a quorum node for the IBM Storage Scale container native cluster.
- Identify if a CNI network configuration is in use for the IBM Storage Scale container native cluster.

Collect configuration before servicing the OpenShift Container Platform cluster

Run an IBM Storage Scale container native must-gather to collect the state before servicing the Red Hat OpenShift cluster. For more information, see [must-gather](#)

Delete one Scale node from IBM Storage Scale container native cluster

To delete the Scale node from IBM Storage Scale container native cluster, complete the following steps:

1. Stop the running operator pod by setting the **replicas** in the deployment to 0.

```
kubectl scale deployment ibm-spectrum-scale-controller-manager -n ibm-spectrum-scale-operator --
replicas=0
```

2. If the failed Red Hat OpenShift node exists, delete the corresponding Scale core pod.

```
kubectl delete pod REPLACE_WITH_CORE_POD_OF_FAILED_OCP_NODE -n ibm-spectrum-scale
```

3. Enter a currently running Scale core pod to run the remaining commands.

```
kubectl -n ibm-spectrum-scale rsh REPLACE_WITH_RUNNING_CORE_POD
```

4. Note the quorum and manager designations as denoted in **mmiscluster**. This information is needed when we attempt to add back a new node.

```
mmiscluster
```

5. Map the affected Red Hat OpenShift node to the GPFS admin interface. It should report a status of **Unknown**.

The GPFS admin interface would have the shortname of the affected Red Hat OpenShift node.

```
mmgetstate -a
```

6. Ensure that most Scale nodes are in the **Active** state. Then, force delete the desired GPFS admin interface from the Scale cluster.

Exercise caution when shutting down GPFS on quorum nodes or deleting quorum nodes from the GPFS cluster. If the number of remaining quorum nodes falls under the requirement for a quorum, then you are unable to perform file system operations.

```
mmdelnode -N REPLACE_WITH_FAILED_GPFS_ADMIN_NODE --force
```

7. Validate that the bad GPFS admin node is deleted from the Scale cluster.

```
mmgetstate -a
```

8. Exit from the Scale core pod.

```
exit
```

Add new Red Hat OpenShift node for use in existing IBM Storage Scale container native cluster

The previous node configurations of the failed node need to be applied to the new Red Hat OpenShift node. So that the IBM Storage Scale container native operator can successfully add it to the existing IBM Storage Scale container native cluster.

1. Ensure the quorum and manager designations are matching what the previous node had in place. Ensure that the GPFS cluster is not in minority of quorum. If the number of remaining quorum nodes falls under the requirement for a quorum, then you are unable to perform file system operations.

Alternatively, if a node is removed but no replacement node is added, ensure that the appropriate number of nodes are assigned to satisfy the quorum.

2. If using CNI, ensure that the network configuration is valid according to CNI documentation. Ensure that the `scale.spectrum.ibm.com/daemon-network` annotation is present on the new Red Hat OpenShift node. For more information, see [Container network interface \(CNI\) configuration](#).
3. Ensure the node selector labels (required for the Scale core pods) from the Cluster CR are present on the newly added Red Hat OpenShift node.
4. Scale up the operator pods by setting the `replicas` in the deployment to 1.

```
kubectl scale deployment ibm-spectrum-scale-controller-manager -n ibm-spectrum-scale-operator -- replicas=1
```

5. The new Scale core pod should be created and go to running in a short amount of time.

Validate IBM Storage Scale container native cluster after removal and replacement of Red Hat OpenShift node

After the removal and replacement of the failed Red Hat OpenShift node has been completed, perform the validation steps. For more information, see [Validating installation](#). Ensure the following:

- All pods are in a running state. Make sure that the CSI pod on the replaced node is running. This validates that file systems are active and running.
- Validate that the IBM Storage Scale cluster has successfully added the node with the desired node designation.
- Validate that the nodes are active in the IBM Storage Scale cluster.

Troubleshooting cluster maintenance

Identifying applications that prevent cluster maintenance

Debug steps to determine when a cluster maintenance action is not being completed.

When do you see this problem?

A drain occurs when a core pod is selected for deletion. Many actions prompt a core pod deletion:

- Pod evictions
 - Red Hat OpenShift Machine Config Operator
 - User-initiated drains
- Pod spec updates
 - Resource requests, for example to change core pod requests for CPU or memory
 - Image updates prompted by a new release

What does this look like?

An update that is driven by pod spec updates does present as core pods that wait for deletion. An update that is driven by Red Hat OpenShift Machine Config Operator (MCO) ceases to update nodes. The signature does look similar between the two scenarios. Use the following steps to determine where to direct the support case.

When to open a support case?

To determine whether the MCO is stuck due to IBM Storage Scale container native, run the following command:

```
kubectl describe daemon -n ibm-spectrum-scale
```

Example output:

```
...
Status:
...
  Cordon And Drain:
    Nodes Cordoned: worker0.example.ibm.com
    Nodes Draining:
      Node: worker0.example.ibm.com
    Ongoing Pod Evictions:
    Pod Evictions Failed:
      myworkload/mypod-0
      myworkload/mypod-1
    Pod Eviction Requests:
      Pods: worker0, worker1, worker2
      Requestor: machine-config-controller
    ...
  Status Details:
    ...
    Pods Waiting To Be Deleted:
      Delete Reason: Eviction of 3 pods requested by machine-config-controller
      Pods: worker0, worker1, worker2
    ...
Events:
  Type      Reason              Age   From      Message
  ----      -
  Normal    PodEvictionRequested  7m9s  Daemon    Eviction of 3 pods requested by machine-config-controller.
  Normal    NodeDraining         6m28s Daemon    Node worker2.example.ibm.com drained by Storage Scale operator. Evicting 6 pods.
  Warning   NodeDrainFailed      87s   Daemon    Failed to evict pod myworkload/mypod-0 on node worker0.example.ibm.com. Reason: Cannot evict pod as it would violate the pod's disruption budget.
  Warning   NodeDrainFailed      87s   Daemon    Failed to evict pod myworkload/mypod-1 on node worker0.example.ibm.com. Reason: Cannot evict pod as it would violate the pod's disruption budget.
```

If the **Pod Evictions Failed:** list is empty and no **NodeDrainFailed** events are present, then IBM Storage Scale container native is blocking the ongoing drain. To resolve it, raise a support ticket to IBM. For more information, see [Gather data about the IBM Storage Scale container native cluster](#).

If pods are listed in **Pod Evictions Failed:**, check the message that is provided in the **NodeDrainFailed** events. The pods that are listed in **Pod Evictions Failed:** block the node drain. Try to solve the problem with the pods that are blocking the node drain. If you cannot solve this problem, raise a support ticket to Red Hat. For more information, see [Gather data about the Red Hat OpenShift Container Platform cluster](#).

Identifying signatures of an ongoing update

Complete the following steps:

1. Check the Daemon status to verify whether any pods are awaiting deletion.

```
kubectl describe daemon -n ibm-spectrum-scale
```

Example output:

```
Status Details:
...
  Pods Waiting To Be Deleted:
    Delete Reason: Eviction of 3 pods requested by machine-config-controller.
    Pods: worker0, worker1, worker2
    Delete Reason: Pod worker1 will restart because image of containers config, gpfs, mmbuildgpl are updated. The node will be rebooted if the kernel modules can not be unloaded properly.
    Pods: worker1
```

2. Check whether any nodes are cordoned.

```
kubectl get nodes
```

Example:

| NAME | STATUS | ROLES | AGE | VERSION |
|-------------------------|--------|--------|-----|-----------------|
| master0.example.ibm.com | Ready | master | 23d | v1.24.0+3882f8f |

| | | | | |
|-------------------------|--------------------------|--------|-----|-----------------|
| master1.example.ibm.com | Ready | master | 23d | v1.24.0+3882f8f |
| master2.example.ibm.com | Ready | master | 23d | v1.24.0+3882f8f |
| worker0.example.ibm.com | Ready,SchedulingDisabled | worker | 23d | v1.24.0+3882f8f |
| worker1.example.ibm.com | Ready | worker | 23d | v1.24.0+3882f8f |
| worker2.example.ibm.com | Ready | worker | 23d | v1.24.0+3882f8f |

```
kubectl describe daemon -n ibm-spectrum-scale
```

Example:

```
Cordon And Drain:
Nodes Cordoned: worker0.example.ibm.com
```

3. Check events in the daemon.

```
kubectl describe daemon -n ibm-spectrum-scale
```

Example event:

```
Warning NodeDrainFailed 87s Daemon Failed to evict pod myworkload/mypod-0 on node
worker0.example.ibm.com. Reason: Cannot evict pod as it would violate the pod's disruption budget.
```

Gather data about your cluster

When opening issue to IBM Support, user need to provide information that helps to perform root cause analysis. The following sections describe the process for gathering data on the cluster.

- [Red Hat OpenShift Container Platform](#)
 - [must-gather](#)
 - [Kernel crash dumps](#)
- [Kubernetes](#)
 - [must-gather](#)
- [Generating GPFS trace reports](#)
- [Generating GPFS kernel thread dumps](#)

Gather data on openshift cluster

- [must-gather](#)
- [Kernel crash dumps](#)

must-gather

The `oc adm must-gather` CLI command is used to collect information about the cluster.

Prerequisites

- Access to the cluster as a user with the `cluster-admin` role
- The OpenShift Container Platform CLI (oc) command installed
- The `rsync` utility installed (optional)

Gather data about the Red Hat OpenShift Container Platform cluster

For issues with the Red Hat OpenShift Container Platform cluster where a ticket must be opened with Red Hat Support, provide the debugging information about the cluster for problem determination. For more information, see [Gathering data about your cluster](#) in Red Hat OpenShift documentation.

Executing a default `must-gather` for OpenShift Container Platform debug does **not** collect information for IBM Storage Scale container native.

Gather data about the IBM Storage Scale container native cluster

To gather data specifically for the IBM Storage Scale container native cluster to provide to IBM Support, point the `must-gather` tool to the `ibm-spectrum-scale-must-gather` image. This collects Kubernetes resources that are associated with the IBM Storage Scale container native cluster as well as retrieving a GPFS snap.

`oc adm must-gather --image` requires the image that is stored in a repository where it can be anonymously pulled (no credentials required). In an airgapped environment, the `ibm-spectrum-scale-must-gather` image must be pulled from the IBM Cloud Container Registry and then uploaded to an internal image registry allowing anonymous pull. For more information about air gap instructions, see [Air-gapped installs](#).

1. In the directory where your must-gather contents are to be stored, enter the must-gather command by using the `ibm-spectrum-scale-must-gather` image:

```
oc adm must-gather --image=icr.io/cpopen/ibm-spectrum-scale-must-gather:v6.0.0.2
```

By default the `gpfs.snap` command is executed with option `--limit-large-files latest` by must-gather that only collects the latest large snap files. The must-gather supports to specify options of `gpfs.snap` command to collect other data, as below:

```
oc adm must-gather --image=icr.io/cpopen/ibm-spectrum-scale-must-gather:v6.0.0.2 -- 'gather --gpfsnap --exclude-merge-logs --limit-large-files 2'
```

Custom options for `gpfs.snap` should be used only under instruction from IBM support.

2. Once completed, a new directory with `must-gather` prefix is created in your working directory. For example:

```
# ls -ltr
drwxr-xr-x 3 root root      229 Jun 14 09:11 must-gather.local.681612165636007567
```

3. Create a compressed file from the must-gather directory that was created in your working directory.

```
tar cvaf must-gather.tar.gz must-gather.local.681612165636007567/
```

Replace the directory name used in this example with your respective must-gather directory.

Kernel crash dumps

Red Hat Enterprise Linux CoreOS (RHCOS) based machines do not support configuring `kdump` or generating kernel crash dumps for Red Hat OpenShift Container Platform 4.6 and earlier. For more information, see [How to configure kdump in Red Hat CoreOS](#) in Red Hat OpenShift documentation.

In some virtual machine installations, it may be possible to generate a vmcore crash dump from the hypervisor.

In lieu of kernel dumps, CoreOS currently recommends using `pstore`, even if only small snippets of diagnostic data can be collected. For more information, see [Using pstore](#) in CoreOS documentation on GitHub.

Gather data on Kubernetes

Gathering debug data on the Kubernetes cluster and define `kthread` dump procedure to collect kernel crash dumps

- [must-gather](#)

Gather Debug Data on Kubernetes cluster

`kubectl` does not have a must-gather command. Follow these steps to collect data from your Kubernetes cluster to send to IBM Support.

1. Run this command to expose the port 6060 of IBM Storage scale container native operator pod on local machine:

```
kubectl port-forward pod/$(kubectl get pod -n ibm-spectrum-scale-operator --no-headers -o custom-columns=":metadata.name" --field-selector=status.phase==Running) -n ibm-spectrum-scale-operator 6060 &
export PORTFORWARDPID=$!
```

2. Run this command to start must-gather:

```
curl https://localhost:6060/mustgather -k -ls | bash
```

By default the `gpfs.snap` command ran with option `--limit-large-files latest` by must-gather that collects the latest large snap files. The must-gather supports to specify options of `gpfs.snap` command to collect other data with environment variable `GPFSSNAP`, for example: `export GPFSSNAP="--exclude-merge-logs --limit-large-files 1"`.

3. A new directory with `scale-must-gather` prefix is created in the working directory.

```
# ls -ltr
drwxrwxrwx 5 root root      112 Nov 21 11:08 scale-must-gather-2024-11-21-11-08-25
```

```
# ls -ltr scale-must-gather-2024-11-21-11-08-25
total 32
-rw-r--r-- 1 root root 247 Nov 21 10:59 version
-rw-r--r-- 1 root root 22220 Nov 21 10:59 api-resources.txt
drwxr-xr-x 15 root root 4096 Nov 21 11:08 cluster-scoped-resources
drwxr-xr-x 8 root root 176 Nov 21 11:08 namespaces
drwxr-xr-x 2 root root 100 Nov 21 11:20 k8sdumps
```

Generating GPFS trace reports

Some issues might require low-level system detail accessible only through the IBM Storage Scale daemon and the IBM Storage Scale Linux kernel trace facilities.

In such instances the IBM Support Center might request such GPFS trace reports to facilitate rapid problem determination of failures.

The level of detail that is gathered by the trace facility is controlled by setting the trace levels using the `mmtracectl` command. For more information, see [mmtracectl command](#) in IBM Storage Scale documentation.

The following steps must be performed under the direction of the IBM Support Center.

1. Enter the following command to access a running `ibm-spectrum-scale-core` pod:

```
kubectl -n ibm-spectrum-scale exec -it \
$(kubectl get pods -n ibm-spectrum-scale -lapp.kubernetes.io/name=core -o
jsonpath='{.items[0].metadata.name}') -- sh
```

The pod must be in **Running** status to connect. It is best to pick a pod running on a node that is not exhibiting issues.

The remaining steps should be completed while connected to this shell running inside the `gpfs` container of this running core pod.

2. Enter the `mmchconfig` command to change the `dataStructureDump` field to point to `/var/adm/ras`. This changes the default location where trace data is stored to a directory that persists on the host machine:

```
mmchconfig dataStructureDump=/var/adm/ras/
```

3. Set desired trace classes and levels.

This part of the process is identical to classic IBM Storage Scale installs. For more information, see [Generating GPFS trace reports](#) in IBM Storage Scale documentation.

```
mmtracectl --set --trace={io | all | def | "Class Level [Class Level ...]}
```

4. Start the trace facility on all nodes by entering the following command:

```
mmtracectl --start
```

5. Re-create the problem.

6. Stop the trace generation as soon as the problem to be captured occurs, by entering the following command:

```
mmtracectl --stop
```

7. Turn off trace generation by entering the following command:

```
mmtracectl --off
```

Generating GPFS kernel thread dumps

If a deadlock is suspected, a kernel threads dump can be used to obtain debug data. The kernel thread dump is a snapshot of all kernel threads. In IBM Storage Scale container native deployment, kernel threads that are captured on each core pod is limited to the threads visible in the same container. It is not sufficient as any thread on the same node that can call into the Storage scale kernel module might be involved in a deadlock.

This document describes the steps that are required to capture all threads currently running in a cluster. Monitoring all threads across nodes helps diagnose thread-related issues, such as deadlocks, by collecting comprehensive logs.

If a deadlock happens and thread capture is not enabled, the system does not collect any thread data. Therefore, you must enable thread capture prior to reproducing the issue to help ensure that data is collected correctly.

Thread collection begins only after the `threadDeadlockCapture` flag is set, which triggers the system to start listening on the socket. The captured thread data is included in `gpfs.snap` that gets taken by a must-gather run. {note}

Steps to Enable Deadlock Threads

1. Enable Thread Capture by using the Cluster Custom Resource (CR)

To begin with, use the following command to patch the Cluster spec:

```
kubectl patch cluster.scale.spectrum.ibm.com ibm-spectrum-scale --type=json -p='[{"op": "add", "path": "/spec/debugConfig", "value": {"threadDeadlockCapture": true}}]'
```

{.codeblock}

This configuration change triggers the creation of a DaemonSet (debug) which runs on all nodes to collect kernel thread information.

2. Verify the DaemonSet is running with all number of pods

Ensure that the required number of pods in the DaemonSet are running and ready:

```
kubectl get ds debug -n ibm-spectrum-scale
```

Check that the DESIRED and CURRENT pod counts match, indicating that the collection agents are running on all nodes.

Steps to Disable Deadlock Threads

If you want to disable the capture mechanism, you can use the as follows:

```
kubectl patch cluster.scale.spectrum.ibm.com ibm-spectrum-scale --type=json -p='[{"op": "add", "path": "/spec/debugConfig", "value": {"threadDeadlockCapture": false}}]'
```

{.codeblock}

Disabling threadDeadlockCapture deletes the DaemonSet and its associated pods. This action removes the thread collection agents from all nodes. As a result, further thread log collection is stopped.

References

- [IBM Storage Scale](#)
- [Red Hat OpenShift or Kubernetes](#)

IBM Storage Scale

- [Administration Guide](#)
- [For Linux on Z: Changing the kernel settings](#)
- [mmchconfig command](#)
- [mmnetverify command](#)
- [Accessing a remote GPFS file system](#)
- [Defining the cluster topology for the installation toolkit](#)
- [Node quorum](#)
- [Deploying the IBM Storage Scale Container Storage Interface driver](#)

Red Hat OpenShift or Kubernetes

- [Display which Pods have the PVC in use](#)
- [Red Hat OpenShift Container Platform 4 now defaults to CRI-O as underlying container engine](#)
- [How to configure kdump in Red Hat CoreOS?](#)
- [Installing and configuring OpenShift Container Platform clusters](#)
- [Customizing nodes](#)
- [Configuring an httpasswd identity provider](#)
- [EU Data Act \(Regulation 2023/2854\)](#)

EU Data Act (Regulation 2023/2854)

Overview

This Data Information Page provides information about data generated by this product. The EU Data Act Terms apply where relevant.

Vital Product Data (VPD) / Machine Readable Product Data (MRPD)

Description

Human-readable information on product data including Serial Number, Machine Type and Model, Date of manufacture, and similar identifying information.

Data Characteristics

- Format: Text files, CSV files, HTTP/HTTPS, JSON or XML files, and other human-readable file types
- Volume: Varies by product, but VPD/MRPD size does not typically change; as of 2025, this data could be up to 4GB
- Generation: Real-time when the product is powered on; most VPD/MRPD data remains static

Storage and Retention

- Storage products can send product data to IBM, sometimes via a service or management console
- Products and consoles can store product data temporarily, deleting based on space management requirements and/or time
- IBM servers store product data for at least 180 days

User Access and Control

- Product data can be obtained by customers through GUIs, CLIs, SNMP, REST interfaces, host commands, and other mechanisms
- Access available through:
 - Service console GUI, CLI, and/or REST interface
 - Product-specific interfaces (GUIs, CLIs, REST interface, SNMP, host commands)

Configuration Data

Description

Human-readable information describing the configuration of the product, identifying parts such as FRUs (Field Replaceable Units) and Feature Codes.

Data Characteristics

- Format: Text files, CSV files, HTTP/HTTPS, JSON or XML files, and other human-readable file types
- Volume: Varies by product and is expected to change with future code product updates and as new features are added; as of 2025, this data could be up to 4GB
- Generation: Real-time when the product is powered on; most configuration/inventory data is stable until the customer upgrades the product

Storage and Retention

- Storage products can send product data to IBM, sometimes via a service or management console
- Products and consoles can store product data temporarily, deleting based on space management requirements and/or time
- IBM servers store product data for at least 180 days

User Access and Control

- Product data can be obtained by customers through GUIs, CLIs, SNMP, REST interfaces, host commands, and other mechanisms
- Access available through:
 - Service console GUI, CLI, and/or REST interface
 - Product-specific interfaces (GUIs, CLIs, REST interface, SNMP, host commands)

Logging Data

Description

Files containing data generated by system use and available to customers, including:

- High-level error logs tracking error information
- Command logs showing information such as read or write commands sent to the storage device
- Access logs tracking users logging into the system and running admin commands

Data Characteristics

- Format: Text files, CSV files, HTTP/HTTPS, JSON or XML files, and other human-readable file types
- Volume: Varies by product and is expected to change with future code product updates and as new features are added; as of 2025, this data could be up to 4GB
- Generation: Real-time when the product is powered on; logging runs continuously and is finite in length, with older data removed as newer data is generated

Storage and Retention

- Storage products can send product data to IBM, sometimes via a service or management console
- Products and consoles can store product data temporarily, deleting based on space management requirements and/or time
- IBM servers store product data for at least 180 days

User Access and Control

- Product data can be obtained by customers through GUIs, CLIs, SNMP, REST interfaces, host commands, and other mechanisms
- Access available through:
 - Service console GUI, CLI, and/or REST interface
 - Product-specific interfaces (GUIs, CLIs, REST interface, SNMP, host commands)

Product documentation in guide format

Table 1. Product documentation in PDF format

| Links |
|---|
| - IBM Storage Scale Container Native Storage Access 5.1.2 |
| - IBM Storage Scale Container Native Storage Access 5.1.3 |
| - IBM Storage Scale Container Native Storage Access 5.1.4 |
| - IBM Storage Scale Container Native Storage Access 5.1.5 |
| - IBM Storage Scale Container Native Storage Access 5.1.6 |
| - IBM Storage Scale Container Native Storage Access 5.1.7 |
| - IBM Storage Scale Container Native Storage Access 5.1.9 |
| - IBM Storage Scale Container Native Storage Access 5.2.0 |
| - IBM Storage Scale Container Native Storage Access 5.2.1 |
| - IBM Storage Scale Container Native Storage Access 5.2.2 |
| - IBM Storage Scale Container Native Storage Access 5.2.3 |
| - IBM Storage Scale Container Native Storage Access 6.0.0 |